第四章　存储器



大　纲

(一)存储器的分类

(二)存储器的层次化结构

(三)半导体随机存取存储器

1. SRAM存储器的工作原理

2.DRAM存储器的工作原理

(四)只读存储器

(五)主存储器与CPU的连接

(六)双口RAM和多模块存储器

(七)高速缓冲存储器(Cache)

1．　程序访问的局部性原理

2．　Cache的基本工作原理

3.Cache和主存之间的映射方式

4. Cache中主存块的替换算法

5.Cache写策略

（八）　虚拟存储器

1. 虚拟存储器的基本概念

2. 页式虚拟存储器

3. 段式虚拟存储器

4. 段页式虚拟存储器

5. TLB（快表）

Contents

1 4.1 概 述

2 4.2 主存储器

3 4.3 高速缓冲存储器

4 4.4 辅助存储器

5 4.5 虚拟存储器

4.1 概 述

回顾：存储器基本术语

- 记忆单元 （存储基元 / 存储元 / 位元） （Cell)
  - 具有两种稳态的能够表示二进制数码0和1的物理器件
- 存储单元 / 编址单位（Addressing Unit)
  - 主存中具有相同地址的位构成一个存储单元，也称为一个编址单位
- 存储体/ 存储矩阵 / 存储阵列（Bank)
  - 所有存储单元构成一个存储阵列
- 编址方式（Addressing Mode)
  - 字节编址、按字编址
- 存储器地址寄存器（Memory Address Register - MAR)
  - 用于存放主存单元地址的寄存器
- 存储器数据寄存器（ Memory Data Register-MDR (MBR) ）
  - 用于存放主存单元中的数据的寄存器

4.1 概 述

一、存储器分类

1. 按存储介质分类

(1) 半导体存储器　双极型　静态MOS型　动态MOS型　易失

(2) 磁表面存储器　磁盘（Disk）、磁带 （Tape）

(3) 磁芯存储器　硬磁材料、环状元件
（破坏性读出）

(4) 光盘存储器　激光、磁光材料

非易失

一、存储器分类

2. 按存取方式分类

(1)存取时间与物理地址无关（随机访问）

- 每个单元读写时间一样，且与各单元所在位置无关。如：内存。
- 随机存储器　在程序的执行过程中 可 读 可 写
- 只读存储器　在程序的执行过程中 只读

(2) 存取时间与物理地址有关（串行访问）

- 顺序存取存储器 (Sequential Access Memory, SAM)

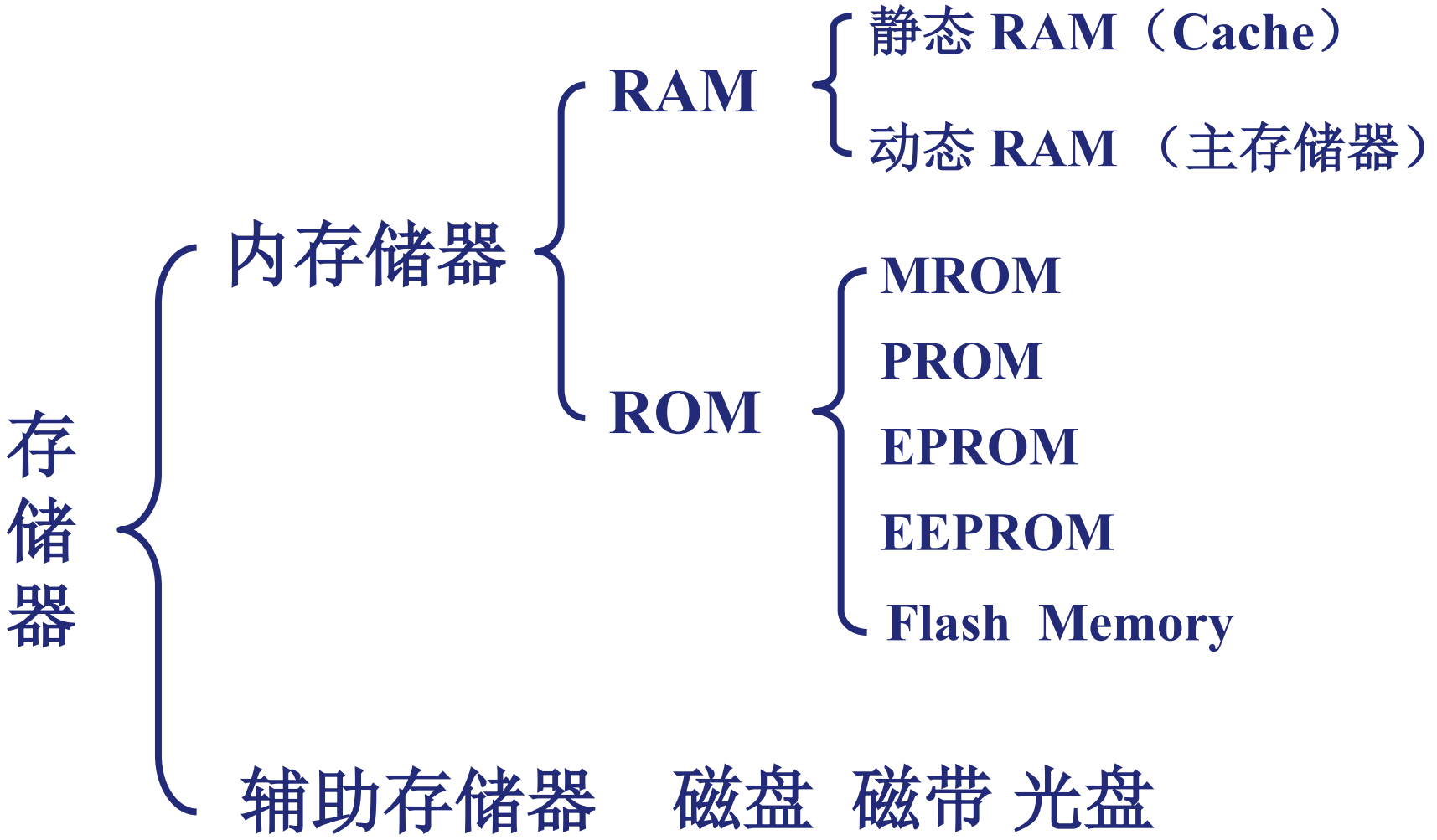
• 数据按顺序从存储载体的始端读出或写入，因而存取时间的长短与信息所在位置有关。例如：磁带。

- 直接存取存储器 (Direct Access Memory, DAM)

• 直接定位到要读写的数据块，在读写某个数据块时按顺序进行。例如：磁盘。

- 相联存储器(Associate Memory/Content Addressed Memory CAM)
  - 按内容检索到存储位置进行读写。例如：快表。

3. 按在计算机中的作用分类



4. 按断电后信息的可保存性分类

- 非易失（不挥发）性存储器(Nonvolatile Memory)
  - 信息可一直保留， 不需电源维持。

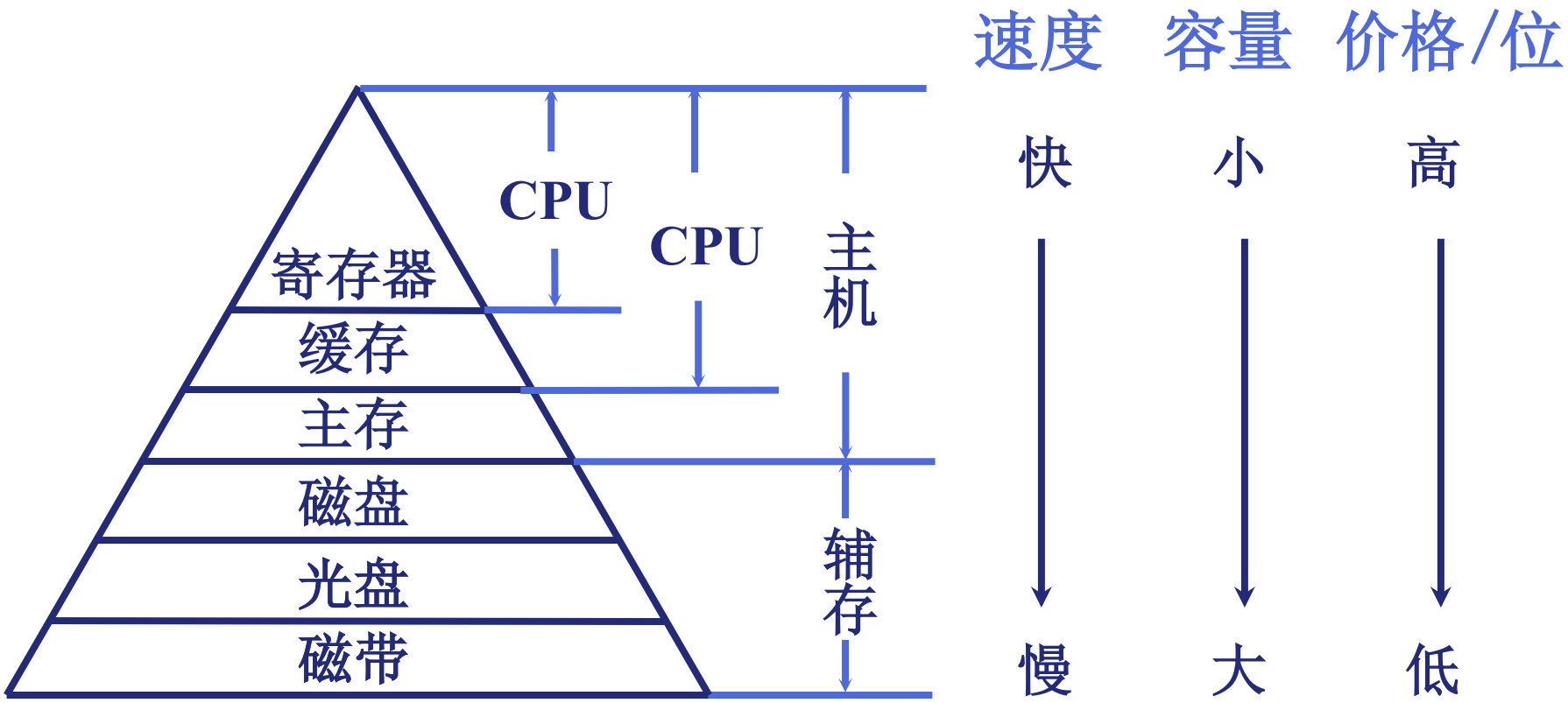
  （如：ROM、磁表面存储器、光存储器等）
- 易失（挥发）性存储器(Volatile Memory)
  - 电源关闭时信息自动丢失。（如：RAM、Cache等）

5. 按功能/容量/速度/所在位置分类

- 寄存器(Register)
  - 封装在CPU内，用于存放当前正在执行的指令和使用的数据
  - 用触发器实现，速度快，容量小（几十个）
- 高速缓存(Cache)
  - 位于CPU内部或附近，用来存放当前要执行的局部程序段和数据
  - 用SRAM实现，速度可与CPU匹配，容量小（几MB）
- 内存储器MM（主存储器Main (Primary) Memory）
  - 位于CPU之外，用来存放已被启动的程序及所用的数据
  - 用DRAM实现，速度较快，容量较大（几GB）
- 外存储器AM (辅助存储器Auxiliary / Secondary Storage)
  - 位于主机之外，用来存放暂不运行的程序、数据或存档文件
  - 用磁表面或光存储器实现，容量大而速度慢

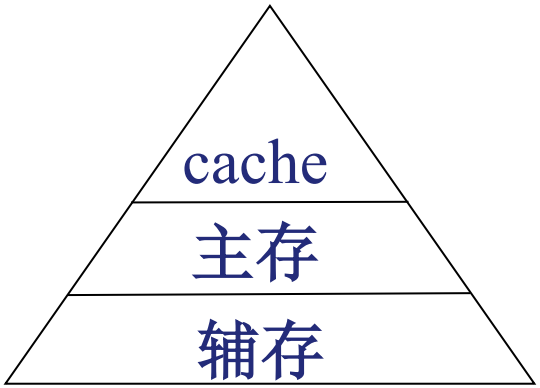
二、存储器的层次结构

1. 存储器三个主要特性的关系

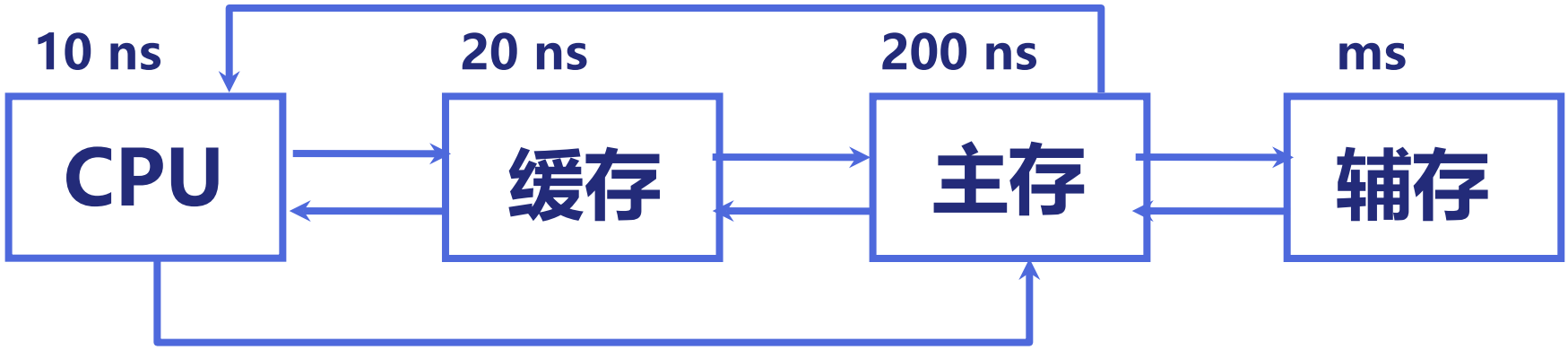


2. 三级存储器结构

- 由高速缓冲存储器、主存储器和外存储器组成
  - 内存储器：CPU能直接访问的存储器
    - 包括高速缓冲存储器和主存储器
  - 外存储器：CPU不能直接访问的存储器，外存储器的信息必须调入内存储器后才能由CPU进行处理



缓存—主存层次和主存—辅存层次

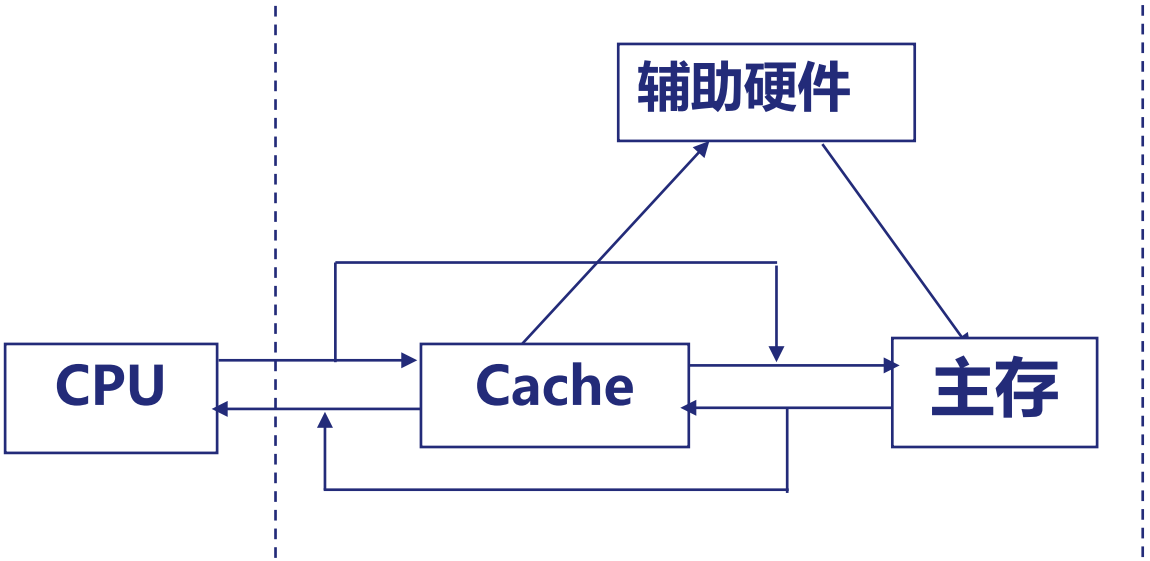


（速度）　　（容量）

缓存－主存　　主存－辅存

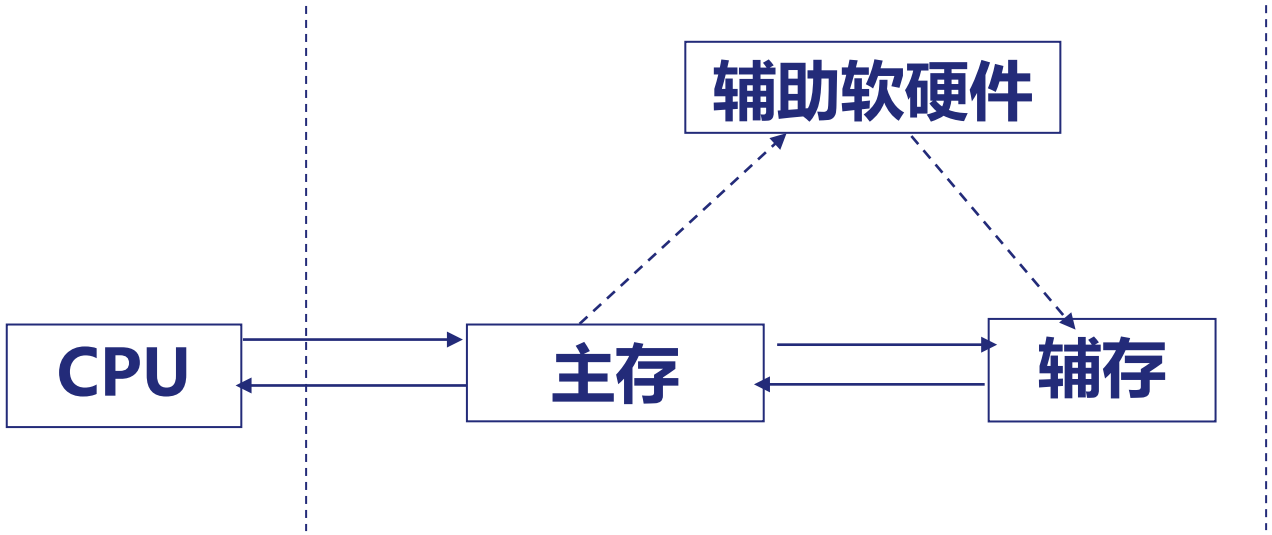
主存储器	虚拟存储器
实地址	虚地址
物理地址	逻辑地址

Cache-主存存储层次（Cache存储系统）



- Cache存储系统是为了解决主存速度不足而提出来的。
- 在Cache和主存之间增加辅助硬件，让它们构成一个整体
- 从CPU看，速度接近Cache，容量是主存的容量，位价格 接近于主存价格
- 由于Cache存储系统全部采用硬件来调度，因此对系统程序员和应用程序员是透明的

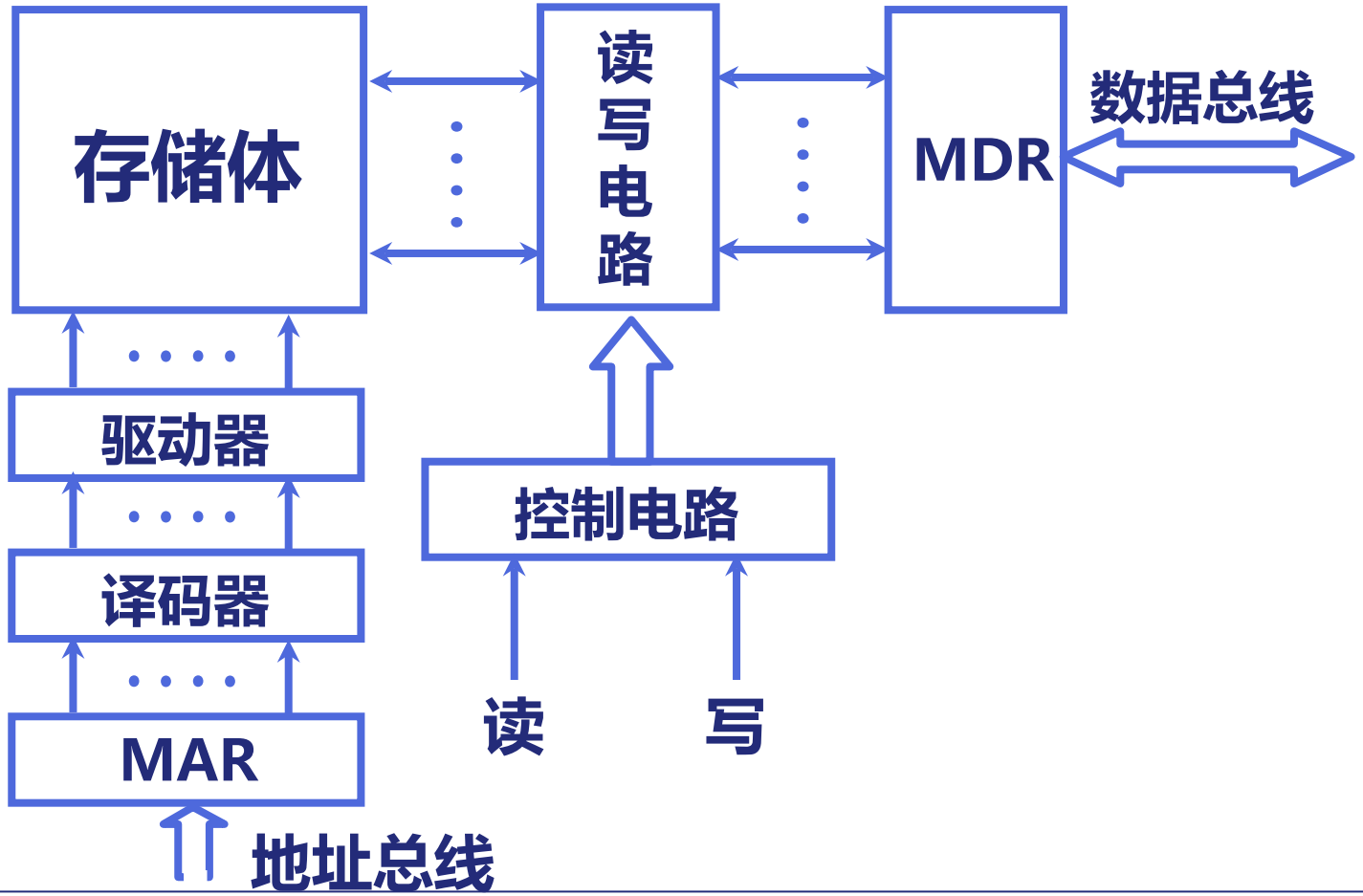
主存 - 辅存存储层次（虚拟存储系统）



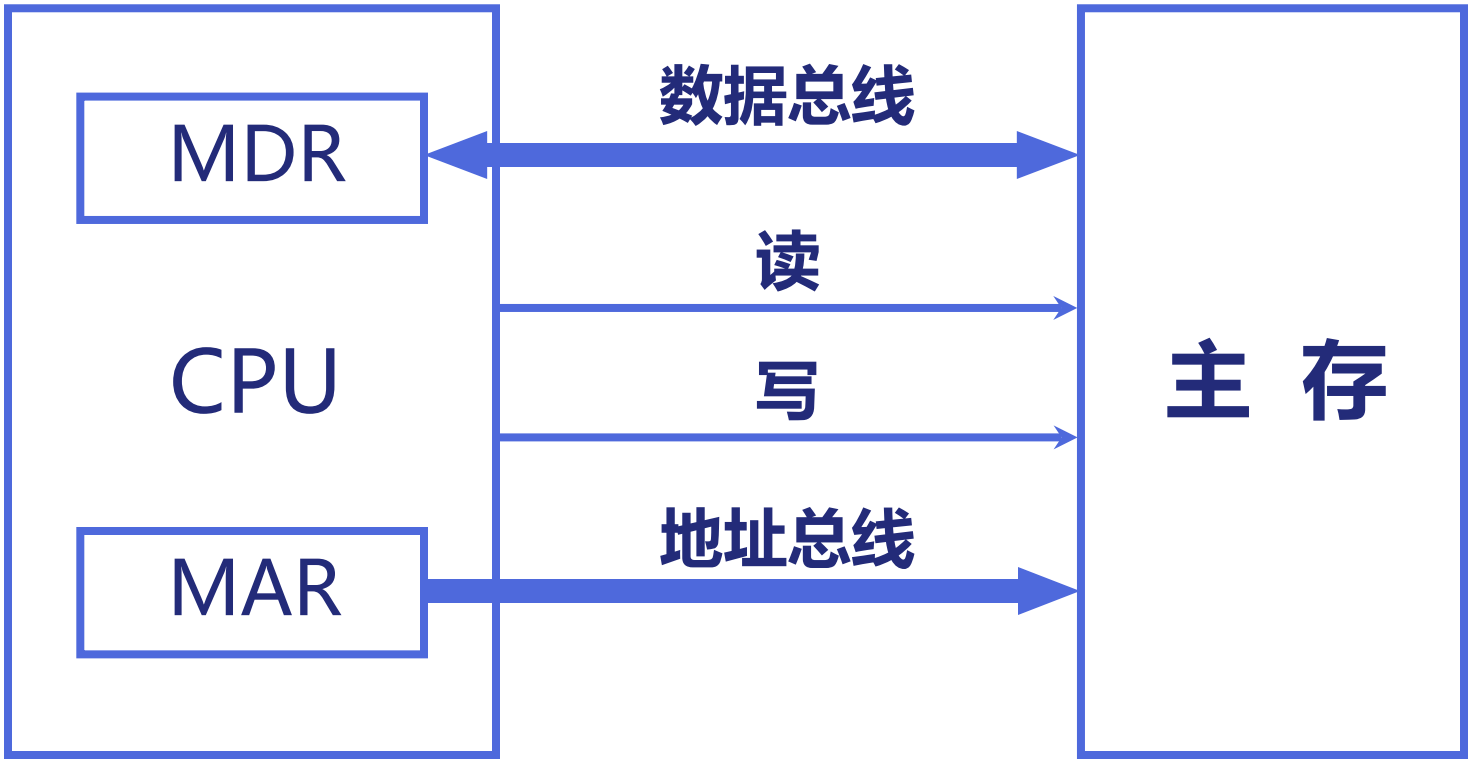
- 虚拟存储器是为了解决主存容量不足而提出来的。
- 在主存和辅存之间，增加辅存的软硬件，让它们构成一个整体
- 从CPU看，速度接近主存的速度，容量是虚拟的地址空间，每位价格是接近辅存的价格
- 由于虚拟存储系统需要通过操作系统来调度，因此对系统程序员是不透明的，但对应用程序员是透明的

一、概 述

1. 主存的基本组成



2. 主存和 CPU 的联系



3. 主存中存储单元地址的分配

高位字节 地址为字地址

字地址	字节地址			
0	0	1	2	3
4	4	5	6	7
8	8	9	10	11

低位字节 地址为字地址

字地址	字节地址	
0	1	0
2	3	2
4	5	4

设地址线 24 根　　按 字节 寻址　2^{24} = 16 M

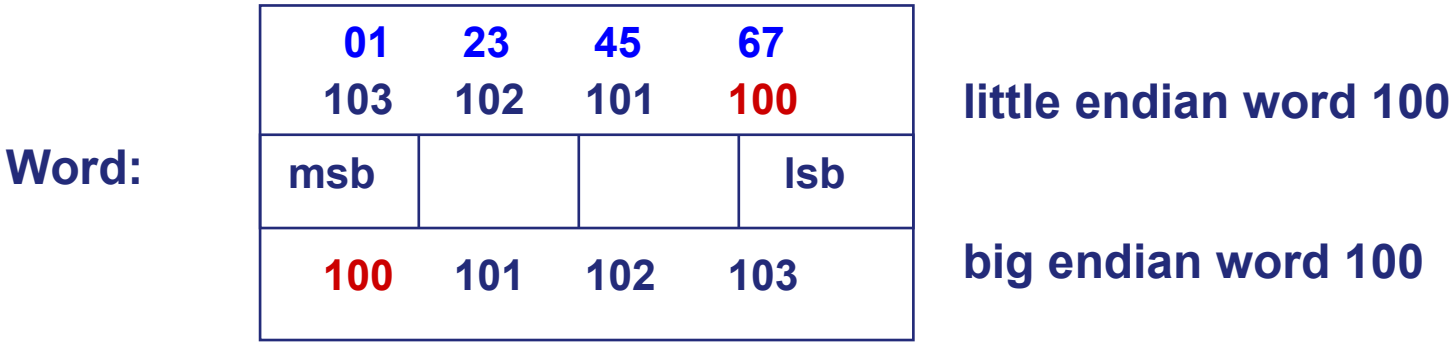
若字长为 16 位　　按 字 寻址　8 M

若字长为 32 位　　按 字 寻址　4 M

数据的存储和排列顺序

- 80年代开始，几乎所有机器都用字节编址
- ISA设计时要考虑的两个问题：
  - **如何从一个字节地址中取到一个32位的字？- 字的存放问题**
  - **一个字能否存放在任何字节边界？- 字的边界对齐问题**

例如，若 int i = 0x01234567，存放在内存100号单元，则用“取数”指令访问100号单元取出 i 时，必须清楚i的4个字节是如何存放的。



大端方式（Big Endian）：MSB所在的地址是数的地址

e.g. IBM 360/370, Motorola 68k, MIPS, Sparc, HP PA

小端方式（ Little Endian）：LSB所在的地址是数的地址

e.g. Intel 80x86, DEC VAX

有些机器两种方式都支持，可以通过特定的控制位来设定采用哪种方式。

BIG Endian versus Little Endian

Ex1: Memory layout of a number ABCDH located in 1000

In Big Endian: →

CD	1001
AB	1000

In Little Endian: →

AB	1001
CD	1000

Ex2: Memory layout of a number 00ABCDEFH located in 1000

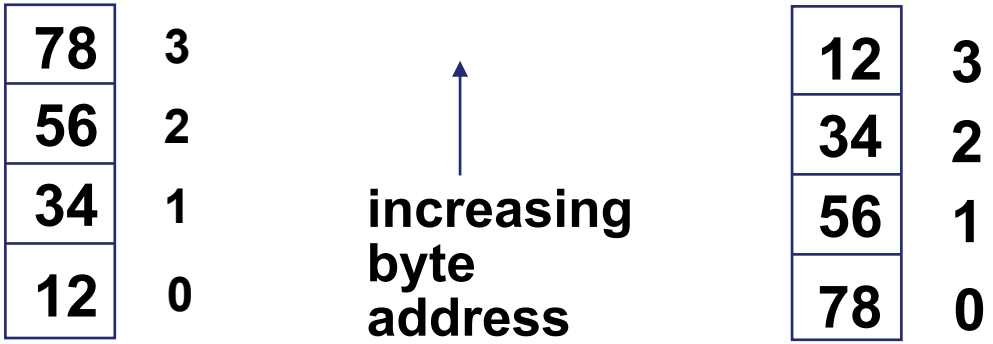
In Big Endian: →

00	1000
AB	1001
CD	1002
EF	1003

In Little Endian: →

00	1003
AB	1002
CD	1001
EF	1000

Byte Swap Problem（字节交换问题）



上述存放在0号单元的数据（字）是什么？12345678H？ 78563412H？

存放方式不同的机器间程序移植或数据通信时，会发生什么问题？

- 每个系统内部是一致的，但在系统间通信时可能会发生问题！
- 因为顺序不同，需要进行顺序转换

音、视频和图像等文件格式或处理程序都涉及到字节顺序问题

ex. Little endian: GIF, PC Paintbrush, Microsoft RTF,etc

Big endian: Adobe Photoshop, JPEG, MacPaint, etc

Alignment(对齐)

Alignment: 要求数据的地址是相应的边界地址

- 目前机器字长一般为32位或64位，而存储器地址按字节编址
- 指令系统支持对字节、半字、字及双字的运算，也有位处理指令
- 各种不同长度的数据存放时，有两种处理方式：
  - 按边界对齐 （假定字的宽度为32位，按字节编址）
    - 字地址：4的倍数（低两位为0）
    - 半字地址：2的倍数（低位为0）
    - 字节地址：任意
  - 不按边界对齐

    坏处：可能会增加访存次数！

Alignment(对齐)

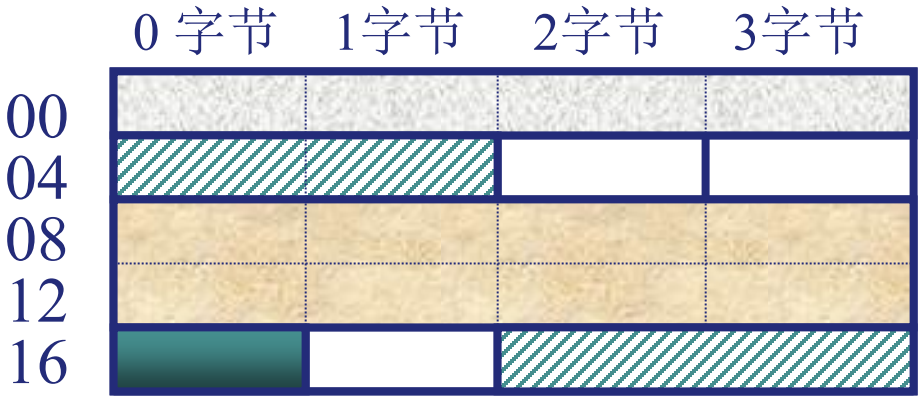
示例　假设数据顺序：字-半字-双字-字节-半字-……

如：int i, short k, double x, char c, short j,……

按边界对齐

x：2个周期

j：1个周期

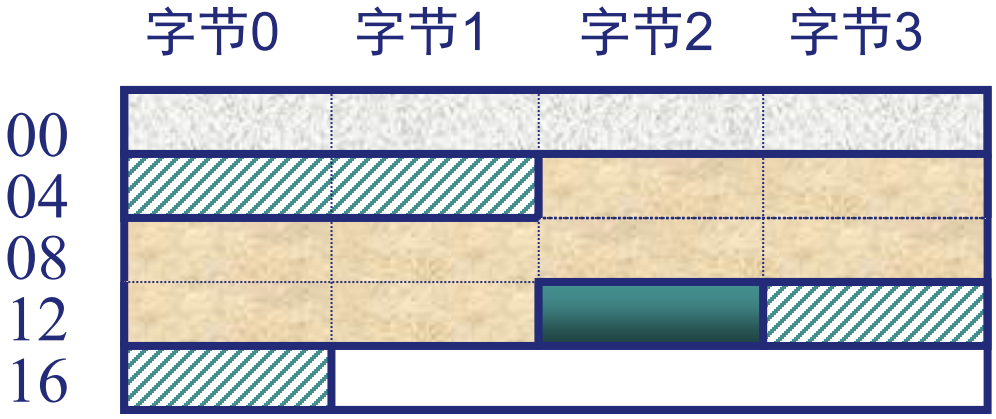


则：&i=0; &k=4; &x=8; &c=16; &j=18;……

边界不对齐

x：3个周期

j：2个周期



增加了访存次数！

则：　&i=0; &k=4; &x=6; &c=14; &j=15;……

4. 主存的性能指标

(1) 存储容量　主存 存放二进制代码的总数量

- 以字编址的计算机：字数与字长的乘积

存储容量 = 存储单元个数×存储字长

64K×16位：64K个存储单元，每个存储单元字长为16位

- 以字节编址的计算机，以字节数来表示

存储容量 = 存储单元个数×存储字长/8

64K×16位用字节数表示：128K字节（128KB）

4. 主存的技术指标

(2) 存储速度

- 存取时间T_A（Memory Access Time）：从CPU送出内存单元的地址码开始，到主存读出数据并送到CPU（或者是把CPU数据写入主存）所需要的时间（单位：ns，1 ns = 10^{-9} s）

读出时间　写入时间

- 存取周期T_{MC}（Memory Cycle Time）：连读两次访问存储器所需的最小时间间隔，它应等于存取时间加上下一存取开始前所要求的附加时间，因此，T_{MC}比T_A大（因为存储器由于读出放大器、驱动电路等都有一段稳定恢复时间，所以读出后不能立即进行下一次访问。）

读周期　写周期

- $T_{MC} > T_A$，因为存储器在读写操作后，要有一段恢复内部状态的复原时间。对于破坏性读出的RAM，存取周期往往比存取时间要大的多，甚至可达到$T_{MC} = 2T_A$，这是因为存储器中的信息读出后需要马上进行再生（重写）。

4. 主存的技术指标

(3) 存储器的带宽:单位时间内存储器存取的信息量

单位：字/秒，字节/秒，位/秒

- 提高主存的带宽的措施
  - 缩短存取周期
  - 增加存储字长
  - 增加存储体

4. 主存的技术指标

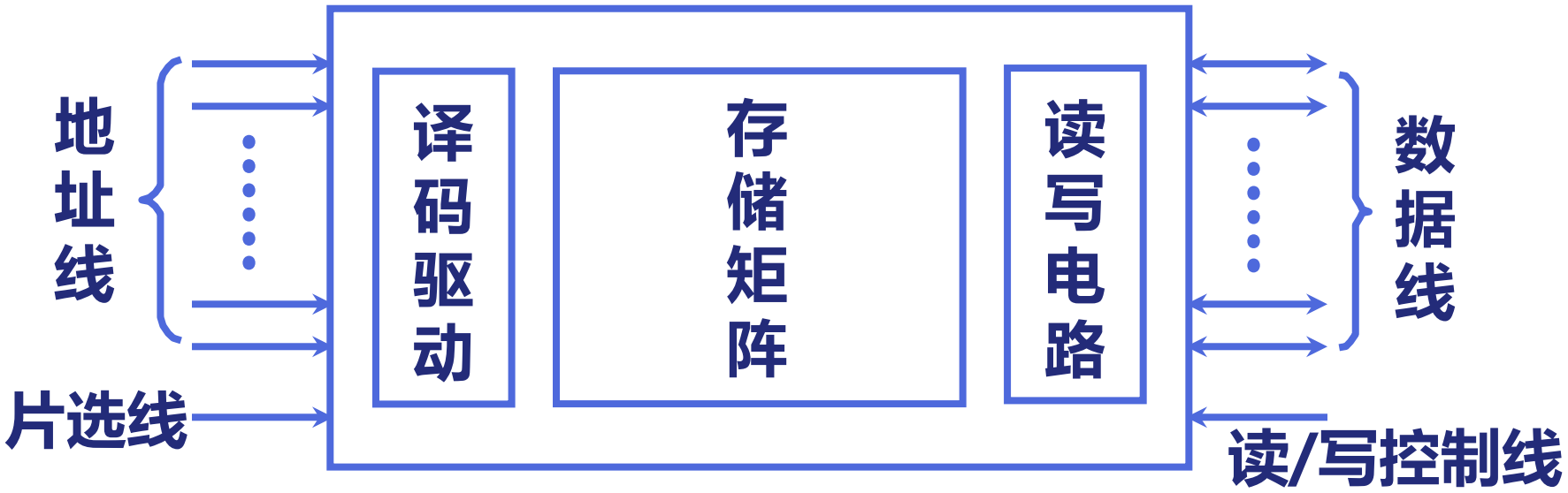
(4) 可靠性：指在规定时间内，存储器无故障读写的概率。

- 平均无故障时间（Mean Time Between Failures，MTBF）用于衡量可靠性。
- MTBF为两次故障之间的平均时间间隔。MTBF越长，说明存储器可靠性越高。

（5）功耗：功耗既反映耗电又反映其发热程度。

二、半导体存储芯片简介

1．半导体存储芯片的基本结构

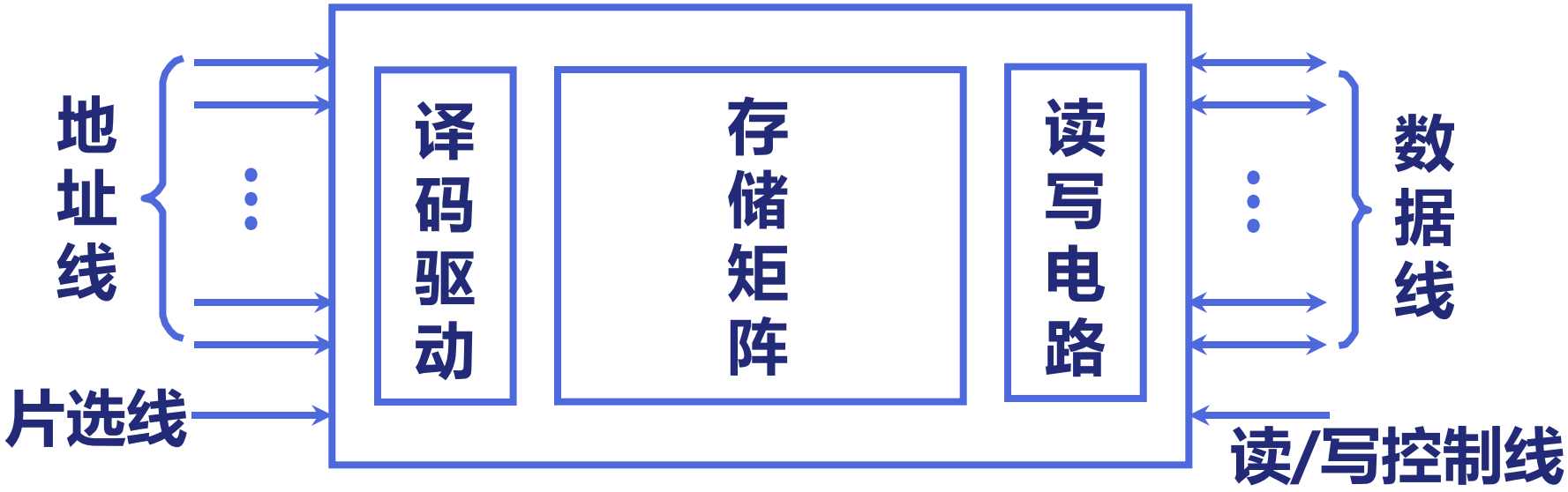


地址线（单向）	数据线（双向）	芯片容量
10	4	1K × 4位
14	1	16K × 1位
13	8	8K × 8位


SAMSUNG
Made in China
2GB 2Rx8 PC2-6400S-666-12-E3
M470T5663QZ3-CF7 0924
SEC 922 HCF7
K4T1G08400
SAMSUNG
Made in China
2GB 2Rx8 PC2-6400S-666-12-E3
M470T5663QZ3-CF7 0924
SEC 922 HCF7
K4T1G08400

二、半导体存储芯片简介

1．半导体存储芯片的基本结构



片选线 $\overline{CS}$ $\overline{CE}$

读/写控制线 $\overline{WE}$（低电平写 高电平读）

$\overline{OE}$（允许读） $\overline{WE}$（允许写）

存储芯片片选线的作用

- 由于一块集成芯片容量有限，要组成一个大的存储器一般需要将多个芯片连接使用。
- 在使用存储器某个地址时，某些芯片需要使用，其它芯片暂时不用，即片选。
- 当芯片的片选信号（$\overline{CS}$）有效时，芯片被选中，此片所连的地址线才有效，才能对它进行读写操作

存储芯片片选线的作用

用16K × 1位 的存储芯片组成 64K × 8位 的存储器

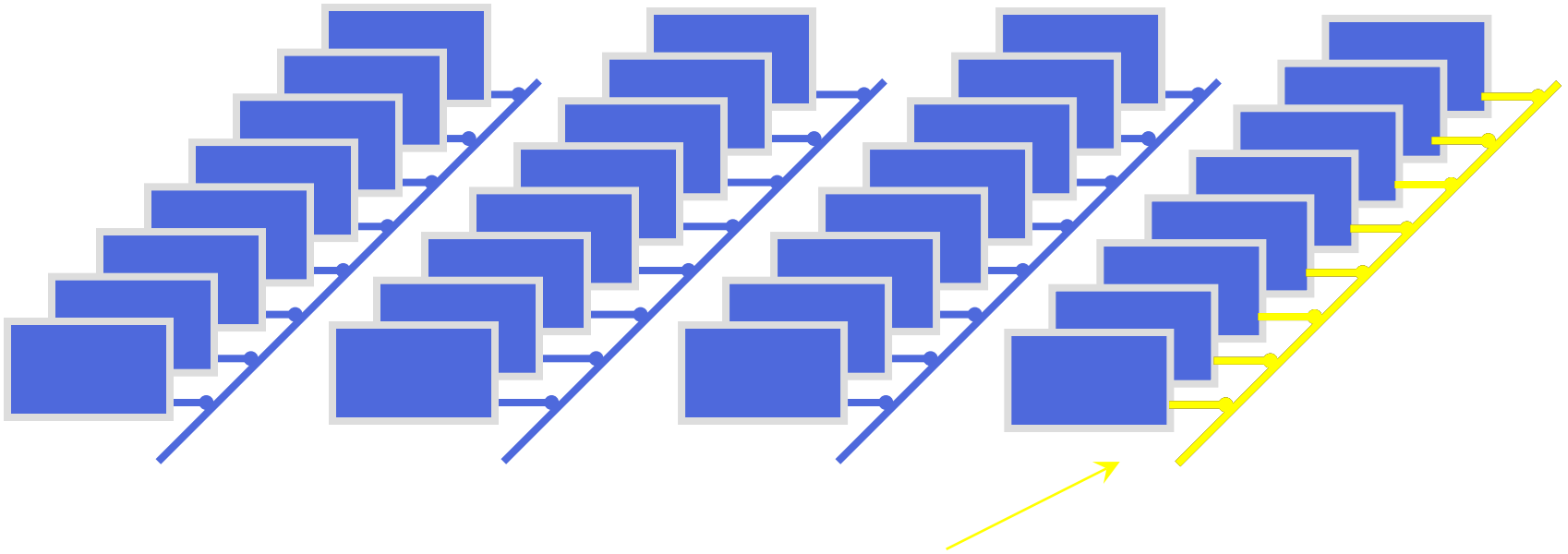
32片

8片 16K × 1位

8片 16K × 1位

8片 16K × 1位

8片 16K × 1位



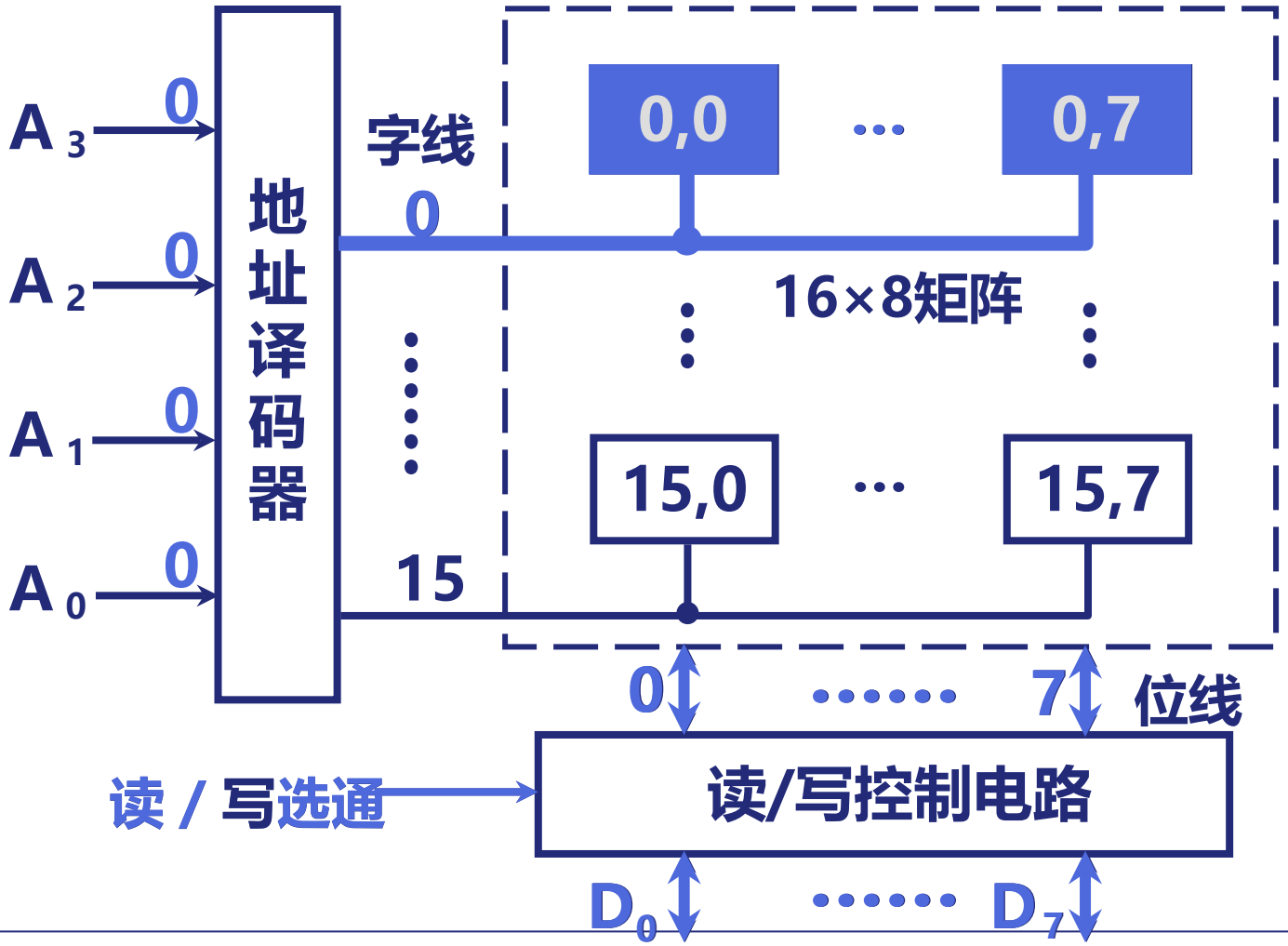
当地址为 65 535 时，此 8 片的片选有效

2. 半导体存储芯片的译码驱动方式

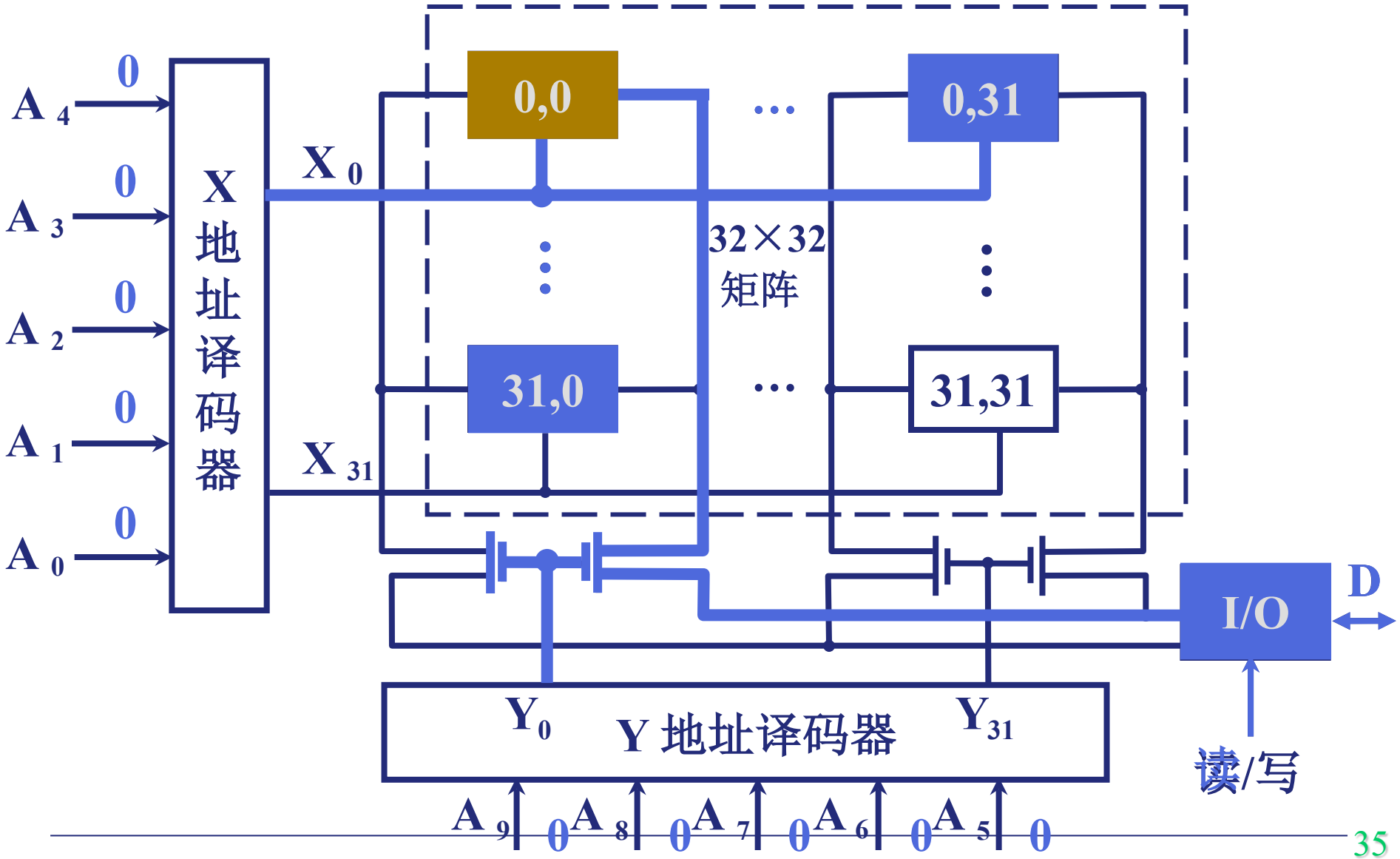
- 地址译码器设计方案
  - 线选法（单译码）：地址译码器只有一个，用一根字线直接选中一个存储单元的各位
    - 当地址线数较大，译码器复杂、开销大，使得存储器成本迅速增加，性能下降
  - 重合法（双译码）：X地址译码器、Y地址译码器

2. 半导体存储芯片的译码驱动方式

(1) 线选法



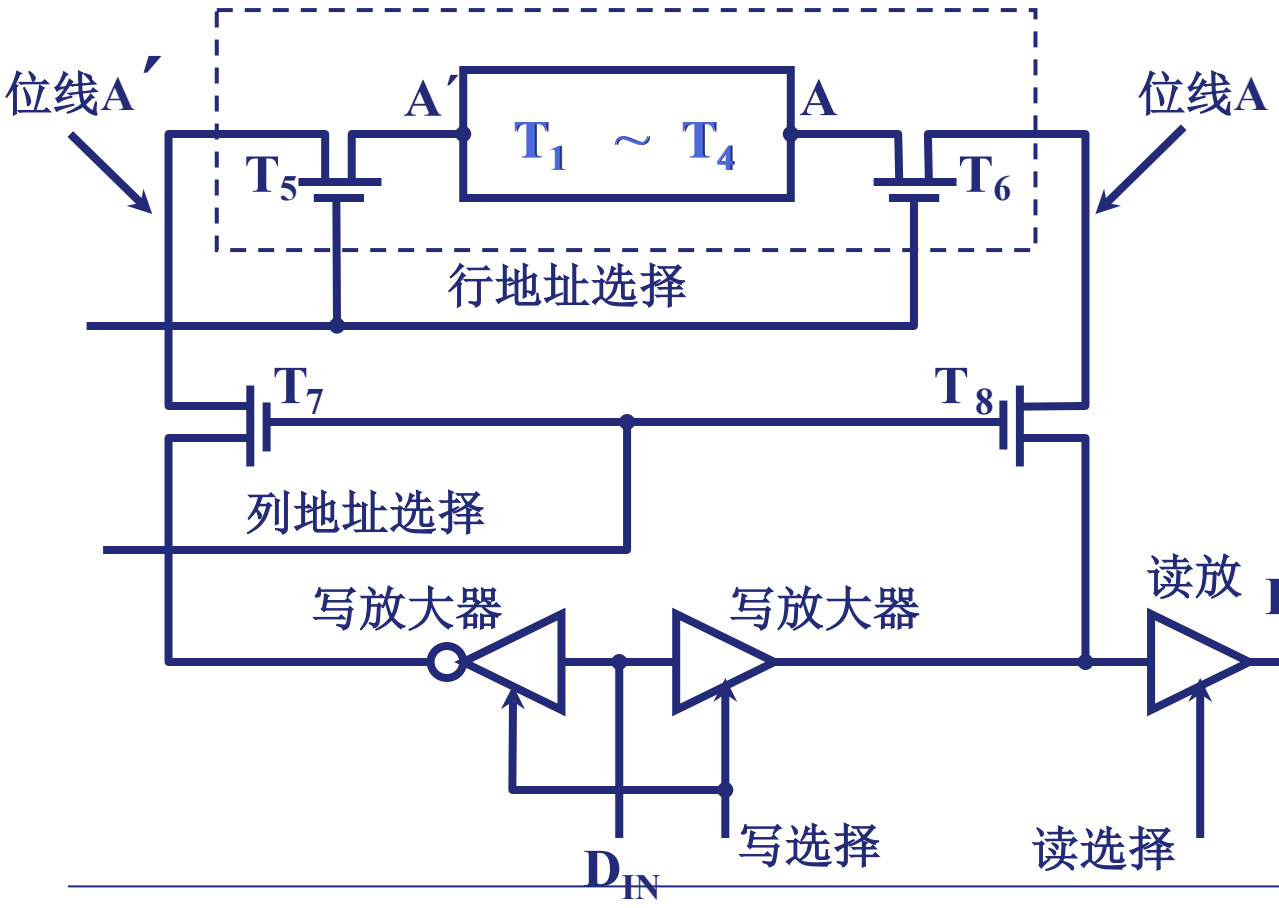
(2) 重合法


A 4
A 3
A 2
A 1
A 0
0
0
0
0
0
X
地
址
译
码
器
X 0
X 31
0,0
0,31
31,0
31,31
32×32
矩阵
I/O
D
读/写
Y 0
Y 地址译码器
Y 31
A 9 0 A 8 0 A 7 0 A 6 0 A 5 0

三、随机存取存储器 (RAM)

1. 静态 RAM (SRAM)

(1) 静态 RAM 基本电路



$T_1 \sim T_4$ 触发器

T_5、T_6 行开关

T_7、T_8 列开关

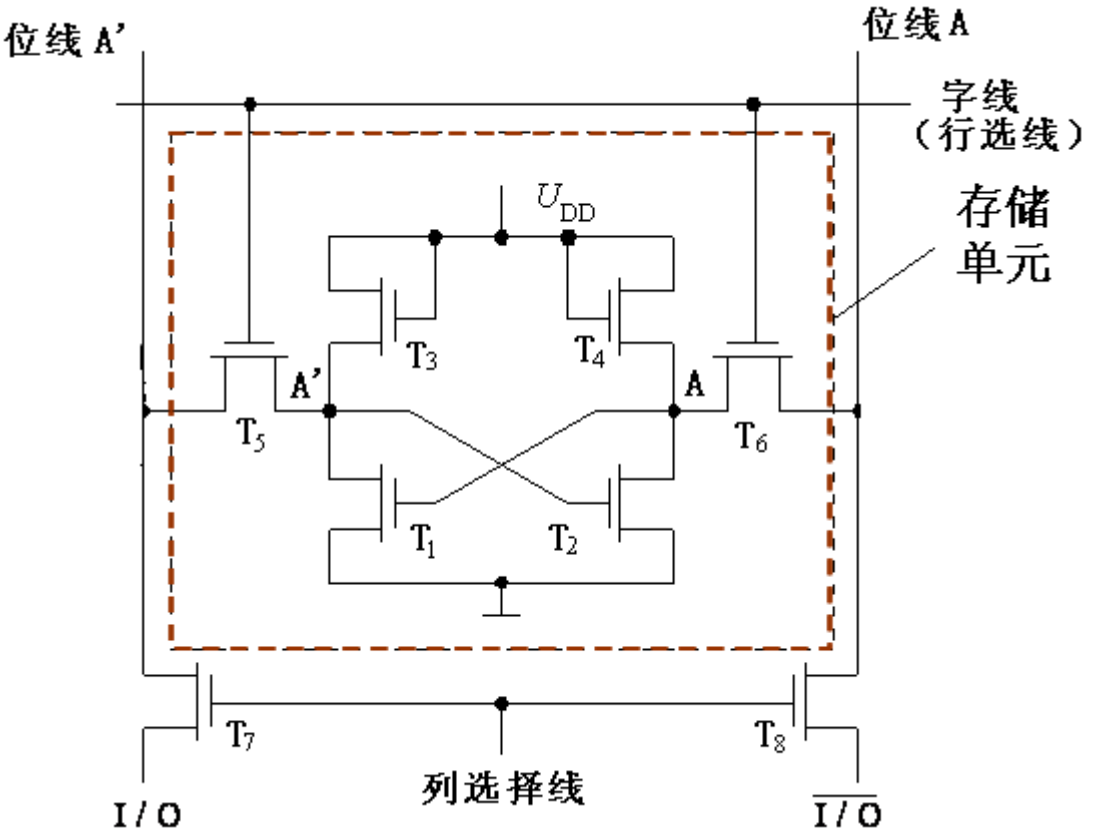
T_7、T_8 一列共用

A 触发器原端

A′ 触发器非端

六管静态MOS管电路

记忆单元



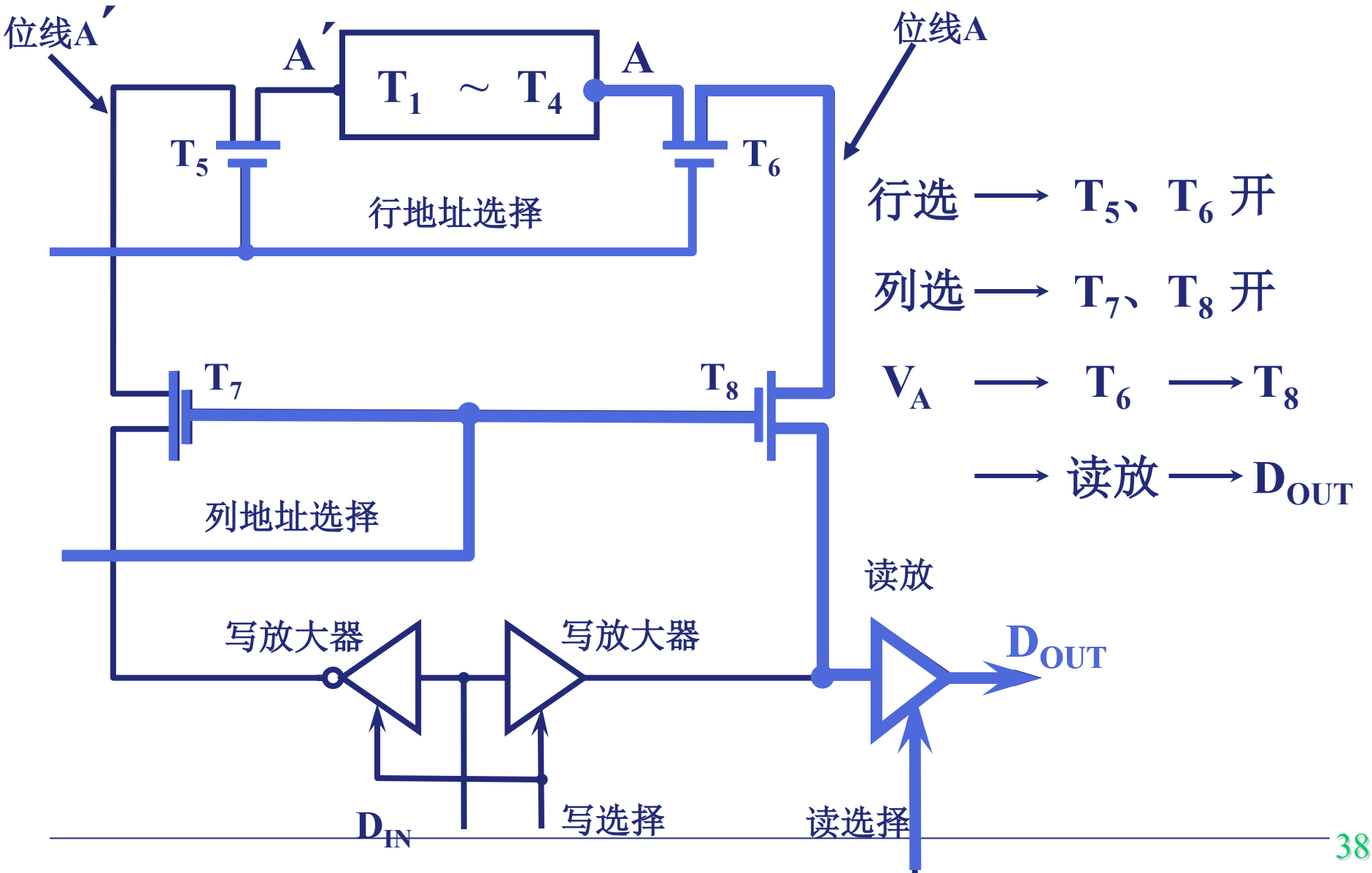
信息存储原理： 看作带时钟的RS触发器

写入时：
- 位线上是被写入的二进位信息0或1
- 置字线为1
- 存储单元(触发器)按位线的状态设置成0或1

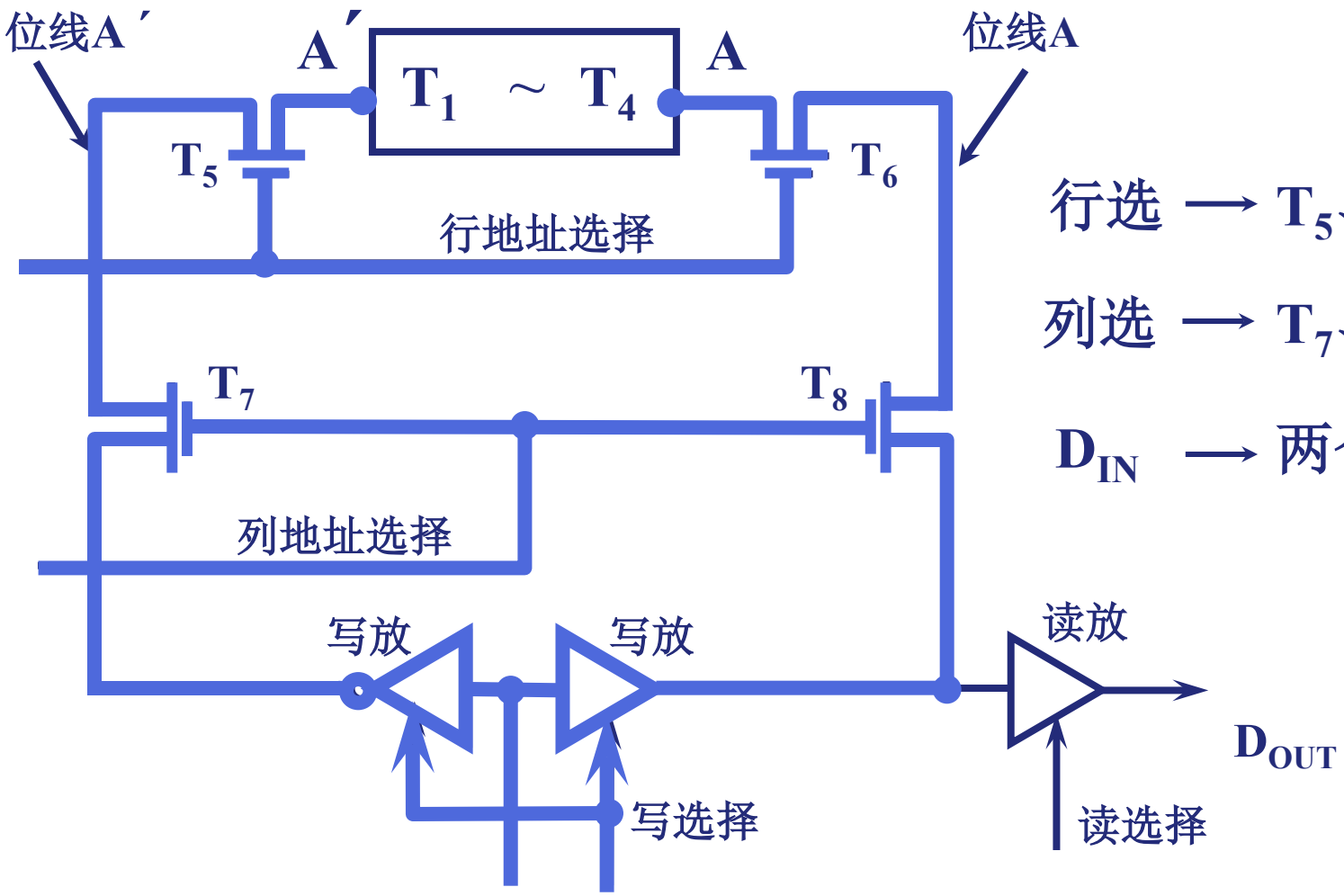
读出时：
- 置2个位线为高电平
- 置字线为1
- 存储单元状态不同，位线的输出不同

SRAM中数据保存在一对正负反馈门电路中，只要供电，数据就一直保持，不是破环性读出，也无需重写。即：无需刷新！

① 静态 RAM 基本电路的 读 操作



② 静态 RAM 基本电路的 写 操作



行选 → T_5、T_6 开

列选 → T_7、T_8 开

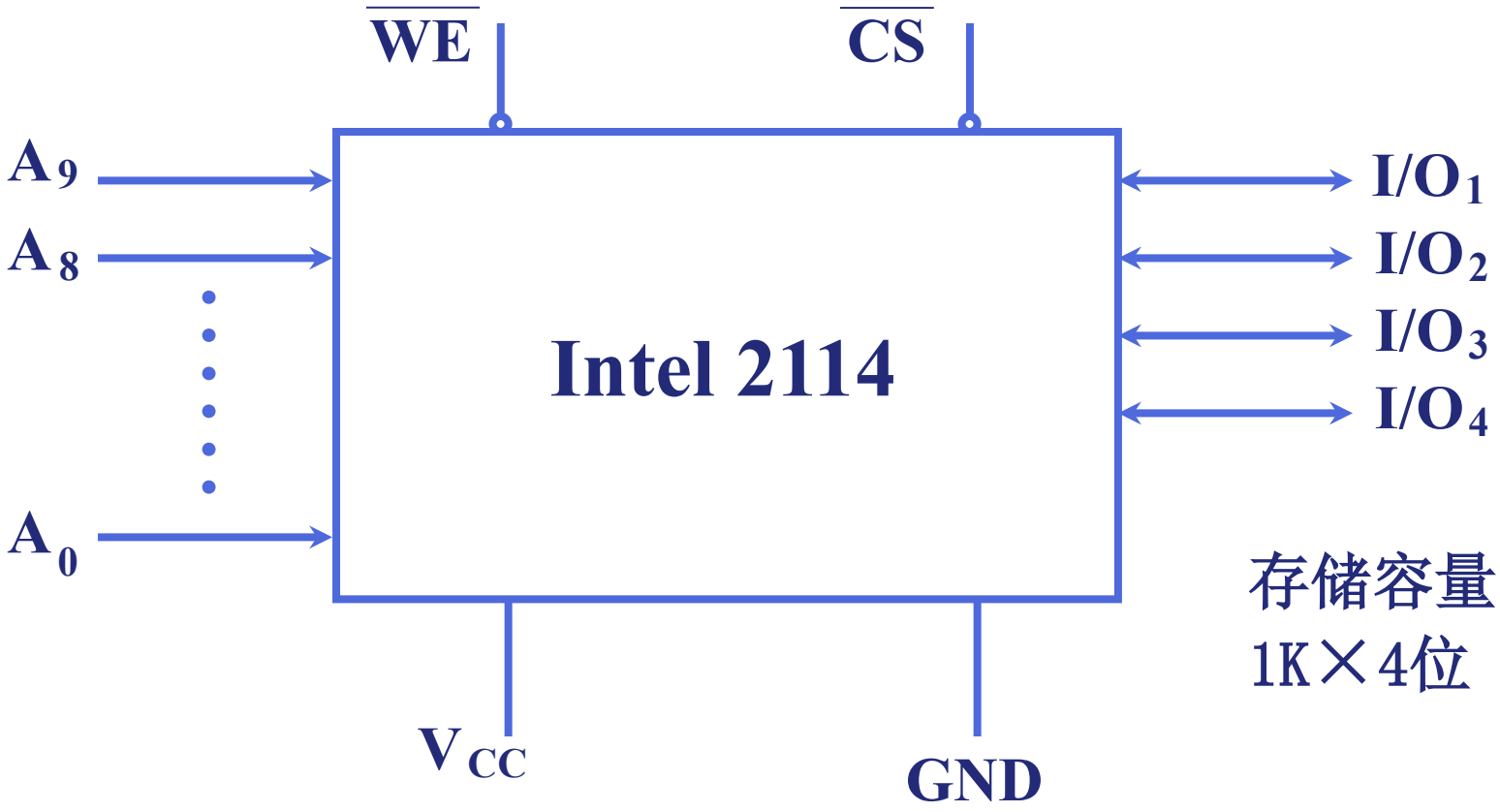
D_{IN} → 两个写放

（左）D_{IN} → 反相 → T_7 → T_5 → A′

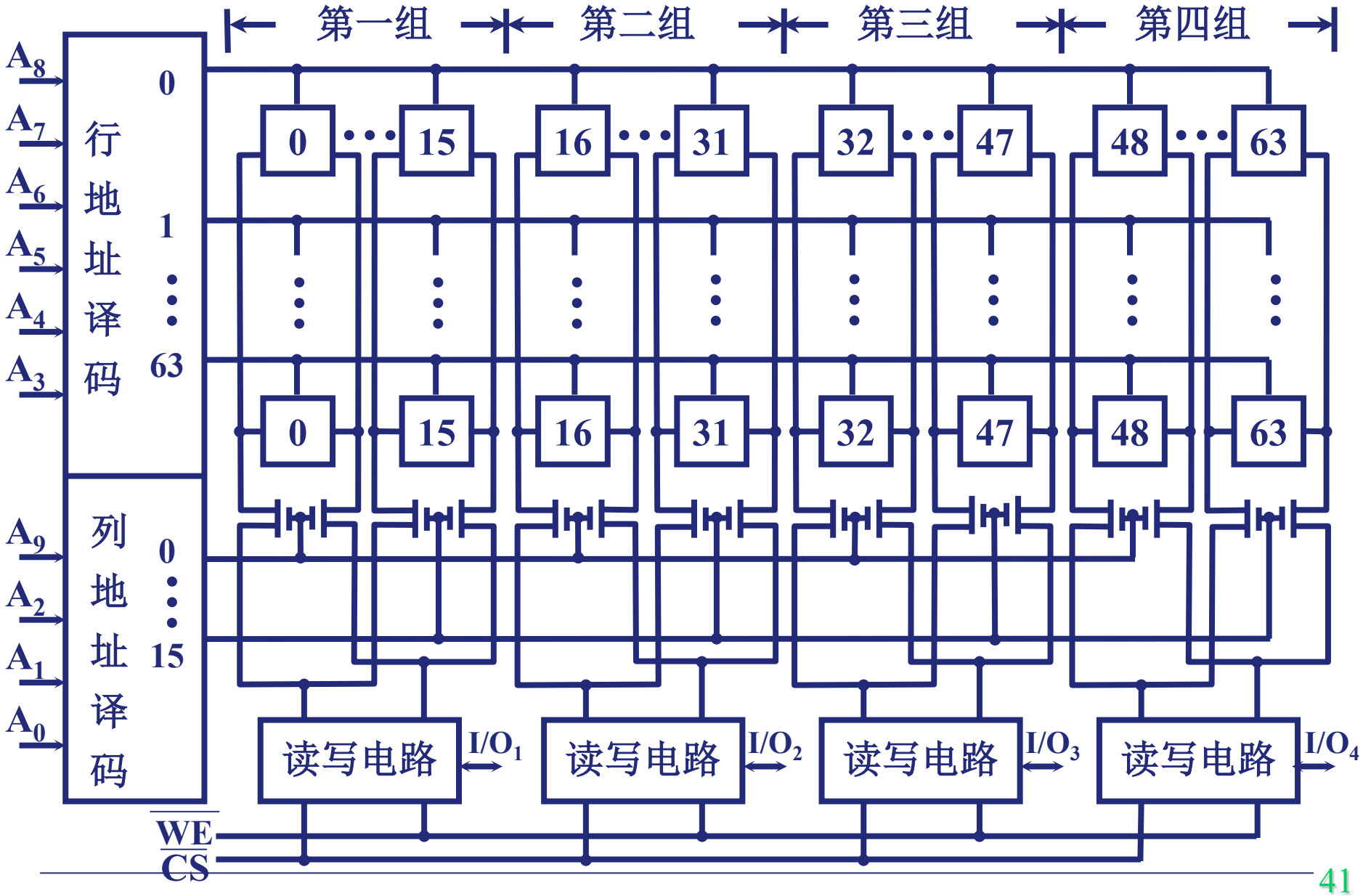
（右）D_{IN} → T_8 → T_6 → A

(2) 静态 RAM 芯片举例

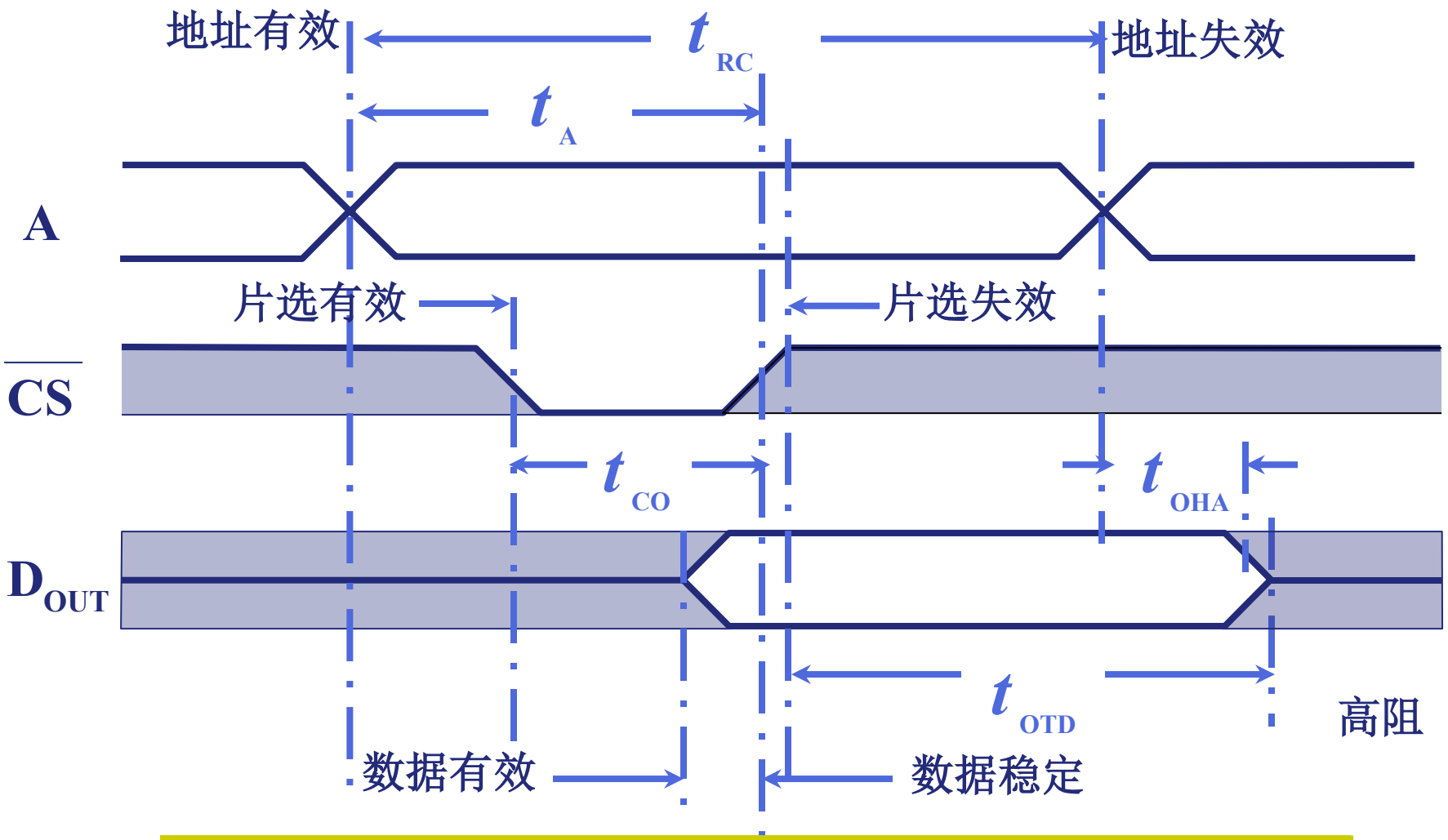
① Intel 2114 外特性



② Intel 2114 RAM 矩阵 (64 × 64) 读

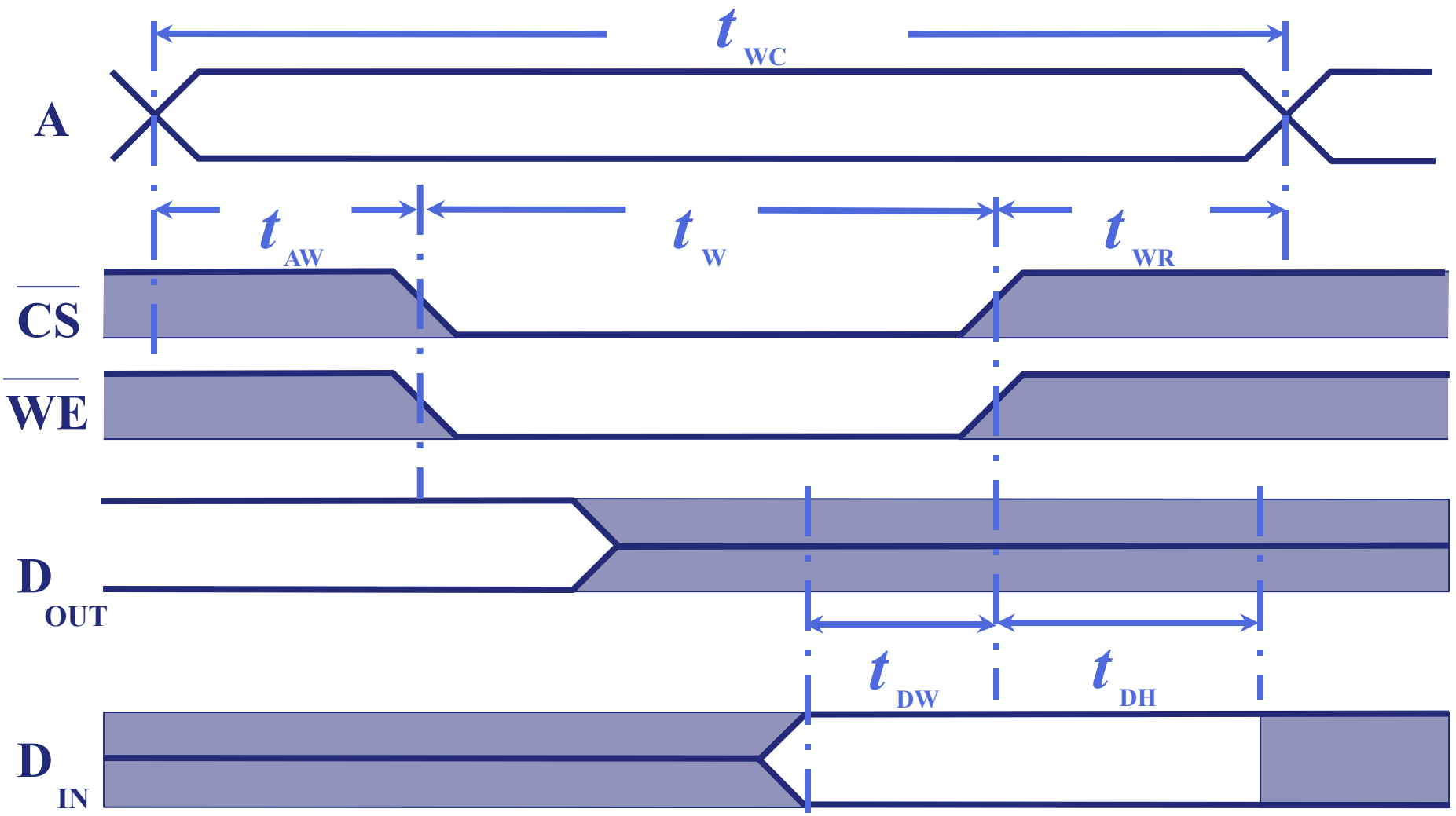


(3) 静态 RAM 读 时序



t_{OHA} 地址失效后的 数据维持时间

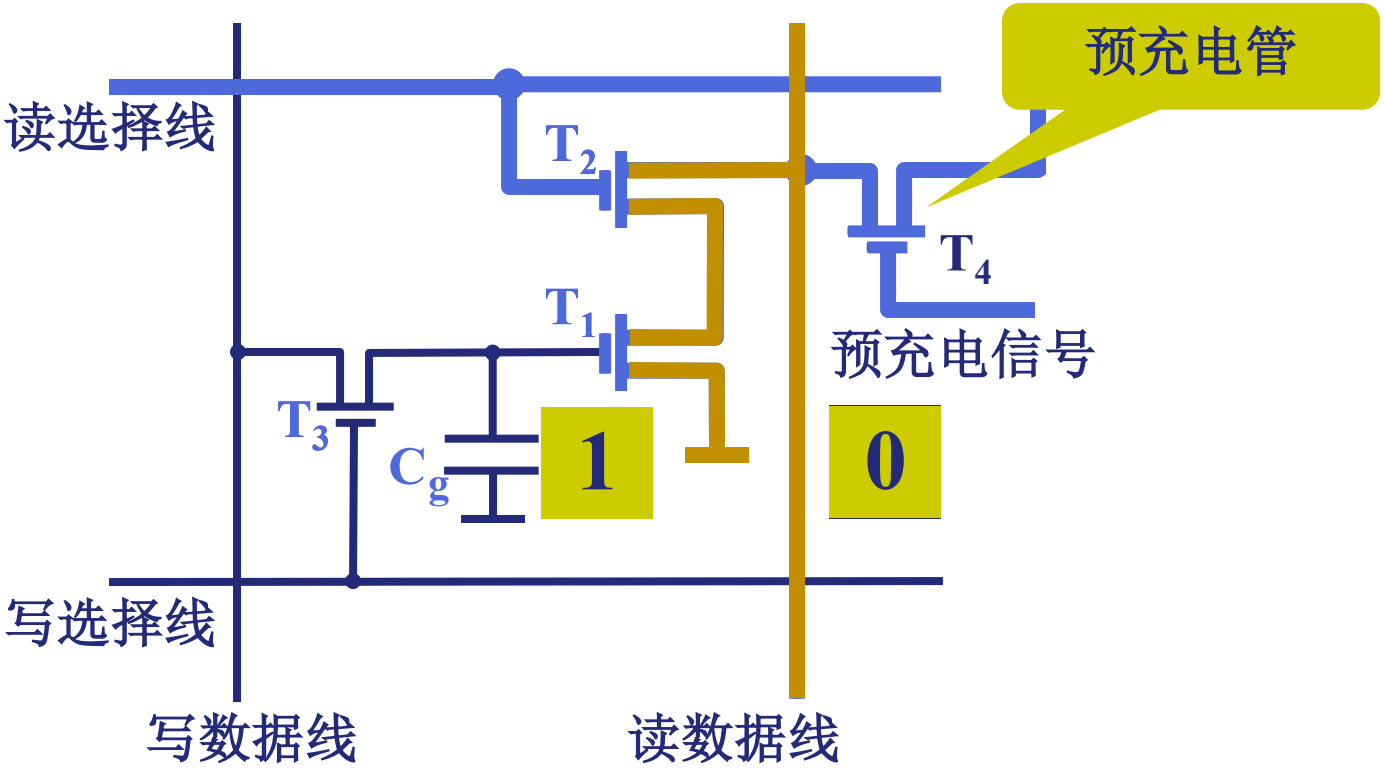
(4) 静态 RAM (2114) 写 时序



t_{DH} $\overline{WE}$ 失效后的数据维持时间

2. 动态 RAM (DRAM)

(1) 动态 RAM 基本单元电路



读出与原存信息相反

写入与输入信息相同

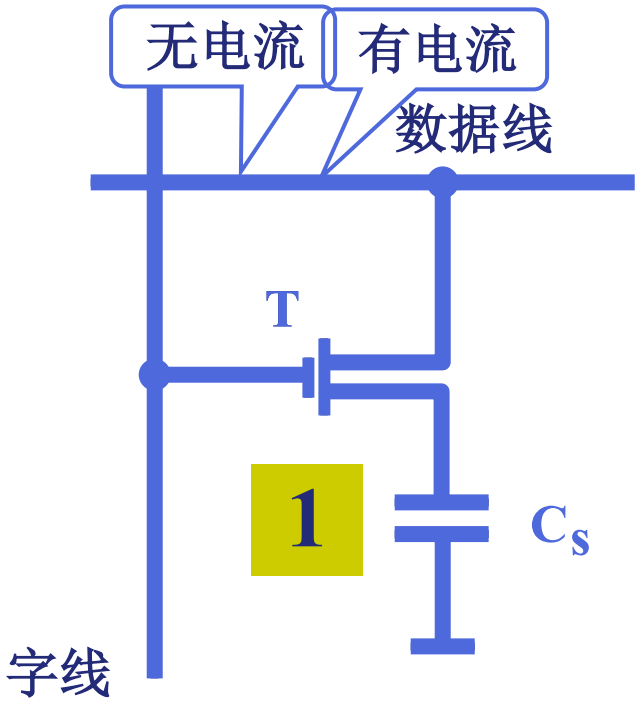
(1) 动态 RAM 基本单元电路

- **构造和表示：**

  信息记忆在电容C_S上，T为门控管，控制数据的进出。其栅极接读/写选择线(字线)，漏和源分别接数据线(位线)和记忆电容C_S 。数据1或0以电容C_S上电荷量的有无来判别。

- **读写原理：**在选择（字）线上加高电平，使T管导通。

  - 写“0”时，在数据线上加低电平，使C_S上电荷对数据线放电；
  - 写“1”时，在数据线上加高电平，使数据线对C_S充电；
  - 读出时，在数据线上有一读出电压。它与C_S上电荷量成正比。

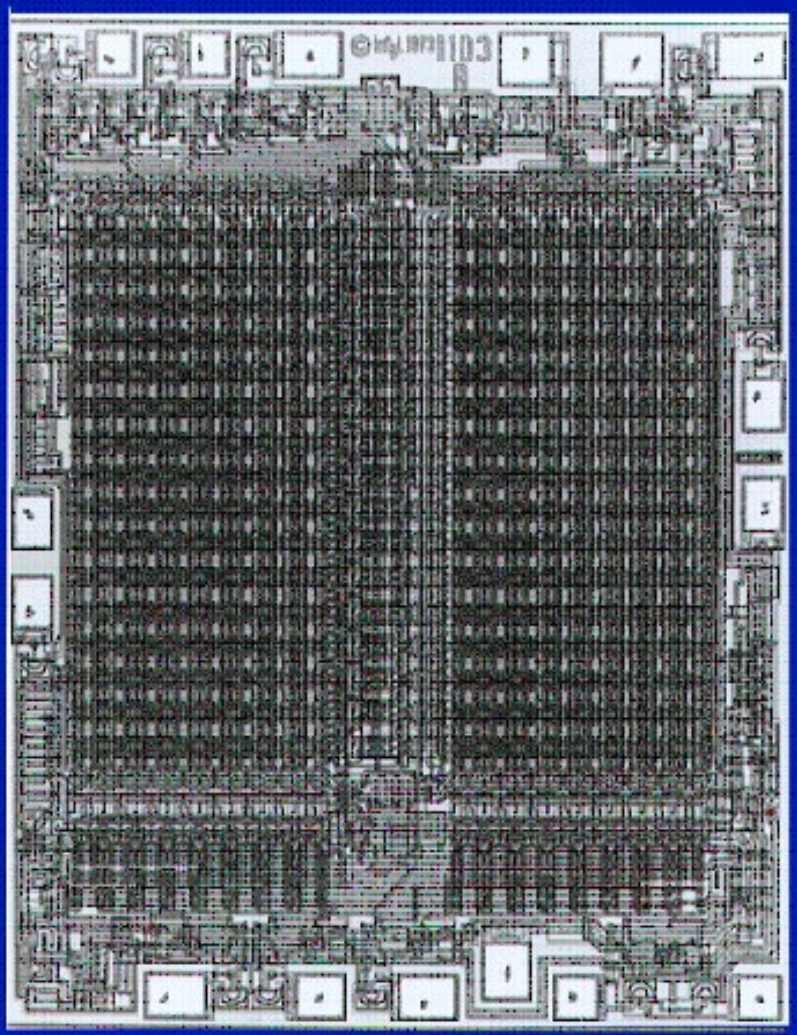


读出时数据线有电流 为 “1”

写入时C_S充电 为 “1” 放电 为”0”

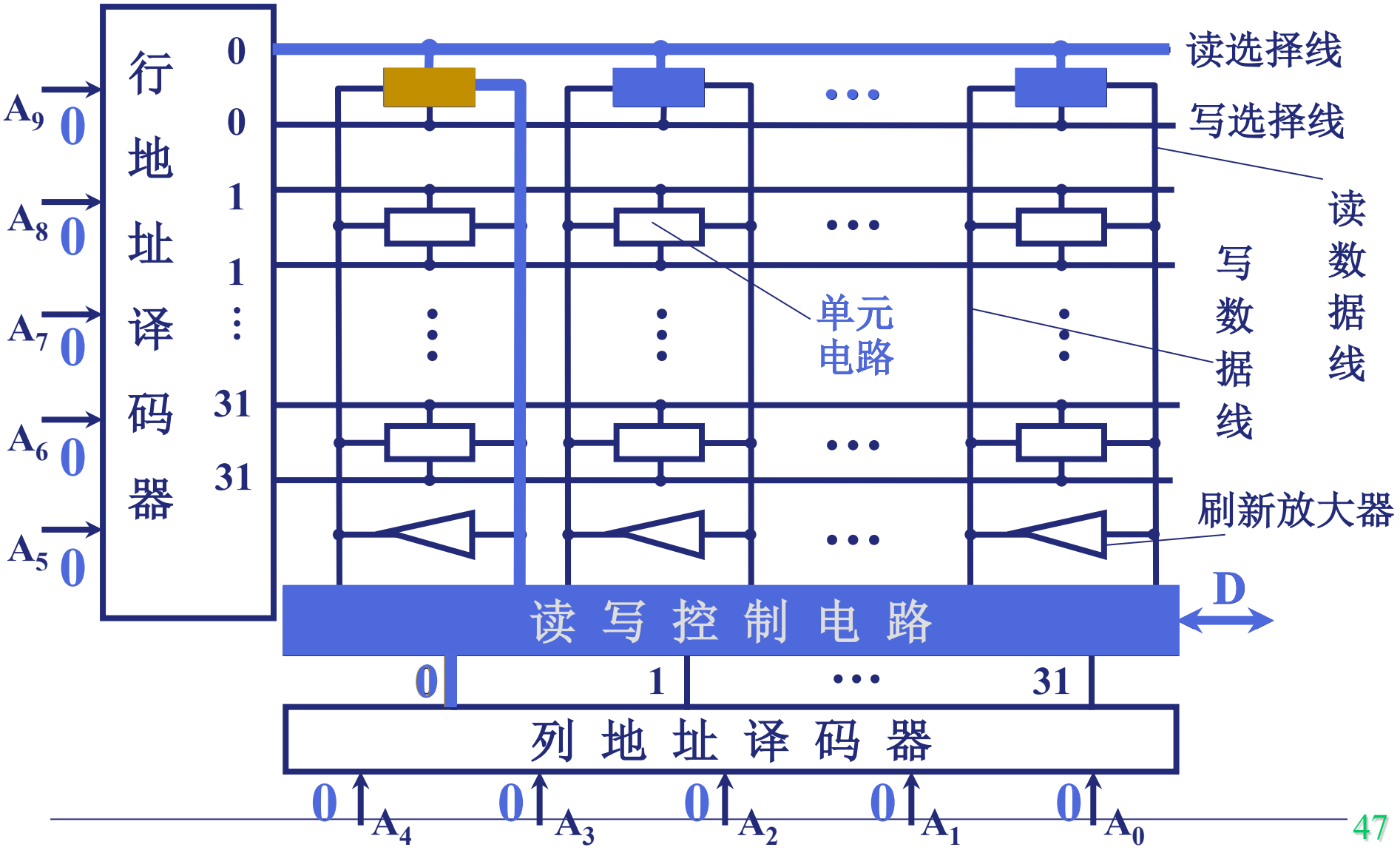
FIRST DRAM (1103 by Intel 1970)

(5 V 8 μm 3 mm^2 2600 μm^2/cell)



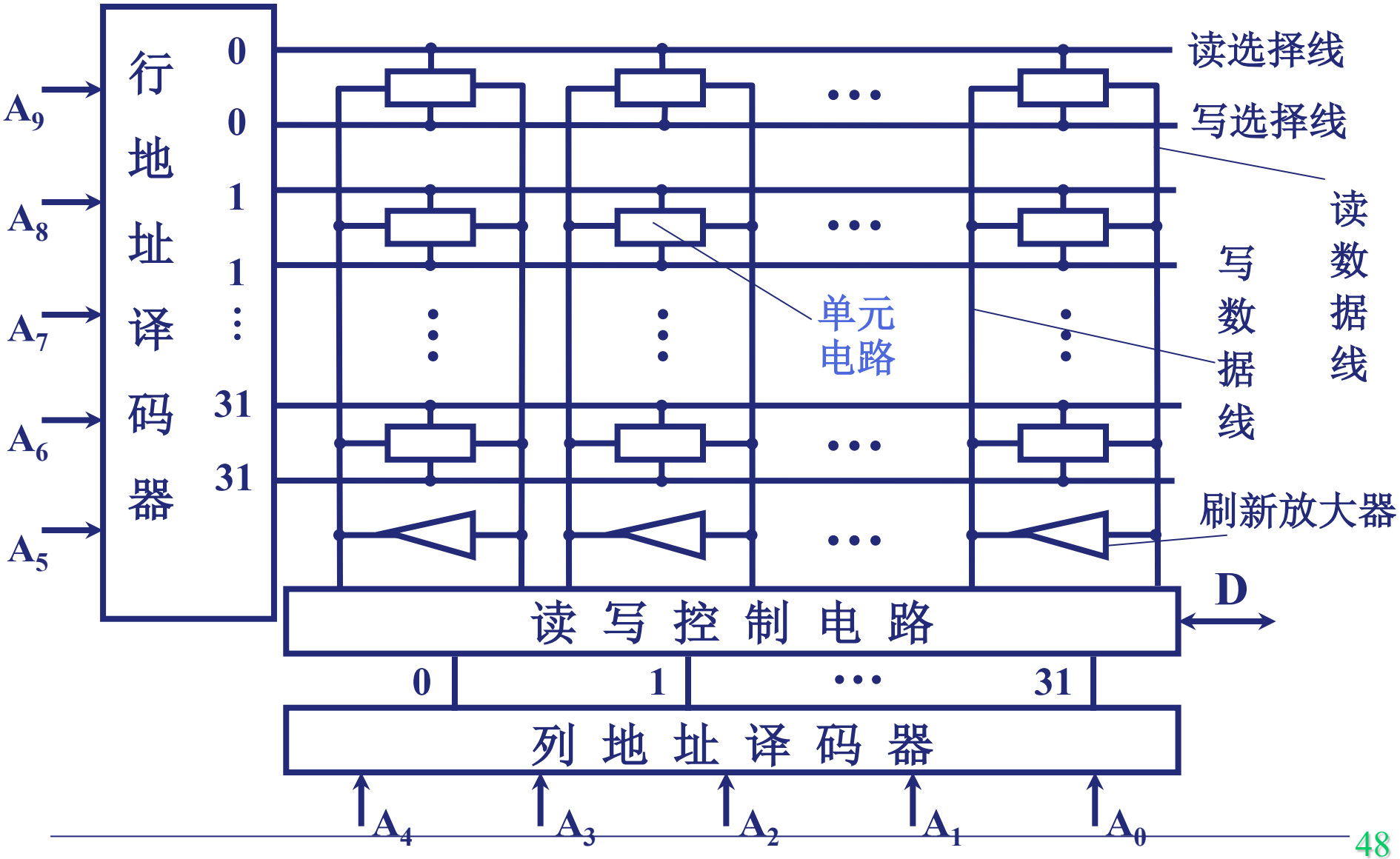
(2) 动态 RAM 芯片举例

① 三管动态 RAM 芯片 (Intel 1103) 读

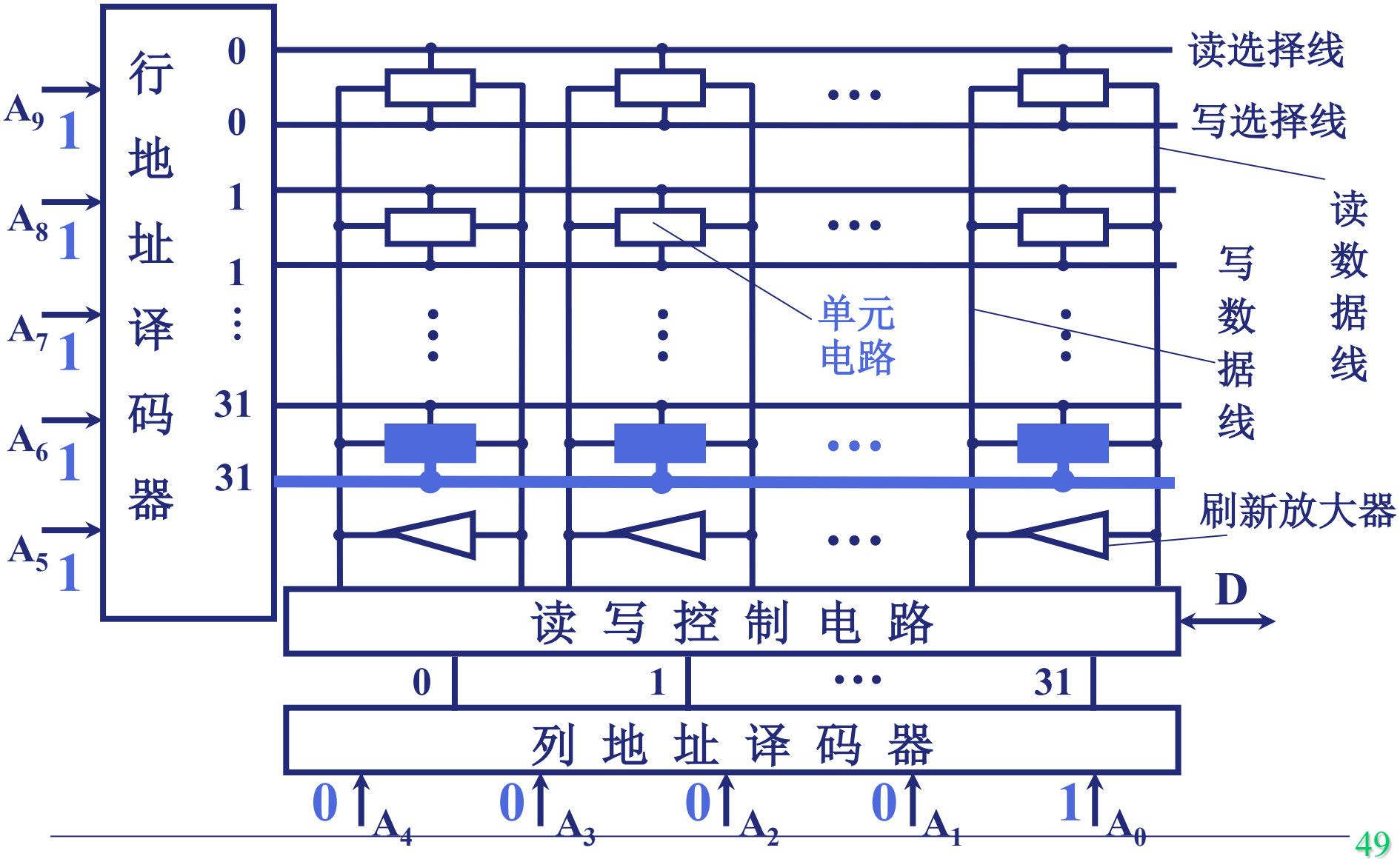


② 三管动态 RAM 芯片 (Intel 1103) 写

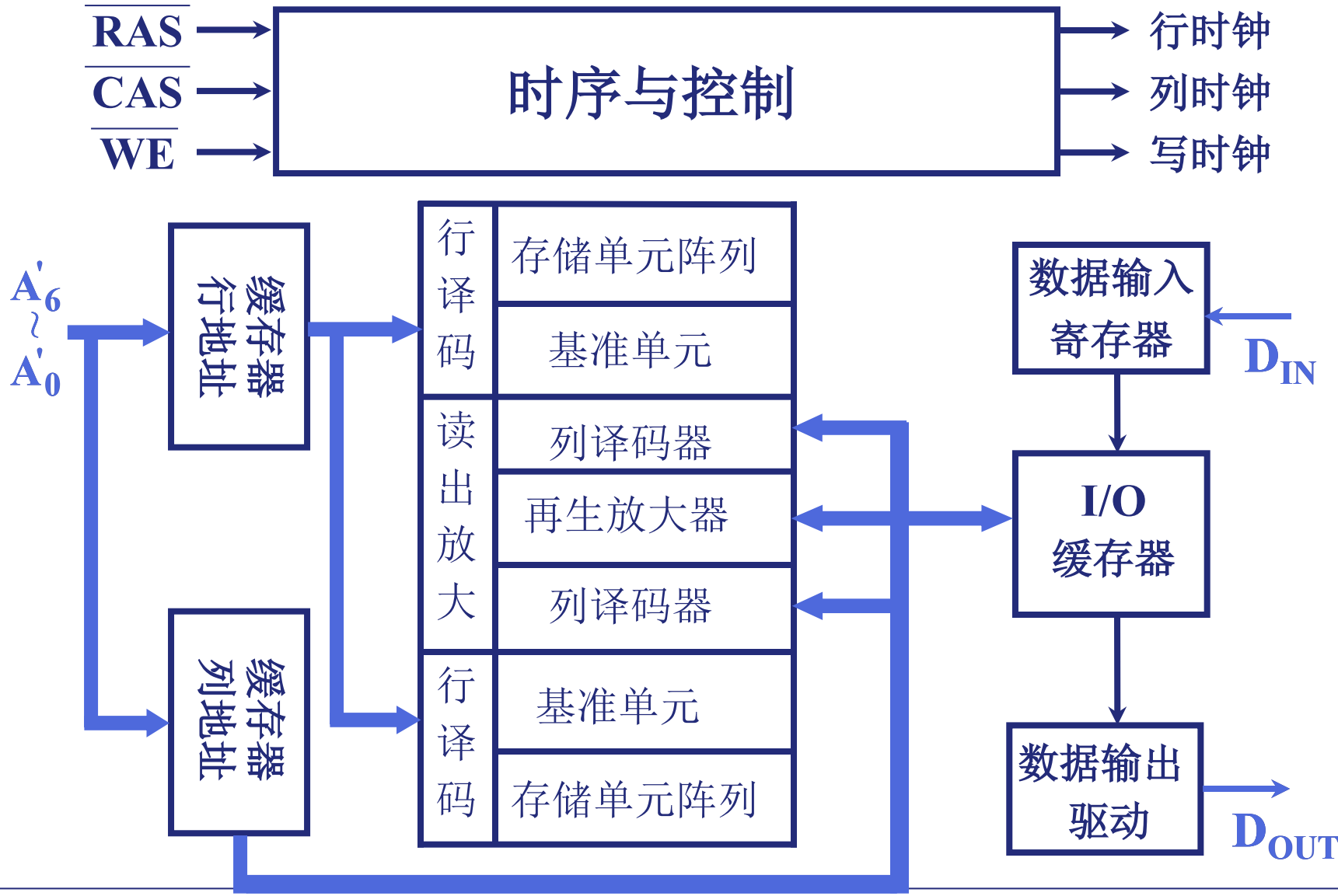
4.2



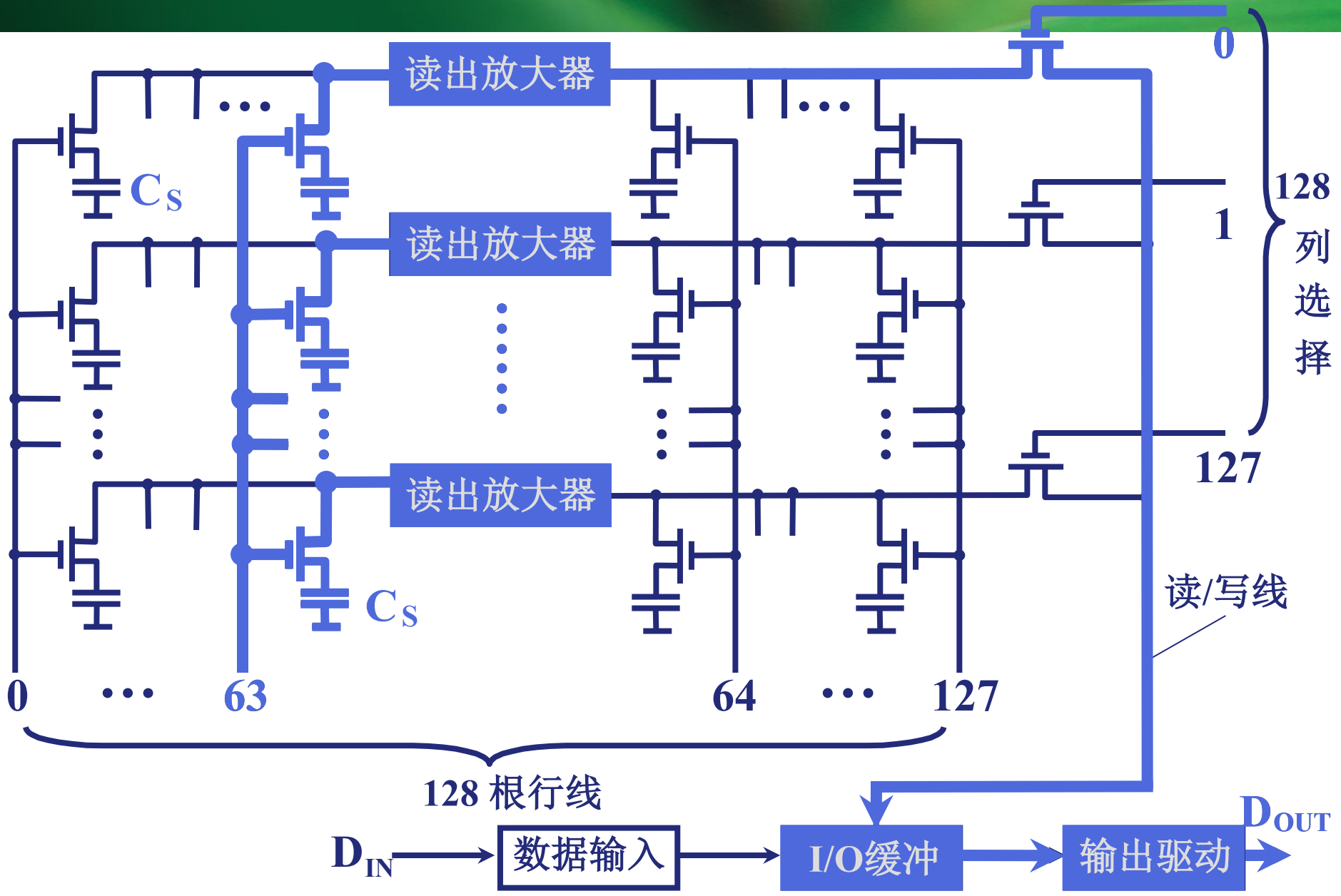
② 三管动态 RAM 芯片 (Intel 1103) 写



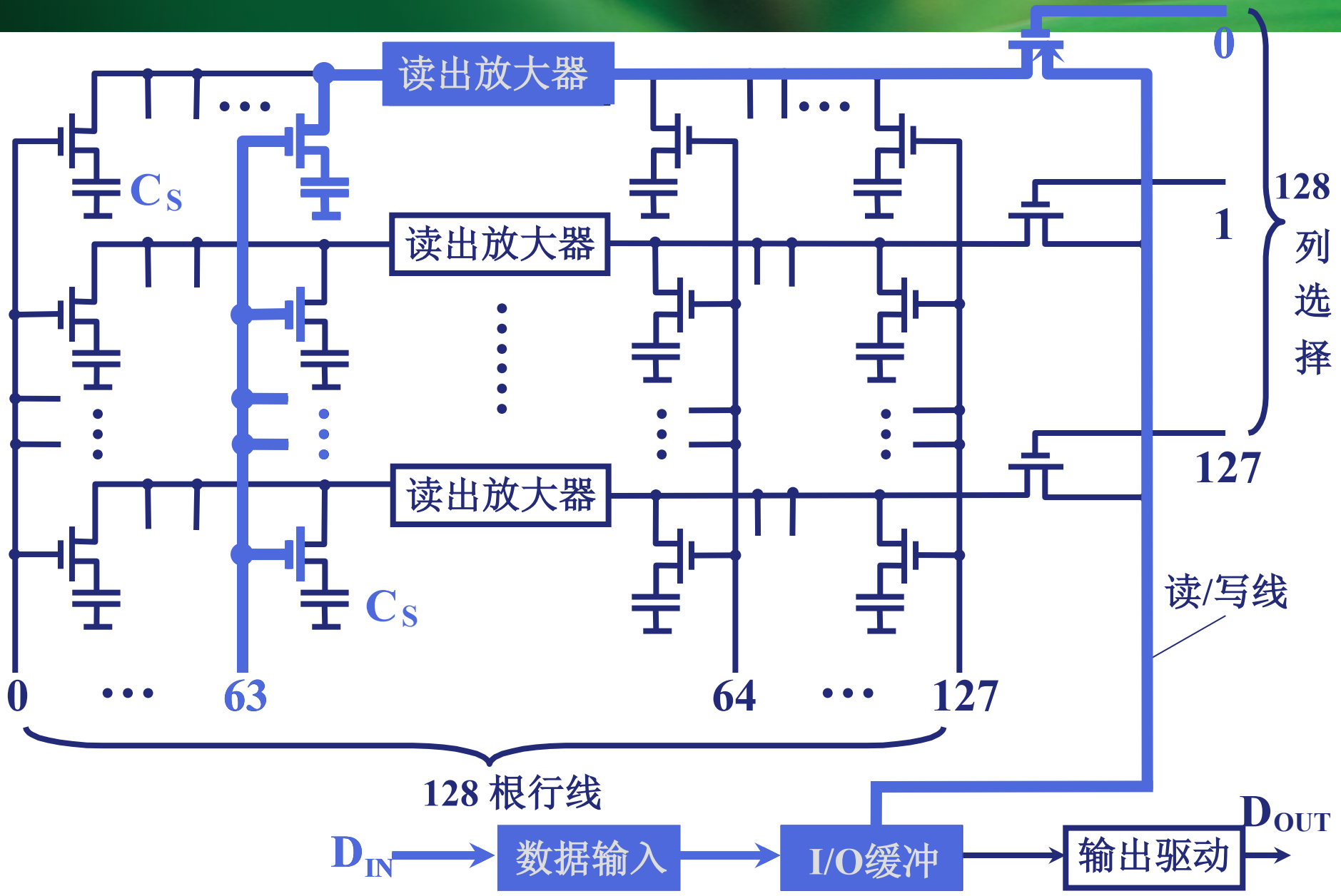
③ 单管动态 RAM 4116 (16K × 1位) 外特性



④ 4116 (16K × 1位) 芯片 读 原理



④ 4116 (16K × 1位) 芯片 写 原理



(3) 动态 RAM 时序

行、列地址分开传送

读时序

行地址 $\overline{\text{RAS}}$ 有效

写允许 $\overline{\text{WE}}$ 有效(高)

列地址 $\overline{\text{CAS}}$ 有效

数据 D_{OUT} 有效

写时序

行地址 $\overline{\text{RAS}}$ 有效

写允许 $\overline{\text{WE}}$ 有效(低)

数据 D_{IN} 有效

列地址 $\overline{\text{CAS}}$ 有效

(4) 动态 RAM 刷新

- 刷新是先将原存信息读出，再由刷新放大器形成原信息并重新写入的再生过程。
  - 刷新过程对CPU是透明的
  - 刷新是按行进行的，每行中的记忆单元同时被刷新，故刷新操作时仅仅需要行地址，不需要列地址。
  - 刷新类似读操作，但有所不同。刷新操作只给电容补充电荷，不需要信息输出。另外，刷新不用加片选，整个存储器中所有芯片同时被刷新。

(4) 动态 RAM 刷新

- 刷新周期：从上次对整个存储器刷新结束到下次对整个存储器全部刷新一遍为止的时间间隔，为电容数据有效保存期的上限（64ms）。
- 有三种刷新方式：集中式、分散式、异步刷新。

① 集中刷新：

前一段时间正常读/写，后一段时间停止读/写，集中逐行刷新。

特点：集中刷新时间长，不能正常读/写（死区），很少使用。

② 分散刷新：

一个存储周期分为两段: 前一段用于正常读/写操作，后一段用于刷新操作。

特点：不存在死区，但每个存储周期加长。很少使用。

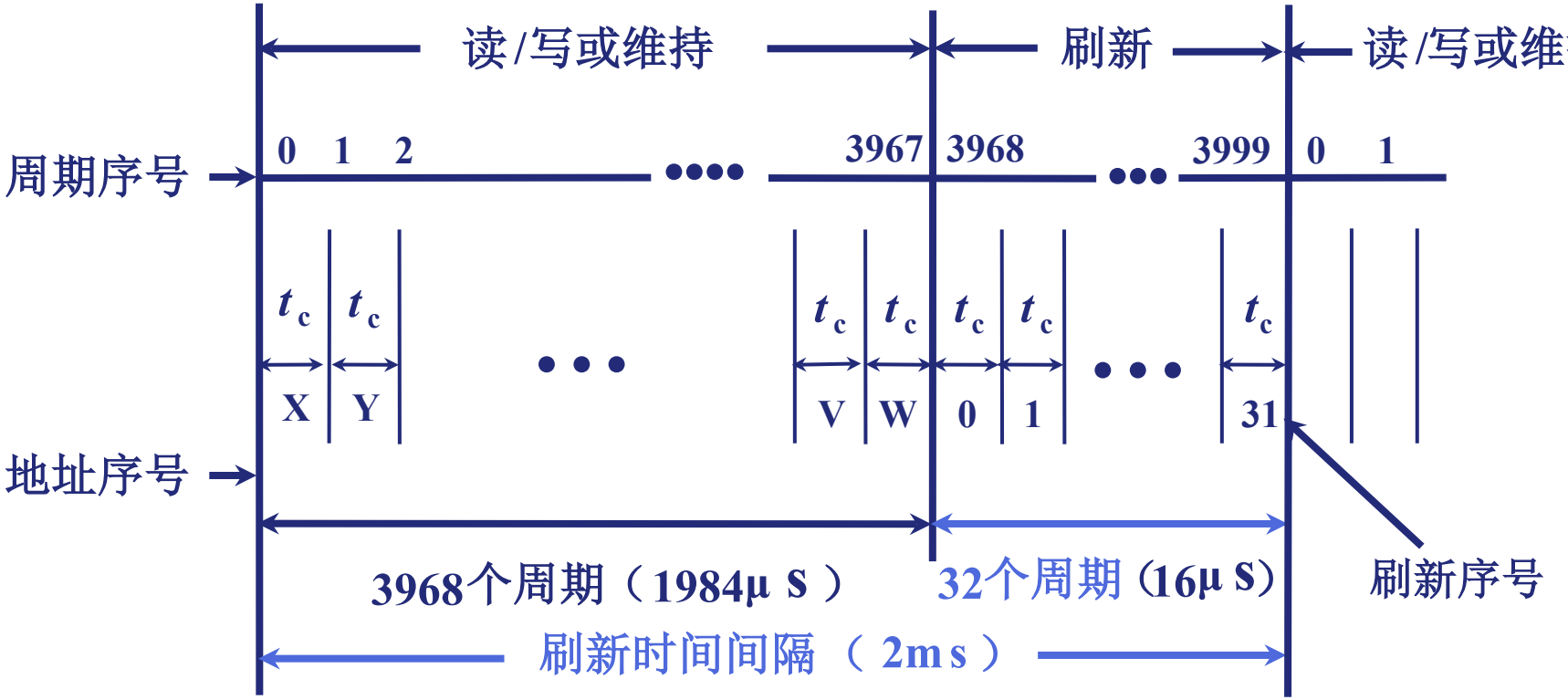
③ 异步刷新：

结合上述两种方式。以4096行为例，在64ms时间内必须轮流对每一行刷新一次，即每隔64ms/4096=15.625μs刷新一行。

特点：结合前两种，效率高，用得较多。

(4) 动态 RAM 刷新

① 集中刷新（存取周期为0.5μs）以 32 × 32 矩阵为例

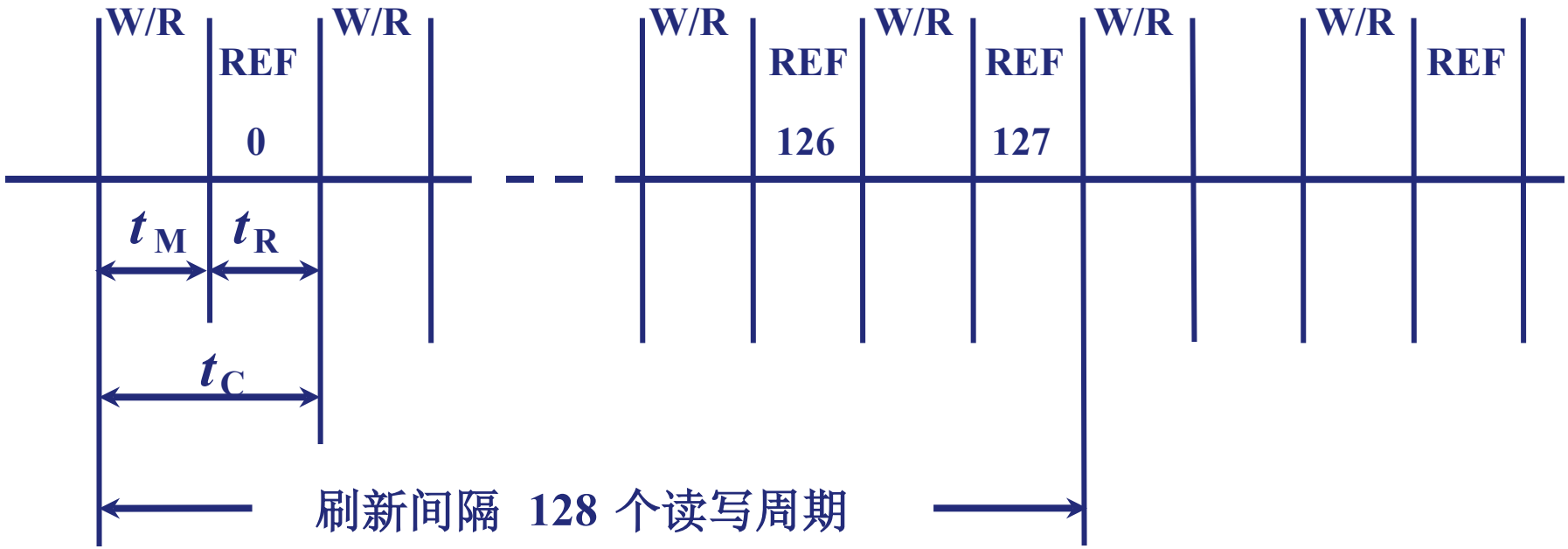


“死区”为 0.5 μs ×32 = 16 μs

“死时间率”为 32/4000 ×100% = 0.8%

② 分散刷新（存取周期为1μs）

以 128 ×128 矩阵为例



$t_C = t_M + t_R$

读写 刷新

无“死区”

(存取周期为 0.5 μs + 0.5 μs)

③ 异步刷新

- 是分散刷新与集中刷新的结合

对于 128 ×128 的存储芯片（存取周期为 0.5μs）

若每隔 2 ms 集中刷新一次　　“死区”为 64 μs

若每隔 15.6 μs 刷新一行

而且每行每隔 2 ms 刷新一次　　“死区”为 0.5 μs

将刷新安排在指令译码阶段，不会出现“死区”

3. 动态 RAM 和静态 RAM 的比较

主存

缓存

	DRAM	SRAM
存储原理	电容	触发器
集成度	高	低
芯片引脚	少	多
功耗	小	大
价格	低	高
速度	慢	快
刷新	有	无

四、只读存储器（ROM）

1. 掩膜 ROM (MROM)

- 以元件的“有/无”表示“1”、“0”。
- 内容由半导体制造厂按用户提出的要求在芯片的生产过程中直接写入的，写入后内容无法改变。

行列选择线交叉处有 MOS 管为“1”

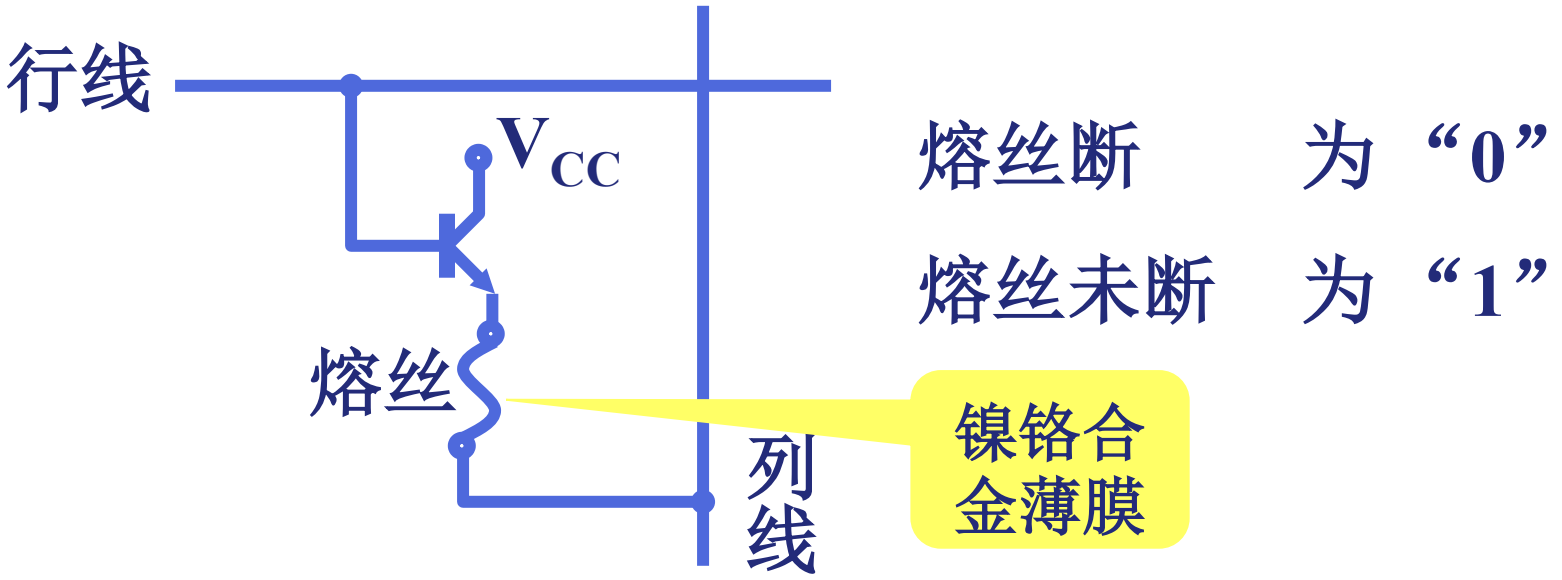
行列选择线交叉处无 MOS 管为“0”

- 优点：可靠性高，形成批量之后价格便宜；
- 缺点：用户对制造厂的依赖性过大，灵活性差。

四、只读存储器（ROM）

2. PROM (一次性编程)

- 允许用户利用专门的设备（编程器）写入自己的程序，但一旦写入后，其内容将无法改变。



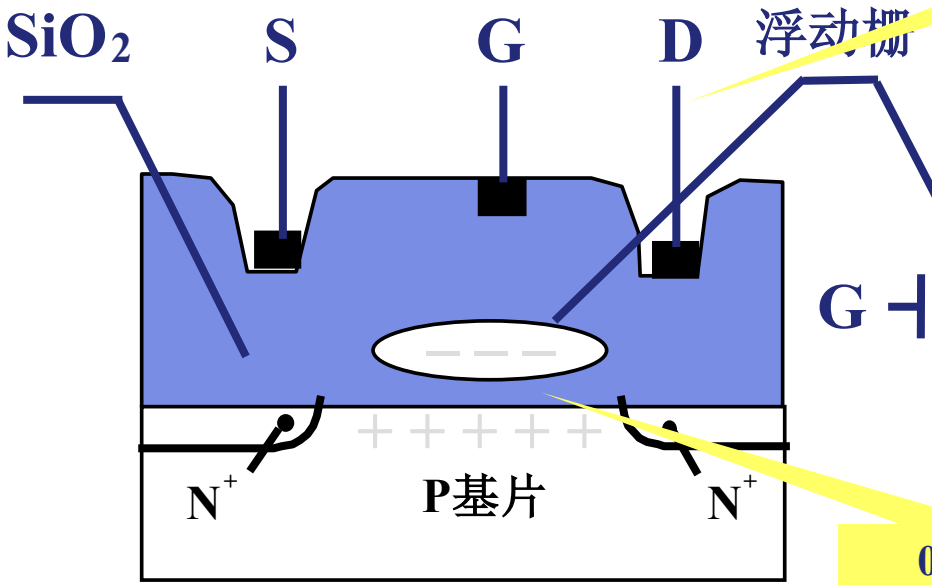
- 产品出厂时，所有记忆单元均制成“0”（或制成“1”），用户根据需要可自行将其中某些记忆单元改为“1”（或改为“0”）。
- 双极型PROM有两种结构，一种是熔丝烧断型，另一种是PN结击穿型，

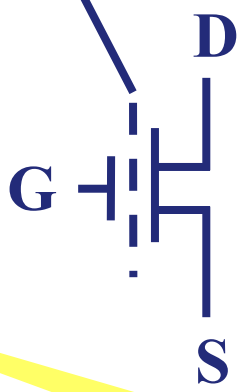
浮动栅雪崩注入型MOS管

(1) N型沟道浮动栅 MOS 电路

25V（写入电压）、50ms宽正脉冲

埋在氧化物绝缘层中，不与外部导通。写入信息前，浮栅上没有电荷，D、S间不导通，MOS呈“1”态





S 源

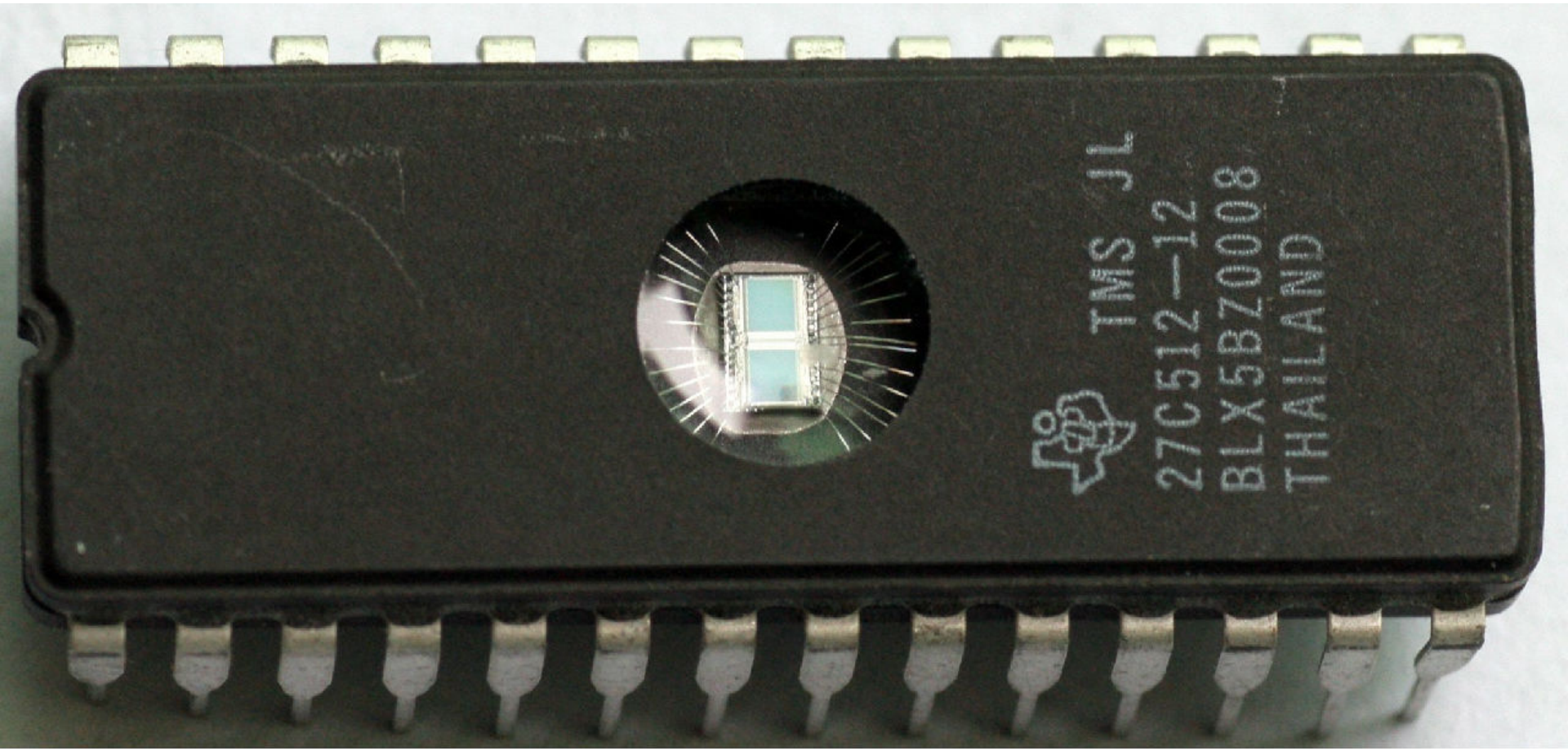
D 漏

紫外线全部擦洗

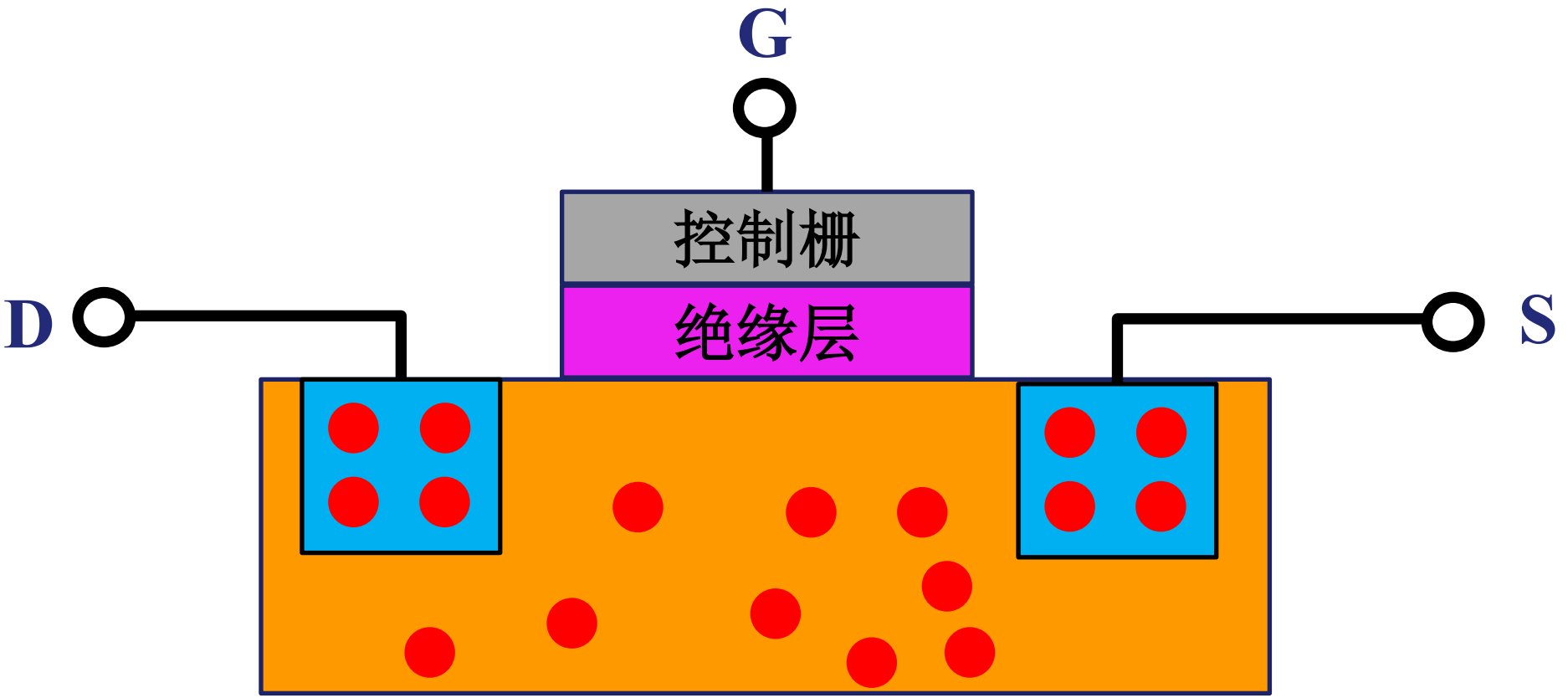
0.05~0.1μm

D 端加正电压　　形成浮动栅　　S 与 D 不导通为 “0”

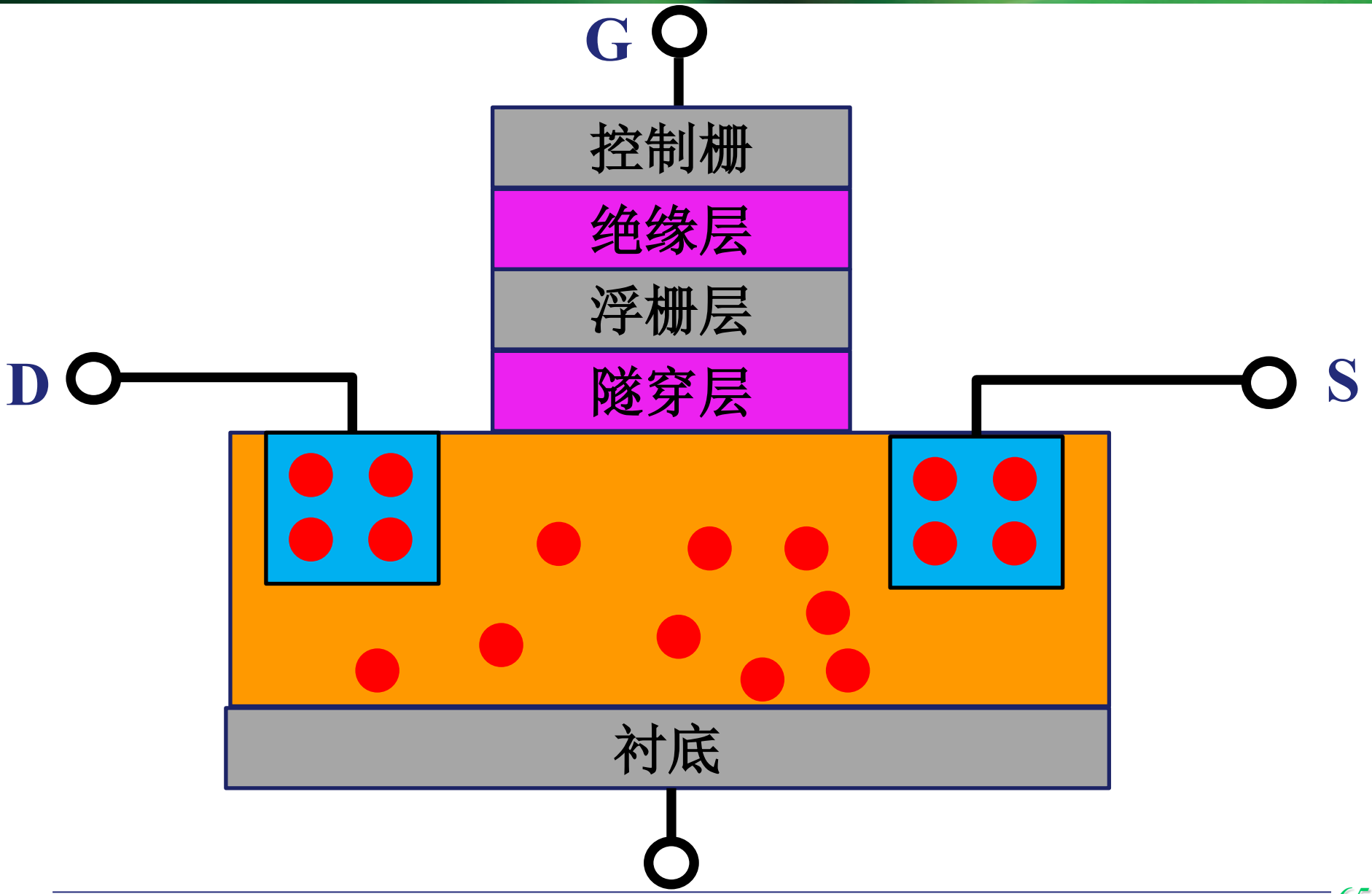
D 端不加正电压　　不形成浮动栅　　S 与 D 导通为 “1”


TMS JL
27C512-12
BLX5BZ0008
THAILAND

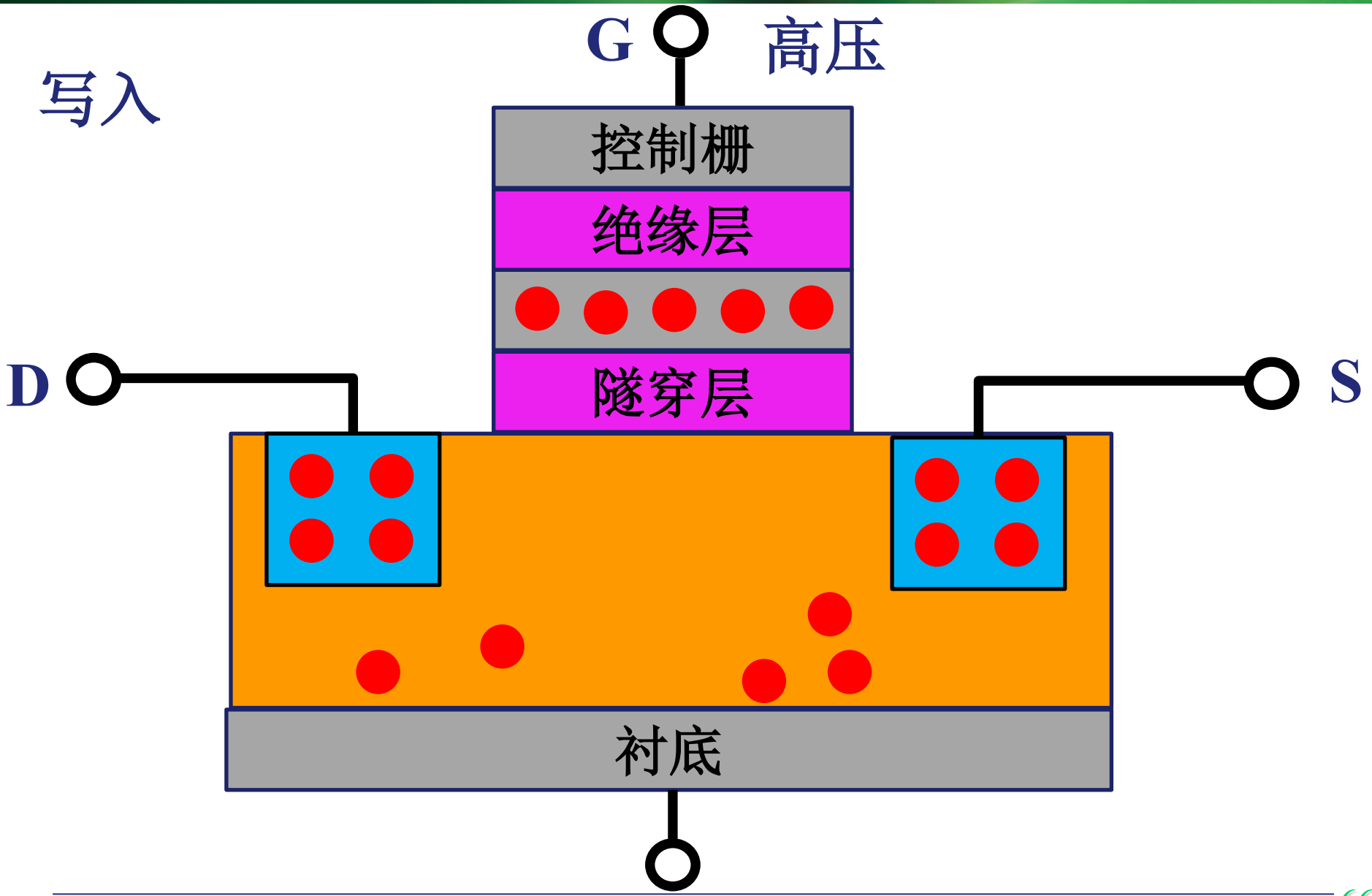
普通晶体管



浮栅晶体管

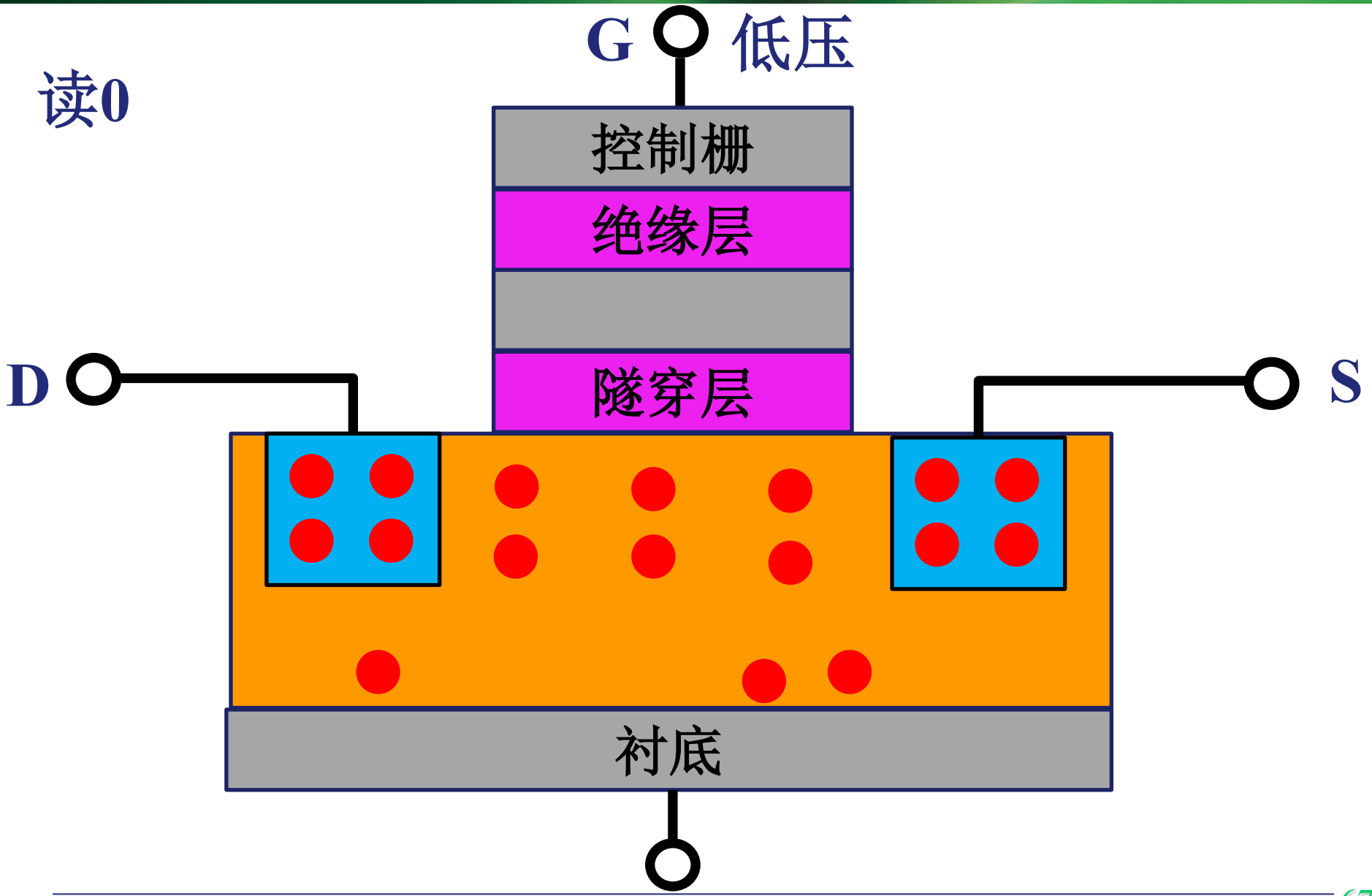


浮栅晶体管



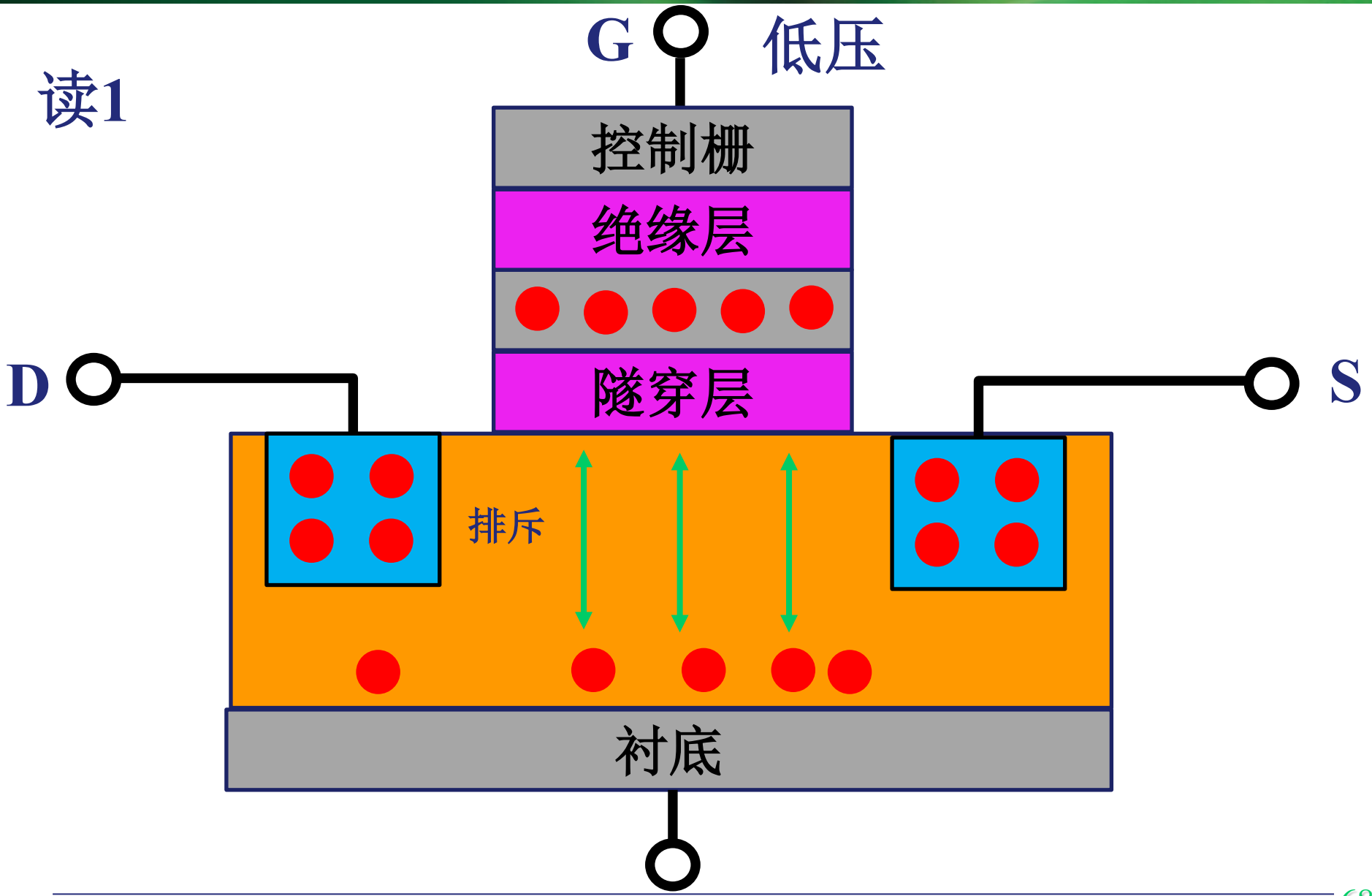
浮栅晶体管

读0



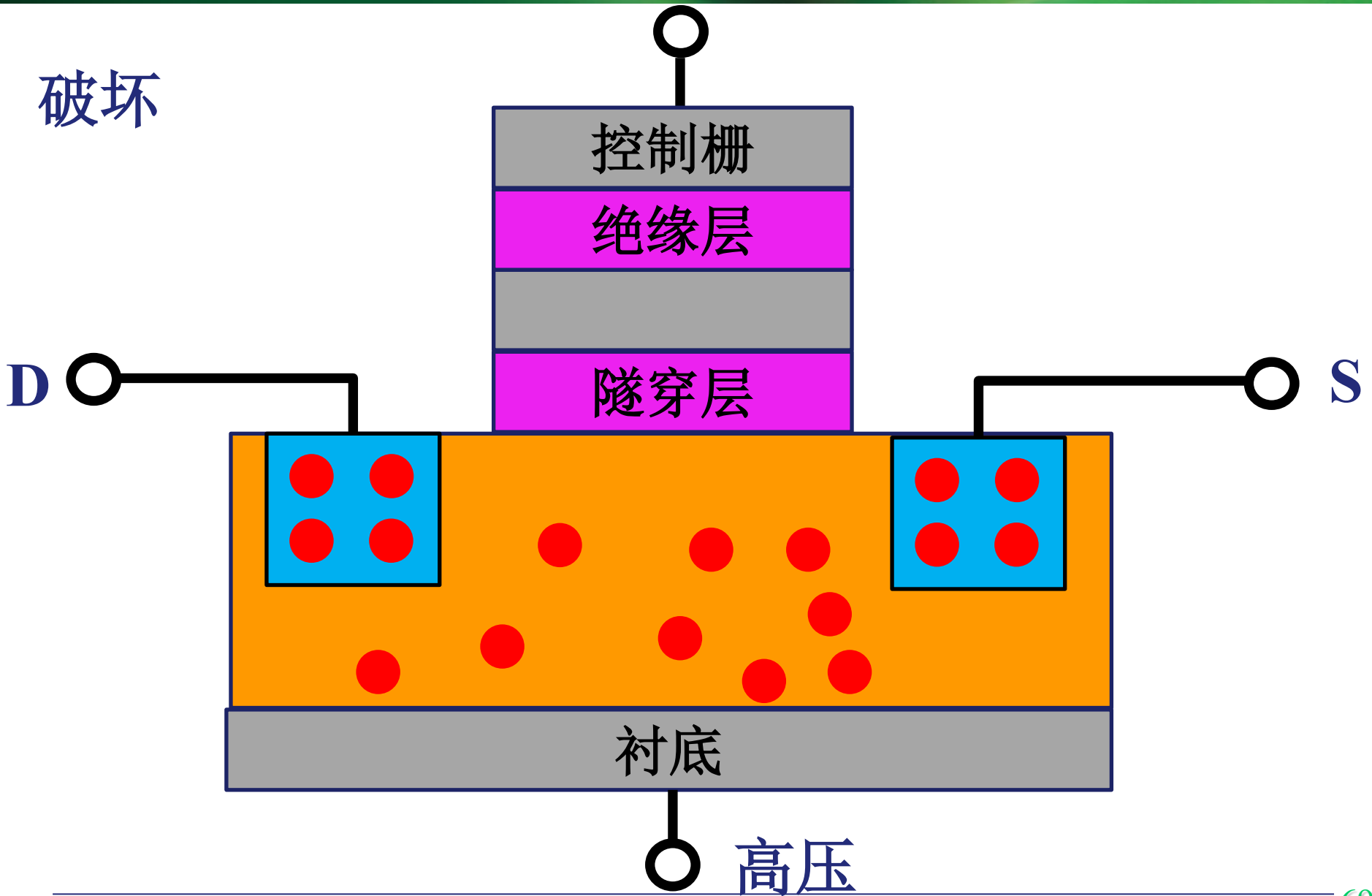
浮栅晶体管

读1



浮栅晶体管

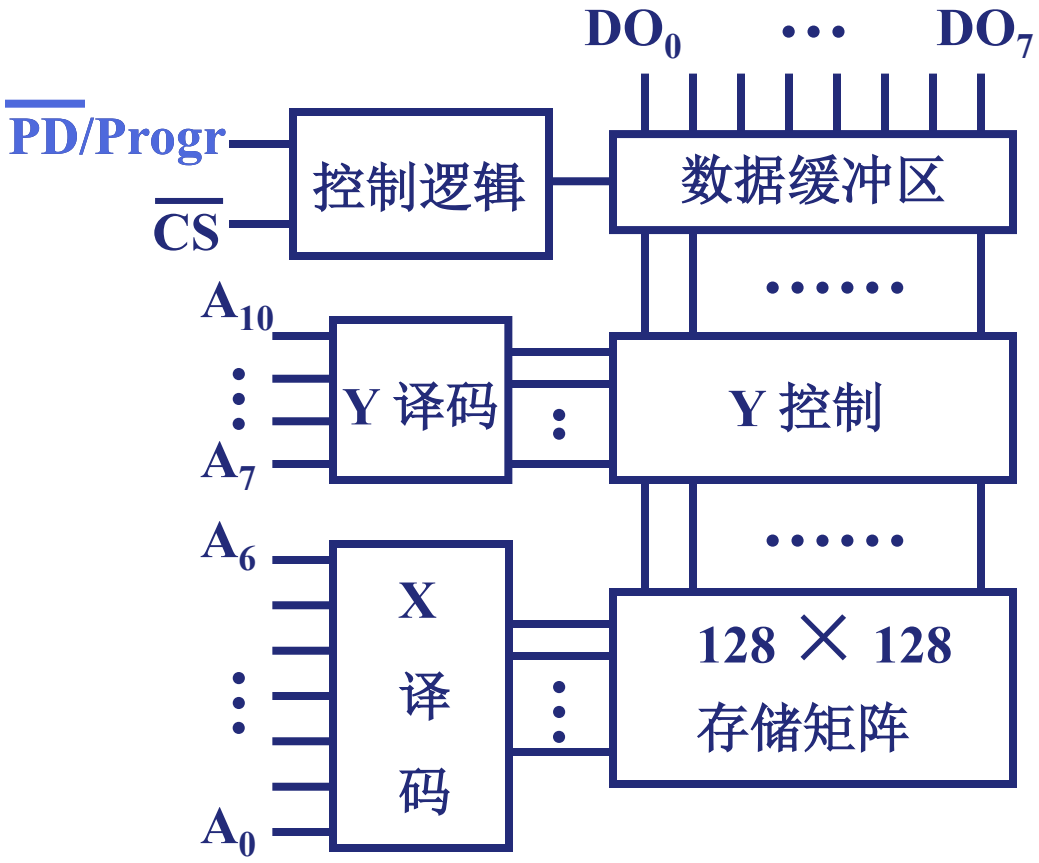
破坏

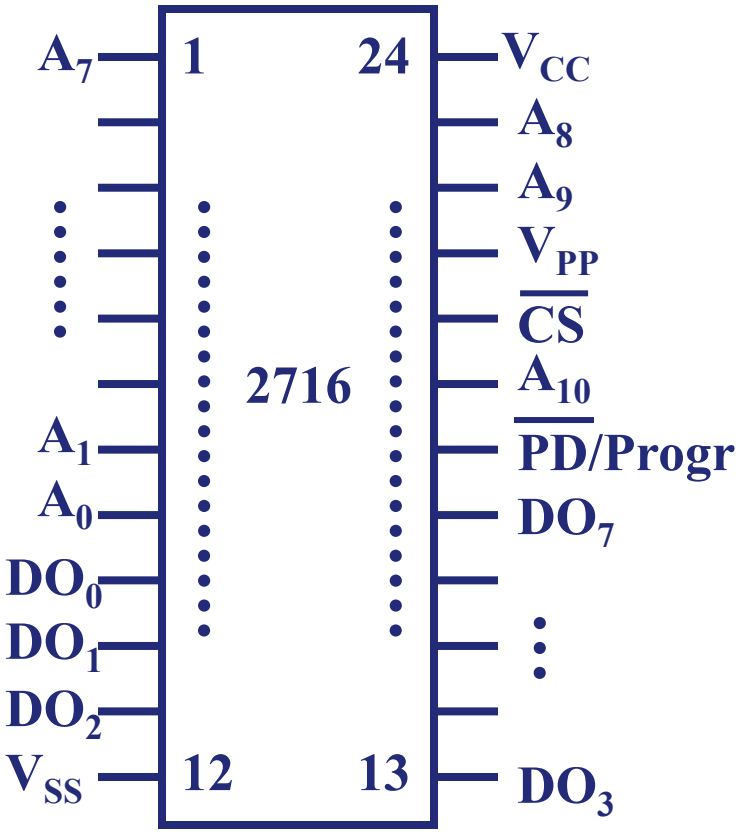


3. EPROM (多次性编程)

- 可由用户利用编程器写入信息，可以多次改写。
- EPROM出厂时，存储内容为全“1”，用户可以根据需要将其中某些记忆单元改为“0”。当需要更新存储内容时可以将原存储内容擦除（恢复全“1”），以便再写入新的内容。
- EPROM可分为：紫外线擦除和电擦除
  - UVEPROM：用紫外线灯制作的擦除器照射存储器芯片上的透明窗口，使芯片中原存内容被擦除。
    - 由于是用紫外线灯进行擦除，所以只能对整个芯片擦除，而不能对芯片中个别需要改写的存储单元单独擦除。
    - 另外，为了防止存储的信息受日光中紫外线成分的作用而缓慢丢失，在UVEPROM芯片写入完成后，必须用不透明的黑纸将芯片上的透明窗口封住。
  - EEPROM是采用电气方法来进行擦除的。
    - 在联机条件下既可以用字擦除方式擦除，也可以用数据块擦除方式擦除。以字擦除方式操作时，能够只擦除被选中的那个存储单元的内容；在数据块擦除操作时，可擦除数据块所有单元的内容。

(2) 2716 EPROM 的逻辑图和引脚





$\overline{PD}$/Progr 功率下降 / 编程输入端　读出时 为 低电平

EPROM不能取代RAM

- 原因

①EPROM的编程次数（寿命）是有限的：

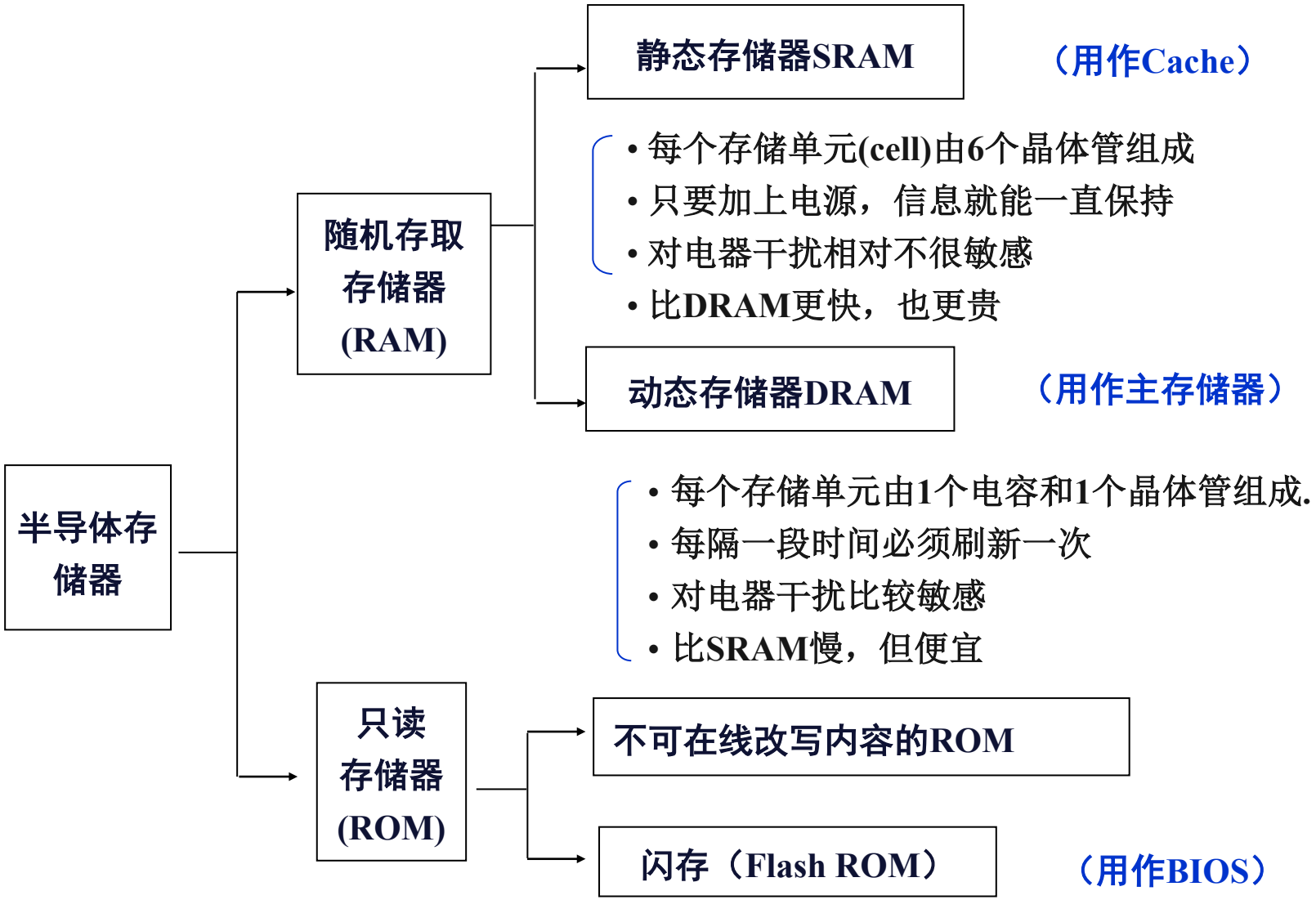
EEPROM因为氧化层被磨损，重复改写次数一般10万次

②写入时间过长，即使对于EEPROM，擦除一个字节大约需要10ms，写入一个字节大约需要10us，比SRAM或DRAM的时间长100－1000倍。

4. Flash Memory (快擦型存储器)

- 闪速存储器是20世纪80年代中期出现的一种快擦写型存储器，它的主要特点是既可在不加电的情况下长期保存信息，又能在线进行快速擦除与重写，兼备了EEPROM和RAM的优点。
- 目前，大多数微型计算机的主板采用闪速存储器来存储BIOS程序，由于BIOS的数据和程序非常重要，不允许修改，故早期主板BIOS芯片多采用PROM或EPROM。

内存芯片的多种类型


半导体存储器
随机存取存储器(RAM)
静态存储器SRAM
（用作Cache）
• 每个存储单元(cell)由6个晶体管组成
• 只要加上电源，信息就能一直保持
• 对电器干扰相对不很敏感
• 比DRAM更快，也更贵
动态存储器DRAM
（用作主存储器）
• 每个存储单元由1个电容和1个晶体管组成.
• 每隔一段时间必须刷新一次
• 对电器干扰比较敏感
• 比SRAM慢，但便宜
只读存储器(ROM)
不可在线改写内容的ROM
闪存（Flash ROM）
（用作BIOS）

五、存储器与 CPU 的连接

1. 存储器容量的扩展：由于单个芯片的存储器容量有限，为了组成较大容量的存储器，需要多个芯片进行组合。
   - 位扩展（增加存储字长）：由$mK\times n_1$的存储器芯片组成$mK\times n_2$的存储器，需（n_2/n_1）片$mK\times n_1$的存储器芯片。
   - 字扩展（增加存储字数量）：由$m_1K\times n$的存储器芯片$m_2K\times n$的存储器，需（m_2/m_1）片$m_1K\times n$的存储器芯片。
   - 字位同时扩展：由$m_1K\times n_1$的存储器芯片组成$m_2K\times n_2$的存储器，需（m_2/m_1）×（n_2/n_1）片$m_1K\times n_1$的存储器芯片。

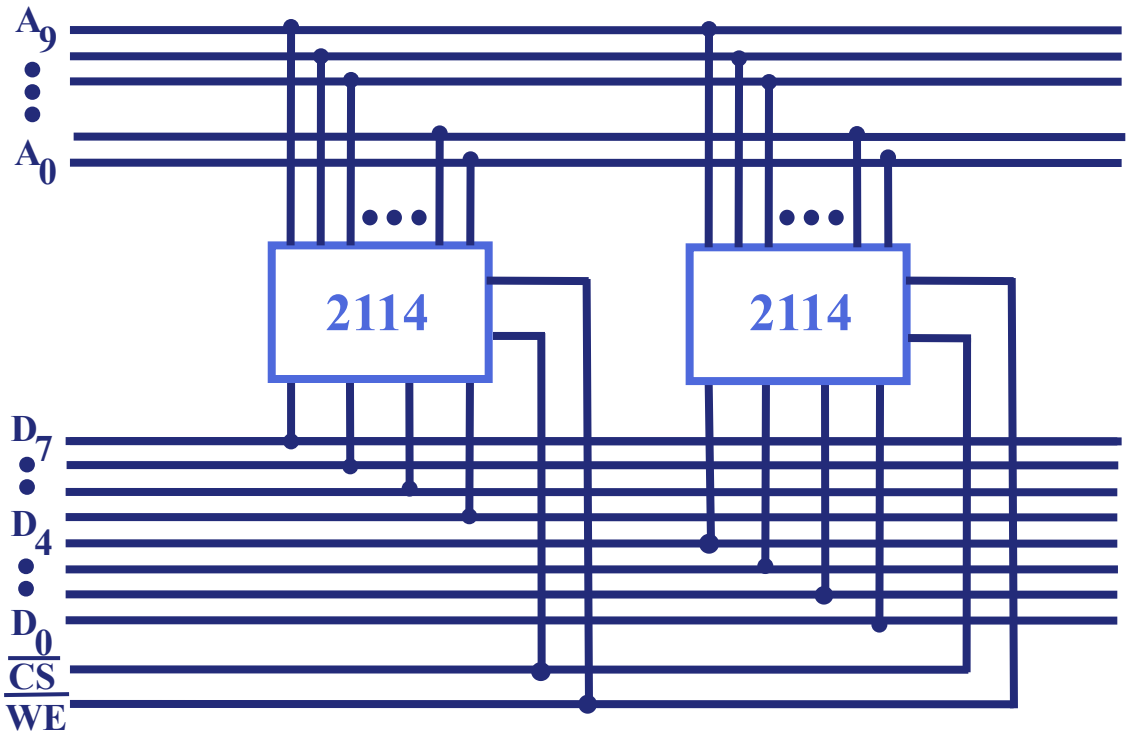
五、存储器与 CPU 的连接

(1) 位扩展（增加存储字长）

用 2片 1K×4位 存储芯片组成 1K×8位 的存储器

10根地址线

8根数据线

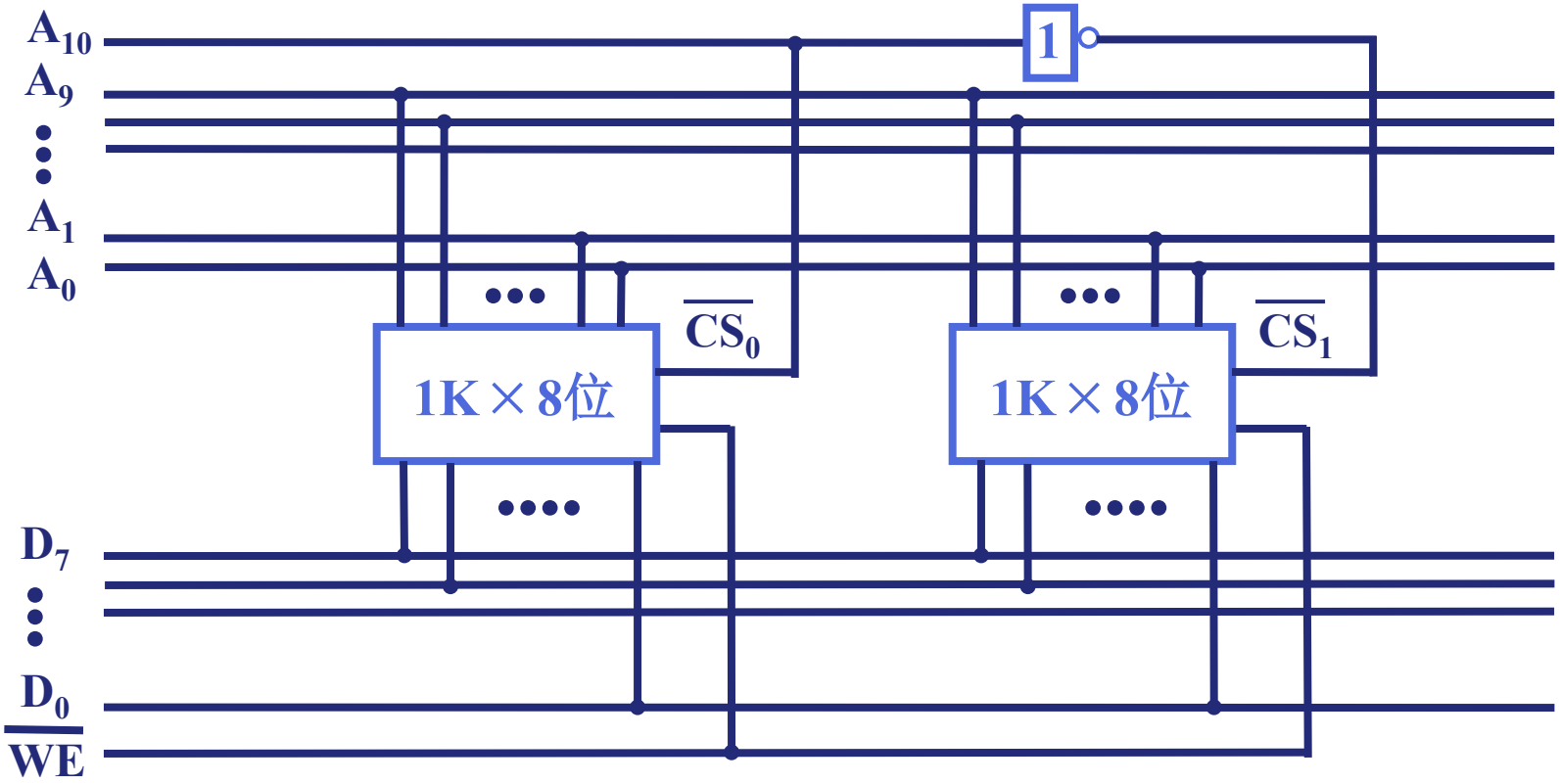


(2) 字扩展（增加存储字的数量）

11根地址线

用 2片 1K×8位 存储芯片组成 2K×8位 的存储器

8根数据线

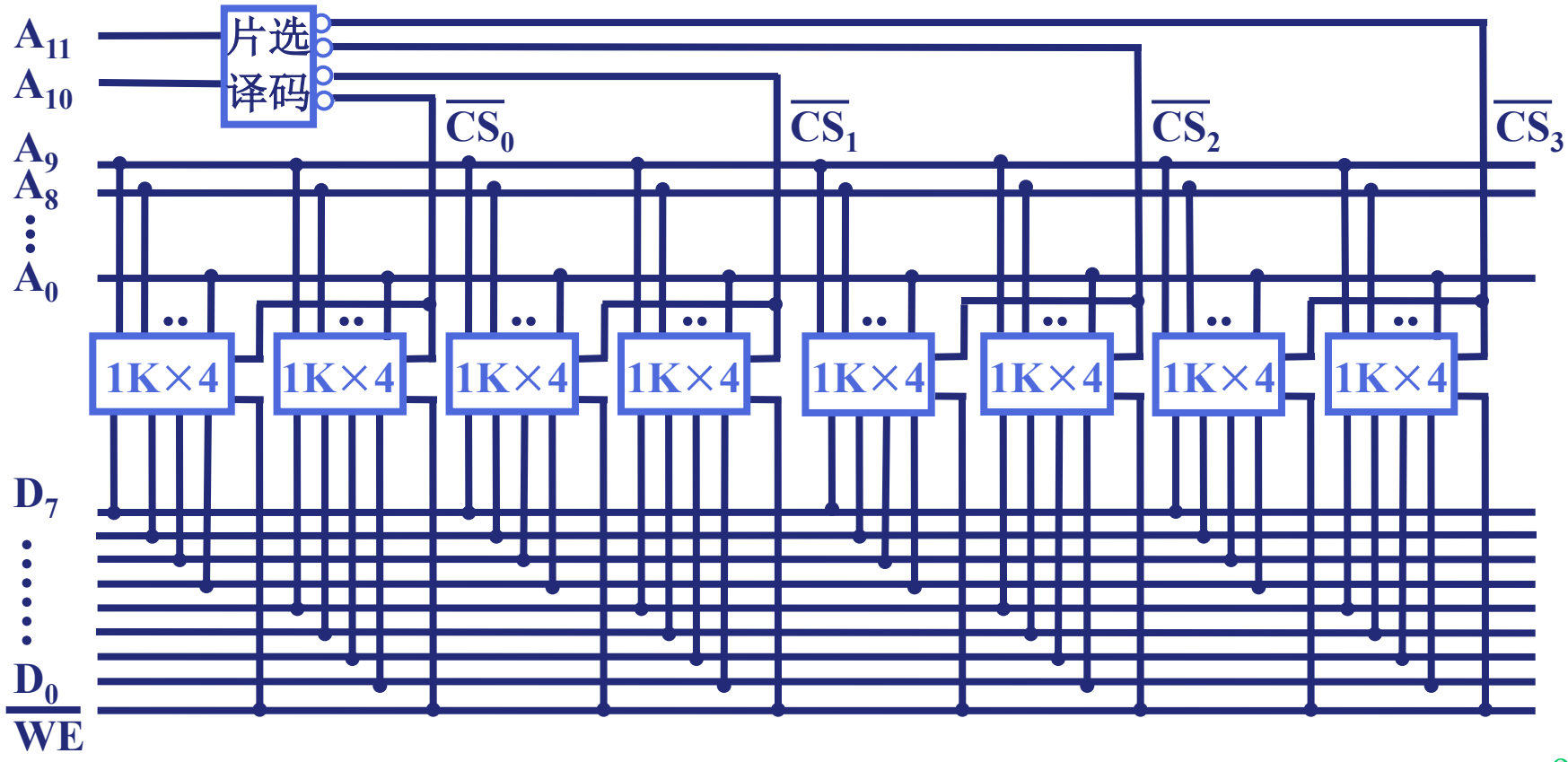


(3) 字、位扩展

用 8片 1K×4位 存储芯片组成 4K×8位 的存储器

12根地址线

8根数据线



2. 存储器与 CPU 的连接

(1)地址线的连接:内部地址线、芯片选择线

CPU地址线往往比存储器地址线多，所以将CPU地址线低位与存储芯片地址相连，CPU地址线高位或做存储芯片扩充，或做他用。

(2) 数据线的连接:数据线对应连接

(3) 读/写线的连接：对应连接

(4) 合理选用芯片

(5) 其他　　时序、负载

例4.1

例1 设CPU有16根地址线、8根数据线，并用$\overline{MREQ}$作为访存控制信号（低电平有效），用WR作为读/写控制信号（高电平为读，低电平为写）。现有下列存储芯片：1 K×4位RAM、 4 K×8位RAM 、 8 K×8位RAM、 2 K×8位ROM、 4 K×8位ROM、 8 K×8位ROM及74138译码器和各种门电路，画出CPU与存储器连接图，要求

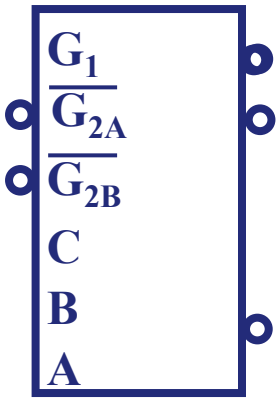
①主存地址空间分配

6000H~67FFH为系统程序区

6800H~6BFFH为用户程序区

②合理选用上述存储芯片，说明各选几片

③ 详细画出存储芯片的片选逻辑图



例4.1 解:

(1) 写出对应的二进制地址码

$A_{15}A_{14}A_{13}$	$A_{11}A_{10}$	…	A_7	…	A_4	A_3	…	A_0	
0 1 1	0 0		0 0 0		0 0 0	0 0		0 0	ROM 2K×8位
⋮									
0 1 1	0 0		1 1 1		1 1 1	1 1		1 1	
0 1 1	0 1		0 0 0		0 0 0	0 0		0 0	1K×8位 RAM
⋮									
0 1 1	0 1		1 1 1		1 1 1	1 1		1 1	

1片2K×8位

2片1K×4位

(2) 确定芯片的数量及类型

(3) 分配地址线

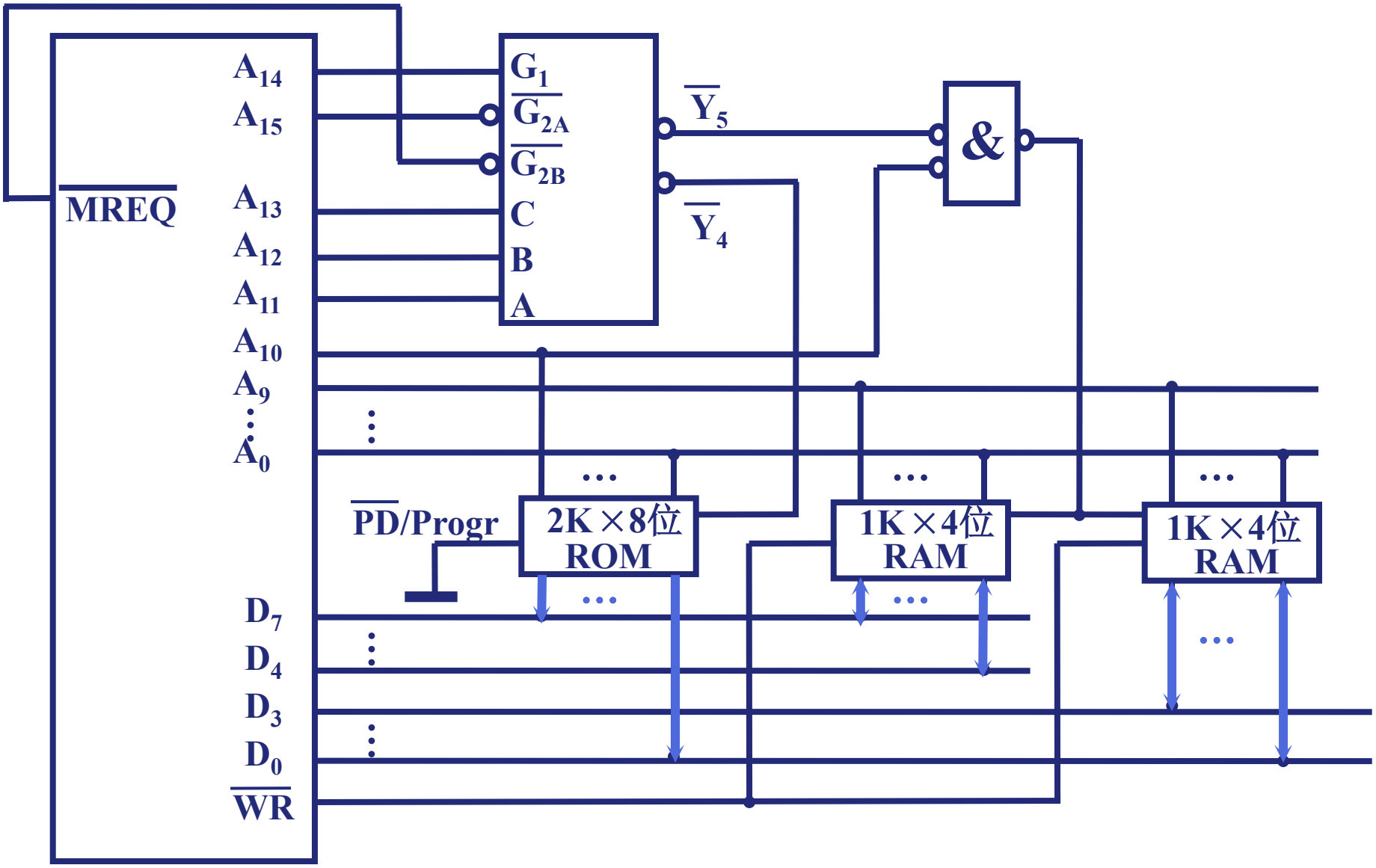
A_{15}	A_{13}	A_{11}	A_{10}	…	A_7	…	A_4	A_3	…	A_0	
0 1	1	0	0	0	0	0	0	0	0	0	1片 ROM
⋮											2K×8位
0 1	1	0	0	1	1	1	1	1	1	1	
0 1	1	0	1	0	0	0	0	0	0	0	2片RAM
⋮											1K×4位
0 1	1	0	1	0	1	1	1	1	1	1	
	C	B	A								

A_{10}~A_0 接 2K×8位 ROM 的地址线

A_9 ~A_0 接 1K×4位 RAM 的地址线

(4) 确定片选信号

例 4.1 CPU 与存储器的连接图



例4.2 假设同前，要求最小 8K为系统程序区，相邻16K为用户程序区。最大 4K为系统程序区工作区。

(1) 写出对应的二进制地址码

(2) 确定芯片的数量及类型

1片 8K × 8位 ROM 2片 8K × 8位 RAM

1片 4K × 8位 RAM

(3) 分配地址线

A_{12}~A_0 接 ROM 和 8K × 8位 RAM 的地址线

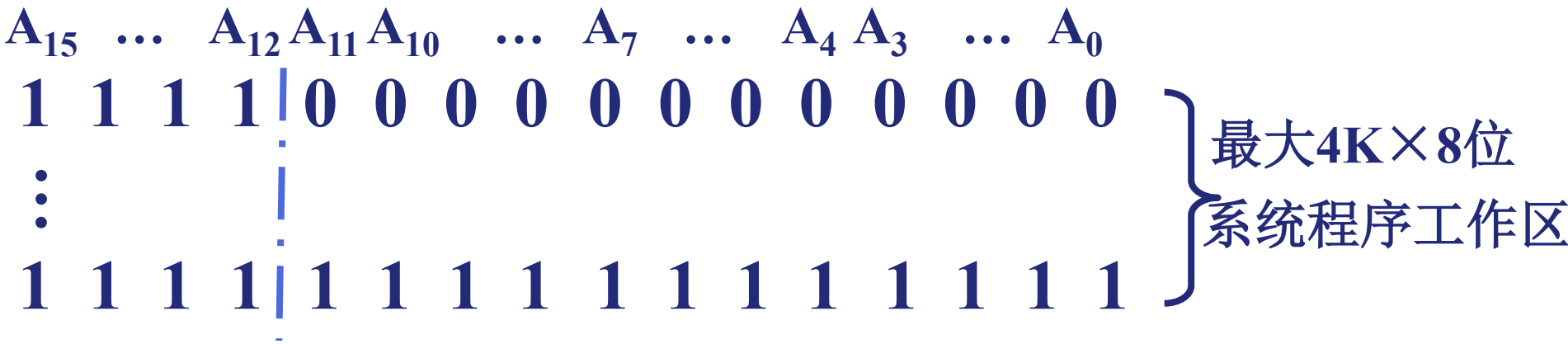
A_{11}~A_0 接 4K × 8位 RAM 的地址线

(4) 确定片选信号

(1) 写出对应的二进制地址码

A_{15} …	A_{12}	A_{11}	A_{10} …	A_7 …	A_4	A_3 …	A_0	
0 0 0	0	0	0	0 0 0 0 0	0	0	0	最小8K×8位 系统程序区
⋮								
0 0 0	1	1	1	1 1 1 1 1	1	1	1	
0 0 1	0	0	0	0 0 0 0 0	0	0	0	相邻16K×8位 用户程序区
⋮								
0 0 1	1	1	1	1 1 1 1 1	1	1	1	
0 1 0	0	0	0	0 0 0 0 0	0	0	0	
⋮								
0 1 0	1	1	1	1 1 1 1 1	1	1	1	

(1) 写出对应的二进制地址码



(2) 确定芯片的数量及类型

- 最小8KB系统程序区：1片8K×8位ROM;
- 与其相邻的16KB用户程序区：选择2片8K×8位RAM;
- 最大4KB系统程序工作区：选择1片4K×8位RAM。

例3：CPU和主存的连接

CPU地址线A15～A0，数据线D7～D0，$\overline{WR}$为读/写信号，$\overline{MREQ}$为访存请求信号。0000H～3FFFH为BIOS区，4000H～FFFFH为用户程序区。用8K×4位ROM芯片和16K×8位RAM芯片构成该存储器，要求说明地址译码方案，并将ROM芯片、RAM芯片与CPU连接。

解：因为0000H～3FFFH为BIOS，故ROM区高两位总是00，低14位为全译码。

ROM区大小为：2^{14}×8位=16K×8位=16KB，ROM芯片数为：

16K×8位 / 8K×4位 = 2×2 = 4，字方向扩展2倍，位方向扩展2倍。

ROM芯片内地址位数为13位，连到CPU低13位地址线A12～A0。

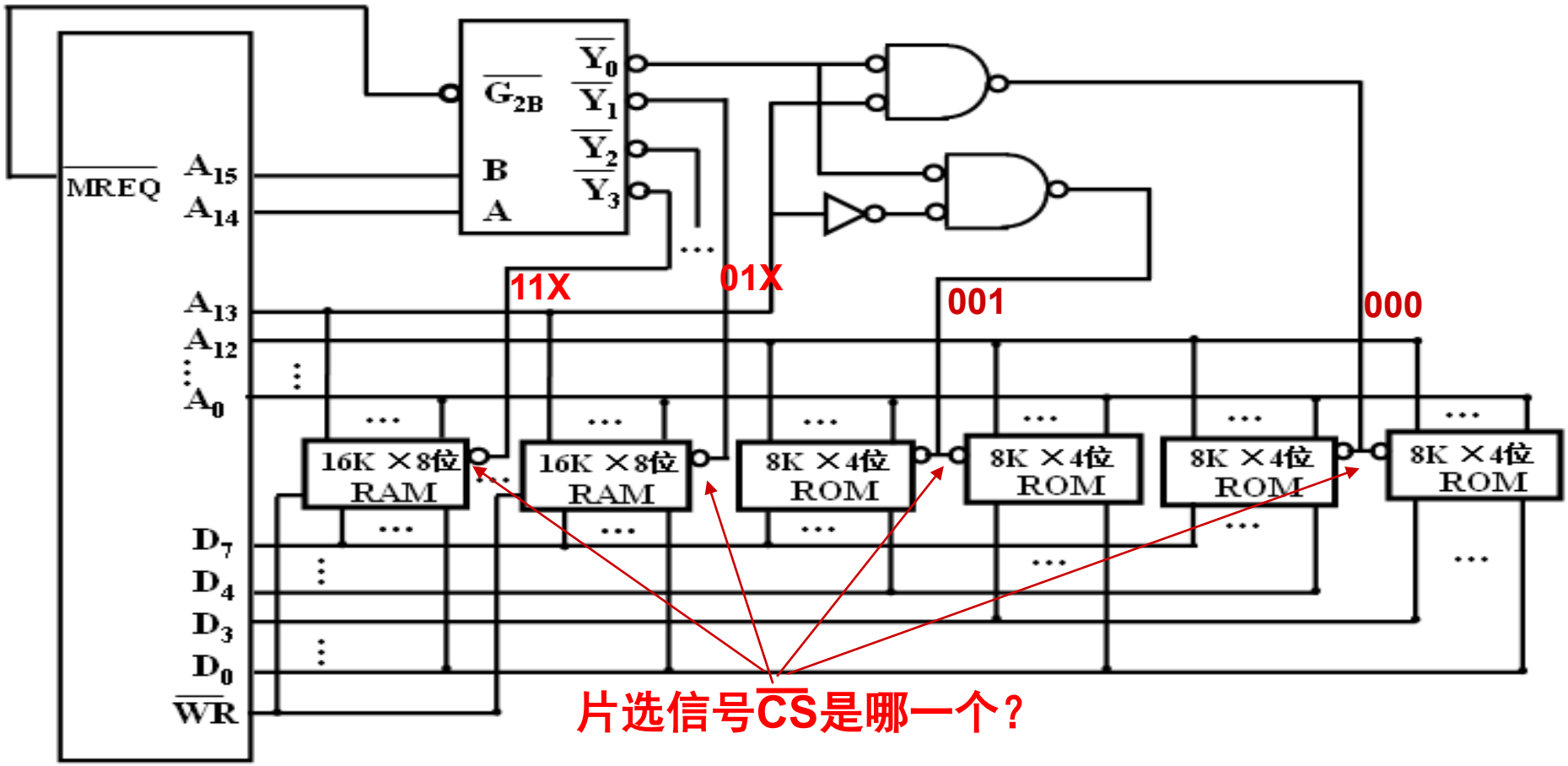
因为4000H～FFFFH为用户程序区，故RAM区高两位是01、10、11，低14位为全译码。RAM区大小为：3×2^{14}×8位=3×16K×8位= 48KB。

RAM芯片数为：

48K×8位 / 16K×8位 = 3×1 = 3，字方向上扩展3倍，位方向上不扩展。

RAM芯片内地址位数为14位，连到CPU低14位地址线A13～A0。

ROM片选由最高三位确定。
RAM片选由最高两位确定。



问题：为什么$\overline{WR}$不连到ROM芯片上？ ROM芯片只读不写

问题：$\overline{MREQ}$信号的作用是什么？ 有效(低电平)时，表示选中主存读写。

练习

CPU有16根地址线（A15～A0），8根数据线（D7～D0），$\overline{\text{MREQ}}$作为访问存储器的控制电平（低电平有效），$\overline{\text{WE}}$作为读写控制电平（$\overline{\text{WE}}$=0时，写允许，$\overline{\text{WE}}$＝1时，读允许）。现有芯片：2114（1K×4位），需要扩展为2KB内存，地址范围为2000H~27FFH，片选信号由74LS138（3－8译码器）译码进行，试画出CPU与3－8译码器及存储芯片的连接。

存储器接口

(1) 8位存储器接口

如果数据总线为8位，则存储器只能按字节编址

(2) 16位存储器接口

16位存储器系统由2个存储体组成，存储体选择通过选择信号BHE实现。

如果要传送一个16位数，则2个存储器被选中

如果要传送一个8位数，则1个存储器被选中

(3) 32位存储器接口

32位存储器系统由4个存储体组成，存储体选择通过选择信号$\overline{BE_3}$~$\overline{BE_0}$实现。

如果要传送一个32位数，则4个存储器被选中

如果要传送一个16位数，则2个存储器被选中

如果要传送一个8位数，则1个存储器被选中

存储器接口

(4) 64位存储器接口

64位存储器系统由8个存储体组成，存储体选择通过选择信号BE7~BE0实现。

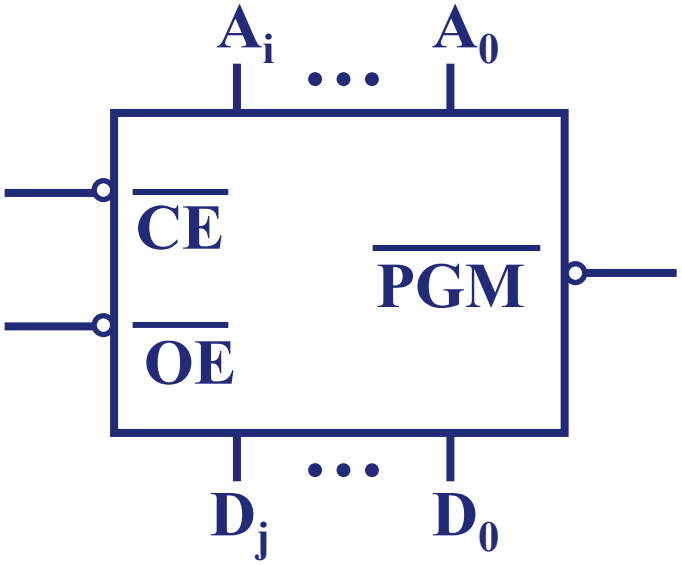
如果要传送一个64位数，则8个存储器被选中

如果要传送一个32位数，则4个存储器被选中

如果要传送一个16位数，则2个存储器被选中

如果要传送一个8位数，则1个存储器被选中

例 4.3　设 CPU 有 20 根地址线，16 根数据线。并用 $\overline{IO/M}$ 作访存控制信号。$\overline{RD}$ 为读命令，$\overline{WR}$ 为写命令。现有 PROM，外特性如下：



$\overline{CE}$　片选信号

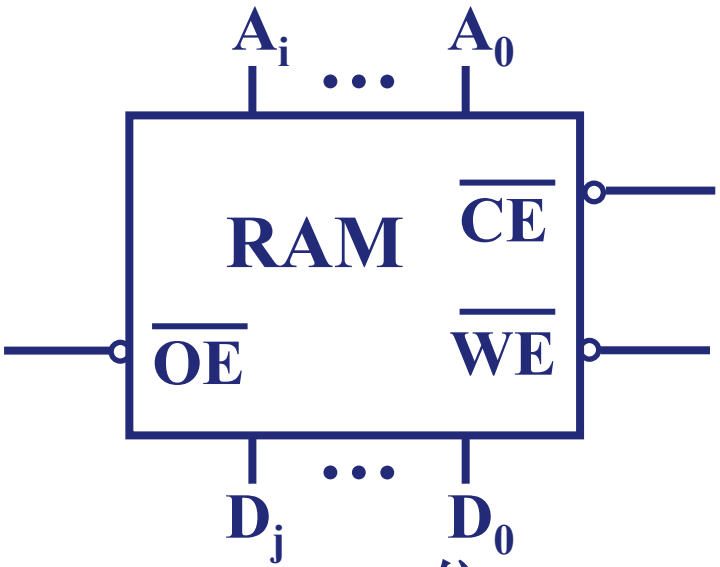
$\overline{OE}$　允许输出

$\overline{PGM}$　可编程端

64K×8位

32K×8位

32K×16位



64K×8位

32K×8位

32K×16位

$\overline{CE}$ 片选信号

$\overline{OE}$ 允许输出

$\overline{WE}$ 允许输入

用 138 译码器及其他门电路（门电路自定）

（1）CPU按字节访问和按字访问的地址范围各多大？

（2）CPU按字节访问时需分奇偶体，且最大64KB为系统程序区，与其相邻的64KB为用户程序区，写出每片存储芯片对应的二进制地址码。

（3）画出CPU与存储芯片连接图。

BHE	A0	访问形式
0	0	字
0	1	奇字节
1	0	偶字节
1	1	不访问

六、存储器的校验

- 数据校验码：是一种常用的带有发现某些错误或带有自动纠错能力的数据编码系统。
- 数据校验码检错纠错方法：“冗余校验”。
- 码距：一种码制中，任意两个合法码字之间的最小距离（不同的二进制位数）称为码距。
- 合理增大码距能提高检错和纠错能力。
- 常用的数据校验码有奇偶校验码、海明校验码和循环冗余校验码（CRC）。

码字和码距

- 码距
  - 由若干位代码组成的一个字叫“码字”
  - 两个码字中具有不同代码的位的个数叫这两个码字间的“距离”
  - 码制中各码字间最小距离为“码距”，它就是这个码制的距离。

  问题：“8421”码的码距是几?

  2（0010）和3（0011）间距离为1，“8421”码制的码距为1。
- 数据校验中“码字”指数据位和校验位按某种规律排列得到的代码
- 常用的数据校验码有：

  奇偶校验码、海明校验码、循环冗余校验码。

合法代码集合

1. {000, 001, 010, 011, 100, 101, 110, 111} 检0位错，纠0位错
2. {000, 011, 101, 110} 检1位错，纠0位错



3. {000, 111} 检1位错，纠1位错



4. {0000, 1111} 检2位错，纠1位错

1000 1100

5. {00000, 11111} 检2位错，纠2位错

11000 11100

码距与检错、纠错能力的关系

编码的纠错 、检错能力与编码的最小距离有关

$$L-1=D+C\ (D\geq C)$$

L： 编码的最小距离　　L = 3

D： 检测错误的位数　　具有 一位 纠错能力

C： 纠正错误的位数

码距与检错、纠错能力的关系（当d≤4）

① 若码距L为奇数，则能发现L-1位错，或能纠正(L-1)/2位错。

② 若码距L为偶数，则能发现L/2位错，并能纠正(L/2-1)位错。

奇偶校验码

基本思想：增加一位奇（偶）校验位并一起存储或传送，根据终部件得到的相应数据和校验位，再求出新校验位，最后根据新校验位确定是否发生了错误。

实现原理：假设数据$B=b_{n-1}b_{n-2}...b_1b_0$从源部件传送至终部件。在终部件接收到的数据为$B'=b_{n-1}'\ b_{n-2}'\ ...b_1'\ b_0'$。

第一步：在源部件求出奇（偶）校验位P。

若采用奇校验，则$P=b_{n-1}\oplus b_{n-2}\oplus...\oplus b_1\oplus b_0\oplus 1$。

若采用偶校验，则$P=b_{n-1}\oplus b_{n-2}\oplus...\oplus b_1\oplus b_0$。

第二步：在终部件求出奇（偶）校验位P'。

若采用奇校验，则$P'=b_{n-1}'\oplus b_{n-2}'\oplus...\oplus b_1'\oplus b_0'\oplus 1$。

若采用偶校验，则$P'=b_{n-1}'\oplus b_{n-2}'\oplus...\oplus b_1'\oplus b_0'$。

第三步：计算最终的校验位P*，并根据其值判断有无奇偶错。

假定P在终部件接受到的值为P''，则$P^*=P'\oplus P''$

① 若P*=1，则表示终部件接受的数据有奇数位错。

② 若P*=0，则表示终部件接受的数据正确或有偶数个错。

奇偶校验码的特点

- 问题：奇偶校验码的码距是几？为什么？
  - 码距d=2。

    在奇偶校验码中，若两个数中有奇数位不同，则它们相应的校验位就不同；若有偶数位不同，则虽校验位相同，但至少有两位数据位不同。因而任意两个码字之间至少有两位不同。
- 特点
  - 根据码距和纠/检错能力的关系，它只能发现奇数位出错，不能发现偶数位出错，而且也不能确定发生错误的位置，不具有纠错能力。
  - 开销小，适用于校验一字节长的代码，故常被用于存储器读写检查或按字节传输过程中的数据校验

    因为一字节长的代码发生错误时，1位出错的概率较大，两位以上出错则很少，所以可用奇偶校验。

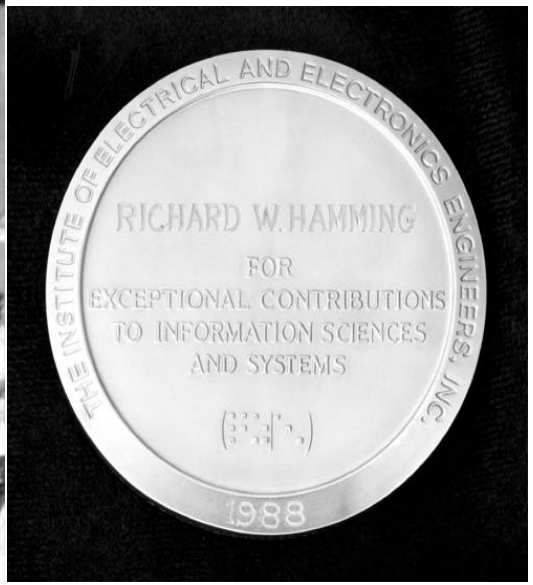
奇偶校验码

- 奇校验：每个码字（包括校验位）中1的个数为奇数
- 偶校验：每个码字（包括校验位）中1的个数为偶数

Richard Hamming (1915.2.11-1998.1.7)

- 1968年Richard W.Hamming因在数值方法，自动编码系统，错误检测和错误纠正码方面的贡献被授予图灵奖
- 1988年因为在信息科学与系统方面的杰出贡献，IEEE授予R.H.一枚奖章，并将此奖章命名为 "the Hamming Medal"





海明校验码

- 海明码：是一种能够纠正一位错误或者检测两位错误并纠正一位错误的线性编码系统。
  - 1950年由贝尔实验室的Richard Hamming首先提出
  - 主要用于存储器校验
- 基本思想：分组校验（**多重奇偶校验**）
  - 将有效信息按某种规律分为若干组，每组安排一个校验位进行奇偶校验。
  - 在数据位组中加入几个校验位，增加了数据代码间的码距。
  - 当某一位发生变化时会引起校验结果发生变化，不同代码位上的错误会得出不同的校验结果。

求信息码的海明校验码

- 设信息码为k位，校验码为r位，海明码为k + r位
- 纠正一位错：
  - r位校验位，可表示2^r个信息
    - 1个信息:无错
    - 2^r - 1个信息:错误定位
      - k位
      - r位
    - $2^r-1 \geq k+r$
- 发现两位错并纠正一位错

r位校验位
- 1位:发现2位错
- 2^{r-1}个信息:错误定位
  - k位
  - r位

- $2^{r-1} \geq k+r$

求信息码的海明校验码（续）

- 编码规律
  - 每个校验位P_i放在海明码中位号为2^{i-1}的位置
  - 海明码的每一位码H_i（包括数据位和校验位）由多个校验位进行校验：

    被校验位的位号 = 校验位位号之和

例　题

例：求信息码01101110的海明码，使之能纠正1位错误，并画出海明校验逻辑电路。

解：<1> 确定校验位位数：

设r为校验位位数，则码字的位数应满足不等式：

$$2^r-1 \geq k+r$$

设r = 3，则$2^3-1 = 7$，k+r = 8 + 3 = 11，不满足

设r = 4，则$2^4-1 = 15$，k+r = 8 + 4 = 12，满足

所以，r最小取4。

例 题（续）

例：求信息码01101110的海明码，使之能纠正1位错误，并画出海明校验逻辑电路。

<2> 确定校验位位置:校验位P_i位于位号为2^{i-1}的位置

因此，P_1、P_2、P_3、P_4位于2^0、2^1、2^2、2^3的位置

1	2	3	4	5	6	7	8	9	10	11	12
P_1	P_2	D_7	P_3	D_6	D_5	D_4	P_4	D_3	D_2	D_1	D_0

例 题（续）

<3> 分组：有4个校验位，将12位分成4组，

被校验位的位号 = 校验位位号之和。

	1	2	3	4	5	6	7	8	9	10	11	12
	P1	P2	D7	P3	D6	D5	D4	P4	D3	D2	D1	D0
			0		1	1	0		1	1	1	0
第一组(p1)1	√		√		√		√		√		√	
第二组(p2)2		√	√			√	√			√	√	
第三组(p3)4				√	√	√	√					√
第四组(p4)8								√	√	√	√	√

例 题（续）

	1	2	3	4	5	6	7	8	9	10	11	12
	P1	P2	D7	P3	D6	D5	D4	P4	D3	D2	D1	D0
	1	1	0	0	1	1	0	1	1	1	1	0
第一组(p1)1	√		√		√		√		√		√	
第二组(p2)2		√	√			√	√			√	√	
第三组(p3)4				√	√	√	√					√
第四组(p4)8								√	√	√	√	√

<4> 校验位的形成:

P_1＝第一组中所有位（除P_1外）求异或＝$D_7 \oplus D_6 \oplus D_4 \oplus D_3 \oplus D_1 = 1$

P_2＝第二组中所有位（除P_2外）求异或＝$D_7 \oplus D_5 \oplus D_4 \oplus D_2 \oplus D_1 = 1$

P_3＝第三组中所有位（除P_3外）求异或＝$D_6 \oplus D_5 \oplus D_4 \oplus D_0 = 0$

P_4＝第四组中所有位（除P_4外）求异或＝$D_3 \oplus D_2 \oplus D_1 \oplus D_0 = 1$

∴ **海明码为110011011110**

例 题（续）

<5> 校验原理

在数据输出存储器时分别求S_1,S_2,S_3,S_4

$S_1=P_1$⊕第一组中所有位求异或$=P_1⊕D_7⊕D_6⊕D_4⊕D_3⊕D_1$

$S_2=P_2$⊕第二组中所有位求异或$=P_2⊕D_7⊕D_5⊕D_4⊕D_2⊕D_1$

$S_3=P_3$⊕第三组中所有位求异或$=P_3⊕D_6⊕D_5⊕D_4⊕D_0$

$S_4=P_4$⊕第四组中所有位求异或$=P_4⊕D_3⊕D_2⊕D_1⊕D_0$

当$S_4S_3S_2S_1=0000$时，则从存储器中取出的数据无错。

否则，$S_4S_3S_2S_1$的二进制编码即为出错位号。

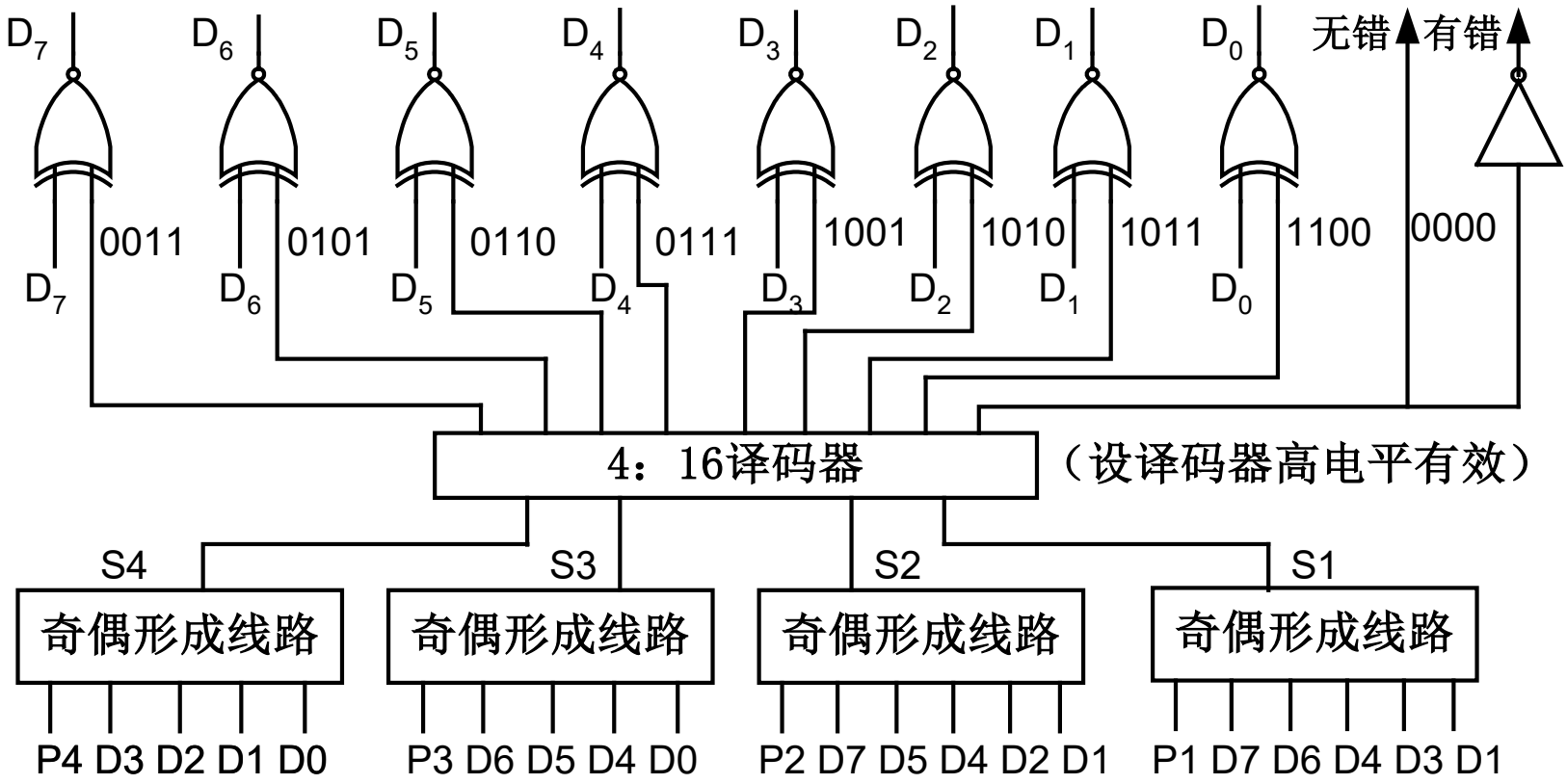
例如：$S_4S_3S_2S_1=1001$，

1	2	3	4	5	6	7	8	9	10	11	12
P_1	P_2	D_7	P_3	D_6	D_5	D_4	P_4	D_3	D_2	D_1	D_0

说明第9位出错，即D_3错，将其取反，即可纠错。

例 题（续）

- 纠正一位出错位的海明校验逻辑电路



例 题（续）

- 进一步思考，本题改为求能够检测两位错误并纠正一位错误的海明校验逻辑电路。

<1> 确定校验位位数：

设r为校验位位数，则码字的位数应满足不等式：

$2^{r-1} \geq k+r$

设r＝4，则$2^{4-1}=8$， k+r＝8＋4＝12，不满足

设r＝5，则$2^{5-1}=16$，k+r＝8＋5＝13，满足

所以，r最小取5。

例 题（续）

- 进一步思考，本题改为求能够检测两位错误并纠正一位错误的海明校验逻辑电路。

<2> 确定校验位位置:校验位P_i位于位号为2^{i-1}的位置

因此，P_1、P_2、P_3、P_4、P_5位于2^0、2^1、2^2、2^3、2^4的位置，但由于共有13个数，所以最大位号为13

1	2	3	4	5	6	7	8	9	10	11	12	13
P_1	P_2	D_7	P_3	D_6	D_5	D_4	P_4	D_3	D_2	D_1	D_0	P_5

例 题（续）

<3> 分组：有5个校验位，将13位分成5组，
被校验位的位号＝校验位位号之和，P5为总校验。

	1	2	3	4	5	6	7	8	9	10	11	12	13
	P1	P2	D7	P3	D6	D5	D4	P4	D3	D2	D1	D0	P5
			0		1	1	0		1	1	1	0	
第一组(p1)1	√		√		√		√		√		√		
第二组(p2)2		√	√			√	√			√	√		
第三组(p3)4				√	√	√	√					√	
第四组(p4)8								√	√	√	√	√	
第五组(p5)13	√	√	√	√	√	√	√	√	√	√	√	√	√

例 题（续）

	1	2	3	4	5	6	7	8	9	10	11	12	13
	P1	P2	D7	P3	D6	D5	D4	P4	D3	D2	D1	D0	P5
	1	1	0	0	1	1	0	1	1	1	1	0	0
第一组(p1)1	√		√		√		√		√		√		
第二组(p2)2		√	√			√	√			√	√		
第三组(p3)4				√	√	√	√					√	
第四组(p4)8								√	√	√	√	√	
第五组(p5)13	√	√	√	√	√	√	√	√	√	√	√	√	√

<4> 校验位的形成:

P_1＝第一组中所有位（除P_1外）求异或＝$D_7 \oplus D_6 \oplus D_4 \oplus D_3 \oplus D_1 = 1$

P_2＝第二组中所有位（除P_2外）求异或＝$D_7 \oplus D_5 \oplus D_4 \oplus D_2 \oplus D_1 = 1$

P_3＝第三组中所有位（除P_3外）求异或＝$D_6 \oplus D_5 \oplus D_4 \oplus D_0 = 0$

P_4＝第四组中所有位（除P_4外）求异或＝$D_3 \oplus D_2 \oplus D_1 \oplus D_0 = 1$

P_5＝第五组中所有位（除P_5外）求异或＝$P_1 \oplus P_2 \oplus D_7 \oplus \ldots \oplus D_0 = 0$

∴ **海明码为1100110111100**

例 题（续）

<5> 校验原理

在存储字读出时分别求S_1,S_2,S_3,S_4,S_5

$S_1=P_1$⊕第一组中所有位求异或$=P_1⊕D_7⊕D_6⊕D_4⊕D_3⊕D_1$

$S_2=P_2$⊕第二组中所有位求异或$=P_2⊕D_7⊕D_5⊕D_4⊕D_2⊕D_1$

$S_3=P_3$⊕第三组中所有位求异或$=P_3⊕D_6⊕D_5⊕D_4⊕D_0$

$S_4=P_4$⊕第四组中所有位求异或$=P_4⊕D_3⊕D_2⊕D_1⊕D_0$

$S_5=P_5$⊕第五组中所有位求异或$=P_1⊕…⊕P_5⊕ D_7⊕…D_0$

S_5=0,则无错或偶数个错

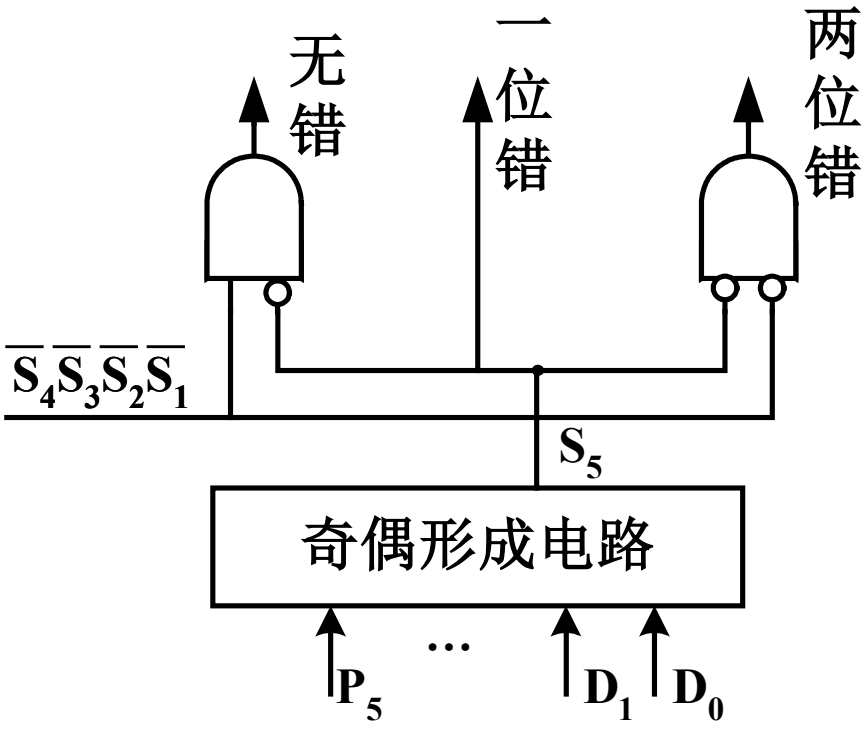
- **$S_4S_3S_2S_1$ =0000, 无错**
- **S_4 ~ S_1 不全为零,两位错**

S_5=1,则有奇数个错

- **S_4 ~ S_1 中只有1个不为0,S_i≠0, 则P_i出错**
- **$S_4S_3S_2S_1$ =0000, 则P_5出错**
- **S_4 ~ S_1 两位以上不为零,则数据位出错且$S_4S_3S_2S_1$的二进制代码即出错位号**

例 题（续）

- 检测两位错误并自动纠正一位错误的附加电路



循环冗余码

循环冗余校验码（Cyclic Redundancy Check），简称CRC码

- 具很强的检错、纠错能力。
- 用于大批量数据存储和传送中的数据校验。是目前磁表面存储器中应用最广泛的一种校验方法，也是多机通信中常用的校验方法
- 可以发现并纠正信息串行读写、存储或传送过程中出现的一位或多位错误。
- 为什么大批量数据不用奇偶校验？

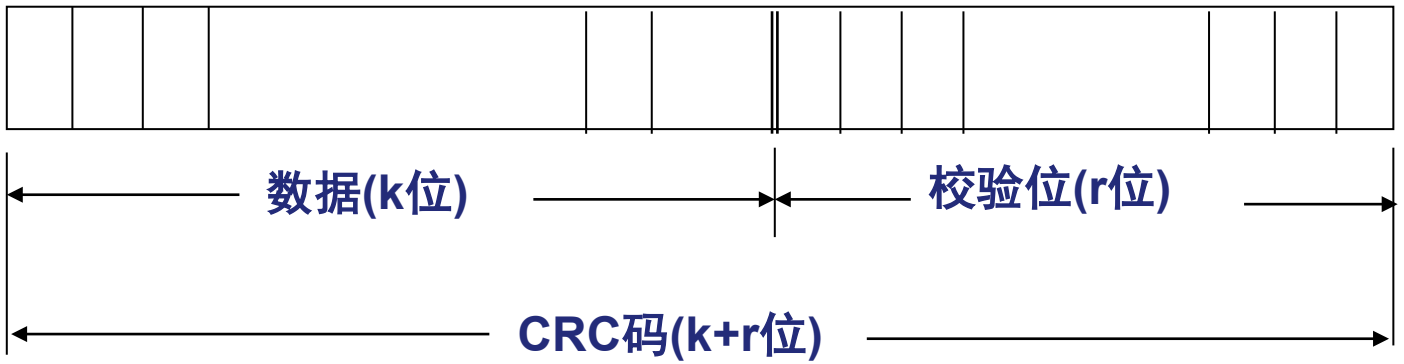
  在每个字符后增加一位校验位会增加大量的额外开销；尤其在网络通信中，对传输的二进制比特流没有必要再分解成一个个字符，因而无法采用奇偶校验码。
- 通过某种数学运算来建立数据和校验位之间的约定关系。

  奇偶校验码和海明校验码都是以奇偶检测为手段的。

CRC码的编码方法

基本思想：

- 数据信息M(x)为一个K位的二进制数据，将M(x)左移r位后，用一个约定的“生成多项式”G(x)相除，G(x)是一个r+1位的二进制数，相除后得到的r位余数就是校验位。校验位拼接到M(x)后，形成一个K+r位的代码，称该代码为循环冗余校验（CRC）码，也称（k+r,k）码。



CRC码的编码方法

- 从k位信息位得到r位校验位的方法：

(1) 将待编码k位有效信息写成多项式M(x)

$$M(x)=C_{k-1}x^{k-1}+C_{k-2}x^{k-2}+\ldots+C_1x+C_0$$

(2) 将M(x)左移r位，得到$M(x)\cdot x^r$。目的是空出r位，以便拼接r位校验位

(3) 选取一个r+1位的生成多项式G(x)。对$M(x)\cdot x^r$做**模2除**运算

$$\frac{M(x)\times x^r}{G(x)}=Q(x)+\frac{R(x)}{G(x)}$$

（要产生r位余数，所以除数应为r+1位）

(4) 将左移r位的待编码信息与余数R(x)做**模2加**，得到CRC码

$$M(x)\cdot x^r+R(x)=Q(x)\cdot G(x)$$

说明CRC码能被生成多项式G(x)整除（此结论是CRC译码纠错的依据）

以上说明CRC码能被生成多项式整除，这是译码与纠错得依据

CRC码的校验方法

- 一个CRC码一定能被生成多项式整除，当数据和校验位一起送到接受端后，只要将接受到的数据和校验位用约定好的同样的生成多项式除，如果能除尽，表明没有发生错误；若除不尽，则表明某些数据位发生了错误，余数将指明出错位所在的位置。

CRC码计算

例：信息码为1100，生成多项式$G(x)=x^3+x+1$，求CRC码。

解：$M(x)=x^3+x^2$　　$G(x)=1011$　　$r=3$

$M(x)\cdot x^3=1100\ 000$

$$\frac{M(x)\cdot x^3}{G(x)}=\frac{1100000}{1011}=1110+\frac{010}{1011}$$

CRC码为1100 010

循环冗余码举例

$X^3 \cdot M(x) \div G(x) = (x^8 + x^4 + x^3) \div (x^3 + 1)$

```
          100111
      ------------
1001 | 100011000
       1001
       ----
        0011
        0000
        ----
         0111
         0000
         ----
          1110
          1001
          ----
           1110
           1001
           ----
            1110
            1001
            ----
             111
```

（模２运算不考虑加法进位和减法借位，上商的原则是当部分余数首位是１时商取１，反之商取０。然后按模２相减原则求得最高位后面几位的余数。这样当被除数逐步除完时，最后的余数位数比除数少一位。这样得到的余数就是校验位，此例中最终的余数有3位。）

校验位为111，CRC码为100011 111。如果要校验CRC码，可将CRC码用同一个多项式相除，若余数为0，则说明无错；否则说明有错。例如，若在接收方的CRC码也为100011 111时，用同一个多项式相除后余数为0。若接收方CRC码不为100011 111时，余数则不为0。

表 4.5 对应 $G(x)=1011$ 的（7，4）循环的出错模式

序号	N_1	N_2	N_3	N_4	N_5	N_6	N_7	余数	出错位
正确	1	1	0	0	0	1	0	000	无
错误	1	1	0	0	0	1	1	001	7
	1	1	0	0	0	0	0	010	6
	1	1	0	0	1	1	0	100	5
	1	1	0	1	0	1	0	011	4
	1	1	1	0	0	1	0	110	3
	1	0	0	0	0	1	0	111	2
	0	1	0	0	0	1	0	101	1

七、提高访存速度的措施

- 采用高速器件
- 采用cache（高速缓冲存储器）
- 采用并行存储器（多体交叉存储器）
- 采用多端口存储器
- 采用相联存储器，加长存储器的字长

CPU的速度比存储器的速度快，假如能同时从存储器取出n条指令，这必然会提高机器的运行速度。

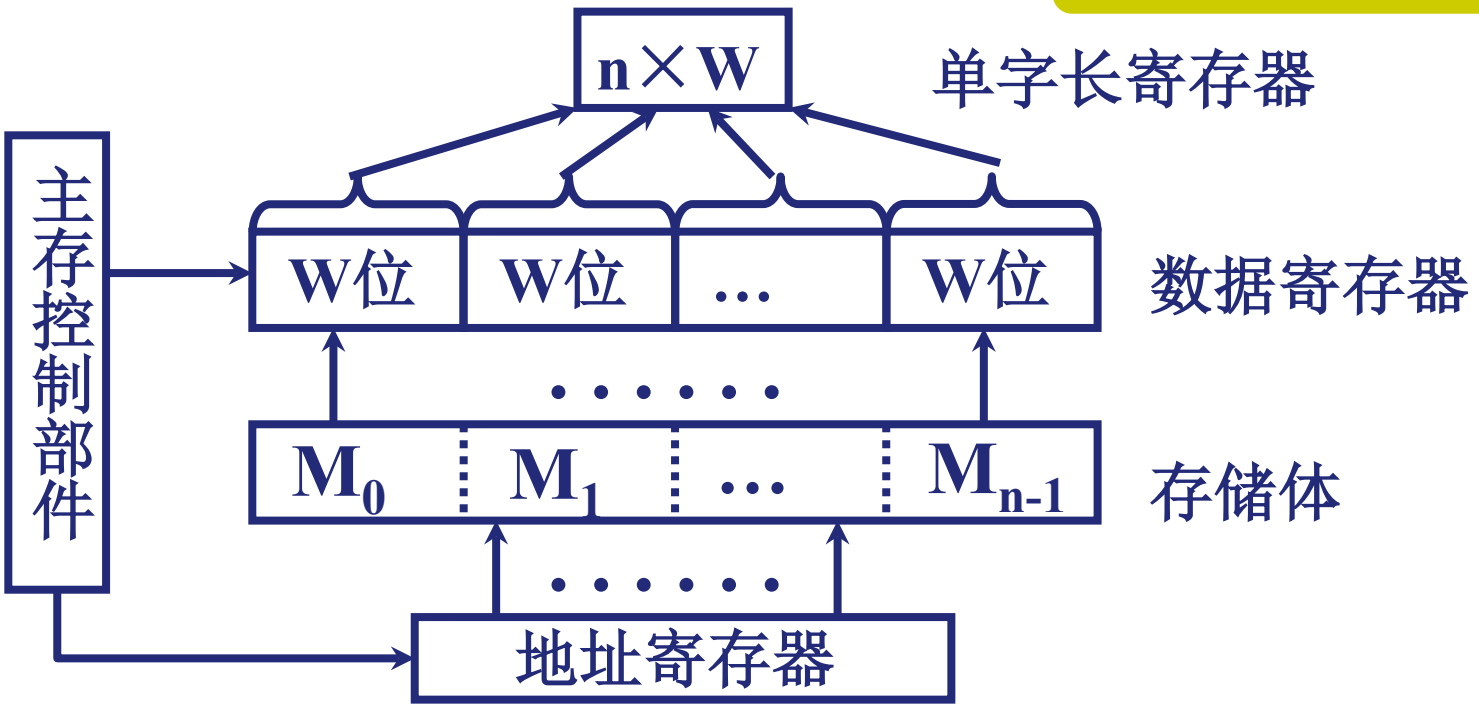
1. 单体多字并行主存系统

- 对于单体存储器，可以在一个存储周期内并行读出多个指令，通过提高整体信息吞吐率来提高访存速度。
- 如果多个并行工作的存储体共有一套地址寄存器和译码电路，按同一地址并行地访问各自的对应单元，也可以将这n个存储体看作一个大存储体，一次访问n个字，也称为单体多字系统。

1. 单体多字并行主存系统

- 例如：CPU输出地址A，则n个存储器中的所有A单元同时被选中。假设每个存储器的字长为w位，则同时访问n×w位。

增加存储器的带宽



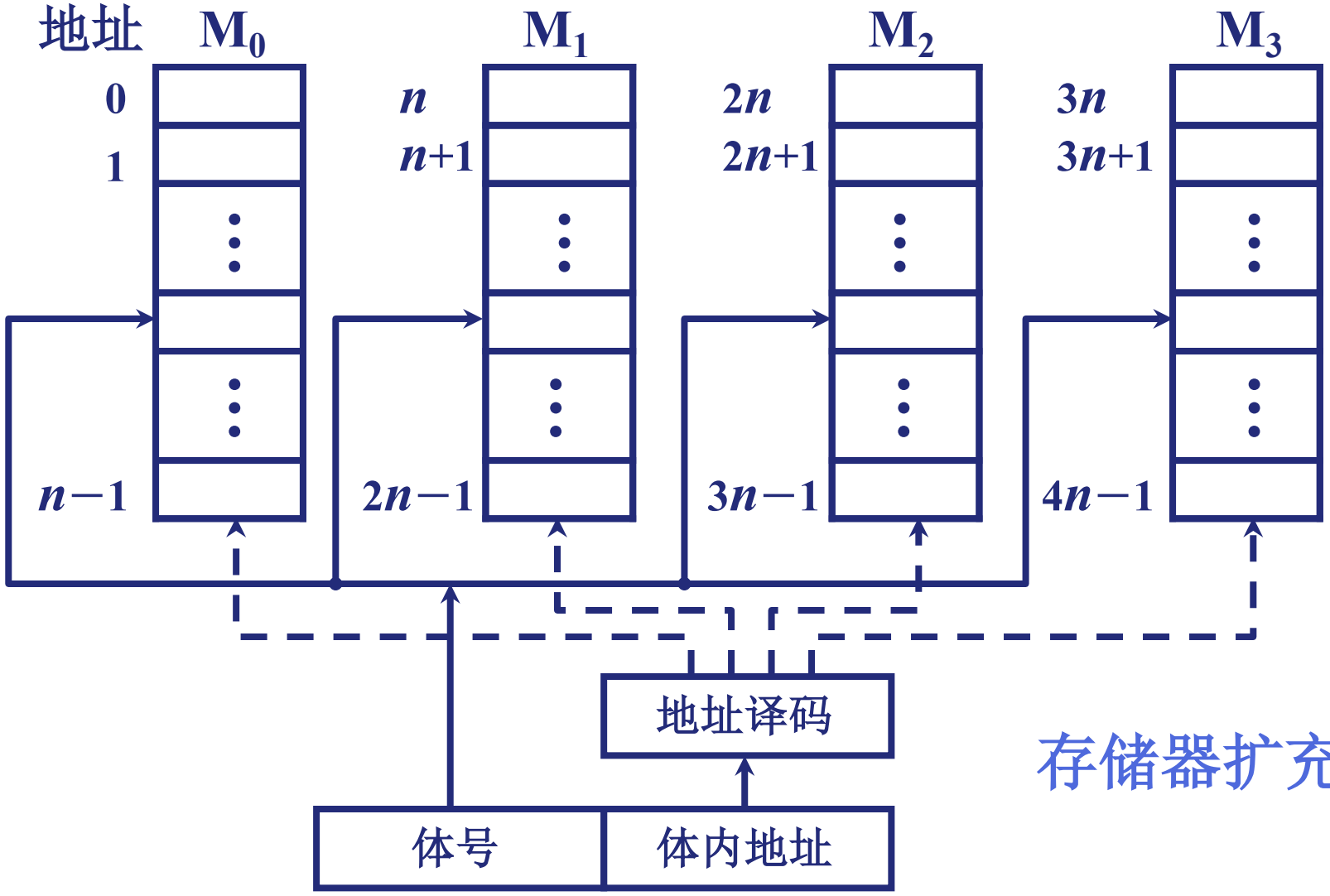
前提：指令和数据在主存中必须是连续存放的，一旦遇到转移指令，或者操作数不能连续存放，这种方法的效果就不明显了。

2. 多体并行系统

- 计算机中大容量的主存，可由多个存储体组成，每个存储体都具有自己的读写线路、地址寄存器和数据寄存器，称为“存储模块”。这种多模块存储器可以实现重叠与交叉存取。

(1) 高位交叉

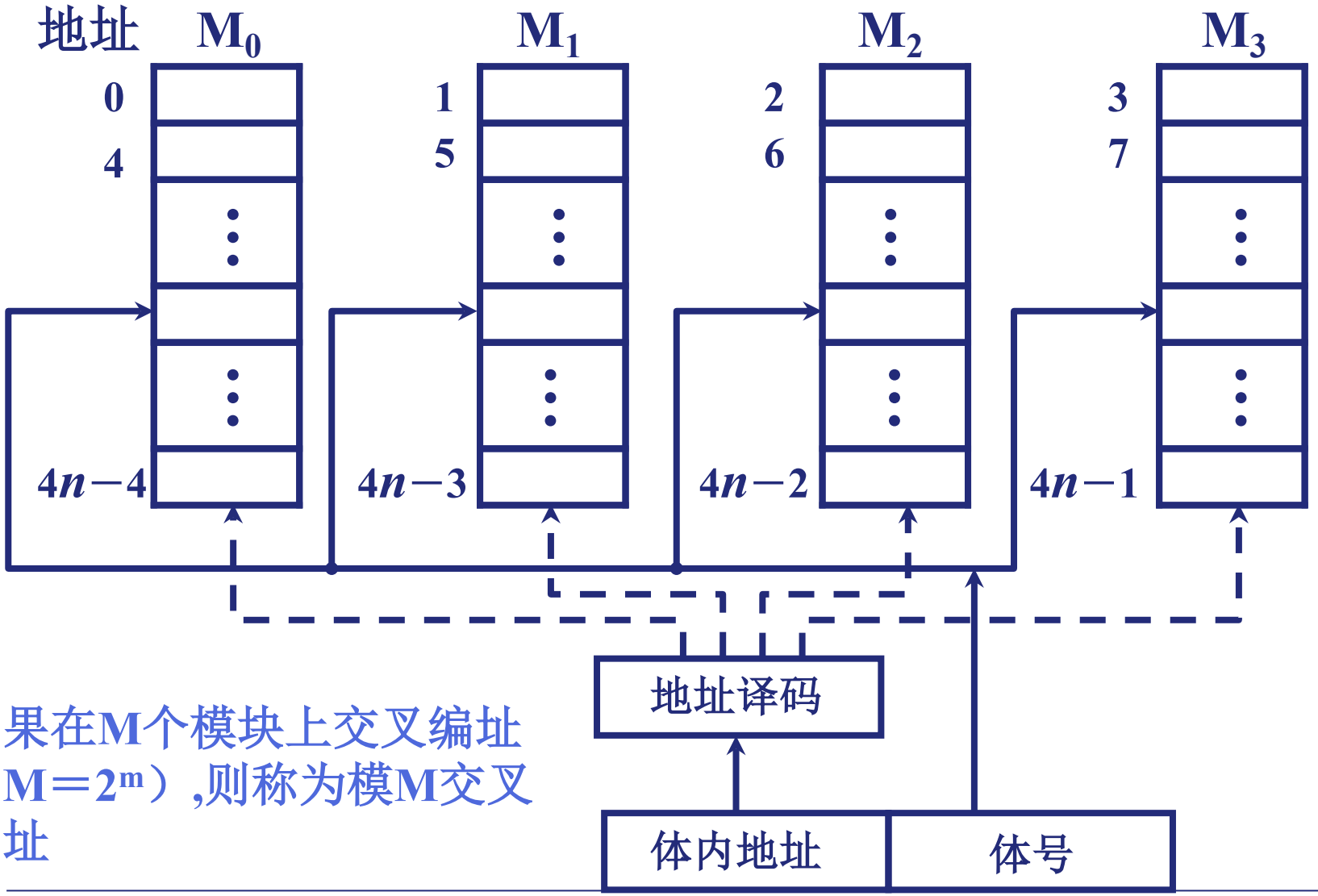
如果不同请求源访问不同的体，可实现各个体并行工作。



存储器扩充容易。

(2) 低位交叉 (各个体轮流编址)

连续地址分布在相邻的不同模块内，同一模块内的地址都不连续



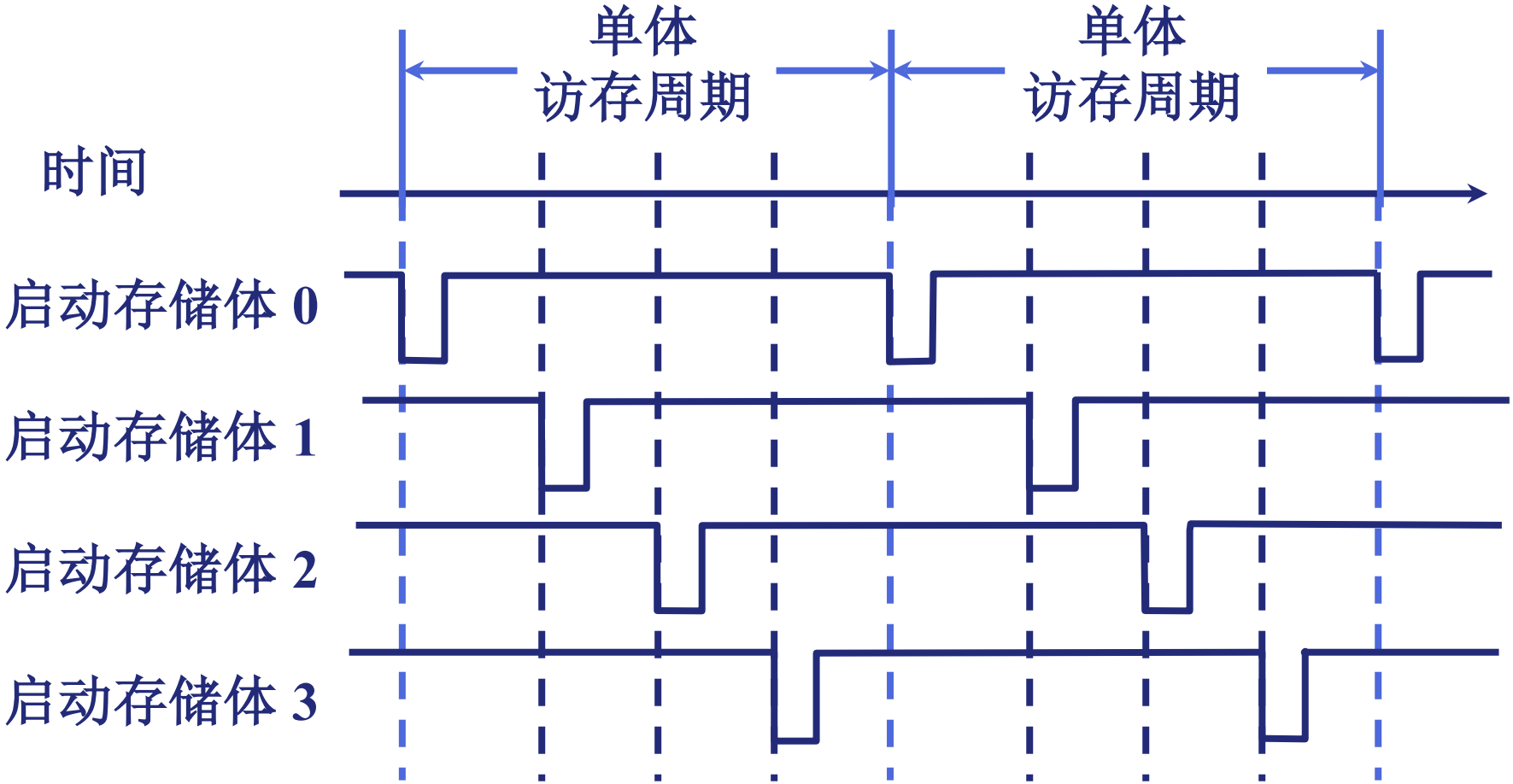
如果在M个模块上交叉编址（$M=2^m$）,则称为模M交叉编址

(2) 低位交叉 (各个体轮流编址)

- 在第一个存储周期的开始时刻启动模块M0，在$T_m/4$、$T_m/2$、$3T_m/4$时刻分别启动模块M1、M2、M3
- 在4个分体完全并行的理想情况下，整个主存的存取周期缩小到原来的1/4，数据传送的平均速度提高到原来的4倍。
- 在理想情况下，如果程序段和数据块都连续地在主存中存放和读取，那么，这种编址方式将大大地提高主存的有效访问速度。
- 但遇到程序转移或随机访问少量数据，访问地址就不一定均匀地分布在多个存储模块之间，这样就会产生存储器冲突而降低了使用率，所以M个交叉模块的使用率是变化的。

低位交叉的特点

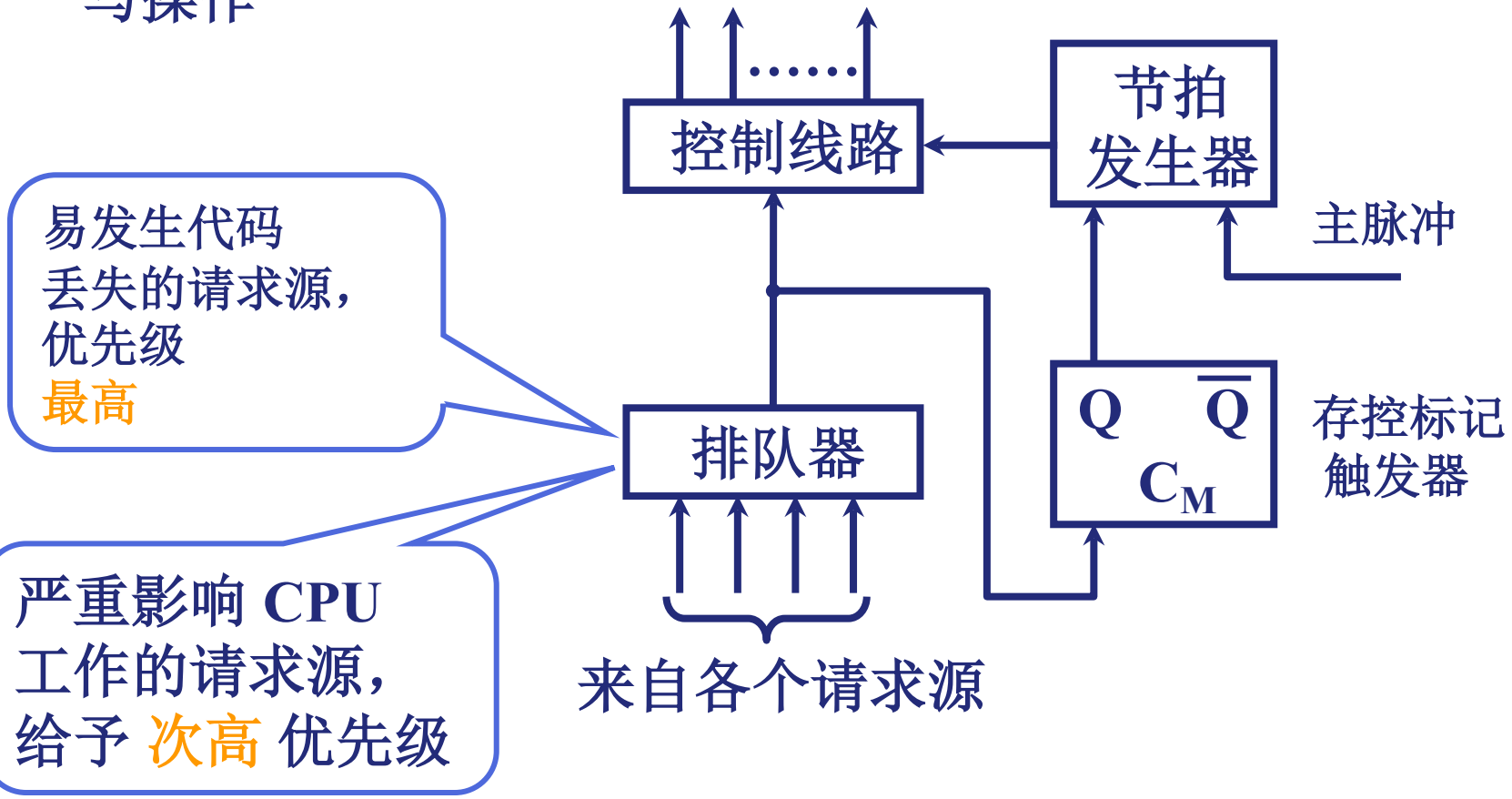
在不改变存取周期的前提下，增加存储器的带宽



假设每个体的存储字长和数据总线宽度一致，假设模块数为n，存取周期为T，总线传输周期为τ，应满足$T=n\tau$

(3) 存储器控制部件（简称存控）

- 多体模块存储器不仅要与CPU交换信息，还要与辅存、I/O设备、I/O处理机交换信息
- 控存功能：合理安排各部件请求访问的顺序；控制主存读/写操作

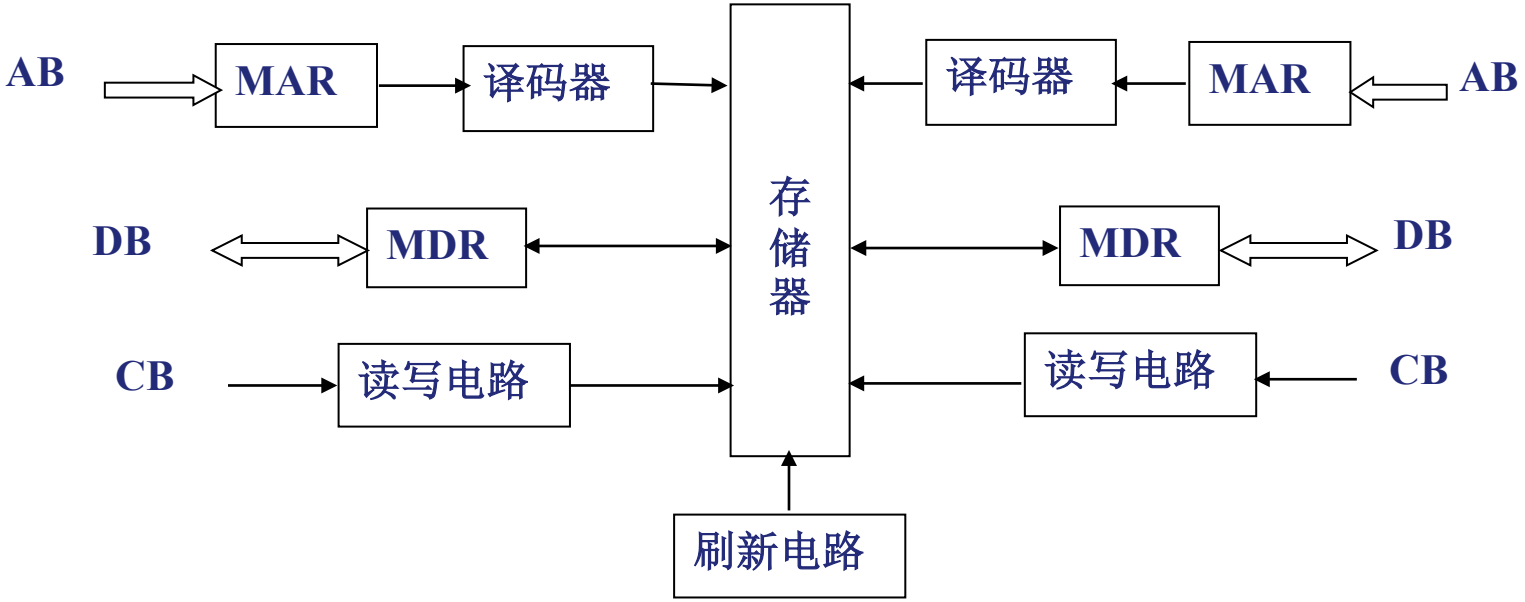


3. 多端口SRAM

- 单端口存储器任一时刻只能接收来自CPU和I/O中一方的访存请求,是串行工作模式。
- **Multi-SRAM：为需要数据共享而设计，**
  - 进行数据共享的多个不同设备需要异步访问保存在同一存储体的信息
- 具有两个或多个端口，不同端口分别连接不同设备，通常具有输出允许$\overline{\text{OE}}$、写允许$\overline{\text{WE}}$和端口允许$\overline{\text{CE}}$三个控制信号
- 对两端口同时读，不仲裁
- 对一个端口读，同时对另一个端口写，或同时对两端口写，需要仲裁。
  - 读写冲突使数据不可预测：读后写、写后读；同时写
  - 解决方法：增加额外读周期；使写操作的多组指定地址只通过一个端口输入

4. 双端口存储器

- 双端口随机存储器DPARAM(Dual-port Access RAM)由两个访问端口，即两套MAR、MDR、地址译码器和读写电路，两个端口分别连接两套独立总线，同时接收来自两方的访存请求，使存储器并行工作。



双端口存储器

- 两个访问端口独立工作互不干扰，只有当两个端口试图在同一时间访问同一地址单元，才会发生冲突。
- 存储器仲裁逻辑根据两端口访问请求到达存储器的微小时间差来决定首先为哪一方服务，被延缓访问的另一方被耽误的时间很短，小于一个存取周期。

4.3 高速缓冲存储器

一、概述

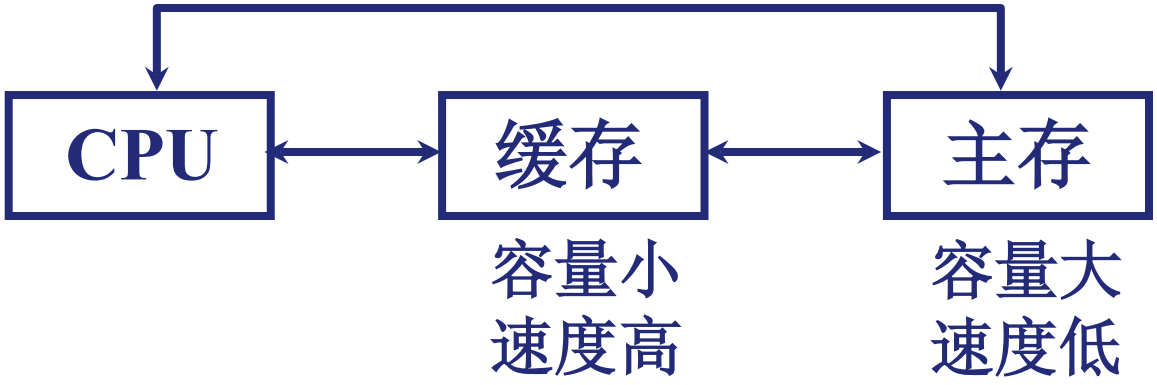
多体并行系统中I/O访存级别高于CPU访存级别，CPU等待

主存速度发展低于CPU速度发展，CPU与DRAM速度间隙每年增加50%

1. 问题的提出

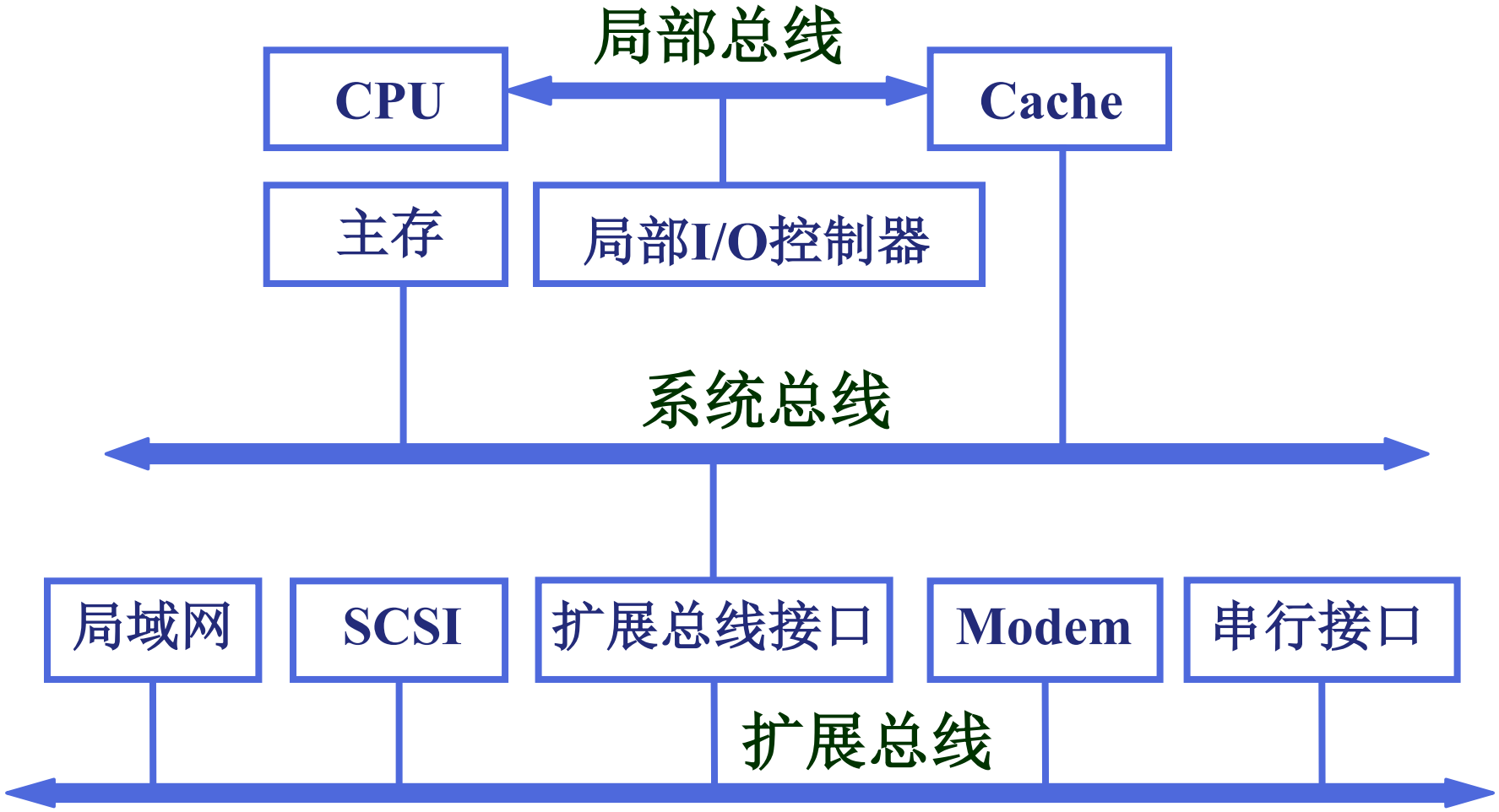
避免 CPU “空等” 现象

CPU 和主存（DRAM）的速度差异



理论依据：程序访问的局部性原理。即：在一个较短的时间间隔内，CPU对局部范围的存储器地址频繁访问，而对此地址范围之外的地址访问很少。

回顾三总线结构

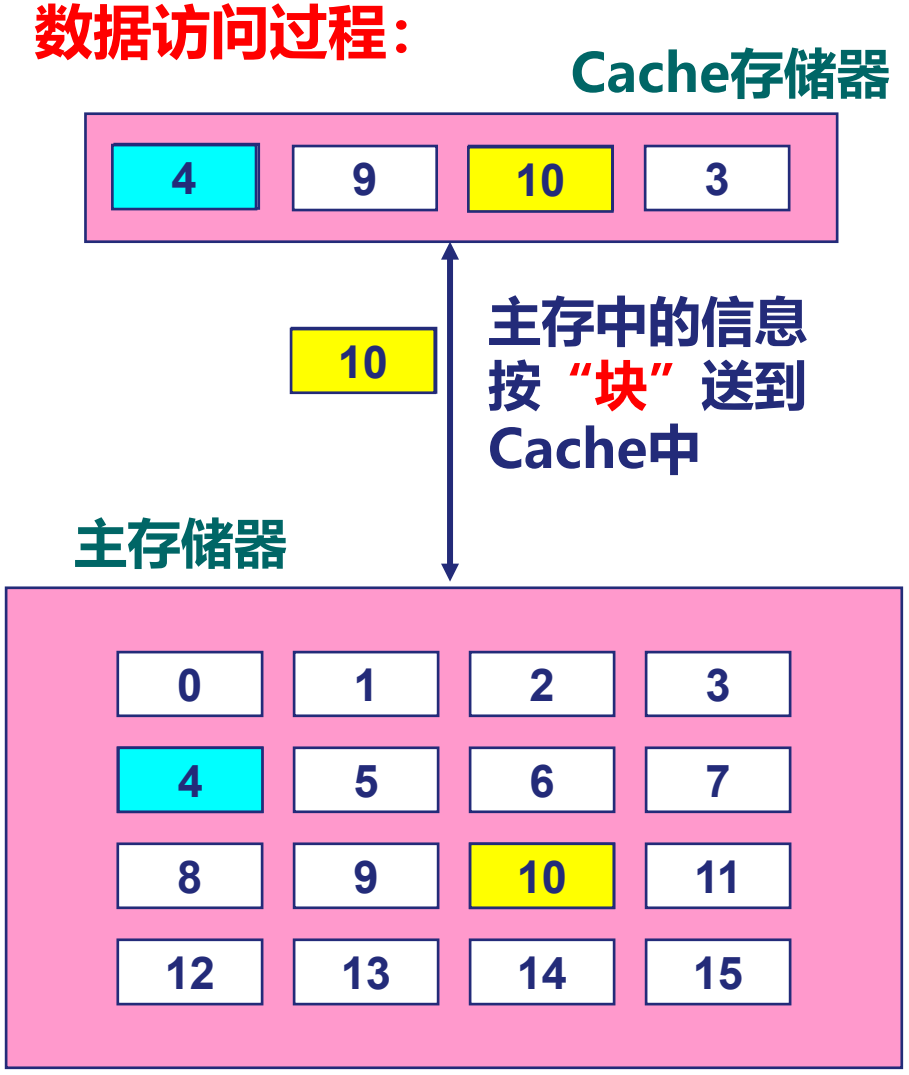

局部总线
CPU
Cache
主存
局部I/O控制器
系统总线
局域网
SCSI
扩展总线接口
Modem
串行接口
扩展总线

程序访问的局部性原理

- 引入Cache的理论依据：程序访问的局部性原理。
  - 大量典型程序的运行情况分析结果表明，在一个较短的时间间隔内，CPU对局部范围的存储器地址频繁访问，而对此地址范围之外的地址访问很少。
- 程序具有访问局部性特征的原因
  - 指令：指令按序存放，地址连续，循环程序段或子程序段重复执行
  - 数据：连续存放，数组元素重复、按序访问
- 程序访问局部性分为空间局部性和时间局部性
- 基于程序访问的局部性使访存要求能快速得到响应
  - 在CPU和主存之间设置一个快速小容量的存储器，其中总是存放最活跃（被频繁访问）的程序块和数据，由于程序访问的局部性特征，大多数情况下，CPU能直接从这个高速缓存中取得指令和数据，而不必访问主存。

Cache(高速缓存)是什么样的？

- Cache是一种小容量高速缓冲存储器，它由SRAM组成。
- Cache直接制作在CPU芯片内，速度几乎与CPU一样快。
- 程序运行时，CPU使用的一部分数据/指令会预先成批拷贝在Cache中，Cache的内容是主存储器中部分内容的映象。
- 当CPU需要从内存读(写)数据或指令时，先检查Cache，若有，就直接从Cache中读取，而不用访问主存储器。



Cache(高速缓存)是什么样的?

问题：要实现Cache机制需要解决哪些问题？

如何分块？

主存块和Cache之间如何映射？

Cache已满时，怎么办？

写数据时怎样保证Cache和MM的一致性？

如何根据主存地址访问到cache中的数据？

主存被分成若干大小相同的块，称为主存块(Block)，Cache也被分成相同大小的块，称为Cache行（line）或槽（Slot）。

……

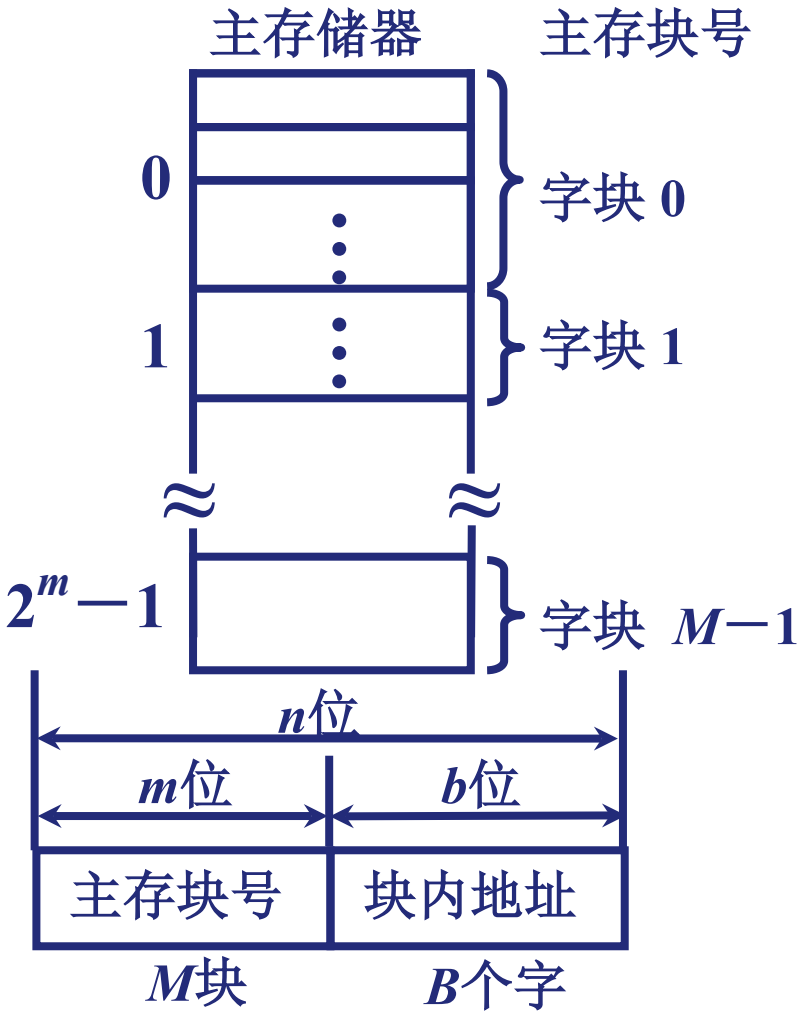
问题：Cache对程序员(编译器)是否透明？为什么？

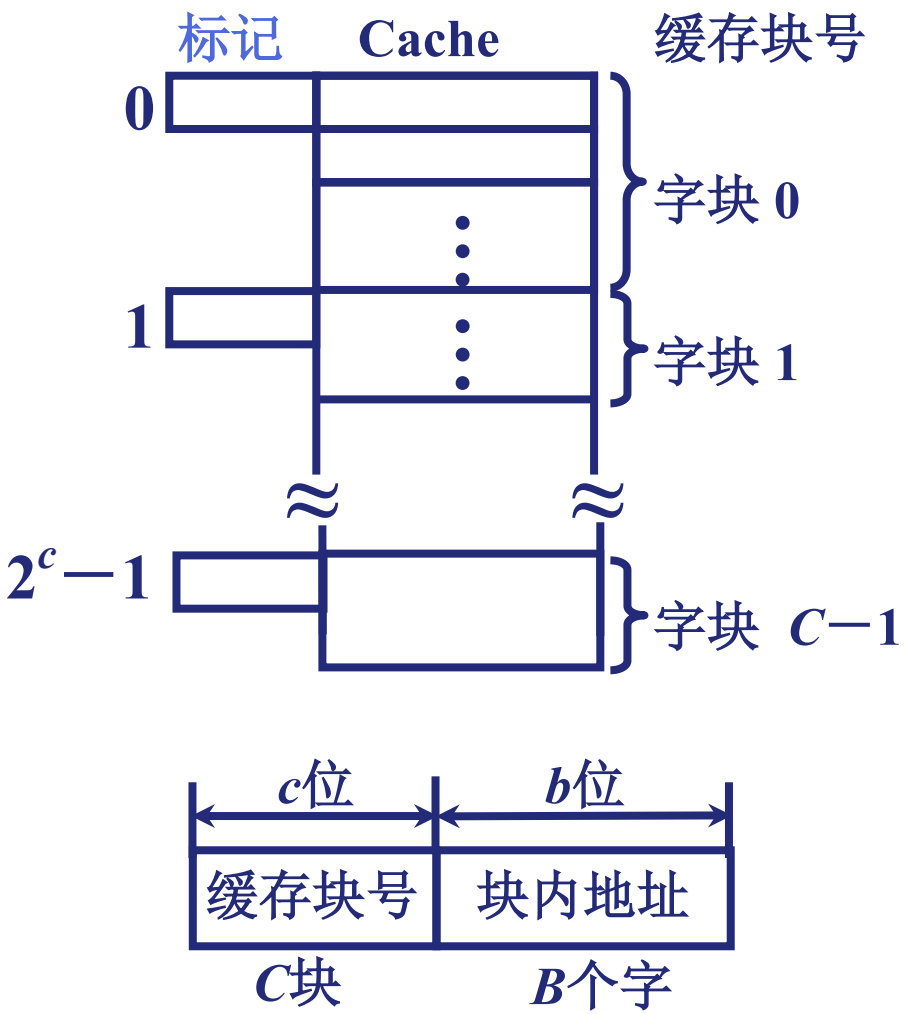
是透明的，程序员(编译器)在编写/生成高级或低级语言程序时无需了解Cache是否存在或如何设置，感觉不到cache的存在。

但是，对Cache深入了解有助于编写出高效的程序！

2. Cache 的工作原理

(1) 主存和缓存的编址





主存和缓存按块存储　　块的大小相同　　B 为块长

(2) 命中与未命中

缓存共有 C 块

主存共有 M 块　$M >> C$

命中(hit)　主存块 调入 缓存

主存块与缓存块 建立 了对应关系

用 标记记录 与某缓存块建立了对应关系的 主存块块号

未命中
(缺失 miss)　主存块 未调入 缓存

主存块与缓存块 未建立 对应关系

(3) Cache 的命中率（hit rate）

CPU 欲访问的信息在 Cache 中的 比率

在程序执行期间，设*Nc*为访问Cache命中的次数，*Nm*为访问主存总次数，命中率为*h*，则

$$h = \frac{Nc}{Nc + Nm}$$

命中率 与 Cache 的 容量 与 块长 有关

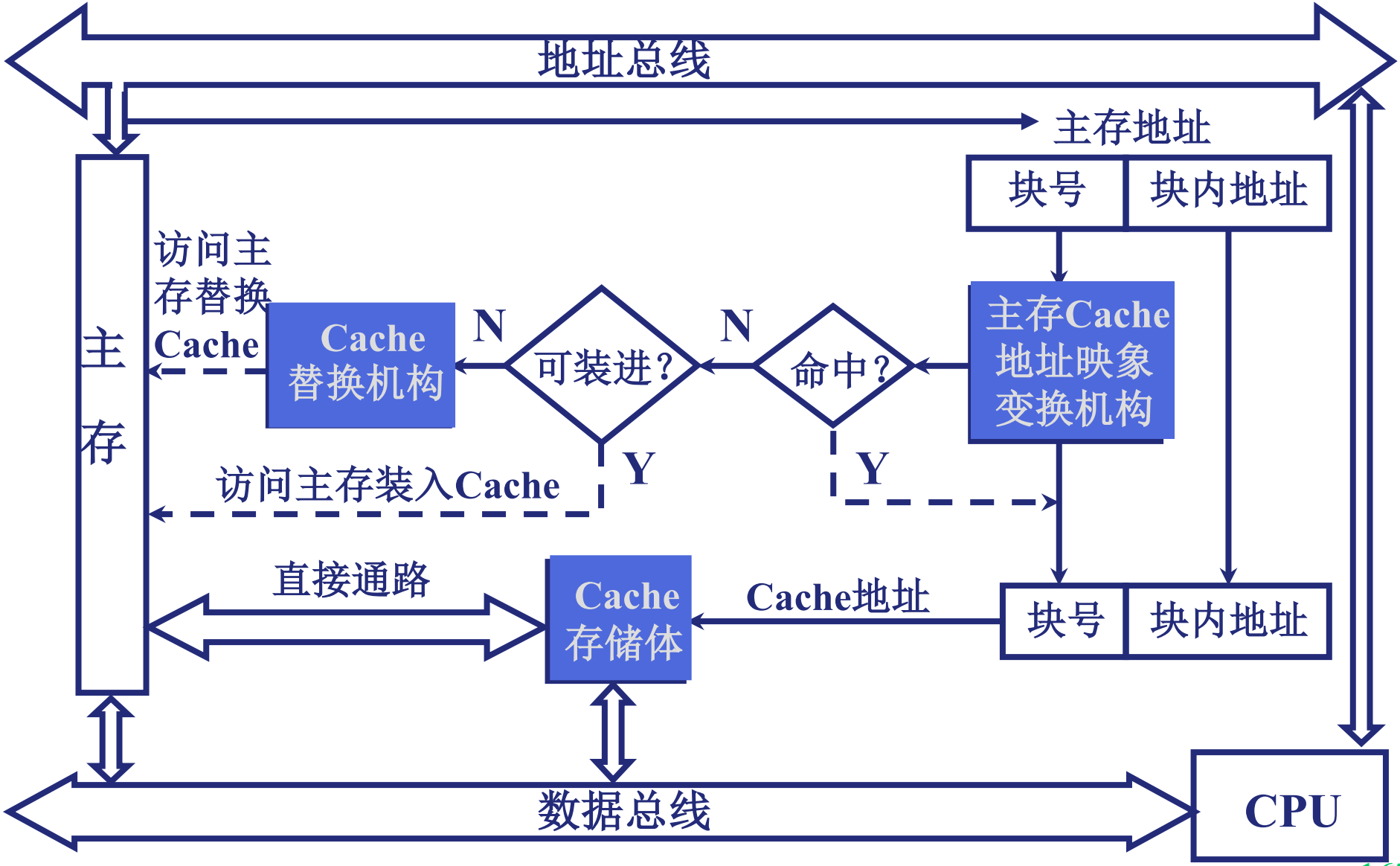
一般每块可取 4 至 8 个字　块长取一个存取周期内从主存调出的信息长度

CRAY_1　16体交叉　块长取 16 个存储字

IBM 370/168　4体交叉　块长取 4 个存储字

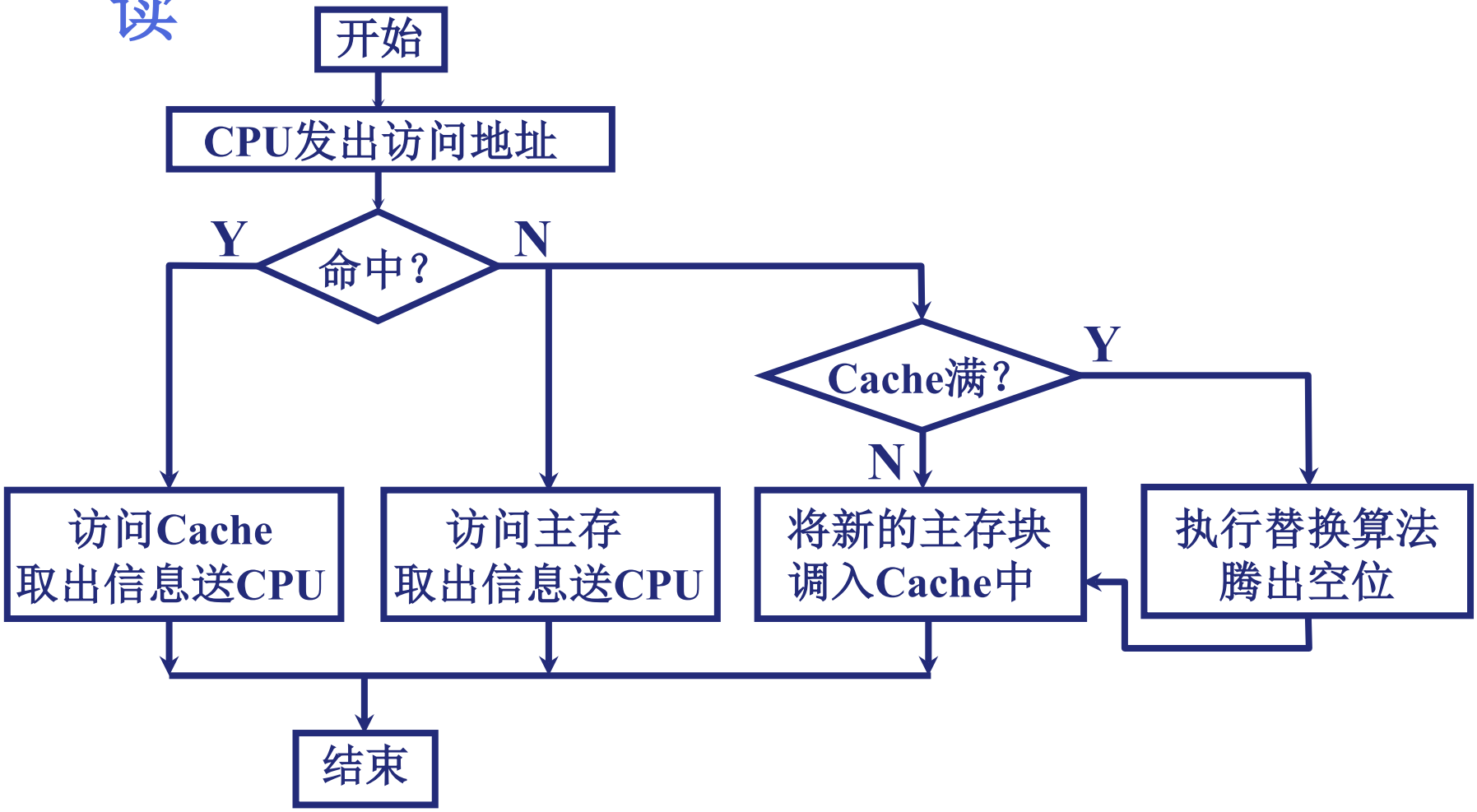
（64位×4 = 256位）

3. Cache 的基本结构



4. Cache 的 读写 操作

读



写　Cache 和主存的一致性

4. Cache 的写操作

- 对Cache块内写入的信息，必须与被映象的主存块内的信息完全一致。
  - 直写（write-through，或称store-through）：随时保证主存内容与Cache的数据始终一致。
    - 但有可能会增加访存次数，因每向Cache写入时，都需向主存写入。
  - 回写（write-back）又称标志交换式（Flag-Swap）：数据每次只是暂时写入Cache，并用标志将该块加以注明，直至该块从Cache替换出时，才写入主存。
    - 这种方法,其速度快，但因主存中的字块未经随时修改，可能失效。
- 信息只写入主存时，同时将相应的Cache块有效位置置“0”,表明此Cache块已失效，需要时从主存调入。还有一种可能，被修改的单元根本不在Cache内，因此写操作只对主存进行。

5. Cache 的改进

(1) 增加 Cache 的级数

片载（片内）Cache

片外 Cache

(2) 统一缓存和分开缓存

指令 Cache　　数据 Cache

与主存结构有关

与指令执行的控制方式有关　　是否流水

Pentium	8K 指令 Cache	8K 数据 Cache
PowerPC620	32K 指令 Cache	32K 数据 Cache

二、Cache 映射(Cache Mapping) 4.3

- 什么是Cache的映射功能?
  - 把访问的局部主存区域取到Cache中时，该放到Cache的何处?
  - Cache槽比主存块少，多个主存块映射到一个Cache槽中
- 地址映象与变换
  - 在主存的地址和Cache地址之间建立一种确定的逻辑关系，也就是根据主存的地址来构成Cache地址。这种地址间的逻辑关系称为映射

三、Cache 映射(Cache Mapping)

- 如何进行映射/映象
  - 把主存空间划分成大小相等的主存块（Block）
  - Cache中存放一个主存块的对应单位称为槽（Slot）或行（line）或块（Block）
  - 将主存块和Cache行按照以下三种方式进行映射
    - 直接(Direct)：每个主存块映射到Cache的固定行中
    - 全相联(Full Associate)：每个主存块映射到Cache的任意行中
    - 组相联(Set Associate)：每个主存块映射到Cache的固定组中的任意一行中

1. 直接映像（direct mapped）

- 直接映像：把主存的每一块映射到Cache中的一个固定的Cache行（槽） 的方式。
- 最简单的地址映像方式，地址变换速度快，但块冲突的概率较高，当程序往返访问两个相互冲突的块中的数据时，Cache的命中率将急剧下降。
- 映射公式：

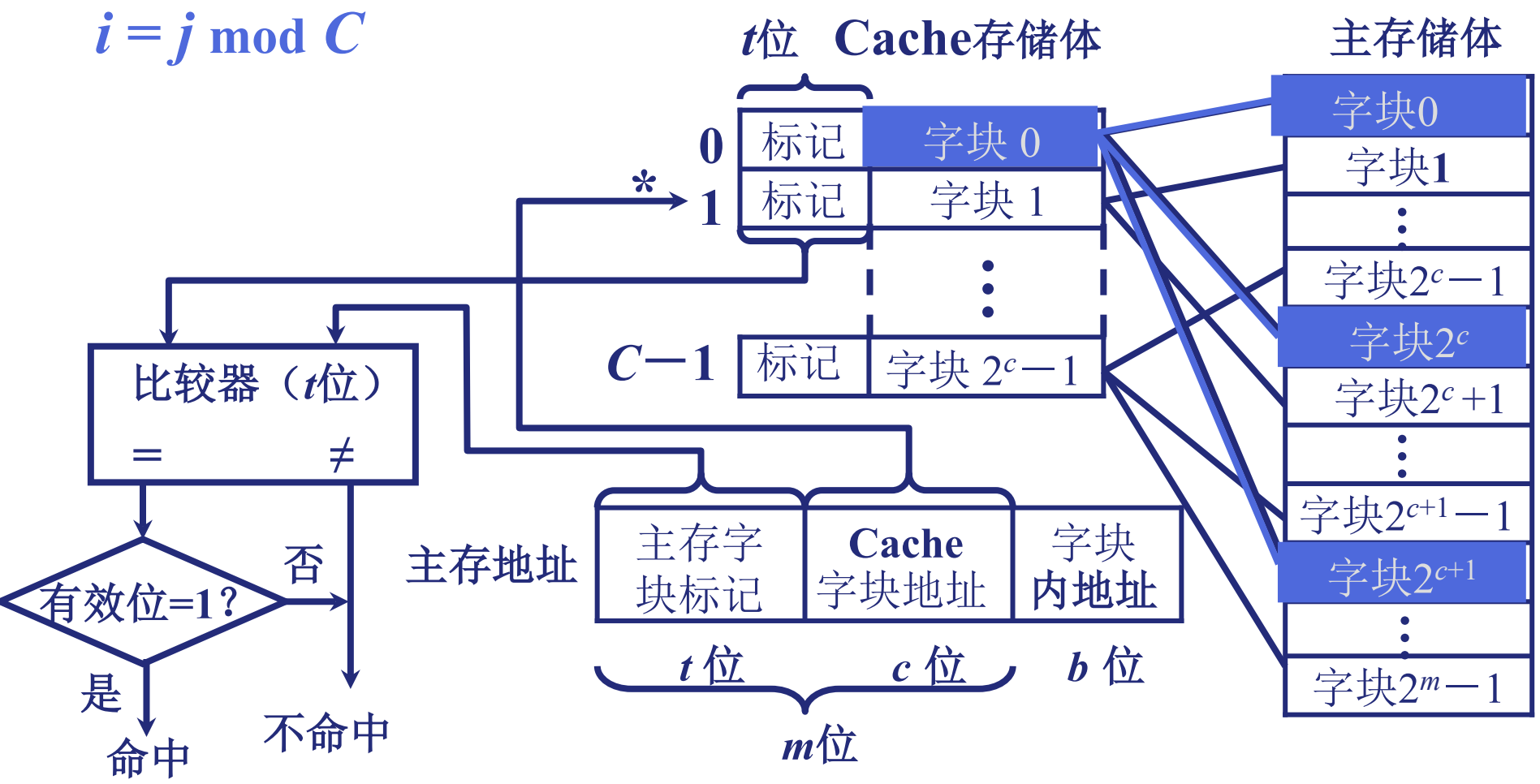
Cache行号=主存块号 mod Cache行数

举例：4=100 mod 16 （假定Cache共有16行）

(说明：主存第100块应映射到Cache的第4行中。)

块（行）都从0开始编号

1. 直接映像



每个缓存块 i 可以和 若干 个 主存块 对应

每个主存块 j 只能和 一 个 缓存块 对应

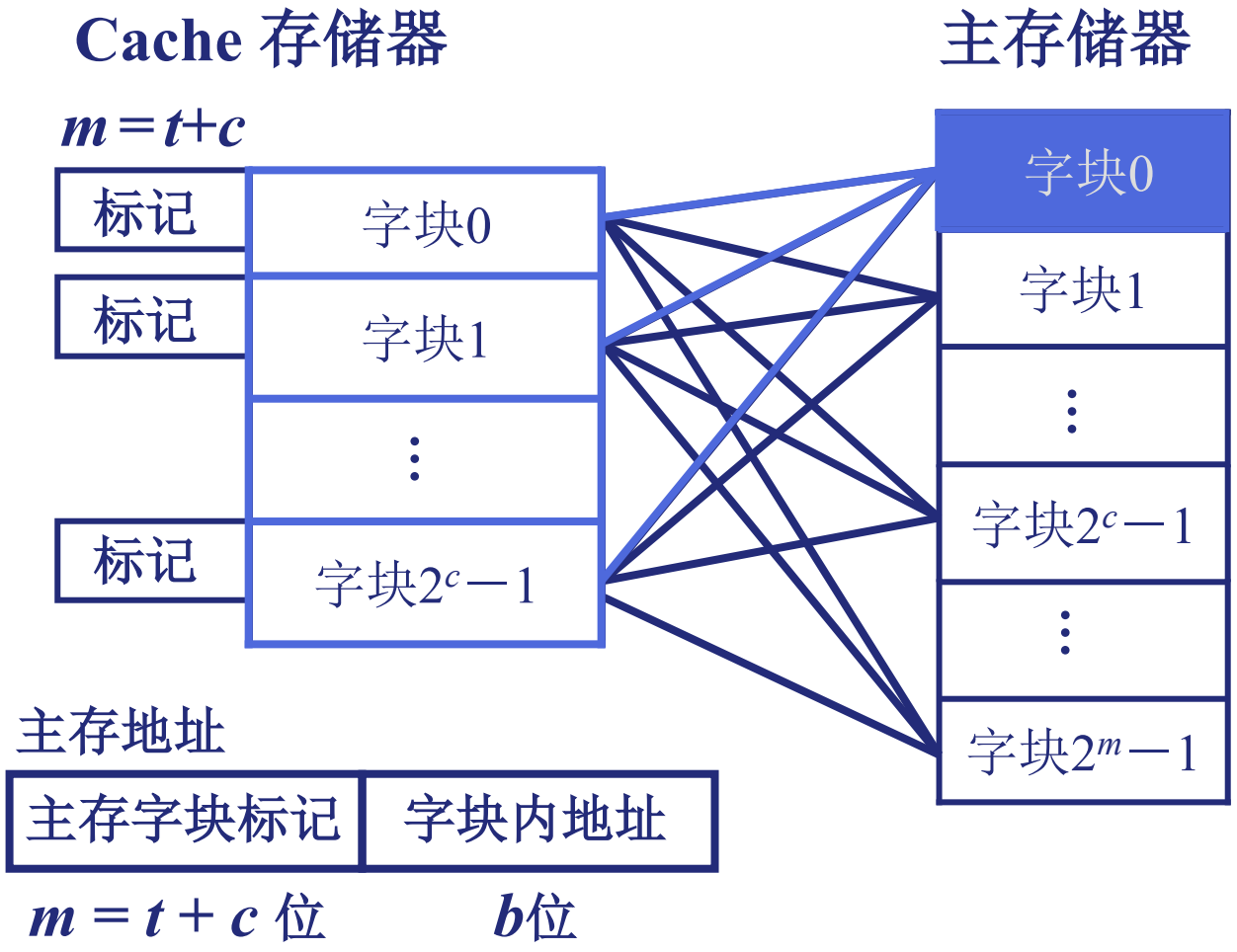
1. 直接映像

- 特点：
  - 容易实现，命中时间短
  - 无需考虑淘汰（替换）问题
  - 但不够灵活，Cache存储空间得不到充分利用，命中率低
  - 例如，需将主存第0块与第16块同时复制到Cache中时，由于它们都只能复制到Cache第0行，即使Cache其它行空闲，也有一个主存块不能写入Cache。这样就会产生频繁的 Cache装入。

2. 全相联映像（fully associative）

- 每个主存块都可映像到任何Cache块的地址映像方式。
- 在全相联映像方式下，主存中存储块的数据可调入Cache中的任意块。
- 命中率实现比较复杂。

2. 全相联映像



主存 中的 任一块 可以映像到 缓存 中的 任一块

3.组相联映射（Set Associative） 4.3

- 组相联映像指的是将存储空间分成若干组，主存块与Cache组之间是直接映像，而与组内各块之间则是全相联映像。
- 将Cache所有行分组，把主存块映射到Cache固定组的任一行中。也即：组间直接映像(模映射)、组内全相联映象（全映射）。映射关系为：

Cache组号=主存块号 mod Cache组数

举例：假定Cache划分为：8K字=8组×2行/组×512字/行

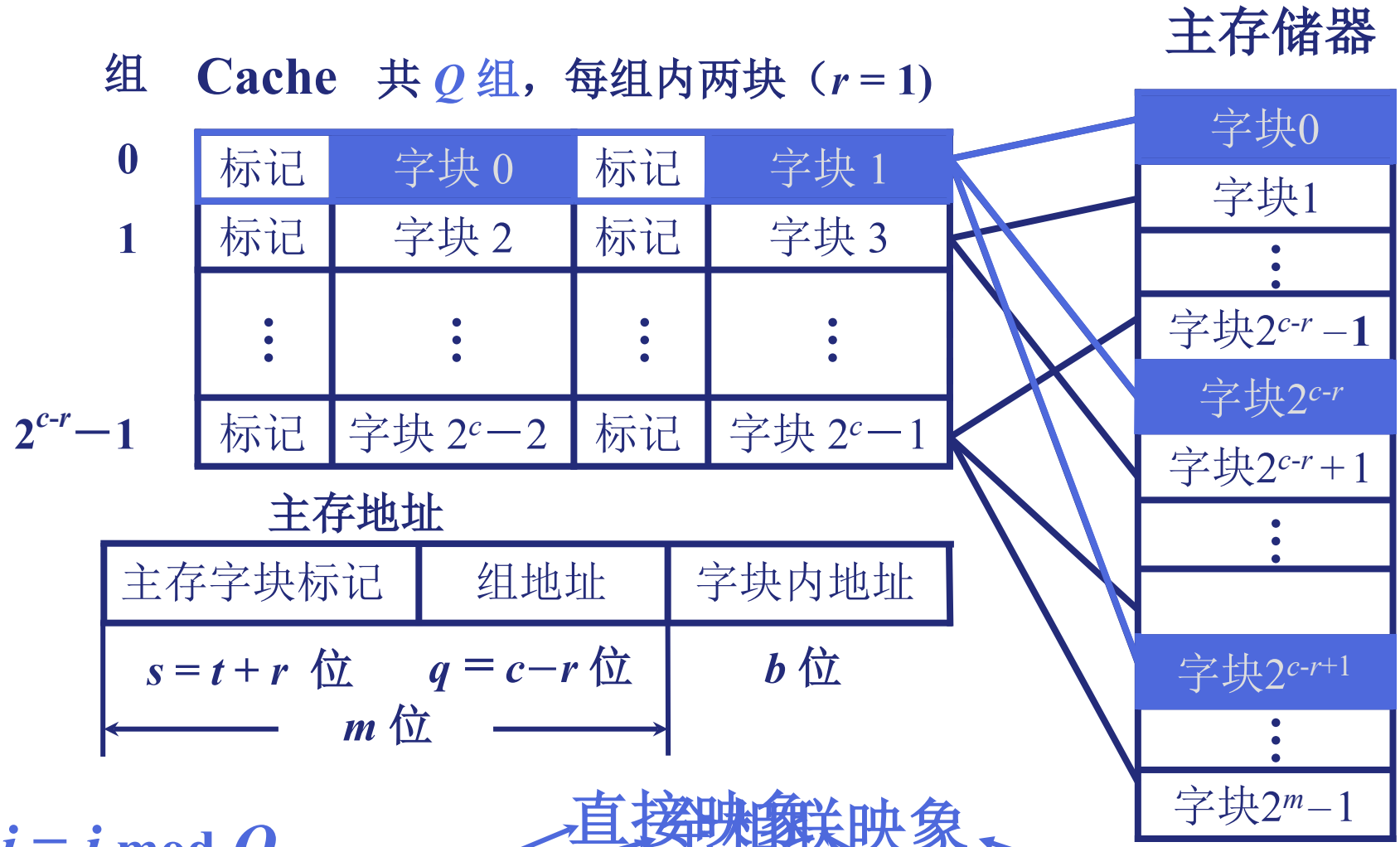
4=100 mod 8

(主存第100块应映射到Cache的第4组的任意行中。)

组相联映射（Set Associative）

- 特点：
  - 结合直接映射和全相联映射的优点。当Cache组数为1时，变为相联映射；当每组只有一个槽时，变为直接映射。
  - 每组2或4行（称为2-路或4-路组相联）较常用。通常每组4行以上很少用。在较大容量的L2 Cahce和L3 Cahce中使用4-路以上。

3. 组相联映像（set associative）



$i = j \bmod Q$

直接映象 全相联映象

某一主存块 j 按模 Q 映射到 缓存 的第 i 组中的 任一块

4. Cache中的位数（容量）

- 由于每个Cache的地址可能对应于存储器中不同的地址，因此需要在Cache中加标记(tag)，标记必须能判断Cache中的字是否为所请求的地址信息。
- 标记只包含存储器地址的高位部分（存储器地址的高位部分用作选定Cache地址）
- Cache中包含一位有效位(valid bit)用于说明Cache块是否含有有效地址
  - 有效位用于判断Cache块中是否有有效信息。例如当处理器启动时Cache是空的，此时标记字段中的值是没有意义的。即使执行了数条指令，Cache中的一些块仍为空，此时也需要用有效位说明这些单元的标记应被忽略。

Cache块内容

有效位	标记	数据

主存地址

直接映射：

标记	Cache行号	块内地址

全相联映射：

标记（主存块号）	块内地址

组相联映射：

标记	Cache组号	块内地址

一个Cache行的内容

有效位	标记	数据

4. Cache的容量

Cache块	有效位	标记	数据

- Cache不仅存储数据，而且存储标记，故Cache中所需总位数是Cache的大小和地址位数的函数。
- 例如： 假设一个32位字节的地址以及一个大小为2^n个字直接映像的Cache，每块有1个字（4个字节），则标记字段位数为32-n-2，其中n为Cache块地址，2位为块内地址。而块大小为1个字（32位）且地址大小也为32位，而这样一个Cache的位数为

$$2^n \times [32+(32-n-2)+1] = 2^n \times (63-n)$$

4. Cache的容量



- 例题 假设一个直接映像的Cache 有64KB数据，地址为32位，块大小为1个字（4个字节），那么该Cache的容量是多少位？

$2^{14} \times [32+(32-14-2)+1] = 2^{14} \times 49 = 784 \times 2^{10} = 784K$位

例. 设某机主存容量为16MB，Cache数据区的容量为16KB。每个字块有8个字，每个字32位。设计一个四路组相联映像（即Cache每组内共有4个字块）的Cache组织，要求：

（1）画出主存地址字段中各段的位数。

解：根据每个字块有8个字，每个字32位，得出主存地址字段中字块内地址字段为5位。

根据Cache容量为16KB=2^{14}B，字块大小为2^5B，得Cache共有2^9块，故c=9。根据四路组相联映像2^r=4，得r=2，则q=c-r=7。

根据主存容量为16MB=2^{24}B，得出主存地址字段中主存字块标记位数为24-7-5=12。

主存地址字段格式如下图所示：主存字块标记组地址字块内地址

12位7位5位

主存字块标记	组地址	字块内地址
12位	7位	5位

设某机主存容量为16MB，Cache数据区的容量为16KB。每个字块有8个字，每个字32位。设计一个四路组相联映像（即Cache每组内共有4个字块）的Cache组织，要求：

（2）设Cache初态为空，CPU依次从主存第0、1、2、…、99号单元读出100个字（主存一次读出一个字），并重复此次序读8次，问命中率是多少？

解：由于每个字块中有8个字，而且初态Cache为空，因此CPU读第0单元时，未命中，必须访问主存，同时将该字所在的主存块调入Cache第0组中的任一块内，接着CPU读1～7号单元时，均命中。同理CPU读第8、16、…、96号单元时均未命中。可见CPU在连续读100个字中共有13次未命中，而后7次循环读100个字全部命中，命中率为：

$$\frac{100\times 8-13}{100\times 8}\times 100\%=98.375\%$$

设某机主存容量为16MB，Cache数据区的容量为16KB。每个字块有8个字，每个字32位。设计一个四路组相联映像（即Cache每组内共有4个字块）的Cache组织，要求：

（3）若Cache的速度是主存容量的6倍，试问有Cache和无Cache相比，速度提高多少倍？

解：根据题意，设主存存取周期为6t，Cache的存取周期为t，没有Cache的访问时间为6t×800，有Cache的访问时间为，则有Cache 和没有Cache相比，速度提高倍数为：

$$\frac{6t \times 800}{t(800-13)+6t \times 13}-1 \approx 4.5$$

命中率对平均访问时间的影响

- 设H是命中率，则平均访问时间$T = HT_C + (1 - H)(T_C + T_M)$
  $= T_C + (1 - H)T_M$

- 例1. 若H=0.85, Tc=1ns, T_M=20ns，则T为多少?
  答： T = 4ns

- 例2. 若命中率H提高到0.95，则结果又如何?
  答： T = 2ns

- 例3. 若命中率为0.99呢?
  答： T = 1.2ns

访存速度与命中率的关系非常大!

The Need to Replace! (何时需要替换？)

- Direct Mapped Cache:
  - 映射唯一，毫无选择，无需考虑替换
- N-way Set Associative Cache:
  - 每个主存数据有N个Cache行可选择，需考虑替换
- Fully Associative Cache:
  - 每个主存数据可存放到Cache任意行中，需考虑替换

结论：若Cache miss in a N-way Set Associative or Fully Associative Cache，则可能需要替换。其过程为：

- 从主存取出一个新块
- 选择一个有映射关系的空Cache行
- 对应的Cache行已被占满而需要调入新的主存块时，必须考虑从cache行中调出一个主存块

三、替换算法

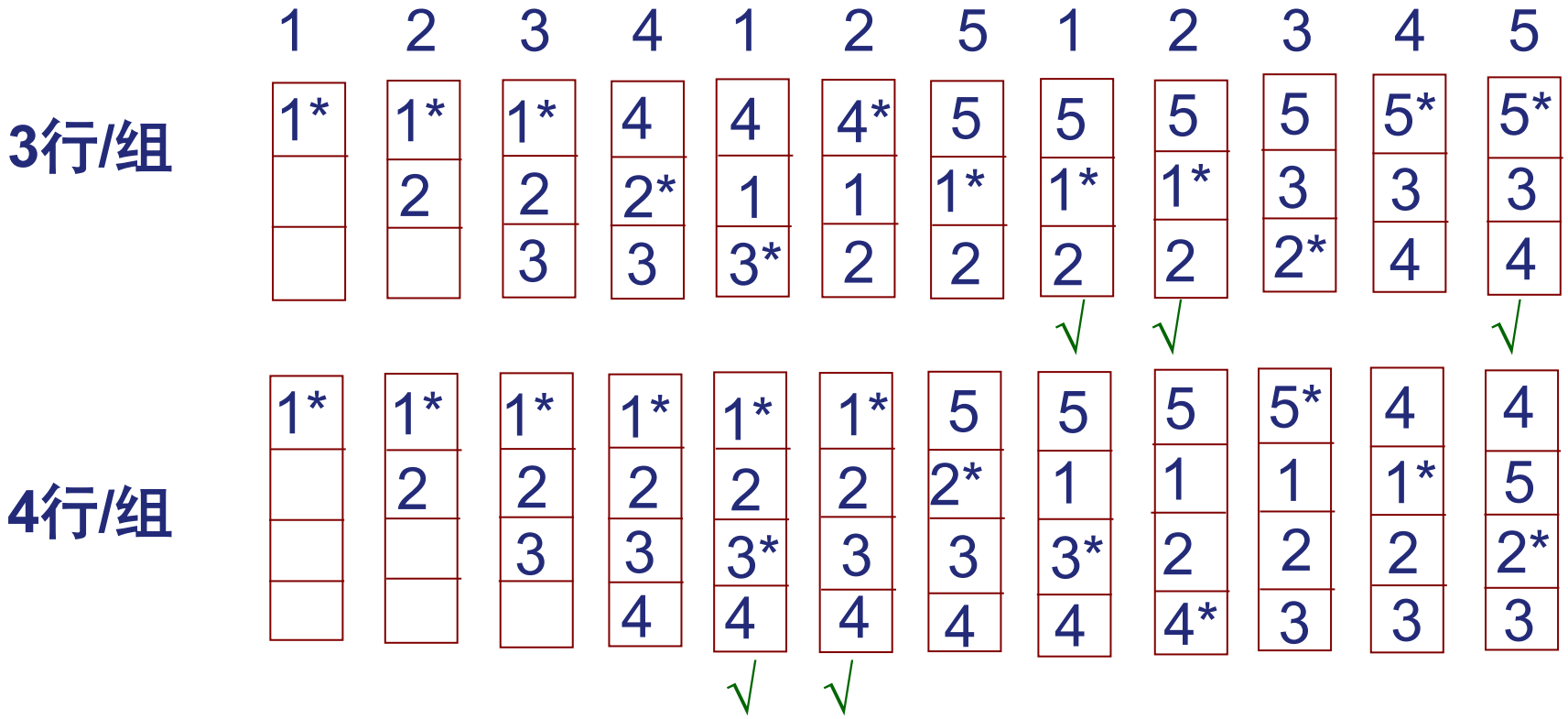
- 选择替换算法的依据是存储器总体的性能，主要是Cache的访问命中率。
- 常用替换算法有：
  - 先进先出FIFO （first-in-first-out）
  - 最近最少使用LRU （ least-recently used）
  - 最不经常使用LFU （ least-frequently used）
  - 随机替换算法（Random）

  等等

替换算法-先进先出（FIFO）

- 总是把最先进入的那一块淘汰掉。

例：假定主存中的5块{1,2,3,4,5}同时映射到Cache同一组中，对于同一地址流，考察3行/组、 4行/组的情况。



由此可见，FIFO不是一种堆栈算法，即命中率并不随组的增大而提高。

替换算法-最近最少用(LRU)

- 总是把最近最少用的那一块淘汰掉。

例：假定主存中的5块{1,2,3,4,5}同时映射到Cache同一组中，对于同一地址流，考察3行/组、 4行/组、 5行/组的情况。

	1	2	3	4	1	2	5	1	2	3	4	5
	1	2	3	4	1	2	5	1	2	3	4	5
		1	2	3	4	1	2	5	1	2	3	4
			1	2	3	4	1	2	5	1	2	3
				1	2	3	4	4	4	5	1	2
							3	3	3	4	5	1
3行/组								√	√			
4行/组					√	√		√	√			
5行/组					√	√		√	√	√	√	√

（自学）替换算法-最近最少用

- 是一种堆栈算法，它的命中率随组的增大而提高。
- 当分块局部化范围(即：某段时间集中访问的存储区)超过了Cache存储容量时，命中率变得很低。极端情况下，假设地址流是1,2,3,4,1 2,3,4,1,……，而Cache每组只有3行，那么，不管是FIFO，还是LRU算法，其命中率都为0。这种现象称为颠簸(Thrashing / PingPong)。
- 该算法具体实现时，并不是通过移动块来实现的，而是通过给每个cache行设定一个计数器，根据计数值来记录这些主存块的使用情况。这个计数值称为**LRU位**。

具体实现

（自学）替换算法-最近最少用

- 计数器变化规则：
  - 每组4行时，计数器有2位。计数值越小则说明越被常用。
  - 命中时，被访问行的计数器置0，比其低的计数器加1，其余不变。
  - 未命中且该组未满时，新行计数器置为0，其余全加1。
  - 未命中且该组已满时，计数值为3的那一行中的主存块被淘汰，新行计数器置为0，其余加1。

1	2	3	4	1	2	5	1	2	3	4	5
0 1	1 1	2 1	3 1	0 1	1 1	2 1	0 1	1 1	2 1	3 1	0 5
	0 2	1 2	2 2	3 2	0 2	1 2	2 2	0 2	1 2	2 2	3 4
		0 3	1 3	2 3	3 3	0 5	1 5	2 5	3 5	0 4	2 3
			0 4	1 4	2 4	3 4	3 4	3 4	0 3	1 3	1 2

（自学）替换算法-其他算法

- 最不经常用（LFU）算法：

  替换掉Cache中引用次数最少的块。LFU也用与每个行相关的计数器来实现。

  （这种算法与LRU有点类似，但不完全相同。）

- 随机算法：

  随机地从候选的cache行中选取一个淘汰，与使用情况无关。

  （模拟试验表明，随机替换算法在性能上只稍逊于基于使用情况的算法。而且代价低！）

程序的局部性原理举例1

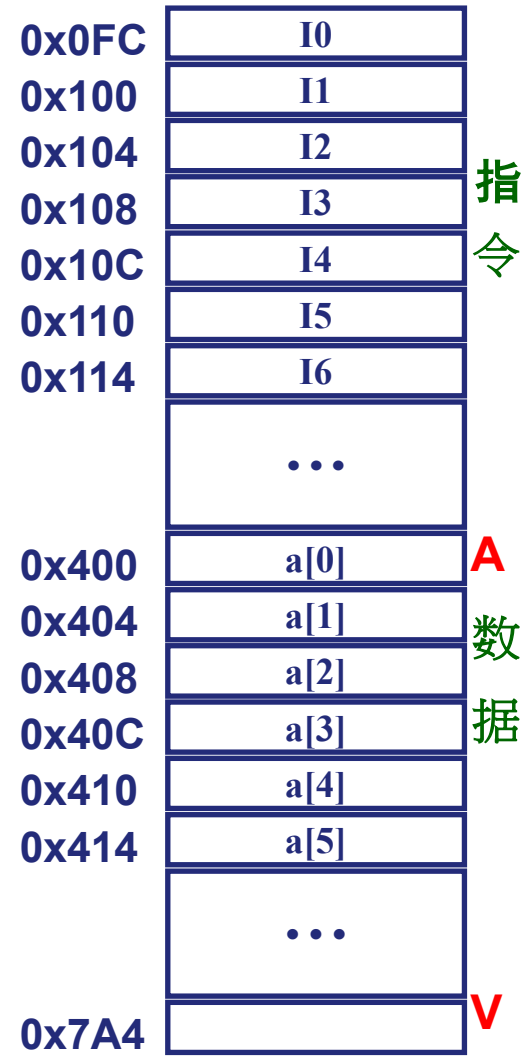
高级语言源程序

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
```

对应的汇编语言程序

```
I0:          sum  <-- 0                    *v = sum;
I1:          ap  <-- A   A是数组a的起始地址
I2:          i   <-- 0
I3:          if (i >= n) goto done
I4:   loop:  t   <-- (ap) 数组元素a[i]的值
I5:          sum <-- sum + t   累计在sum中
I6:          ap  <-- ap + 4    计算下个数组元素地址
I7:          i   <-- i + 1
I8:          if (i < n) goto loop
I9:   done:  V  <-- sum    累计结果保存至地址v
```

主存的布局:



每条指令4个字节；每个数组元素4字节

指令和数组元素在内存中均连续存放

sum, ap ,i, t 均为通用寄存器；A，V为内存地址

程序的局部性原理举例1

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
*v = sum;
```

问题：指令和数据的时间局部性和空间局部性各自体现在哪里?

指令：　0x0FC（I0）

…

→0x108（I3）

→0x10C（I4）

…

→0x11C（I8）

→0x120（I9）

循环n次

若n足够大，则在一段时间内一直在局部区域内执行指令，故循环内指令的时间局部性好；

数据：只有数组在主存中：

0x400→0x404→0x408

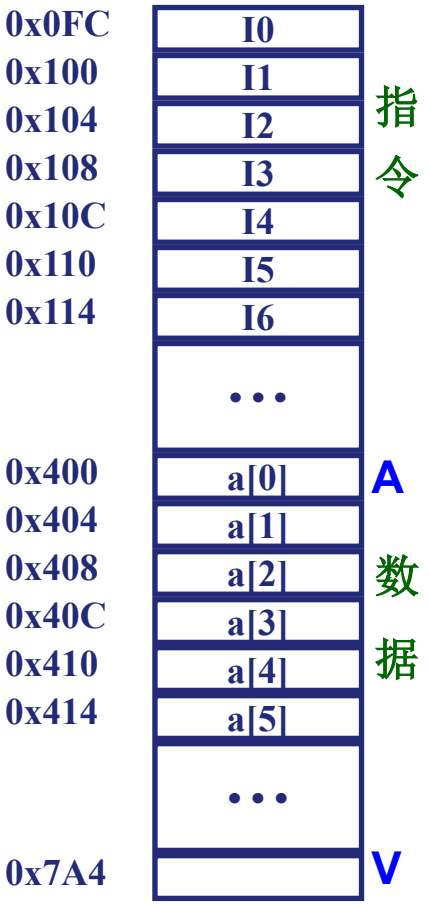
→0x40C→……→0x7A4

按顺序执行，故程序空间局部性好！

数组元素按顺序存放，按顺序访问，故空间局部性好；

每个数组元素都只被访问1次，故没有时间局部性。

主存的布局：



程序的局部性原理举例2

以下哪个对数组a引用的空间局部性更好？时间局部性呢？变量sum的空间局部性和时间局部性如何？对于指令来说，for循环体的空间局部性和时间局部性如何？

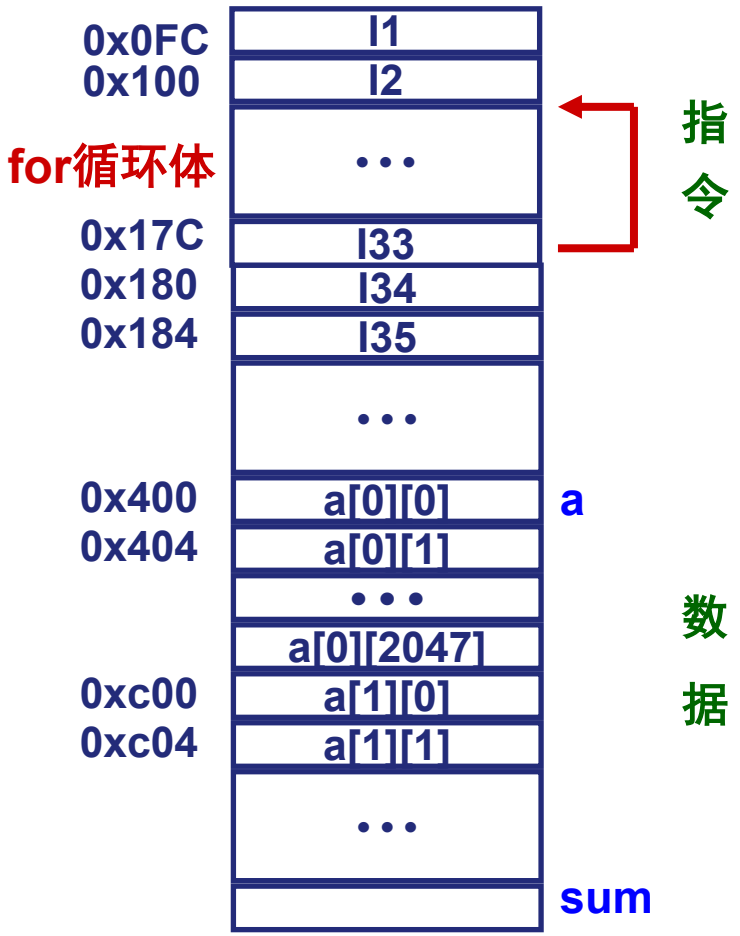
程序段A:

```
int sumarrayrows(int a[M][N])
{
   int i, j, sum=0;
       for  (i=0; i<M, i++)
              for (j=0; j<N, j++)  sum+=a[i][j];
       return sum;
}
```

程序段B:

```
int sumarraycols(int a[M][N])
{
    int i, j, sum=0;
        for  (j=0; j<N, j++)
               for (i=0; i<M, i++)  sum+=a[i][j];
         return sum;
}
```

M=N=2048时主存的布局:



数组在存储器中按行优先顺序存放

程序的局部性原理举例2

程序段A的时间局部性和空间局部性分析

(1) **数组a**：访问顺序为a[0][0], a[0][1] ,……, a[0][2047]; a[1][0], a[1][1],…… ,a[1][2047]; ……，与存放顺序一致，故空间局部性好！

因为每个a[i][j]只被访问一次，故时间局部性差！

(2) **变量sum**：单个变量不考虑空间局部性；每次循环都要访问sum，所以其时间局部性较好！

(3) **for循环体**：循环体内指令按序连续存放，所以空间局部性好！

循环体被连续重复执行2048x2048次，所以时间局部性好！



实际上 优化的编译器使循环中的sum分配在寄存器中，最后才写回存储器！

程序的局部性原理举例2

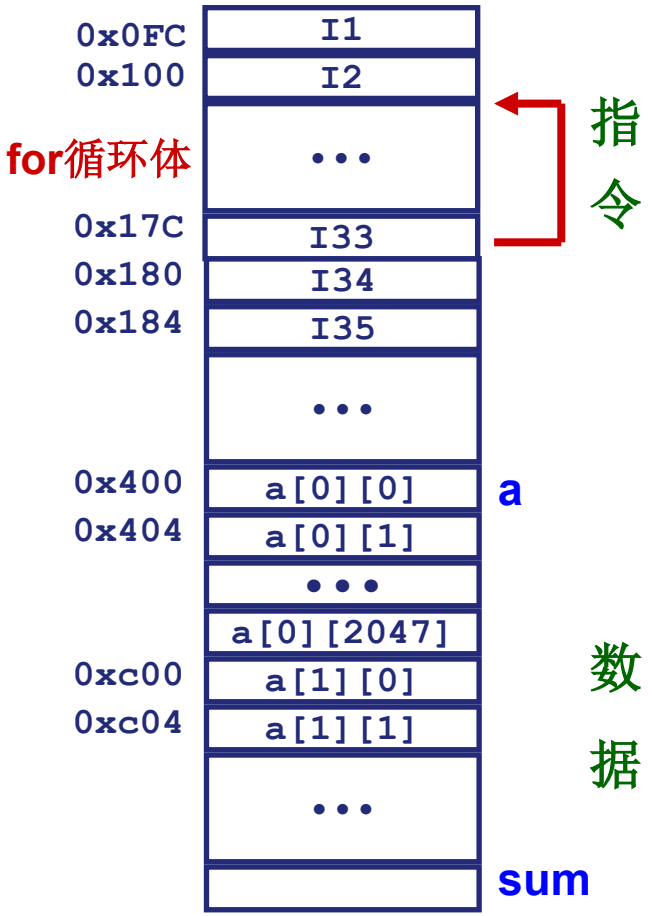
程序段B的时间局部性和空间局部性分析

(1) 数组a：访问顺序为a[0][0], a[1][0] ,……, a[2047][0];
a[0][1],a[1][1],…… ,a[2047][1];……，与存放顺序不一致，每次跳过2048个单元，若交换单位小于2KB，则没有空间局部性！

(时间局部性差，同程序A)

(2) 变量sum： (同程序A)

(3) for循环体： (同程序A)



实际运行结果(2GHz Intel Pentium 4):

程序A：59,393,288 时钟周期

程序B：1,277,877,876 时钟周期

程序A比程序B快 21.5 倍!!

Cache和程序性能举例

- 举例：某32位机器主存地址空间大小为256 MB，按字节编址。指令cache和数据cache均有8行，主存块为64B，数据cache采用直接映射。假定编译时i, j, sum均分配在寄存器中，数组a按行优先方式存放，其首址为320。

程序 A：

```
int a[256][256];
……
int sum_array1 ( )
{
        int i, j, sum = 0;
        for ( i = 0; i < 256; i++)
                for (j = 0; j < 256; j++)
                        sum += a[i] [j];
        return sum;
}
```

程序 B：

```
int a[256][256];
……
int sum_array2 ( )
{
        int i, j, sum = 0;
        for ( j = 0; j < 256; j++)
                for ( i = 0; i < 256; i++)
                        sum += a[i] [j];
        return sum;
 }
```

（1）不考虑用于一致性和替换的控制位，数据cache的总容量为多少？

（2）a[0][31]和a[1][1]各自所在主存块对应的cache行号分别是多少？

（3）程序A和B的数据访问命中率各是多少？哪个程序的执行时间更短？

Cache和程序性能举例

- 举例：某32位机器主存地址空间大小为256 MB，按字节编址。指令cache和数据cache均有8行，主存块为64B，数据cache采用直接映射。假定编译时i, j, sum均分配在寄存器中，数组a按行优先方式存放，其首址为320。

(1) 主存地址空间大小为256MB，因而主存地址为28位，其中6位为块内地址，3位为cache行号（行索引），标志信息有28-6-3=19位。在不考虑用于cache一致性维护和替换算法的控制位的情况下，数据cache的总容量为：
8×(19+1+64×8)=4256位=532字节 。

(2) a[0][31]的地址为320+4×31=444，[444/64]=6（取整），因此a[0][31]对应的主存块号为6。6 mod 8=6，对应cache行号为6。

或: 444=0000 0000 0000 0000 000 110 111100B，中间3位110为行号（行索引），因此，对应的cache行号为6。 a[1][1]对应的cache行号为：
[(320+4×(1×256+1))/64] mod 8=5。

(3) A中数组访问顺序与存放顺序相同，共访问64K次，占4K个主存块；首地址位于一个主存块开始，故每个主存块总是第一个元素缺失，其他都命中，共缺失4K次，命中率为1-4K/64K=93.75%。

方法二：每个主存块的命中情况一样。对于一个主存块，包含16个元素，需访存16次，其中第一次不命中，因而命中率为15/16=93.75%。

B中访问顺序与存放顺序不同，依次访问的元素分布在相隔256×4=1024的单元处，它们都不在同一个主存块中,cache共8行，一次内循环访问16块，故再次访问同一块时，已被调出cache，因而每次都缺失，命中率为0。

4.4 辅助存储器

一、概述

1. 特点：不直接与 CPU 交换信息

2. 磁表面存储器的技术指标

(1) 记录密度

道密度 D_t：沿磁盘半径方向长度上的磁道数

道密度＝磁道数/存储区域的长度

单位：道/英寸（TPI）；道/毫米（TPM）

位密度 D_b：位密度是磁道单位长度上可以记录的二进制代码数位。

位密度＝磁道容量/内圈的周长

单位：位/英寸（BPI）；位/毫米（BPM）

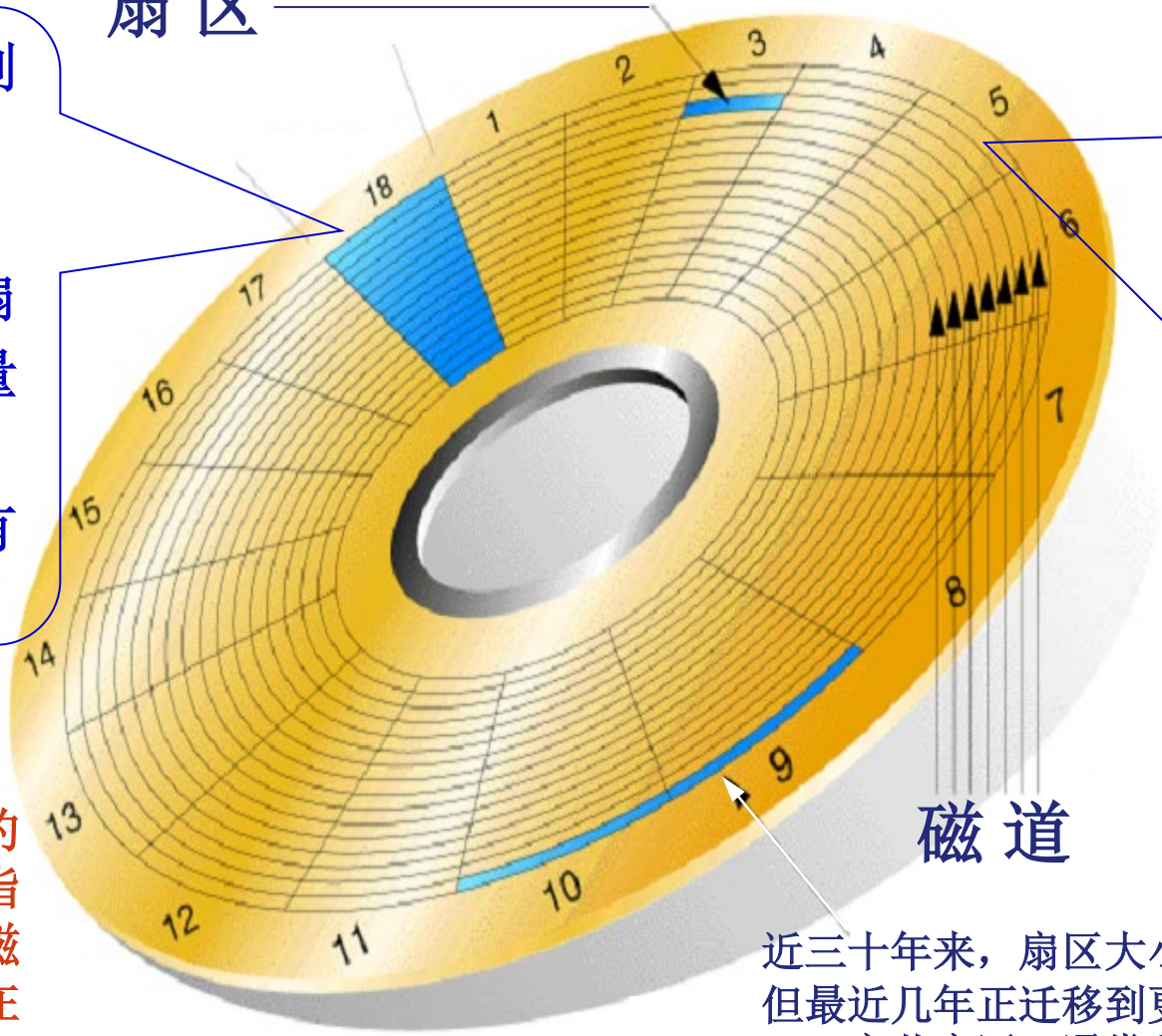
几个概念

- 记录面：磁盘片表面称为记录面。
- 磁道：记录面上一系列同心圆称为磁道。它的编址是由外向内依次编号，最外一个同心圆叫做零磁道。
- 柱面：所有记录块上半径相等的磁道的集合称为柱面，一个磁盘组的柱面数等于其中一个记录面上的磁道数。
- 扇区：将每一个记录面分成若干个区域，每一个区域称为一个扇区。
- 记录块：将每一个磁道分成若干个段，每段称为一个记录块或扇段。

磁盘的磁道和扇区

每个磁道被划分为若干段（段又叫扇区），每个扇区的存储容量为512字节。每个扇区都有一个编号

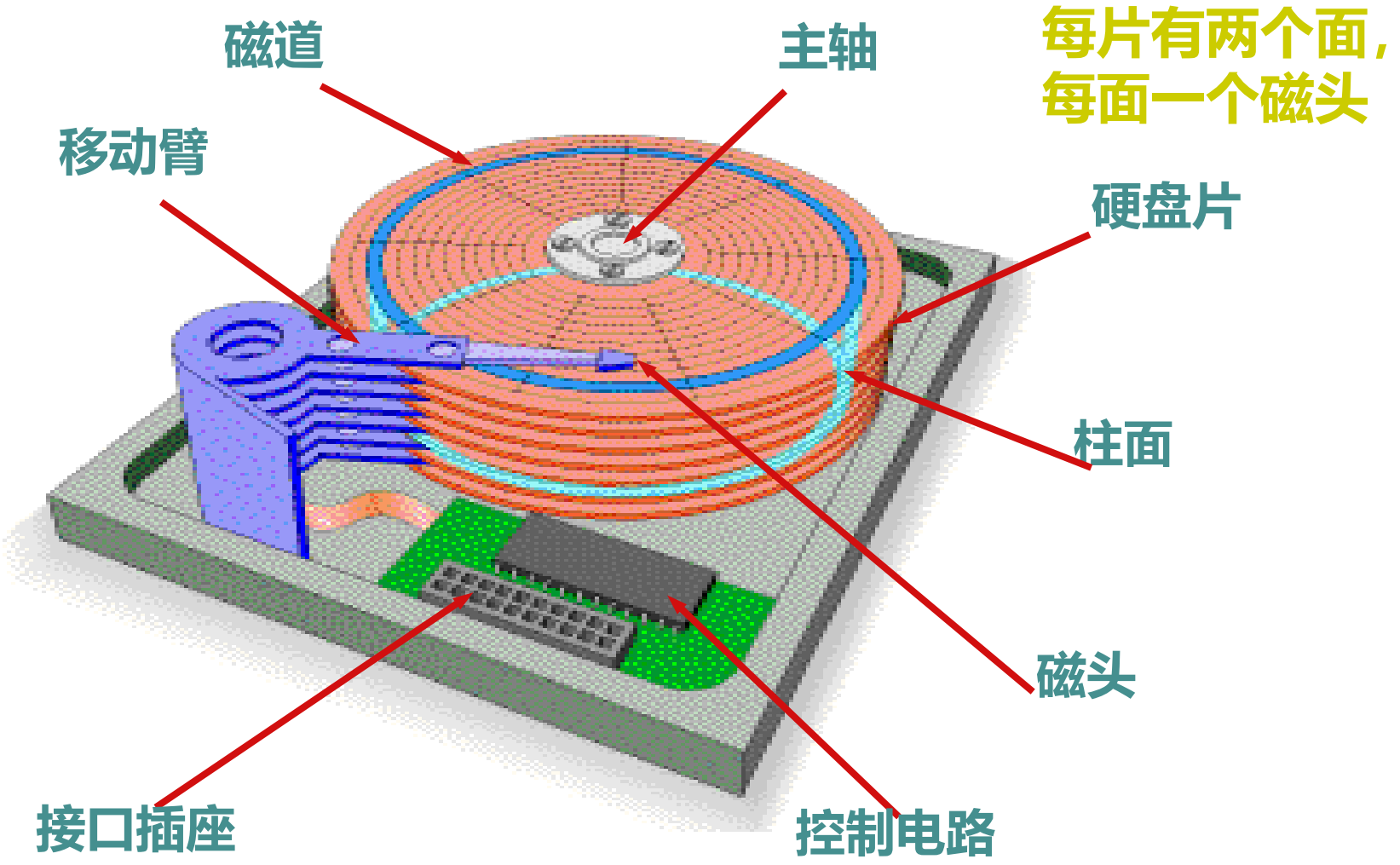
磁盘表面被分为许多同心圆，每个同心圆称为一个磁道。每个磁道都有一个编号，最外面的是0磁道



注：所谓磁盘的格式化操作，指在盘面上划分磁道和扇区，并在扇区中填写扇区号等信息的过程

近三十年来，扇区大小一直是512字节。但最近几年正迁移到更大、更高效的4096字节扇区，通常称为4K扇区。国际硬盘设备与材料协会（IDEMA）将之称为高级格式化。

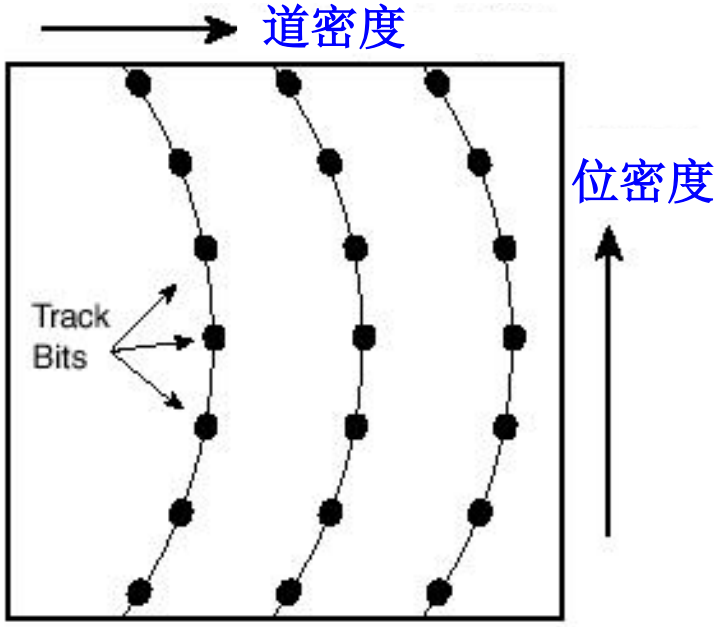
磁盘驱动器



磁道号就是柱面号、磁头号就是盘面号

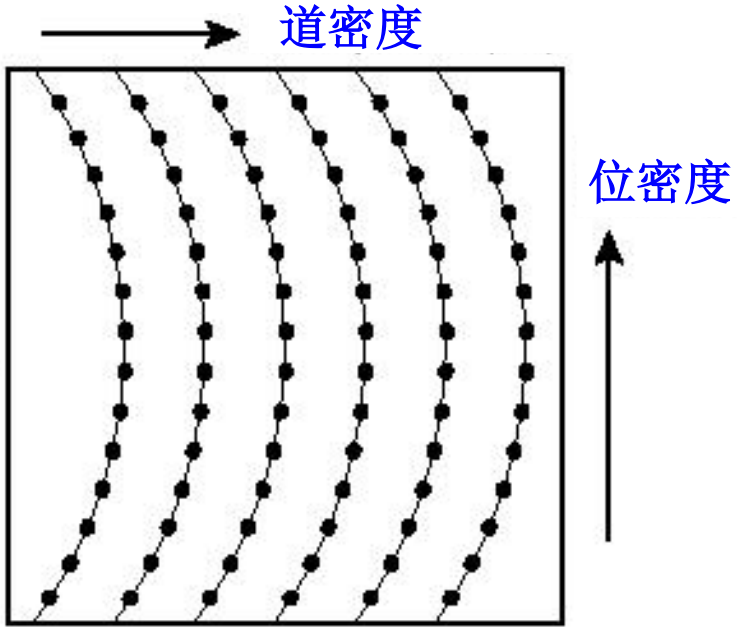
如何增大磁盘片的容量？

- 提高盘片上的信息记录密度！
  - 增加磁道数目——提高磁道密度
  - 增加扇区数目——提高位密度，并采用可变扇区数



低密度存储示意图

早期的磁盘所有磁道上的扇区数相同，所以位数相同，内道上的位密度比外道位密度高



高密度存储示意图

现代磁盘磁道上的位密度相同，所以，外道上的扇区数比内道上扇区数多，使整个磁盘的容量提高

磁盘磁道的格式

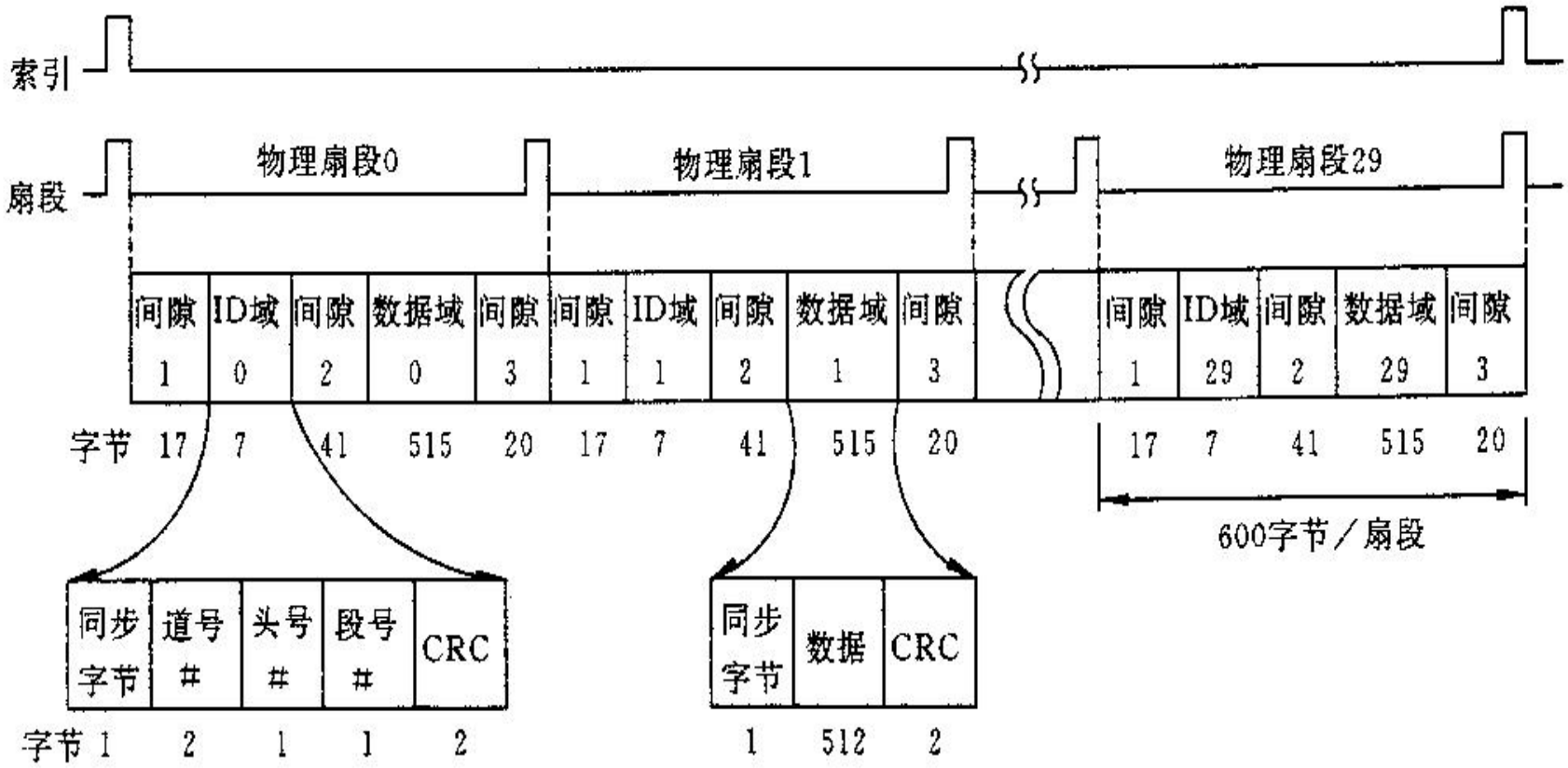


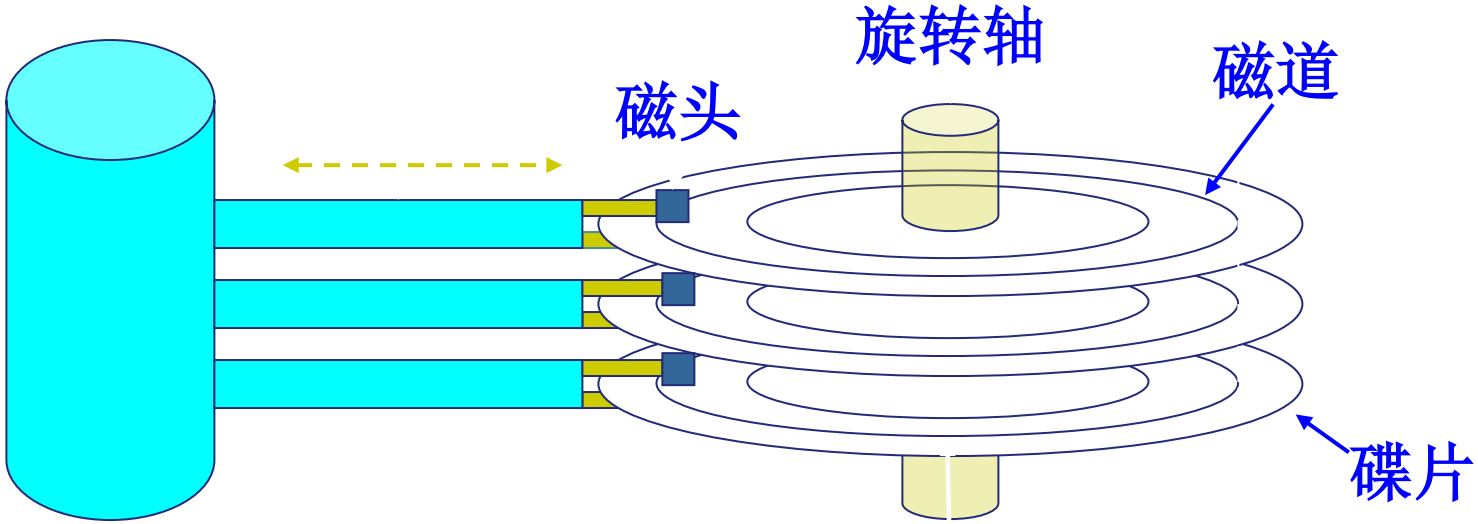
图 5.2 温彻斯特磁盘磁道格式(Seagate ST506)

在此例中，每个磁道包含30个固定长度的扇段，每个扇段有600个字节(17+7+41+515+20=600)。

（2）存储容量：磁盘存储器可以存储的总字节数

- 格式化容量：是可按照某个特定记录格式利用的磁化单元数，也是用户真正可以使用的容量
  - 格式化容量＝记录面数×每面的磁道数×扇区数×记录块的字节数，
  - 单位：KB或MB。
- 非格式化容量：是磁记录表面可以利用的磁化单元总数。
  - 非格式化容量＝记录面数×每面的磁道数×磁道容量，
  - 单位:KB或MB。
- 关系：格式化容量一般为非格式化60%～70%

平均存取时间



硬盘的操作流程如下：

所有磁头同步寻道（由柱面号控制）→ 选择磁头（由磁头号控制）→

被选中的磁头等待扇区到达磁头下方（由扇区号控制）→ 读写该扇区中的数据

2. 磁表面存储器的技术指标

（3）平均寻址时间：从读写命令发出后，磁头从某一起始位置出发移动到新的记录位置，到开始从盘片表面读出或写入信息所需要的时间。

- 寻道时间：将磁头定位到所要求的磁道上所需要的时间
- 等待时间：等待磁道上需要访问的信息到达磁头下的时间
- 平均寻址时间＝平均寻道时间＋平均等待时间。
  - 平均寻道时间＝（最大寻道时间＋最小寻道时间）/2；
  - 平均等待时间＝磁盘旋转一周所需时间的一半，即1/2×（1/转速）。

- 磁盘上的信息以扇区为单位进行读写，平均存取时间为：

  T = 寻道时间 + 旋转等待时间 + 数据传输时间（忽略不计）

  - 寻道时间——磁头寻找到指定磁道所需时间(大约5ms)
  - 旋转等待时间——指定扇区旋转到磁头下方所需要的时间(大约4～6ms) (转速： 4200 / 5400 / 7200 / 10000rpm)
  - 数据传输时间——(大约0.01ms / 扇区)

2. 磁表面存储器的技术指标

（4）数据传输率：磁盘存储器在单位时间能向主机传送的字节数。

- $D_r = D_b \times V$
- 如果磁盘的旋转速度为每秒n转，每条磁道的容量为N个字节，则数据传输率

$$Dr = nN（B/s）$$

(5) 误码率:出错信息位数与读出信息的总位数之比

- CRC循环冗余校验码

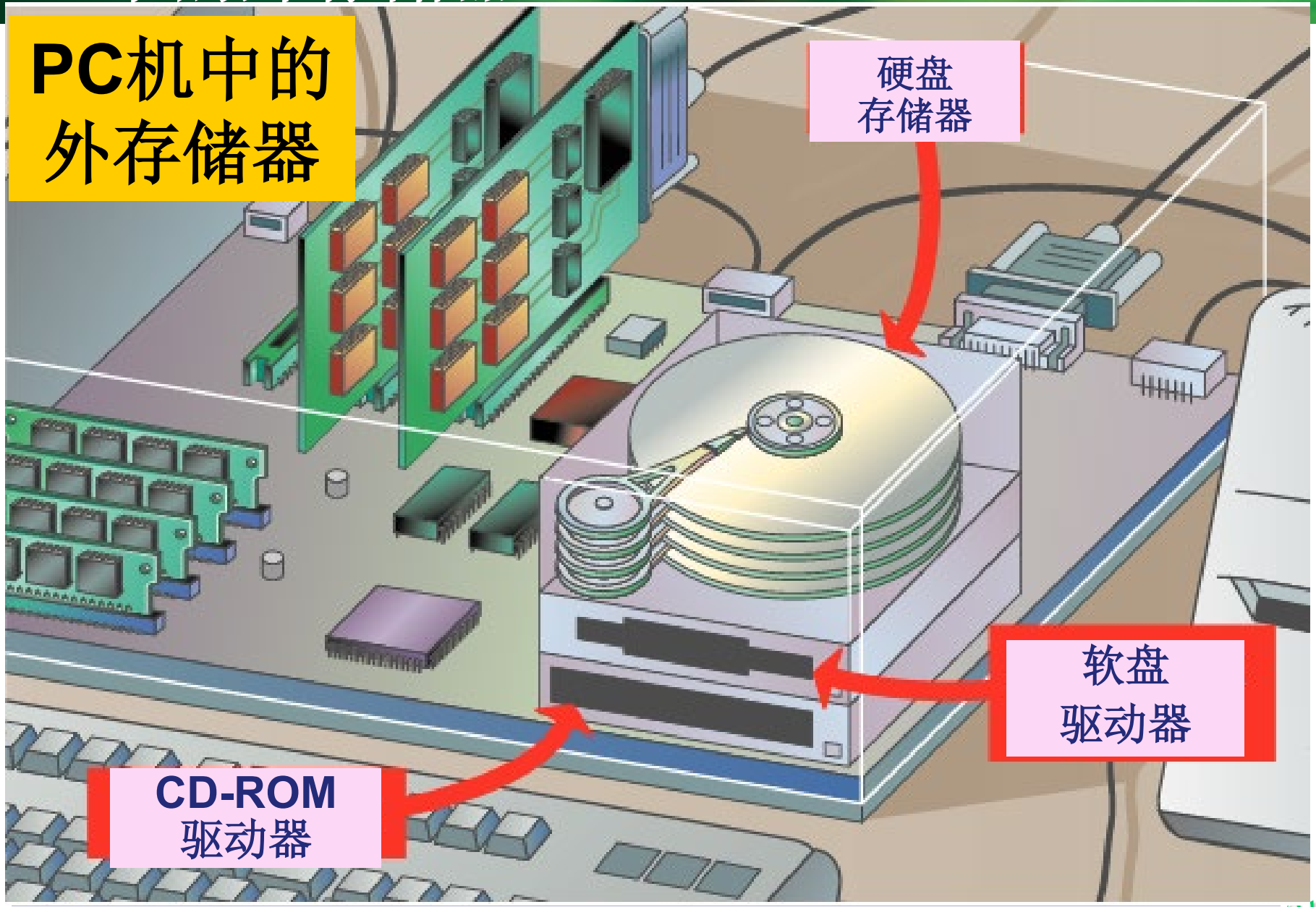
3、记录的地址格式

- 磁盘地址格式：对于活动头磁盘组，磁盘地址由记录面号（也称为磁头号）、磁道号和扇区号组成。一台主机如果配有几台磁盘机，则还要给它们编号，因此，磁盘地址格式为：

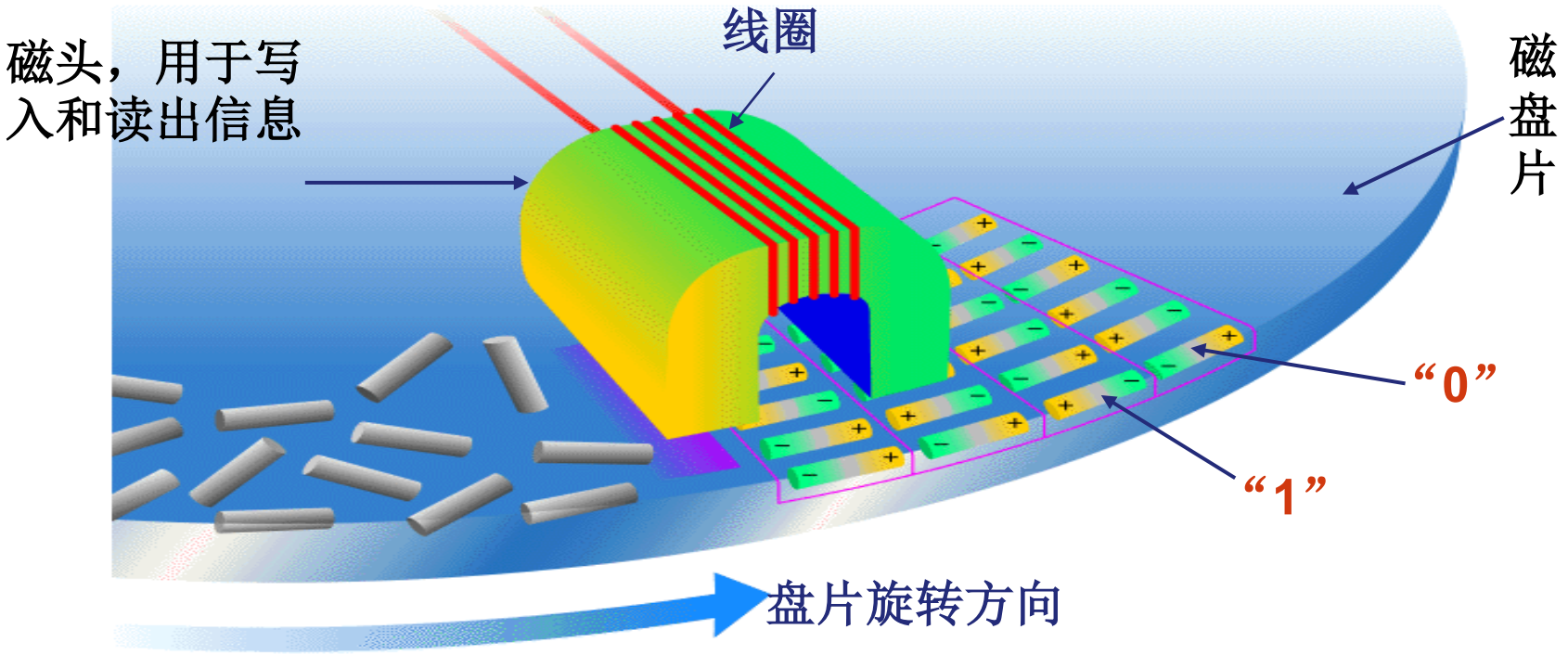
驱动器号	磁道号（柱面号）	记录面号（磁头号）	扇区号

- 注意磁道号（柱面号）与记录面号的顺序。
  - 如果有一个较大的文件，在某磁道、某记录面的所有扇区内存放不下时，应先改变记录面号（即换成与该磁道号对应的另一记录面存放），这样可避勉磁头的机械运动影响存取速度。只有当磁道号（柱面号）对应的所有记录都存放不下时，才改变磁道号（柱面号）。

PC中的外存储器



二、磁记录原理和记录方式



写1：线圈通以正向电流，使呈N-S状态

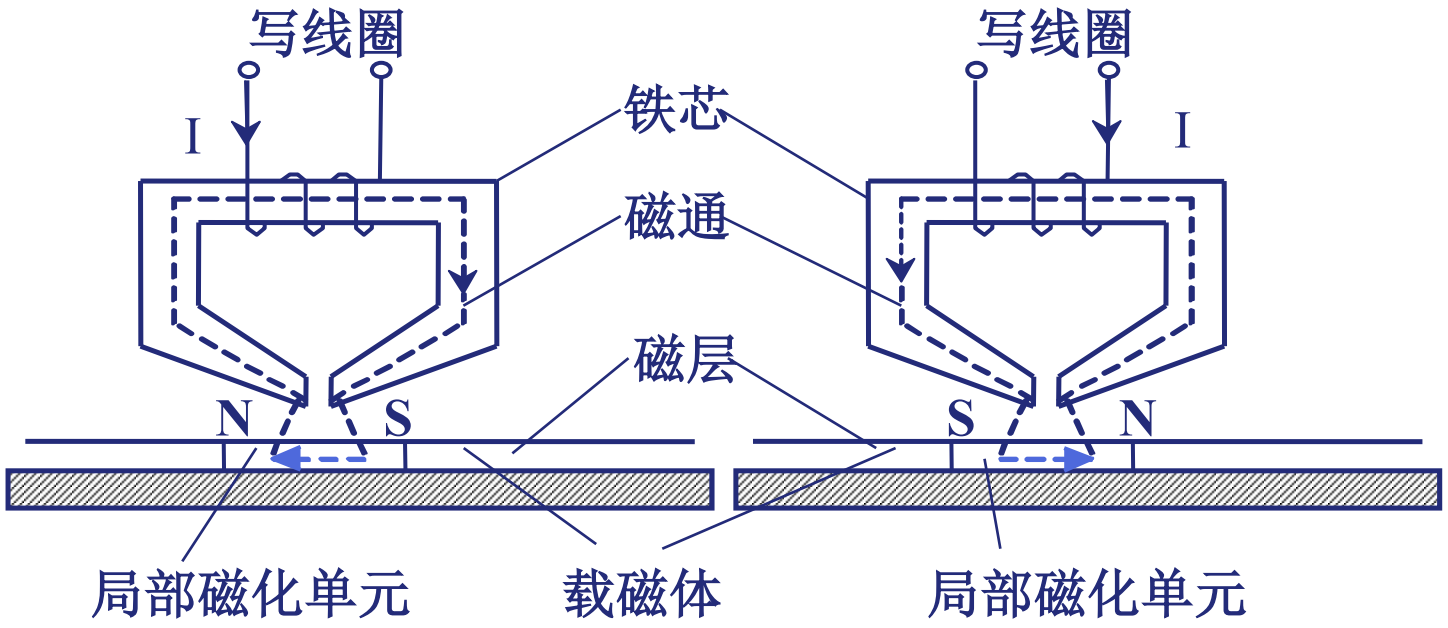
写0：线圈通以反向电流，使呈S-N状态

} 不同的磁化状态被记录在磁盘表面

读时：磁头固定不动，载体运动。因为载体上小的磁化单元外部的磁力线通过磁头铁芯形成闭合回路，在铁芯线圈两端得到感应电压。根据不同的极性，可确定读出为0或1。

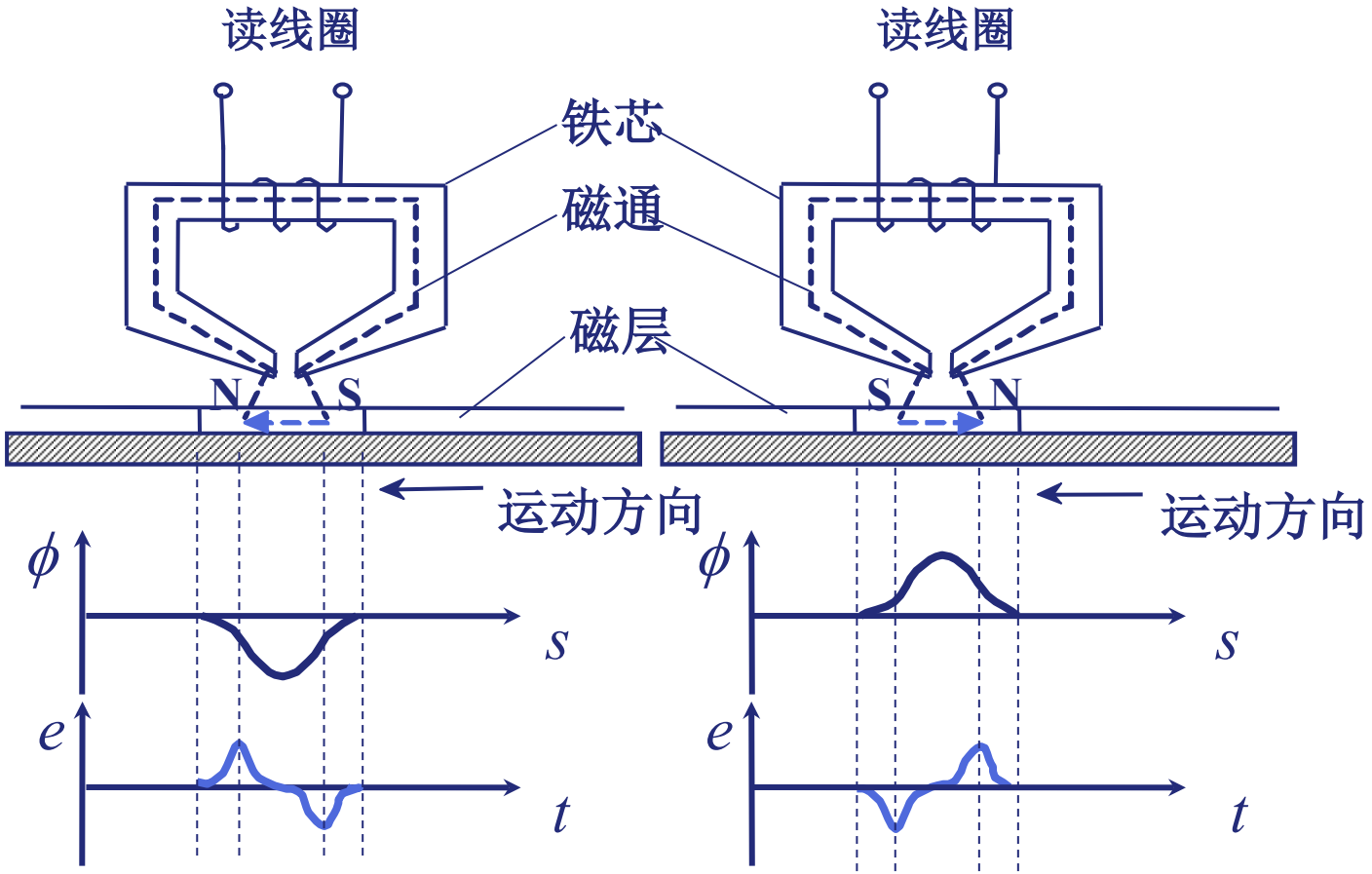
1. 磁记录原理

写



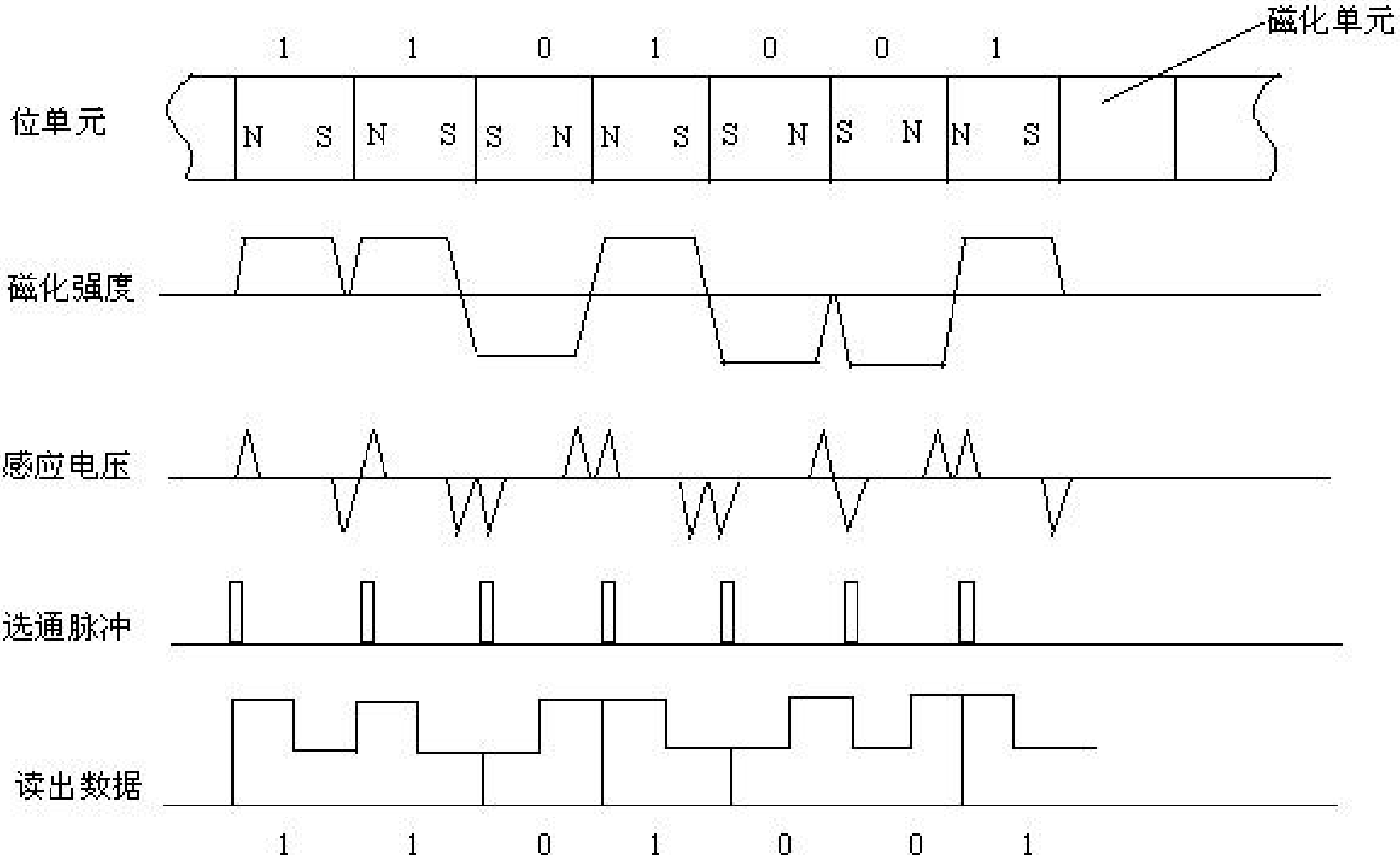
写入“0”　　写入“1”

读



读出“0”　　读出“1”

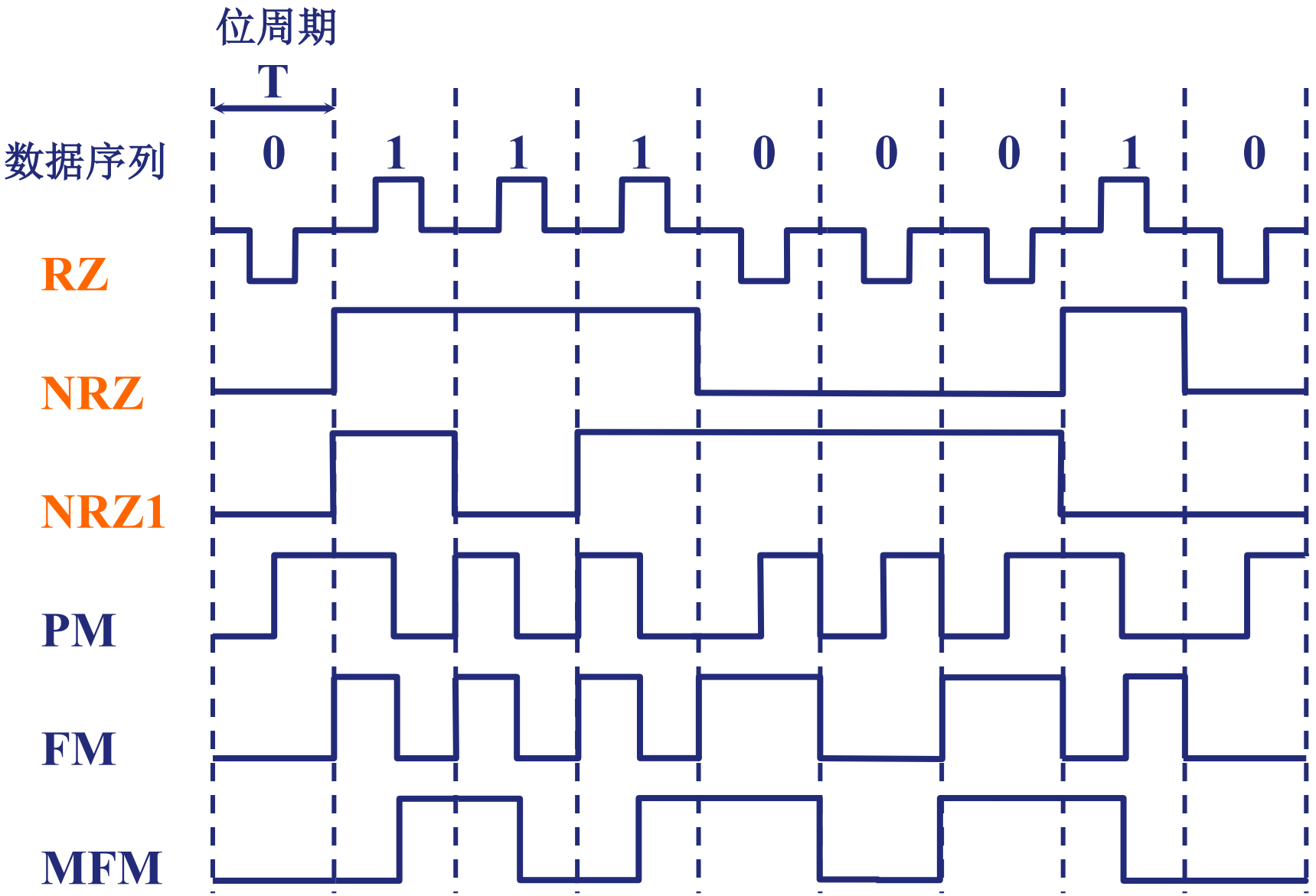
磁表面信息读出过程


磁化单元
1 1 0 1 0 0 1
位单元
N S N S S N N S S N S N N S
磁化强度
感应电压
选通脉冲
读出数据
1 1 0 1 0 0 1

2. 磁表面存储器的记录方式

- 磁记录方式是一种编码方式，即按照某种规律，把待写入的二进制信息转换成磁表面相应的磁化状态。实际就是将二进制信息转换成对应的写电流脉冲序列，写入电流波形的组成方式称为记录方式。
- 归零制（RZ，Return to Zero）记录方式中，写“1”时磁头线圈中加正向脉冲，写“0”时加负向脉冲。由于脉冲电流均要回到零，故称归零制
- 不归零制（NRZ，Non Return to Zero）：磁头线圈中始终有电源。写“1”时有正向电流，写“0”时有负向电流。由于磁头中电流不回到零，故称为不归零制。
- 见“1”就翻的不归零 - （NRZ - 1）这是一种改进的不归零制，记录“1”时，在位周期开始写电流改变方向；而记录“0”时，写电流方向维持不变，所以称之为见“1”就翻的不归零制。

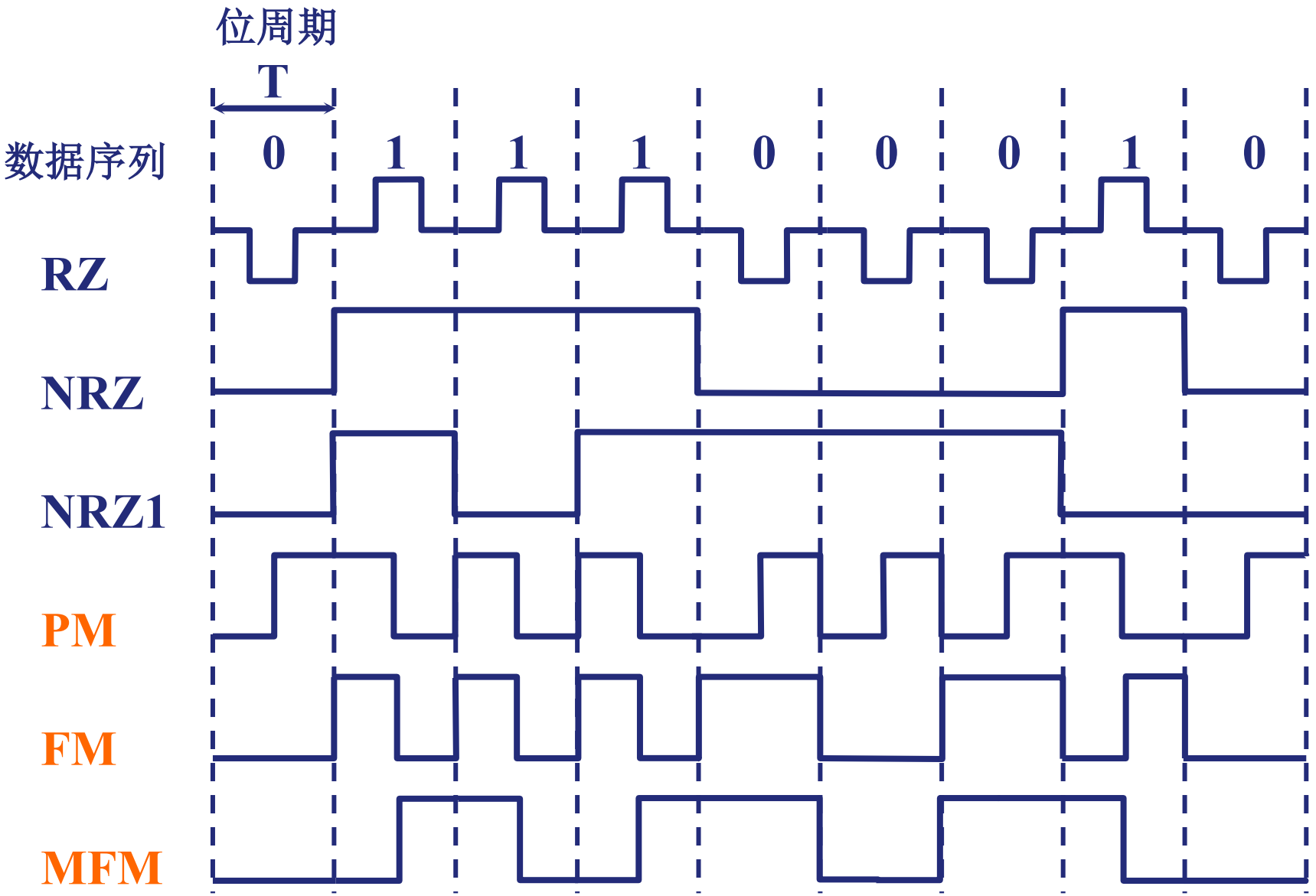
2. 磁表面存储器的记录方式



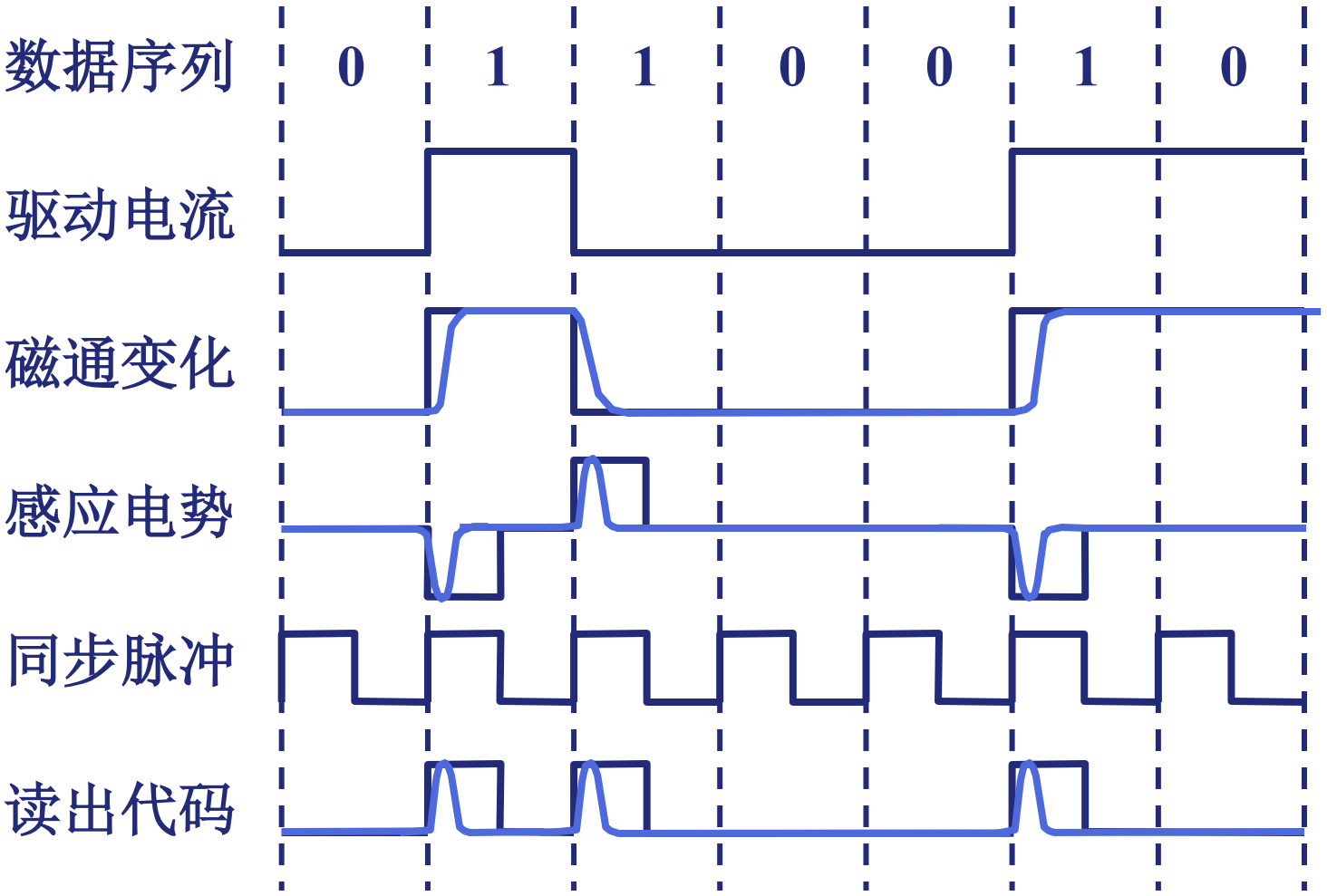
2. 磁表面存储器的记录方式

- 调相制（PE，Phase Encoding）：写“1”时，磁头线圈中的电流先正后负；写“0”时，电流先负后正。
- 调频制（FM，Frequency Modulation）：写“1”时，磁头线圈中的电流在位周期开始时改变一次方向，在位周期中间还要改变一次方向；写“0”时，磁头线圈中的电流只在位周期开始时改变一次方向。（差分曼彻斯特）
- 改进的调频制（MFM） 是对FM制改进，编码规则：记录“1”时，写电流在位周期中间改变方向；记录单独的一个“0”时，写电流不改变方向；记录连续的两个“0”时，写电流在位周期边界改变方向。

2. 磁表面存储器的记录方式



例 NRZ1 的读出代码波形



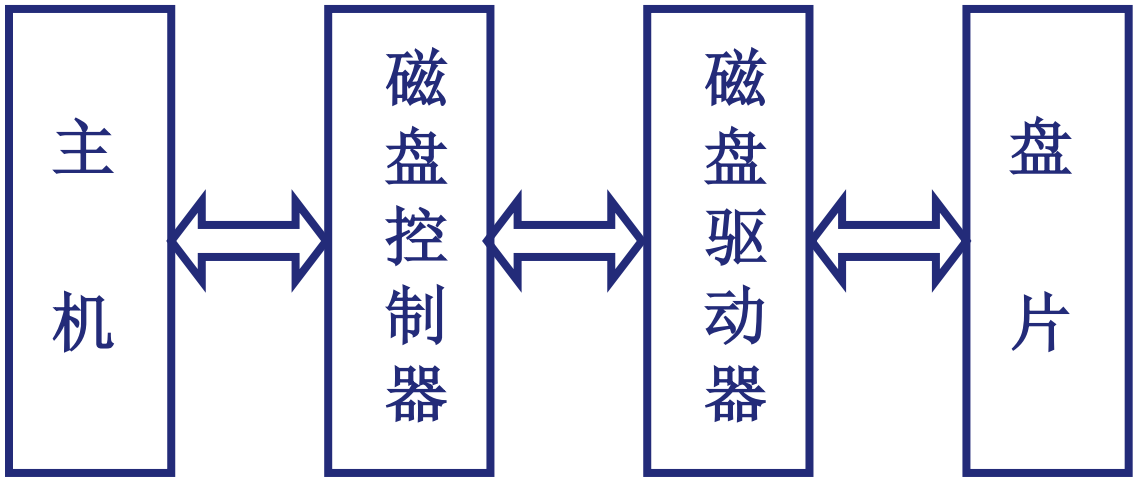
三、硬磁盘存储器

1. 硬磁盘存储器的类型

(1) 固定磁头和移动磁头

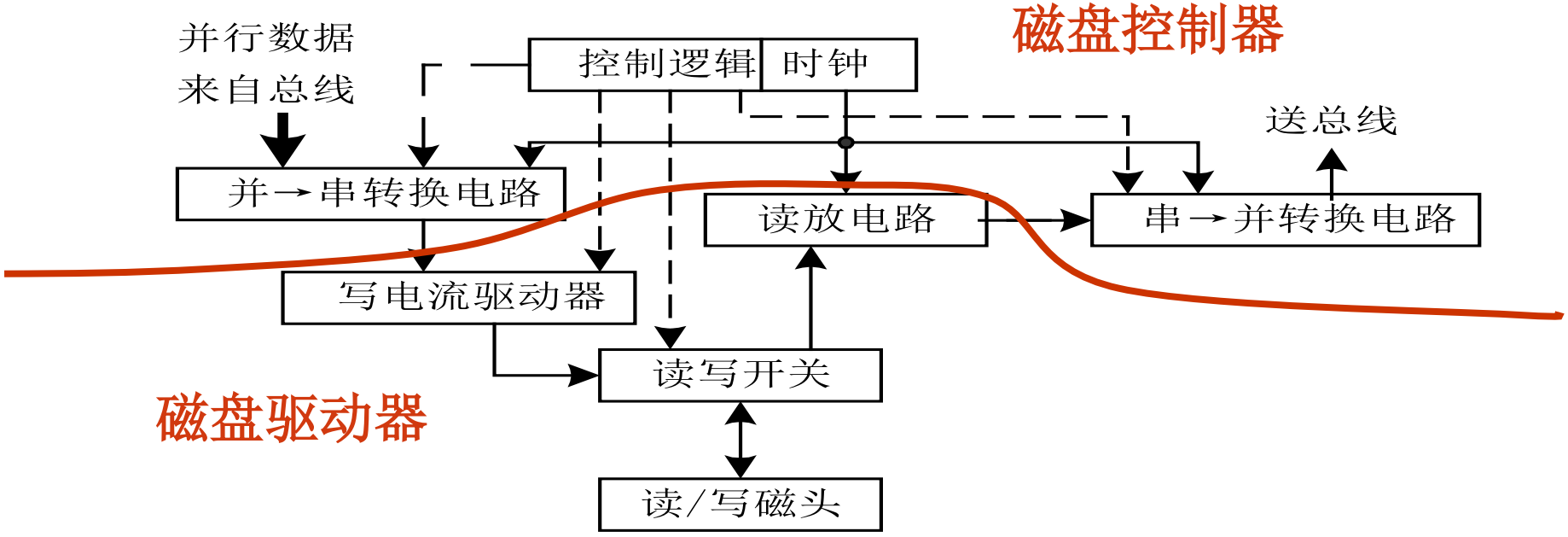
(2) 可换盘和固定盘

2. 硬磁盘存储器结构



硬盘存储器的组成

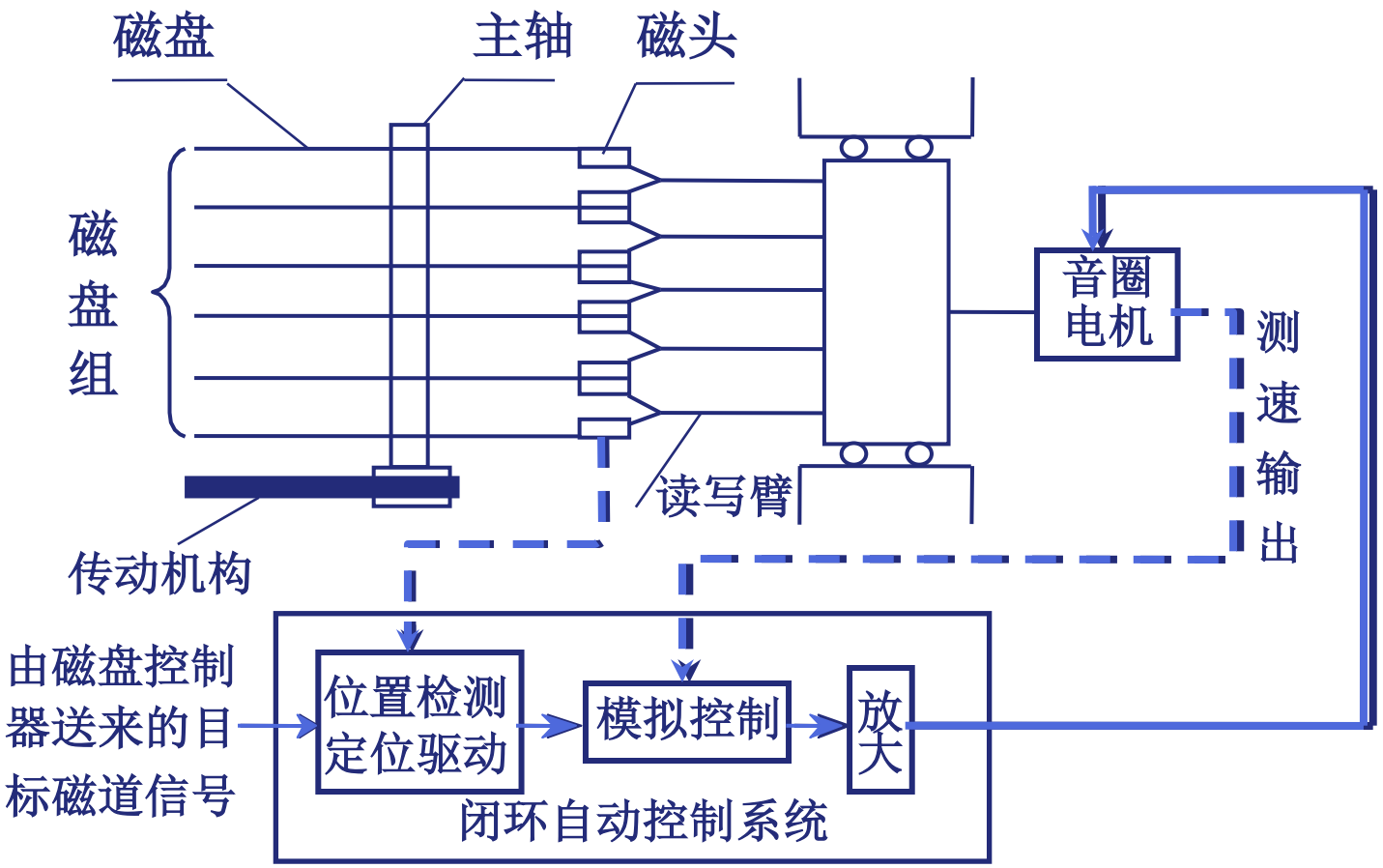
- 磁记录介质：用来保存信息
- 磁盘驱动器：包括读写电路、读/写转换开关、读写磁头与磁头定位伺服系统等
- 磁盘控制器：包括控制逻辑、时序电路、“并→串”转换和“串→并”转换电路等。（用于连接主机与盘驱动器）



硬盘存储器的逻辑结构

(1) 磁盘驱动器

主轴　　定位驱动　　数据控制



(2) 磁盘控制器

磁盘控制器 是

主机与磁盘驱动器之间的 接口 { 对主机 通过总线
对硬盘（设备）

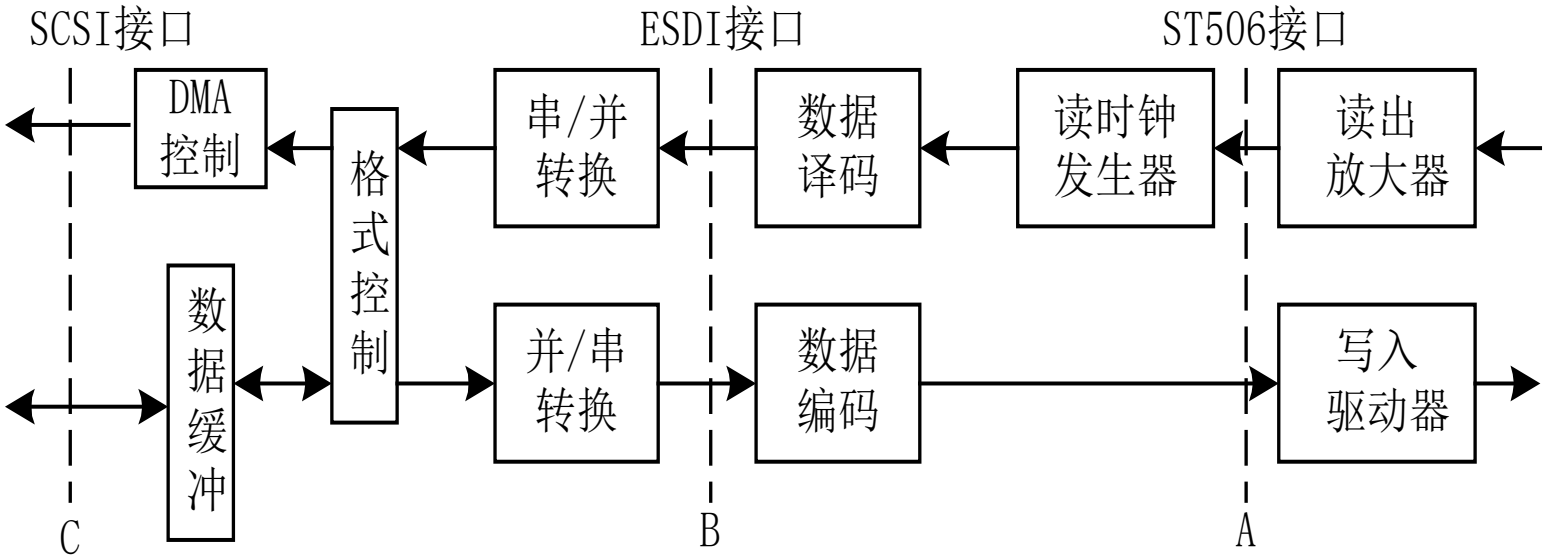
- 接受主机发来的命令，转换成磁盘驱动器的控制命令
- 实现主机和驱动器之间的数据格 式转换
- 控制磁盘驱动器读写

硬盘控制器的逻辑结构

- 磁盘控制器是主机与磁盘驱动器之间的接口
- 磁盘控制器与磁盘驱动器之间并没有明确的界线
  (可以在 A点 / B点 / C点)

通过总线与主机连接

通过接口电缆与磁盘驱动器连接



服务器大多使用SCSI接口

PC机前几年大多使用IDE(ATA)接口

近两年PC机开始大量使用SATA接口

(3) 盘片

磁层由非矩形剩磁特性的导磁材料（如氧化铁、镍钴合金）构成。导磁材料制成的磁胶涂敷或镀在载磁体上，厚度通常在0.1~5μm。载磁体可以是金属合金（硬质载磁体）或者塑料（软质载磁体）

冗余磁盘阵列(RAID)

- 系统总体性能的提高不匹配
  - 处理器和主存性能改进快
  - 辅存性能性能改进慢
- 所用措施：RAID-Redundant Arrays of Inexpensive Disk （磁盘冗余阵列）
- **RAID的基本思想：**

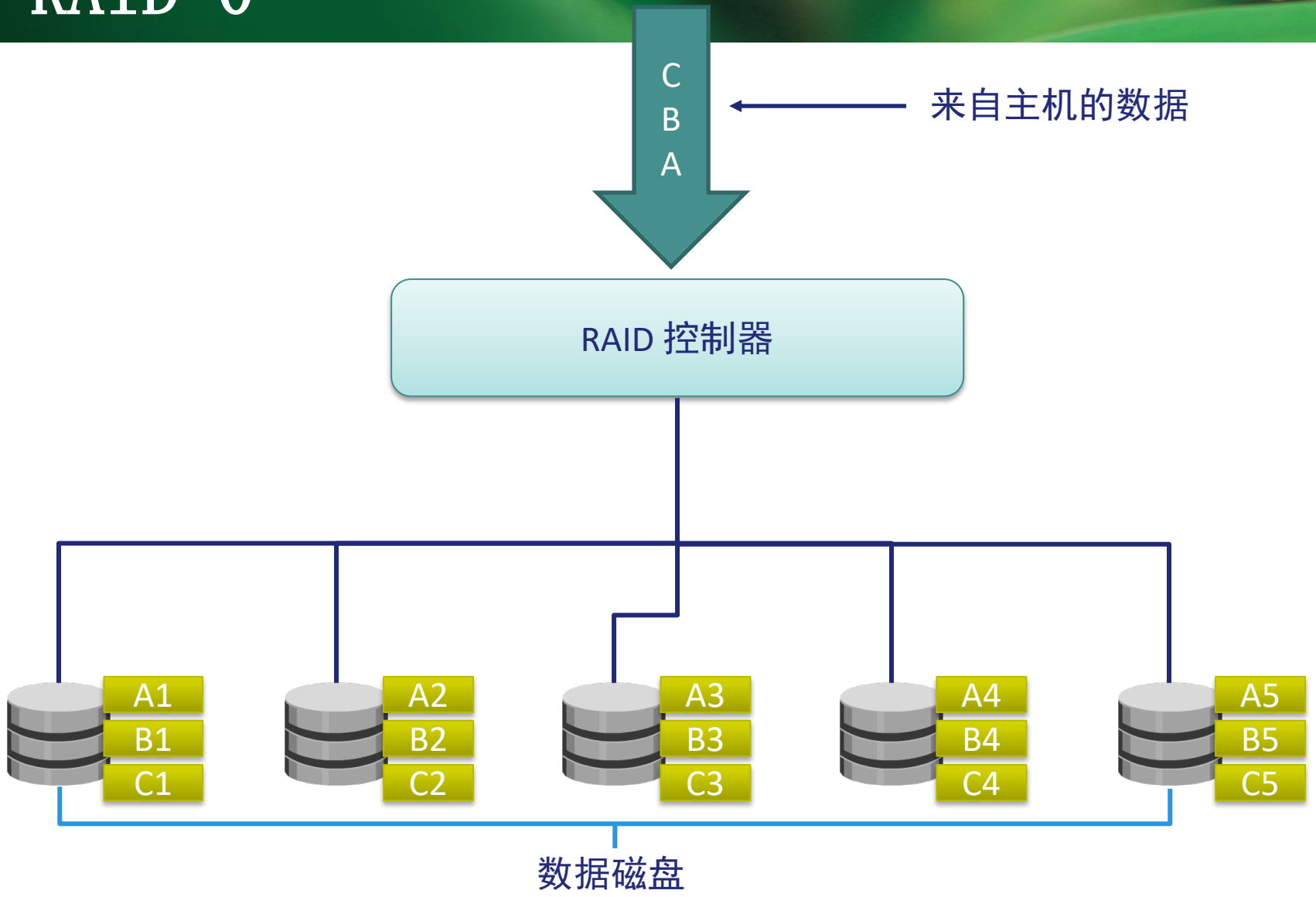
  将多个独立操作的磁盘按某种方式组织成磁盘阵列(Disk Array)，以增加容量，利用类似于主存中的多体交叉技术，将数据存储在多个盘体上，通过使这些盘并行工作来提高数据传输速度，并用冗余(redundancy)磁盘技术来进行错误恢复(error correction)以提高系统可靠性。
- **RAID特性：**

  （1）RAID是一组物理磁盘驱动器，在操作系统下被视为一个单个逻辑驱动器。

  （2）数据分布在一组物理磁盘上。

  （3）冗余磁盘用于存储校验信息，保证磁盘万一损坏时能恢复数据。
- **RAID级别**
  - **目前已知的RAID方案分为8级（0-7级），以及RAID10（结合0和1级）和RAID30（结合0和3级）和 RAID50 （结合0和5级） 。但这些级别不是简单地表示层次关系，而是表示具有上述3个共同特性的不同设计结构。**

RAID 0

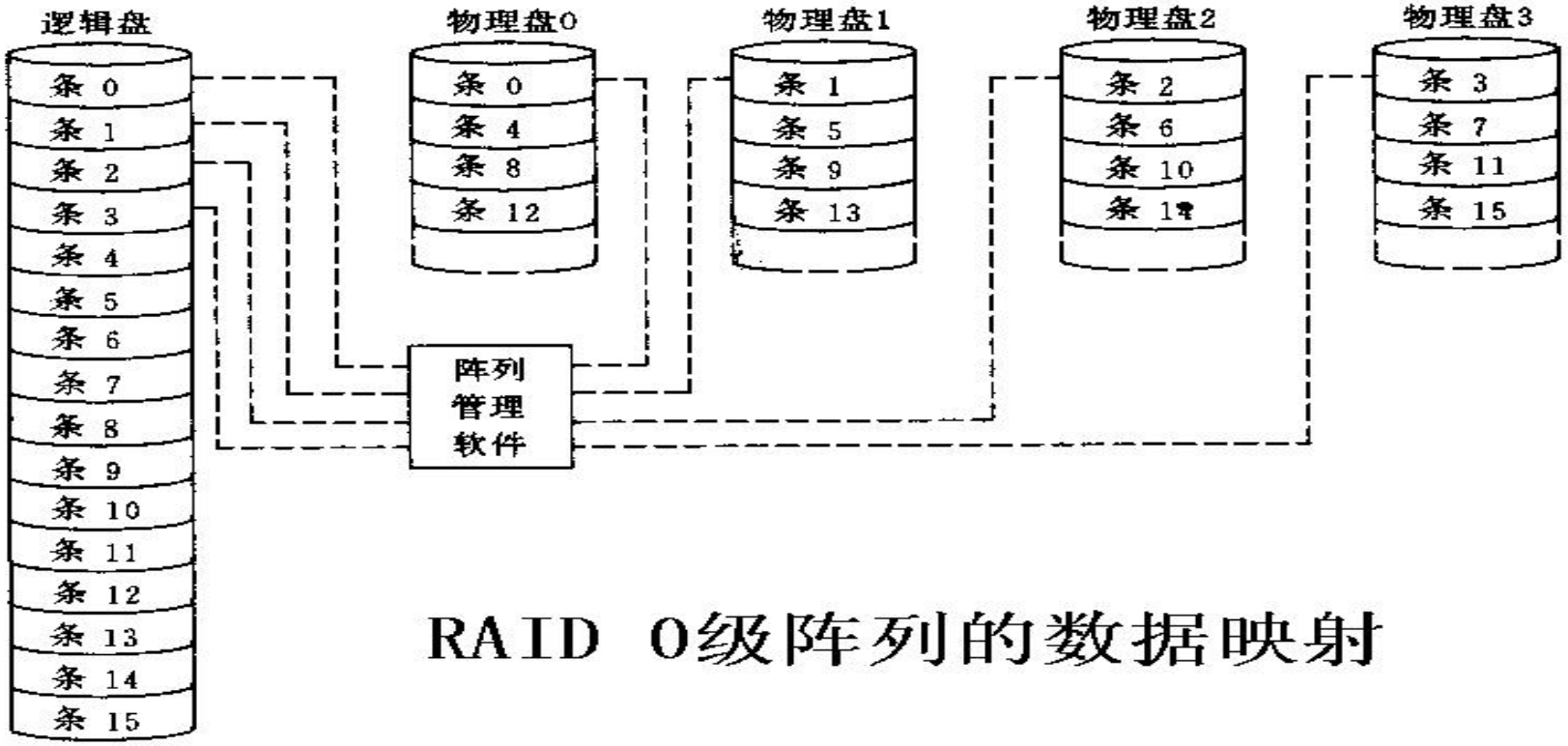

C
B
A
来自主机的数据
RAID 控制器
A1
B1
C1
A2
B2
C2
A3
B3
C3
A4
B4
C4
A5
B5
C5
数据磁盘

冗余磁盘阵列（ RAID 0 ）

- 不遵循特性(3)，所以无冗余。适用于容量和速度要求高的非关键数据存储的场合
  - 与单个大容量磁盘相比有两个优点：

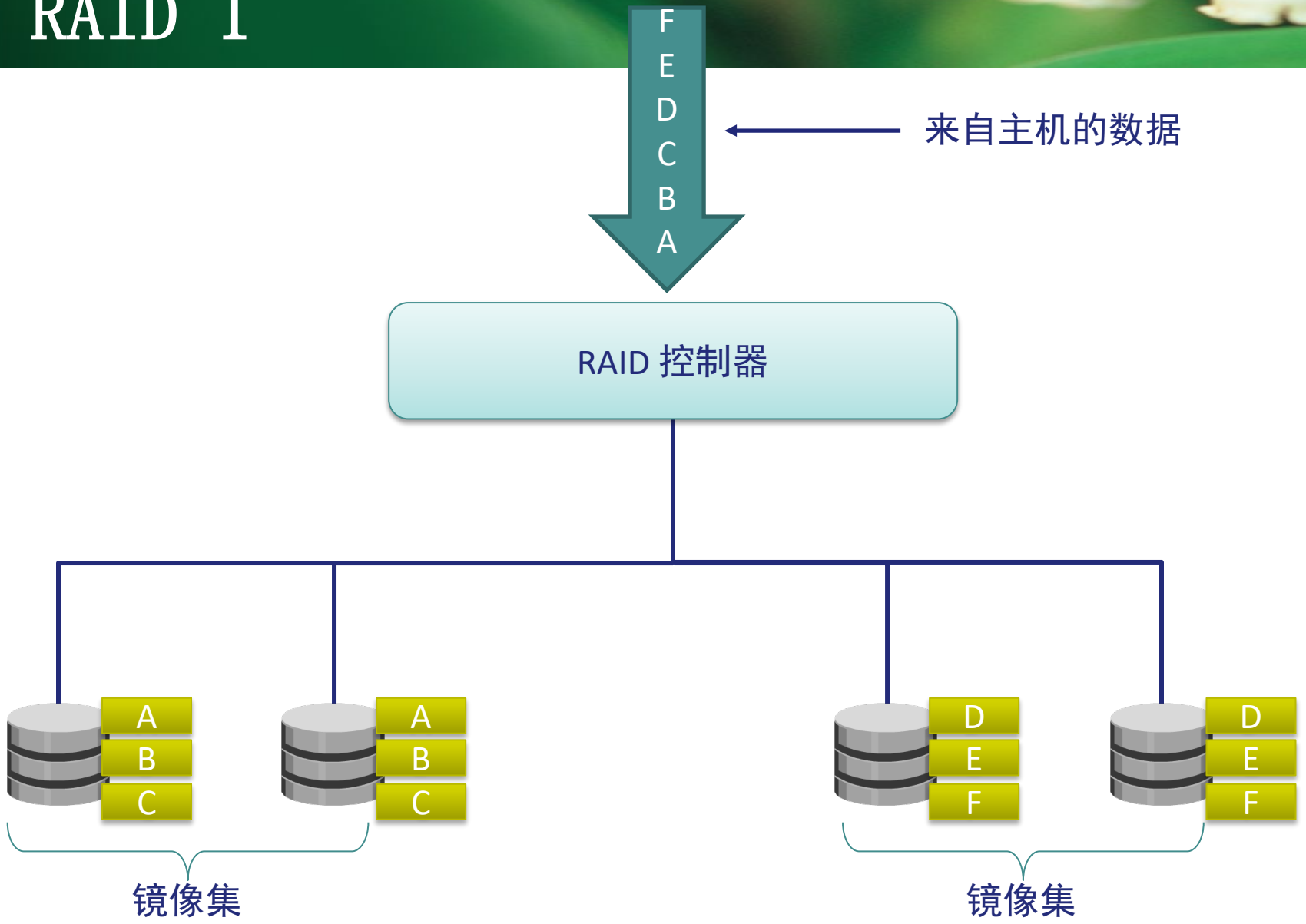
(1) 连续分布或大条区交叉分布时，如果两个I/O请求访问不同盘上的数据，则可并行发送。减少了I/O排队时间。具有较快的I/O响应能力。

(2) 小条区交叉分布时，同一个I/O请求有可能并行传送其不同的数据块(条区)，因而可达较高的数据传输率。例如，可以用在视频编辑和播放系统中，以快速传输视频流



RAID 0级阵列的数据映射

RAID 1



冗余磁盘阵列（ RAID 1 ）

- 镜像盘实现1对1冗余(100% redundancy)

（1）读：一个读请求可由其中一个定位时间更少的磁盘提供数据。

（2）写：一个写请求对应的两个磁盘并行更新。故写性能由两次中较慢的一次写来决定，即定位时间更长的那一次。

（3）检错：数据恢复很简单。当一个磁盘损坏时，数据仍能从另一个磁盘中读取。

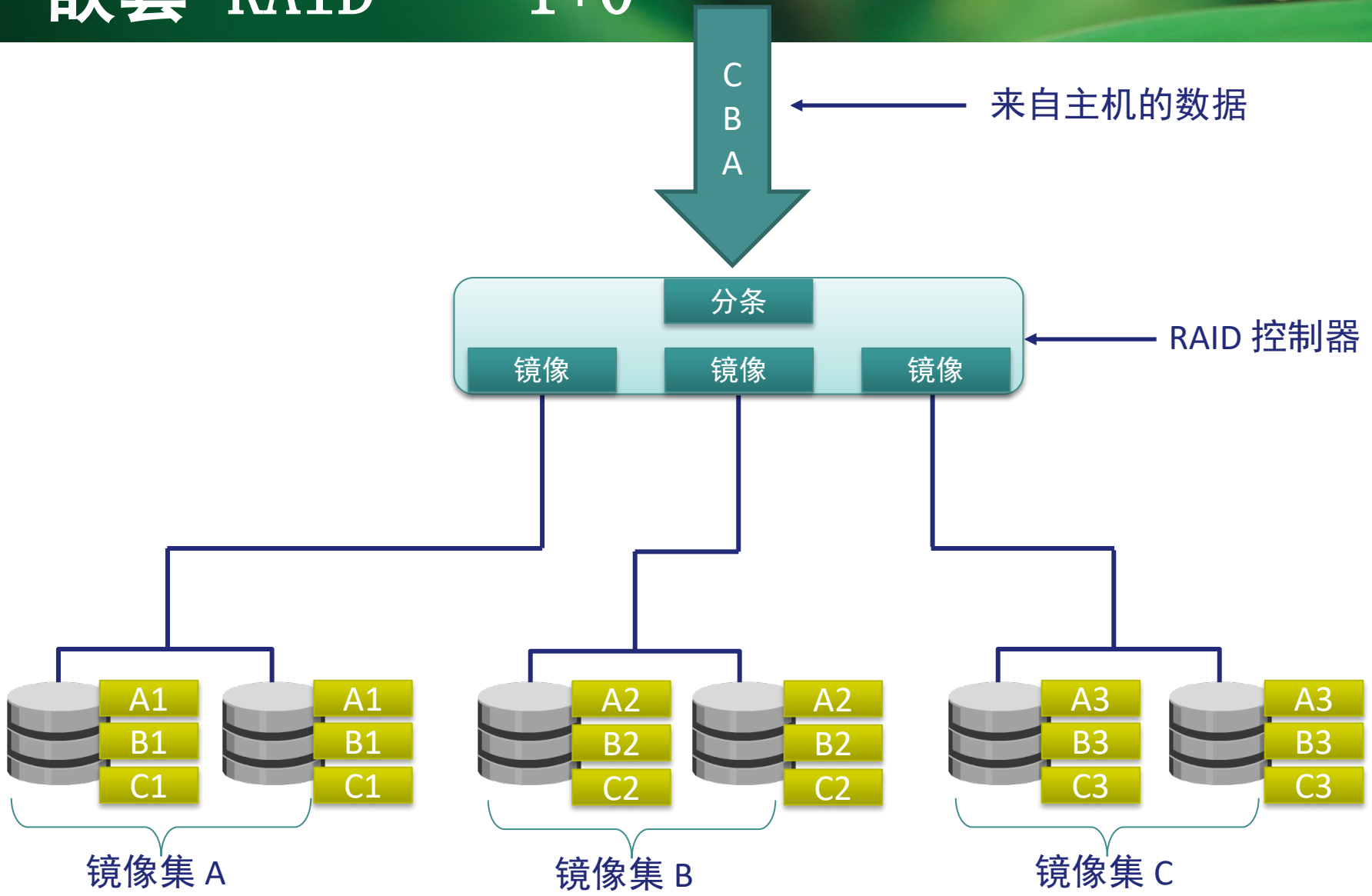
- 特点；可靠性高，但价格昂贵。

常用于可靠性要求很高的场合，如系统软件的存储，金融、证券等系统。



(b) RAID 1(镜像)

嵌套 RAID - 1+0

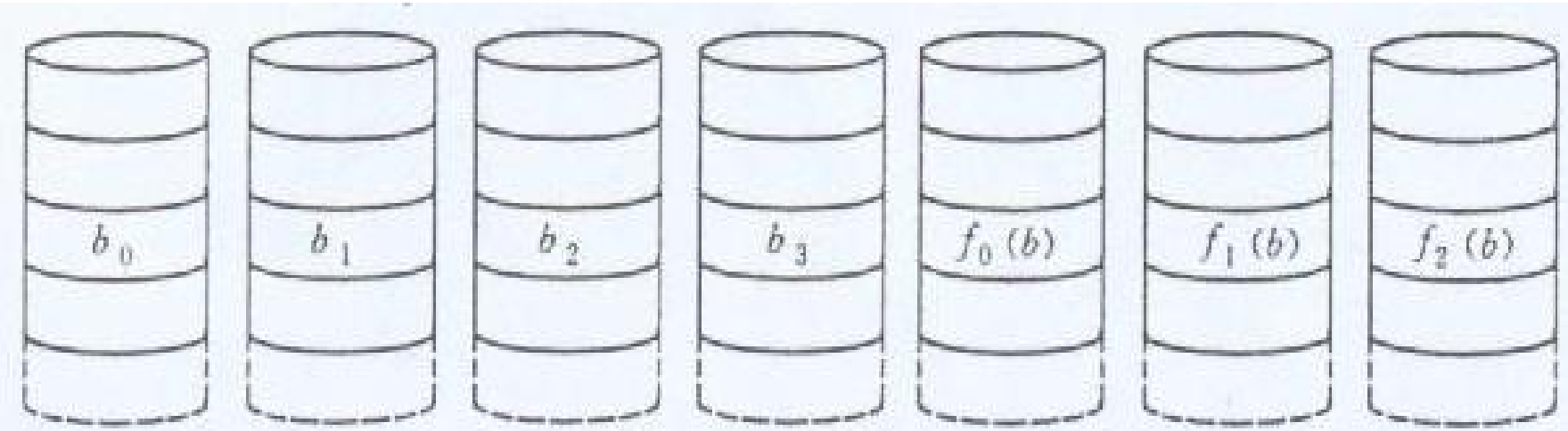

C
B
A
来自主机的数据
分条
镜像
镜像
镜像
RAID 控制器
A1
B1
C1
A1
B1
C1
A2
B2
C2
A2
B2
C2
A3
B3
C3
A3
B3
C3
镜像集 A
镜像集 B
镜像集 C

冗余磁盘阵列（ RAID2 ）

- 用海明校验法生成多个冗余校验盘，实现纠正一位错误、检测两位错误的功能。
- 采用条区交叉分布方式，且条区非常小（有时为一个字或一个字节）。这样，可获得较高的数据传输率，但I/O响应时间差。
- 采用海明码，虽然冗余盘的个数比RAID1少，但校验盘与数据盘成正比。所以冗余信息开销还是太大，价格也较贵
- 读操作性能高（多盘并行）。
- 写操作时要同时写数据盘和校验盘。

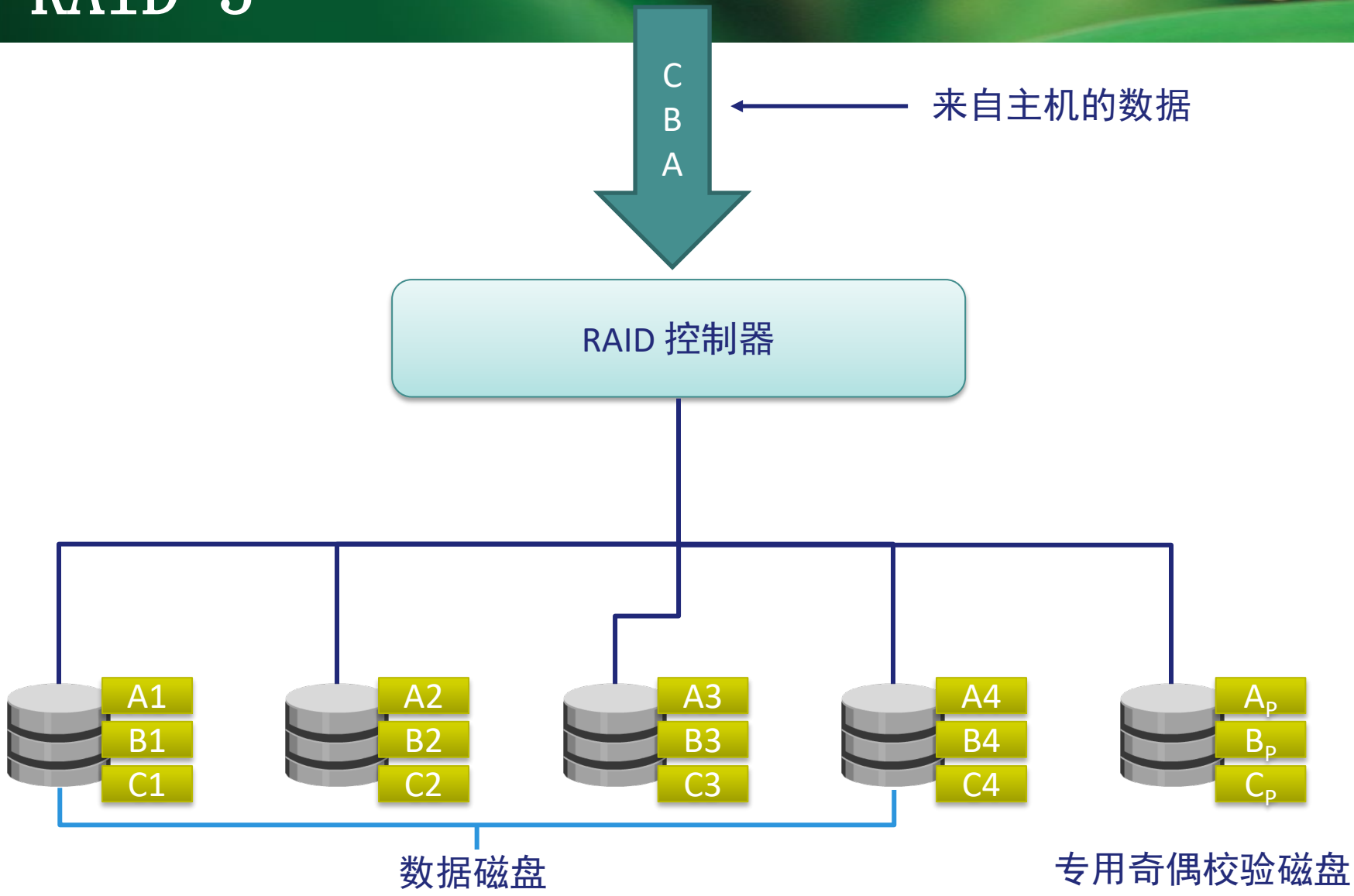
RAID2已不再使用！

为什么？



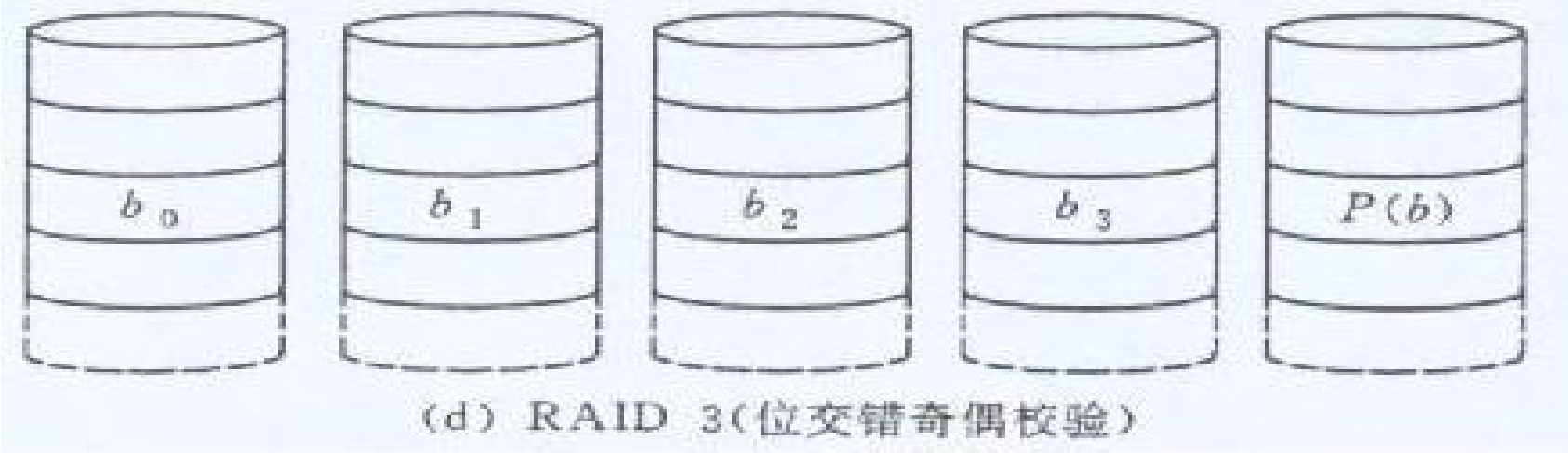
(c) RAID 2(冗余用于海明码)

RAID 3


C
B
A
来自主机的数据
RAID 控制器
A1
B1
C1
A2
B2
C2
A3
B3
C3
A4
B4
C4
A_P
B_P
C_P
数据磁盘
专用奇偶校验磁盘

冗余磁盘阵列（ RAID 3 ）

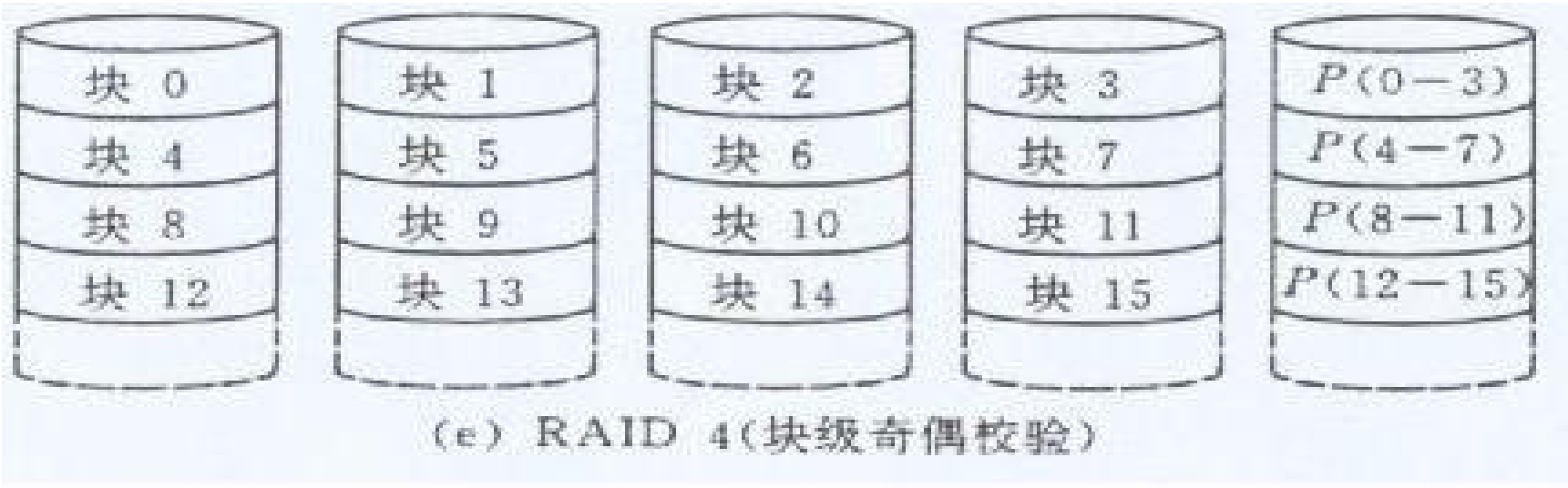
- 采用奇偶校验法生成单个冗余盘。
- 与RAID 2相同，也采用条区交叉分布方式，并使用小条区。这样，可获得较高的数据传输率，但I/O响应时间差。为什么？
- 用于大容量的 I/O请求的场合，如：图像处理、CAD 系统中。
- 某个磁盘损坏但数据仍有效的情况，称为简化模式。此时损坏的磁盘数据可以通过其它磁盘重新生成。数据重新生成非常简单，这种数据恢复方式同时适用于RAID3、4、5级。



(d) RAID 3(位交错奇偶校验)

冗余磁盘阵列（ RAID 4 ）

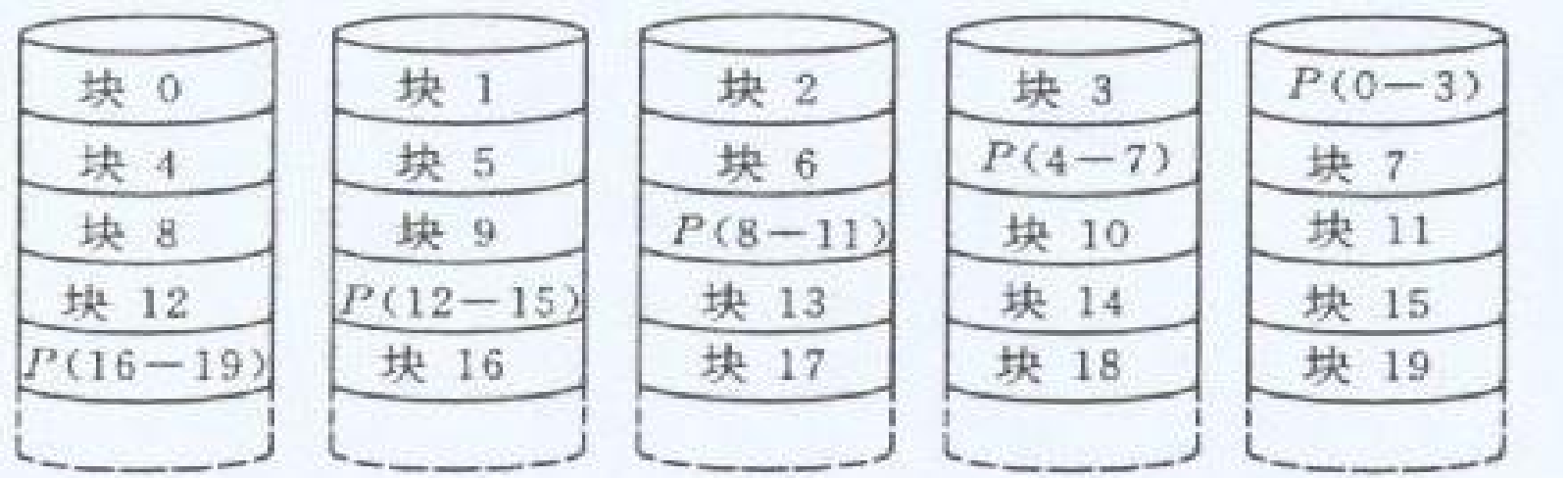
- 用一个冗余盘存放相应块（较大的数据条区）的奇偶校验位。
- 采用独立存取技术，每个磁盘的操作独立进行，所以可同时响应多个I/O请求。因而它适合于要求I/O响应速度块的场合。
- 对于写操作，校验盘成为I/O瓶颈，因为每次写都要对校验盘进行。
  - 少量写（只涉及个别磁盘）时，有“写损失”，因为一次写操作包含两次读和两次写
  - 大量写（涉及所有磁盘的数据条区）时，则只需直接写入奇偶校验盘和数据盘。因为奇偶校验位可全部用新数据计算得到。而无须读原数据



(e) RAID 4(块级奇偶校验)

冗余磁盘阵列（ RAID 5 ）

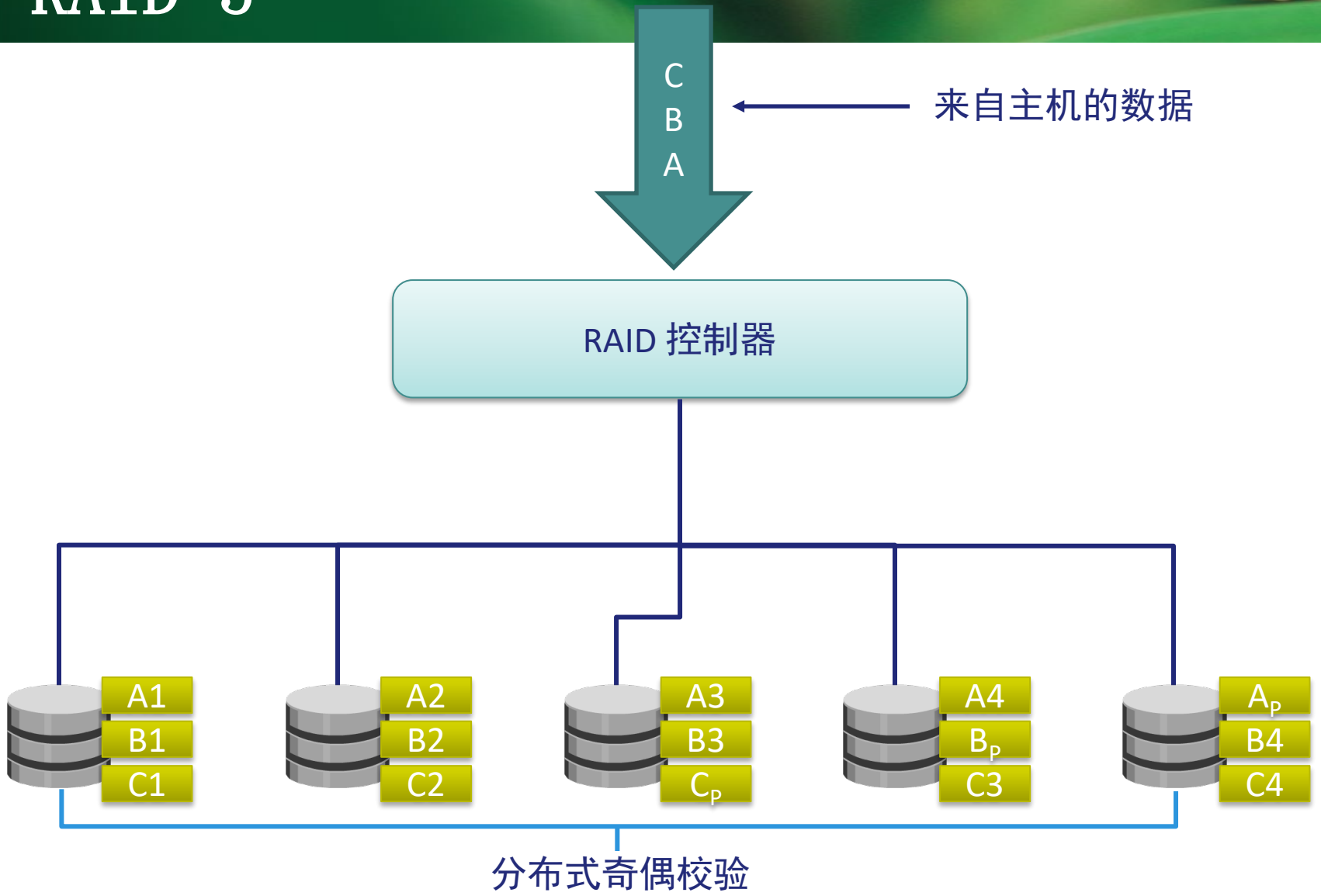
- 与RAID 4组织方式类似，只是奇偶校验块分布在各个磁盘中，所以所有磁盘地位等价，这样可提高容错性，并且避免了使用专门校验盘时潜在的I/O瓶颈。
- 与RAID 4一样，采用独立的存取技术，因而有较高的I/O响应速度。
- 小数据量的操作可以多个磁盘并行操作
- 成本不高但效率高，所以被广泛使用



(f) RAID 5(块级分布式奇偶校验)

P块为校验块，分布在不同的磁盘中

RAID 5


C
B
A
来自主机的数据
RAID 控制器
A1
B1
C1
A2
B2
C2
A3
B3
C_P
A4
B_P
C3
A_P
B4
C4
分布式奇偶校验

冗余磁盘阵列（ RAID 6 ）

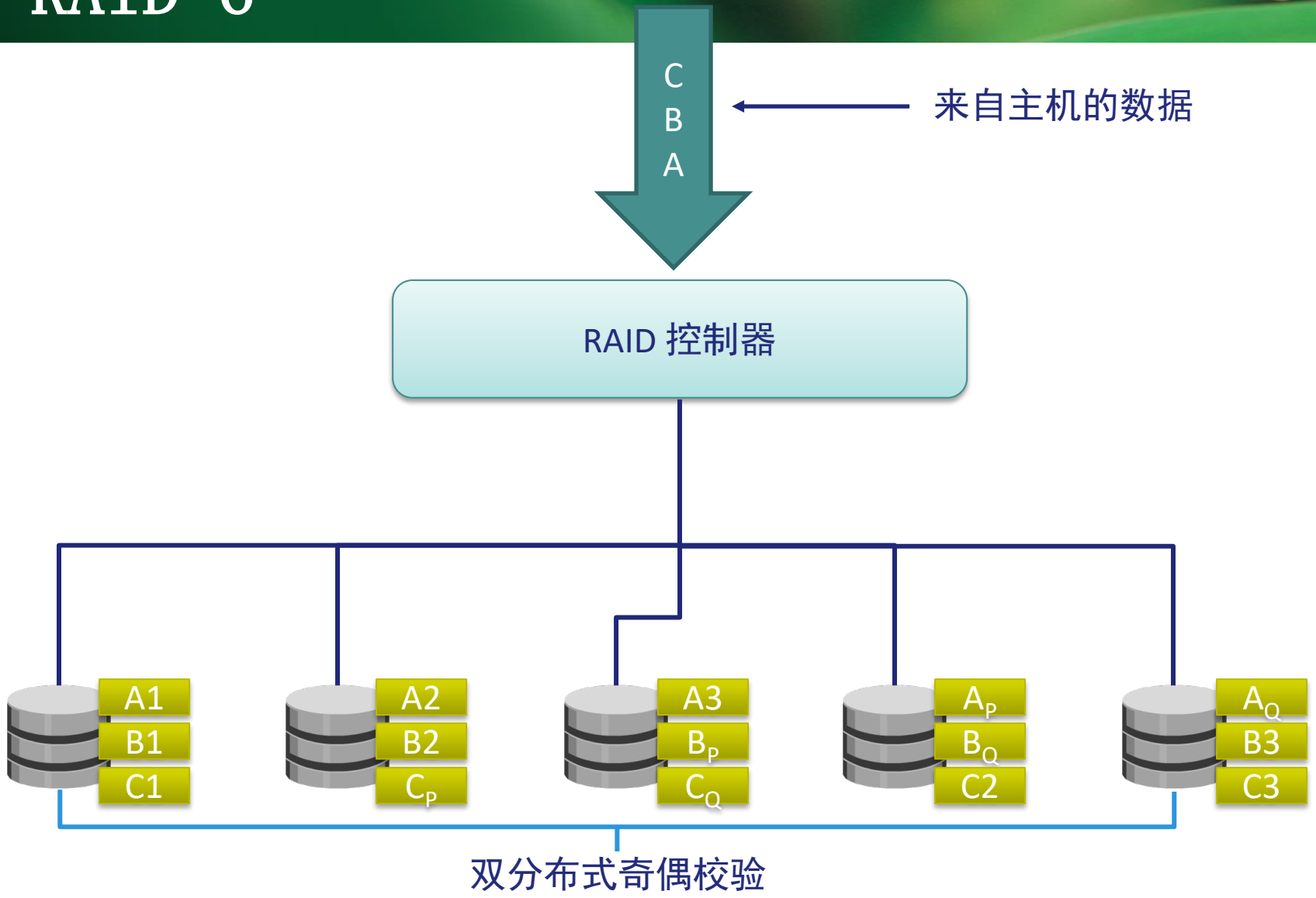
- 冗余信息均匀分布在所有磁盘上，而数据仍以块交叉方式存放
- 双维块交叉奇偶校验独立存取盘阵列，容许双盘出错
- 它是对RAID 5的扩展，主要是用于要求数据绝对不能出错的场合
- 由于引入了第二种奇偶校验值，对控制器的设计变得十分复杂，写入速度也比较慢，用于计算奇偶校验值和验证数据正确性所花费的时间比较多
- RAID 6级以增大开销的代价保证了高度可靠性



RAID 6 级双维块交叉奇偶校验独立存取盘阵列示意图

P0代表第0条区的奇偶校验值，而PA代表数据块A的奇偶校验值

RAID 6


C
B
A
来自主机的数据
RAID 控制器
A1
B1
C1
A2
B2
C_P
A3
B_P
C_Q
A_P
B_Q
C2
A_Q
B3
C3
双分布式奇偶校验

冗余磁盘阵列（ RAID 7 ）

- 带Cache的盘阵列
- 在RAID6的基础上，采用Cache技术使传输率和响应速度都有较大提高
- Cache分块大小和磁盘阵列中数据分块大小相同，一 一对应
- 有两个独立的Cache，双工运行。在写入时将数据同时分别写入两个独立的Cache，这样即使其中有一个Cache出故障，数据也不会丢失
- 写入磁盘阵列以前，先写入Cache中。同一磁道的信息在一次操作中完成
- 读出时，先从Cache中读出，Cache中没有要读的信息时，才从RAID中读

Cache和RAID技术结合，弥补了RAID的不足（如：分块的写请求响应性能差等），从而以高效、快速、大容量、高可靠性，以及灵活方便的存储系统提供给用户

四、软磁盘存储器

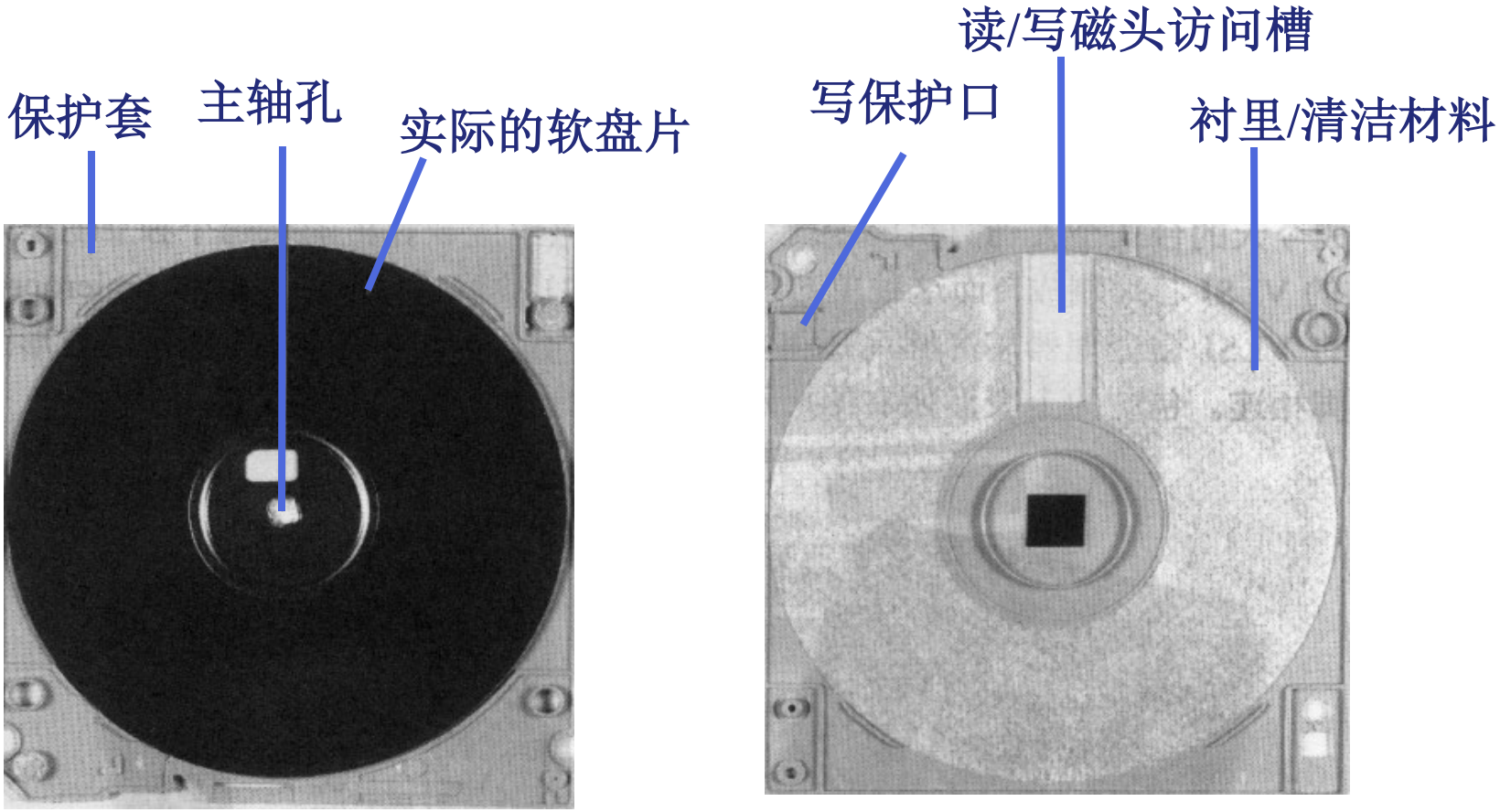
4.4

1. 概述

	硬盘	软盘
速度	高	低
磁头	固定、活动	活动
	浮动	接触盘片
盘片	固定盘、盘组 大部分不可换	可换盘片
价格	高	低
环境	苛刻	

2. 软盘片

由聚酯薄膜制成



五、光盘

4.4

1. 概述

采用光存储技术　　利用激光写入和读出

- 第一代光存储技术　采用非磁性介质　不可擦写
- 第二代光存储技术　采用磁性介质　可擦写

2. 光盘的存储原理

只读型和只写一次型　热作用（物理或化学变化）

可擦写光盘　　热磁效应

六、固态硬盘（SSD）

4.4

1. 概述

利用闪存技术存储　　浮动栅

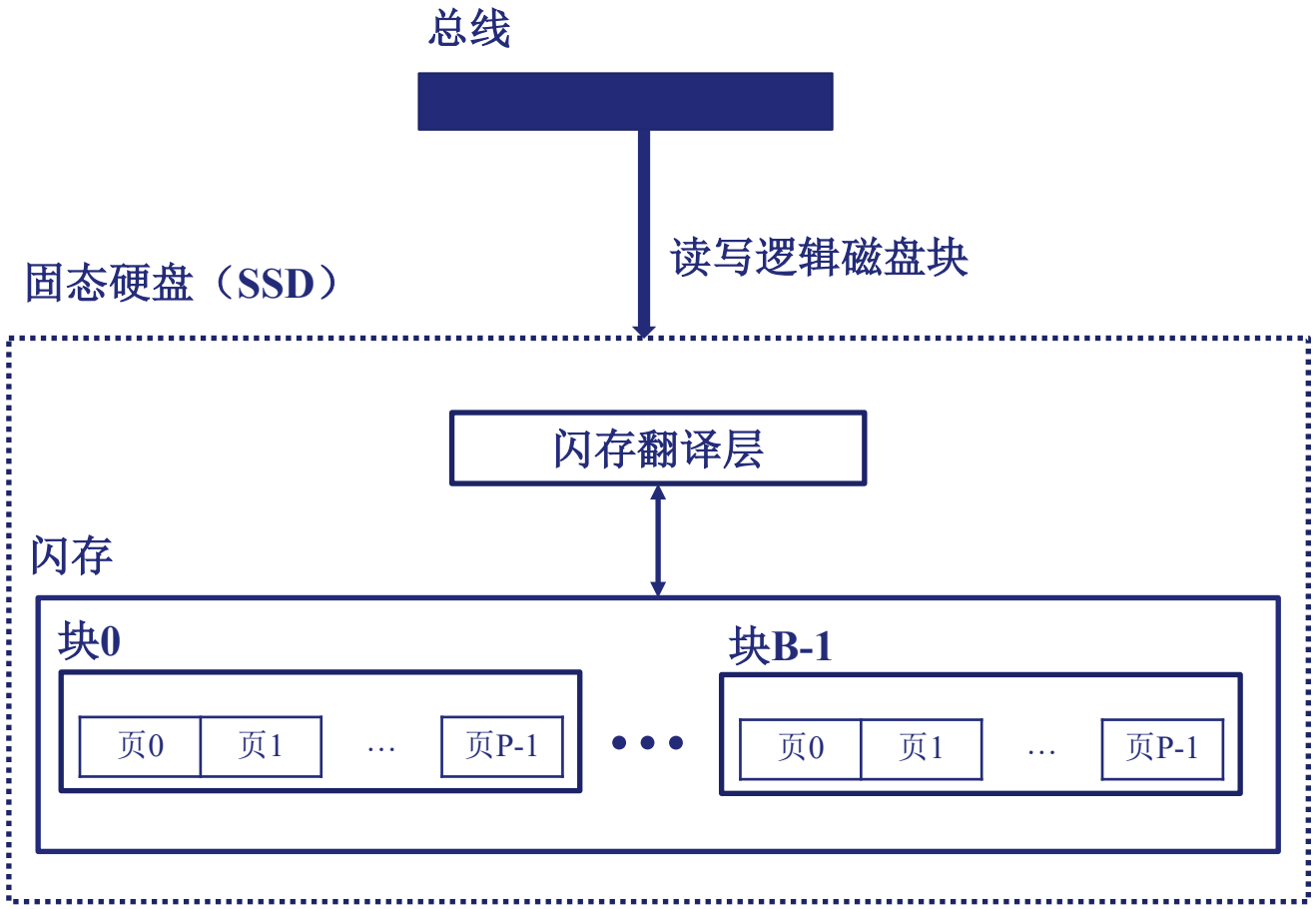
本质上与U盘没有差别 速度更快，集成度、性能更好

2. 固态硬盘的存储原理

一个或多个闪存芯片和闪存翻译层构成

固态硬盘 { 闪存芯片 → 机械驱动器；闪存翻译层 → 磁盘控制器 } 机械硬盘

固态硬盘



- 一个闪存由B块组成，每块由P页组成；
- 块的大小为16-512KB；
- 每块由32-128页组成，页的大小是512-4KB。

- 数据以页为单位读写；
- 在一页所属的整个块被擦除后，才能写这一页。

4.5 虚拟存储器

1. 概述

(1) 虚拟存储器：建立在主存与辅存物理结构基础之上，由附加硬件装置以及操作系统存储管理软件组成的一种存储体系。

- 主存和辅存统一编址，形成一个庞大存储空间，在这个大空间中，用户可以自由编程。
- 编好的程序由计算机操作系统装入辅助存储器中，程序运行时，附加的辅助硬件机构和存储管理软件会把辅存的程序一块块自动调入主存，由CPU执行或从主存调出。

(2) 虚地址和实地址

- 虚地址（虚拟地址、逻辑地址）：用户编程时指令地址允许涉及到辅存的空间范围。虚地址对应的存储空间称为“虚拟空间”或叫“逻辑空间”。

- 实地址（主存地址、物理地址）：实际的主存储器单元的地址实地址对应“主存空间”或称“物理空间”。

(3) 主存-辅存层次与cache-主存层次的比较

- 存储层次之间基本信息传递单位：块
  - cache-主存层次每次传递定长信息块，长度只有几十字节
  - 虚拟存储器信息块以多种形式划分，由页、段等，长度均在几百字节至几十万字节左右
- 目标：速度、容量
  - cache-主存层次：用快速存储元件弥补主存与CPU间速度差距
  - 虚拟存储器：弥补主存容量不足的问题，具有容量大和程序编址方便的优点。
- cache比主存快5~10倍，主存比辅存快100~1000倍
- cache-主存体系中CPU与cache和主存都建立了直接访问的通路；而辅存与CPU间没有直接通路，一旦在主存中不命中，则只能从辅存调度信息块到主存，但辅存与CPU速度差异太大，（调度为毫秒级），因此CPU将改执行另一个程序，调度完成后再返回原程序继续工作。

2. 虚拟存储器的结构

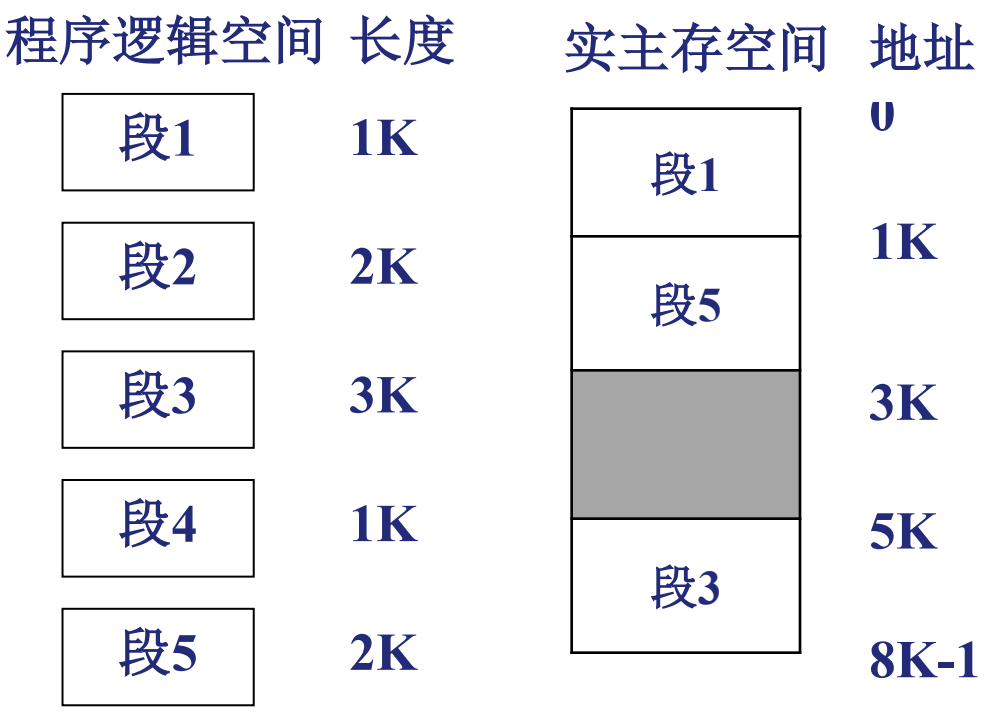
(1) 段式虚拟存储器：内存和辅存调入调出基本单位是段。

- 段是利用程序的模块化性质，按照程序的逻辑结构划分的多个相对独立的部分，例如，过程、子程序、一个数组、一张表等都可以作为一个段。段长度因程序而异，每段被安排一个段号。
- 段式虚拟存储器用段表来指明各段在主存中的位置。
- 当某段被调入内存，有了相应的物理地址，操作系统将虚存中的段号和该段在内存中的首地址记录下来建立一张段表，段表指明各段在内存中的位置及各段的段长。

2. 虚拟存储器的结构

(1) 段式虚拟存储器

- 逻辑地址与物理地址转换：根据该程序的段号查找段表，得到该段首地址，将段首地址与段内偏移地址相加得到物理地址。

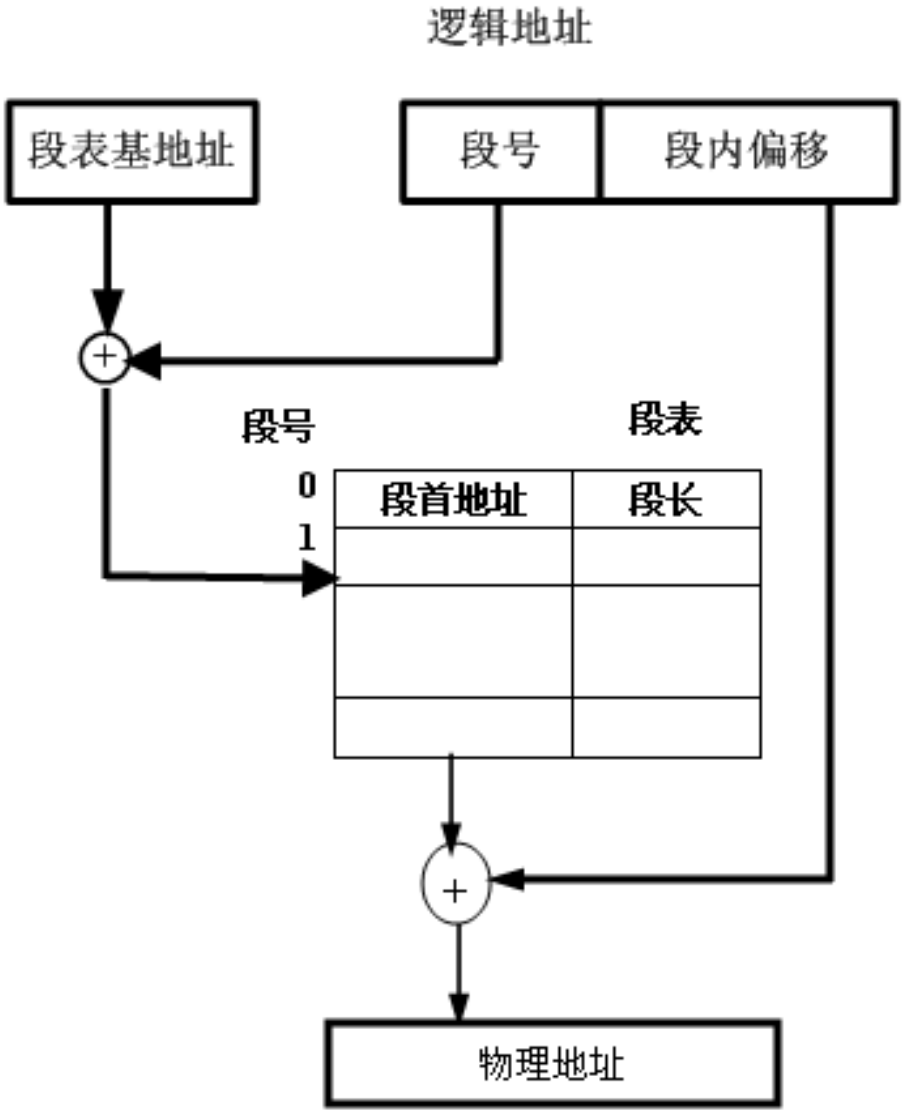


段表

段号	起始地址	装入位	段长
1	0	1	1K
2		0	
3	5K	1	3K
4		0	
5	1K	1	2K

（1）段式虚拟存储器

逻辑地址与物理地址转换：



2. 虚拟存储器的结构

(1) 段式虚拟存储器

- 优点：段的逻辑独立性使它易于编译、管理、修改和保护，还可以实现多道程序共享。
- 缺点：段的长度各不相同，起点终点不定，不利于调用时在内存中找到合适的空间，常导致在段间留下许多零碎的空余空间，因此不能很好地利用内存存储空间，从而造成浪费。

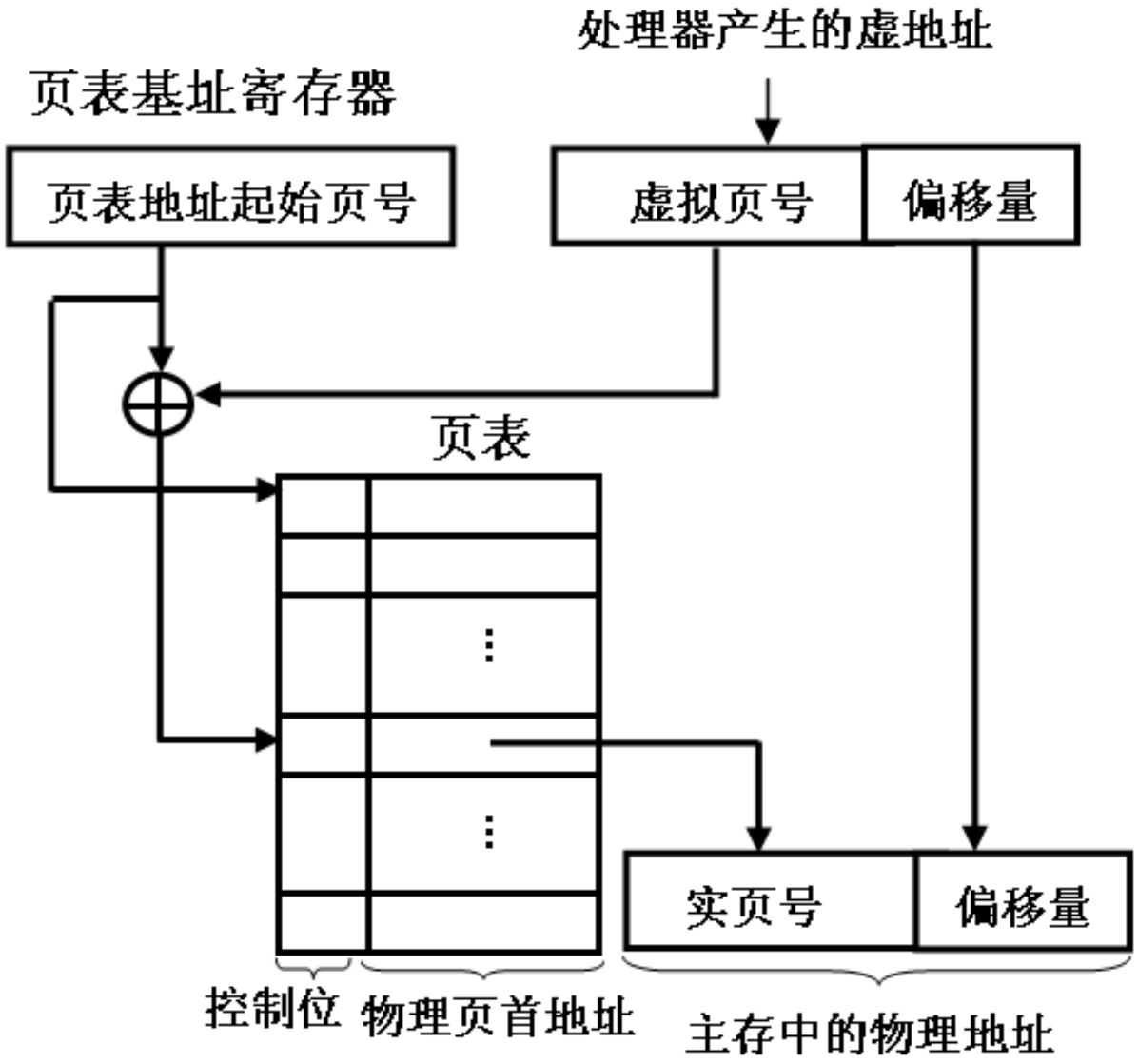
2. 虚拟存储器的结构

(2)页式虚拟存储器：将虚拟空间分成页，称为虚页或逻辑页，主存空间也分成同样大小的页，称为实页或物理页。

- 设虚页号为0, 1, 2, … , *m* ，实页号为0 , 1 , 2 , … , *l* , 则 m > l
- 虚地址分为两个字段：高位段为虚页号，低位段为页内字地址
- 虚地址到实地址变换由页表实现。
  - 页表中，对应每一个虚拟页号有一个表目，表目内容至少包含该虚拟页所在的主存页面号，用作主存地址的高字段；与虚拟地址的页内地址字段相拼接，产生完整的实主存地址，据此访问主存。

(2) 页式虚存

逻辑地址与物理地址转换:



(2)页式虚拟存储器

- 优点：页面起始地址和终止地址是固定的，便于构造页表，新页调入内存也很容易掌握，比段式管理的空间浪费小；
- 缺点：由于页不是逻辑上独立的实体，所以在处理、保护和共享上都不如段式方便。

例

设主存容量4MB，虚存容量1GB，页面大小为4KB。

（1）写出主存地址格式

（2）写出虚拟地址格式

（3）页表长度为多少？

（4）画出虚实地址转换图。

2. 虚拟存储器的结构

(3)段页式虚拟存储器：段页式存储管理系统把程序按逻辑结构分段，再把每段分成固定大小的若干页。

- 优点：程序对主存的调入调出是按页面进行的，但又可以按段实现共享和保护。因此兼具了页式和段式系统的优点。
- 缺点：地址映射过程中需要多次查表。
- 虚地址转换成物理地址是通过一个段表和一组页表来进行定位的。
- 段表中每个表目对应一个段，每个表目有一个指向该段的页表的起始页号及该段的控制保护信息。由页表指明该段各页在主存中的位置以及是否已装入、已修改等标志。

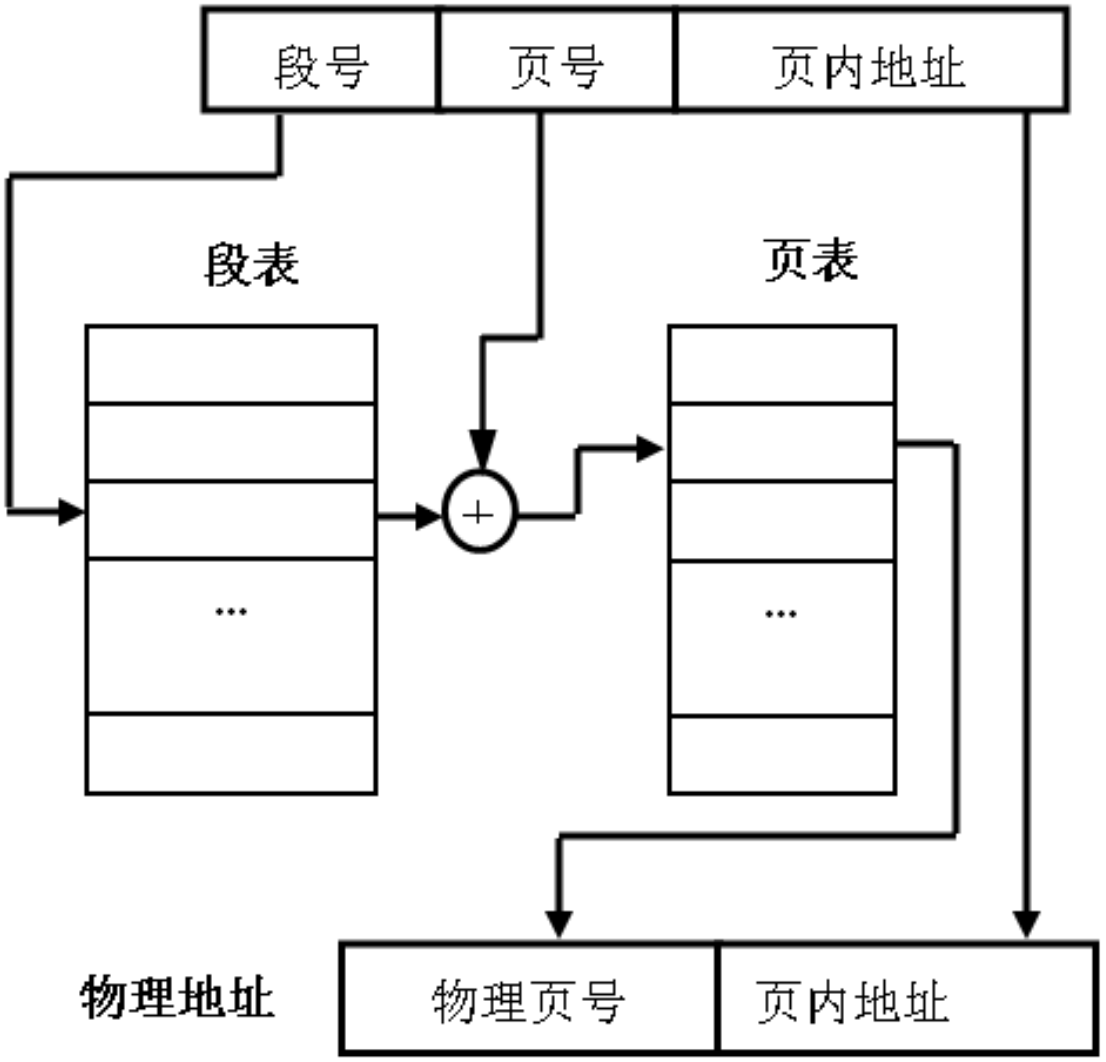
(3)段页式虚拟存储器

- 如果有多个用户在机器上运行，称为多道程序，多道程序的每一道需要一个基号（用户标识号），可以由基号指明该道程序的段表起点（存放在基址寄存器中）。
- 虚拟地址格式：

基号D	段号S	页号P	页内地址d

(3)段页式虚拟存储器

逻辑地址与物理地址的转换



2. 快表TLB

- 是一个专用的高速缓冲器，用于存放近期经常使用的页表项
- 其内容是页表部分内容的一个副本
- 快表和慢表同时查，快表中有，就能很快找到对应的物理页号送入实主存地址寄存器，从而做到虽采用虚存，但访主存速度几乎没有下降


Thank You !